

OfferExchange

nano

October 1, 2026

Contents

1	Abstract offer-exchange model	1
1.1	Isabelle/HOL Setup	1
1.2	Price	2
1.3	Offers	2
1.4	The ledger	3
1.5	Maximum unrestricted exchange	3
1.6	Valid offers	4
1.7	Trades	4
1.8	Posting an offer	5
1.9	Crossing a posted sell-wheat offer	6
1.10	Properties	11
2	Exchange arithmetic	15
2.1	The model and its main properties	15
2.1.1	Fixed-width C++ arithmetic	15
2.1.2	Wide multiplication	16
2.1.3	Checked division	17
2.1.4	Price error bound	18
2.1.5	Price-error application	19
2.1.6	Offer value	21
2.1.7	Protocol-29 amount examples	24
2.1.8	Exchange before price-error thresholds	25
2.1.9	Full exchange	30
2.1.10	Protocol-facing exchange interface	30
2.1.11	Main properties	33
2.2	Wide multiplication	35
2.3	Checked division	35
2.4	Price error bound	39
2.5	Price-error application	41
2.6	Offer value	44
2.7	Exchange before price-error thresholds	45
2.8	Full exchange	59

3	Offer adjustment and liabilities	67
3.1	Offer adjustment and offer liabilities	67
3.2	Protocol compatibility of the unlimited-cap projections	70
3.3	Reusable normalization and replay results	74
4	Minimal implementation posting slice	82
4.1	Posting-state invariant facts	86
4.2	Repaired adjustment arithmetic	87
4.3	Posting refinement against the specification	94
5	Layered model of the wide arithmetic helpers	101
5.1	Definitions	101
5.1.1	Examples	102
5.2	The 128-bit family	102
5.2.1	Examples	104
5.3	Wide multiplication	104
5.3.1	Examples	104
5.4	Reading the word arithmetic as integer arithmetic	105
5.5	Characterizations	106
5.6	Agreement with the collapsed model	107
5.7	Wide multiplication agrees with the collapsed model	107
5.8	The 128-bit family: word arithmetic as integer arithmetic . .	108
5.9	The 128-bit family: characterizations	109
5.10	The 128-bit family agrees with the collapsed model	110
6	Posting an offer and then crossing it	110
6.1	Parties	111
6.2	State-dependent adjustment and offer liabilities	113
6.3	Posting an offer	114
6.4	Crossing the posted offer	120
6.5	Executable offer lifecycle	122
6.6	The whole process	125
6.7	Intended properties	126
6.7.1	Rounding favors the offer that stays	127
6.7.2	Maximal positive normal crossing	127
6.7.3	Coverage implies adjustment stability	130
6.7.4	Takeability after adjusting the maker's sheep limit . .	130
6.7.5	Takeability after maker-state changes	132
6.7.6	Exact amount when a posted offer is fully taken . . .	134
6.7.7	Exact amount when an incoming offer is fully taken .	135
6.8	Proofs and counterexamples	136
6.8.1	Rounding favors the offer that stays	136
6.8.2	Unchanged-maker adjustment stability	136
6.8.3	Release, replay and capacity lemmas for posted offers	140

6.8.4	Maximality of positive normal crossings with both re-pairs	149
6.9	An options-parametric exact-integer abstraction	155
6.10	Strict-send positivity at the repaired record	161
6.11	Counterexample-first probes for the repaired strict modes . .	164
6.11.1	The reservation anomaly, replayed end to end	169
6.11.2	A sheep-limit adjustment can make an offer untakeable	172
6.12	Continuous specification and fixed-width refinement	173
6.12.1	Continuous normal exchange	173
6.12.2	Refinement relation	174
6.12.3	Real and integer bridge	175
6.12.4	Primary-unit proximity	177
6.12.5	End-to-end refinement	178
6.13	An unconstrained-maker benchmark, kept for reference	178
6.13.1	The shipped protocol differs from the benchmark	180
6.13.2	The exact receive cap repairs the walkthrough, not the benchmark comparison	180
6.14	Proofs and counterexamples: fully taken posted offers	181
7	Strict-mode contracts across the exact-cap boundary	187
7.1	Strict send	188
7.2	Strict receive, and simultaneous zero in general	189
8	Protocol-29 property catalogue	191
8.1	The exact-integer image at the protocol interface	194
8.2	Strict-mode contracts through the protocol mapping	195
8.3	Properties protocol 29 does not promise	197
9	Stored offers	197
9.1	Lifecycle results used here	198
9.2	Persistent liability compatibility	199
9.3	Unlimited replay of a protocol-28 adjustment	200
9.4	Preventative adjustment at protocol 29	201
9.5	Successful-cross safety infrastructure	201
9.6	No phantom full take	203
9.7	Existential safe full take	203
9.8	Positive remainder provenance and recursive takeability . . .	204
10	Protocol-29 takeability and limit-adjustment stability	206
11	Protocol-29 takeability and limit stability for ManageBuy	208
11.1	Stored-offer provenance for the common posting core	208
11.2	Takeability and limit stability	209

12 Exact-cap protocol migration	210
12.1 Counterexample-first executable probes	211
12.2 Dependency map for the migration proofs	215
12.3 Offers posted at protocol 28	215
12.4 Reachability boundary	217
13 The migration ladder at the real protocol boundary	217
14 The ManageBuy route at protocol 29	220
14.1 Counterexample-first executable probes	221
14.2 ManageBuy-created offers crossed at protocol 29	222

1 Abstract offer-exchange model

```
theory Offer-Exchange-Specification
imports
  Main
  HOL.Rat
  HOL-Library.Monad-Syntax
begin
```

In this theory we formalize the offer exchange process with the goal of maximum simplicity, as opposed to following the C++ implementation. Ideally, we will later modify the C++ implementation to follow this specification.

1.1 Isabelle/HOL Setup

In this section we do a little technical setup that can be ignored if you mostly care about the semantics of the offer exchange and not Isabelle/HOL technicalities.

Longer ordered validation chains use the guarded conditional notation below. The first true guard determines the result, and the final otherwise clause is the fallback. This is input syntax only: Isabelle immediately translates it to ordinary nested conditionals.

nonterminal *cond-cases*

syntax

```
-cond :: cond-cases  $\Rightarrow$  'a
  ((cond {/(2 -)/}) [12] 62)
-cond-when :: [bool, 'a, cond-cases]  $\Rightarrow$  cond-cases
  ((when -  $\Rightarrow$  / - / | -) [0, 13, 12] 12)
-cond-otherwise :: 'a  $\Rightarrow$  cond-cases
  ((otherwise  $\Rightarrow$  / -) [13] 12)
```

translations

```
-cond (-cond-when condition result rest)
   $\rightarrow$  CONST If condition result (-cond rest)
-cond (-cond-otherwise result)
   $\rightarrow$  result
```

This theory enables Isabelle's standard coercion from *int* (unbounded integer) to *rat* (unbounded-precision rational). An integer-valued ledger amount or price component used in a rational expression is therefore promoted automatically. This keeps exact prices written as ordinary quotients instead of obscuring the formulas with explicit conversion functions.

```
declare [[coercion-enabled]]
declare [[coercion of-int :: int  $\Rightarrow$  rat]]
```

1.2 Price

datatype *ledger-price* =
 Ledger-Price (*price-n*: int) (*price-d*: int)

definition *rat-price* :: *ledger-price* $\Rightarrow$ *rat*
 where
 rat-price price = *price-n price* / *price-d price*

declare [[*coercion rat-price*]] — We implicitly coerce *ledger-price* to rational.

definition *ledger-price-well-formed* :: *ledger-price* $\Rightarrow$ *bool*
 where
 ledger-price-well-formed price $\longleftrightarrow 0 < \text{price-n price} \wedge 0 < \text{price-d price}$

ledger-price is the price representation stored in the ledger. Its coercion through *rat-price* gives the corresponding exact rational value whenever a rational expression requires it.

1.3 Offers

record *offer-attributes* =
 is-passive :: *bool*

record *sell-wheat-offer* = *offer-attributes* +
 wheat-amount :: *int*
 sheep-per-wheat :: *ledger-price*

record *sell-sheep-offer* = *offer-attributes* +
 sheep-amount :: *int*
 wheat-per-sheep :: *ledger-price*

definition *sell-wheat-offer-well-formed* :: *sell-wheat-offer* $\Rightarrow$ *bool*
 where
 sell-wheat-offer-well-formed offer $\longleftrightarrow$
 $0 < \text{wheat-amount offer}$
 $\wedge \text{ledger-price-well-formed } (\text{sheep-per-wheat offer})$

definition *sell-sheep-offer-well-formed* :: *sell-sheep-offer* $\Rightarrow$ *bool*
 where
 sell-sheep-offer-well-formed offer $\longleftrightarrow$
 $0 < \text{sheep-amount offer}$
 $\wedge \text{ledger-price-well-formed } (\text{wheat-per-sheep offer})$

The maker sells wheat and buys sheep, while the taker sells sheep and buys wheat. Both roles therefore have balances for both assets, while their selling liabilities, buying liabilities, and trustline limits follow their respective directions.

1.4 The ledger

definition *int64-max* :: *int*
where
int64-max = $2^{63} - 1$

We implicitly assume that all non-computed integer values are *int64-max* at most.

record *balances* =
wheat-balance :: *int*
sheep-balance :: *int*

record *maker-state* = *balances* +
wheat-selling-liabilities :: *int*
sheep-limit :: *int*
sheep-buying-liabilities :: *int*

record *taker-state* = *balances* +
sheep-selling-liabilities :: *int*
wheat-limit :: *int*
wheat-buying-liabilities :: *int*

record *ledger* =
maker-state :: *maker-state*
taker-state :: *taker-state*

definition *maker-state-well-formed* :: *maker-state* $\Rightarrow$ *bool*
where
maker-state-well-formed *maker* $\longleftrightarrow$
 $0 \leq \text{wheat-selling-liabilities } \text{maker}$
 $\wedge \text{wheat-selling-liabilities } \text{maker} \leq \text{wheat-balance } \text{maker}$
 $\wedge 0 \leq \text{sheep-balance } \text{maker}$
 $\wedge 0 \leq \text{sheep-buying-liabilities } \text{maker}$
 $\wedge \text{sheep-balance } \text{maker} + \text{sheep-buying-liabilities } \text{maker} \leq \text{sheep-limit } \text{maker}$

definition *maker-available-wheat* :: *maker-state* $\Rightarrow$ *int*
where
maker-available-wheat *maker* =
 $\text{wheat-balance } \text{maker} - \text{wheat-selling-liabilities } \text{maker}$

definition *maker-sheep-headroom* :: *maker-state* $\Rightarrow$ *int*
where
maker-sheep-headroom *maker* =
 $\text{sheep-limit } \text{maker} - \text{sheep-balance } \text{maker} -$
 $\text{sheep-buying-liabilities } \text{maker}$

1.5 Maximum unrestricted exchange

record *exchanged-amounts* =
wheat-to-taker :: *int*

sheep-to-maker :: *int*

definition *max-unrestricted-exchange* **where**

```

max-unrestricted-exchange offer = do {
  let p = sheep-per-wheat offer;
  let feasible = { wheat .  $\exists$  sheep .
    wheat =  $\lceil \text{sheep}/p \rceil \wedge \text{wheat} \leq \text{wheat-amount offer}$ 
     $\wedge \text{sheep} = \lfloor \text{wheat} * p \rfloor$ 
     $\wedge 0 < \text{wheat} \wedge \text{wheat} \leq \text{int64-max}$ 
     $\wedge 0 < \text{sheep} \wedge \text{sheep} \leq \text{int64-max}$ 
     $\wedge \text{wheat} * p \leq \text{int64-max}$  — NOTE: seems weird
  };
  let max-wheat = Max feasible;

  if feasible = {} then (wheat-to-taker = 0, sheep-to-maker = 0)
  else (wheat-to-taker = max-wheat, sheep-to-maker =  $\lfloor \text{max-wheat} * p \rfloor$ )
}
```

max-unrestricted-exchange corresponds to the maximum trade possible, measured in wheat, assuming an unrestricted counterparty.

1.6 Valid offers

TODO: is this enforced upstream of `OfferExchange.cpp`?

definition *valid-offer* **where**

```

valid-offer offer ledger  $\longleftrightarrow$  do {
  let max-exch = max-unrestricted-exchange offer;
  let maker-state = maker-state ledger;

  if (max-exch = (wheat-to-taker = 0, sheep-to-maker = 0))
  then
    False
  else
    wheat-balance maker-state  $\geq$ 
      wheat-to-taker max-exch + wheat-selling-liabilities maker-state
     $\wedge$  sheep-limit maker-state  $\geq$ 
      sheep-balance maker-state + sheep-to-maker max-exch
      + sheep-buying-liabilities maker-state
}
```

An offer is valid when its maximum unrestricted exchange is nonzero and the maker can reserve the corresponding selling and buying liabilities.

1.7 Trades

TODO: do we need this record? Why not just use *exchanged-amounts*?

record *trade* = *exchanged-amounts* +
resting-offer-stays :: *bool*

A *trade* records a positive whole-unit transfer and whether the resting offer has greater capped capacity (meaning it is not fully consumed and stays on the book).

definition *effective-sheep-per-wheat* :: 'a *exchanged-amounts-scheme* $\Rightarrow$ *rat*
where
effective-sheep-per-wheat exchange =
sheep-to-maker exchange / *wheat-to-taker exchange*

For a trade with a positive wheat amount, *effective-sheep-per-wheat* is the realized number of sheep sent per wheat received.

definition *price-error-within-bound* ::
ledger-price $\Rightarrow$ 'a *exchanged-amounts-scheme* $\Rightarrow$ *bool*
where
price-error-within-bound price exchange $\longleftrightarrow$
 $\text{abs } (\text{effective-sheep-per-wheat exchange} - \text{price}) \leq (\text{price} :: \text{rat}) / 100$

For a positive whole-unit trade, *price-error-within-bound* compares its effective sheep-per-wheat price with the posted ledger price. It accepts the trade when their absolute difference is at most one percent of the posted price. A trade that fails this test is rejected.

1.8 Posting an offer

definition *reserve-sell-wheat-offer-liabilities* ::
sell-wheat-offer $\Rightarrow$ *maker-state* $\Rightarrow$ *maker-state*
where
reserve-sell-wheat-offer-liabilities offer maker = *do* {
let amounts = *max-unrestricted-exchange offer*;
maker(
wheat-selling-liabilities :=
wheat-selling-liabilities maker + *wheat-to-taker amounts*,
sheep-buying-liabilities :=
sheep-buying-liabilities maker + *sheep-to-maker amounts*) } }

datatype *post-outcome* =
Malformed
| *Line-Full*
| *Underfunded*
| *No-Offer*
| *Created sell-wheat-offer maker-state*

definition *post-sell-wheat-offer* ::
sell-wheat-offer $\Rightarrow$ *maker-state* $\Rightarrow$ *post-outcome*
where
post-sell-wheat-offer offer maker = *do* {
let amounts = *max-unrestricted-exchange offer*;
cond {

$offer < 0 \Rightarrow$
 $\quad$ when $\neg$ ledger-price-well-formed (sheep-per-wheat offer) $\vee$ wheat-amount
 $\quad$ Malformed
 $\quad$ | when maker-sheep-headroom maker $\leq 0 \vee$
 $\quad$ maker-sheep-headroom maker $<$ sheep-to-maker amounts $\Rightarrow$
 $\quad$ Line-Full
 $\quad$ | when maker-available-wheat maker $<$ wheat-to-taker amounts $\Rightarrow$
 $\quad$ Underfunded
 $\quad$ | when wheat-amount offer $= 0 \vee$ wheat-to-taker amounts $= 0 \Rightarrow$
 $\quad$ No-Offer
 $\quad$ | when $\neg$ price-error-within-bound
 $\quad$ (sheep-per-wheat offer) amounts $\Rightarrow$
 $\quad$ No-Offer
 $\quad$ | otherwise $\Rightarrow$ do {
 $\quad$ let posted = offer(wheat-amount := wheat-to-taker amounts);
 $\quad$ Created posted (reserve-sell-wheat-offer-liabilities posted maker)}} }

The line-full branch also rejects nonpositive sheep headroom, matching the immediate C++ check `maxWheatReceive == 0` after capacities are clamped. The maximum unrestricted exchange is the full execution selected against an unrestricted active taker, for which the resting offer does not stay. Applying the price-error threshold before reserving liabilities ensures that every *Created* offer admits that execution. Failure is *No-Offer* because the offer is syntactically well formed but has no acceptable full unrestricted execution at whole-unit precision.

1.9 Crossing a posted sell-wheat offer

definition *taker-state-well-formed* :: *taker-state* $\Rightarrow$ *bool*

where

taker-state-well-formed taker $\longleftrightarrow$
 $0 \leq$ sheep-selling-liabilities taker
 $\wedge$ sheep-selling-liabilities taker $\leq$ sheep-balance taker
 $\wedge 0 \leq$ wheat-balance taker
 $\wedge 0 \leq$ wheat-buying-liabilities taker
 $\wedge$ wheat-balance taker + wheat-buying-liabilities taker $\leq$ wheat-limit taker

definition *taker-available-sheep* :: *taker-state* $\Rightarrow$ *int*

where

taker-available-sheep taker =
 sheep-balance taker - sheep-selling-liabilities taker

definition *taker-wheat-headroom* :: *taker-state* $\Rightarrow$ *int*

where

taker-wheat-headroom taker =
 wheat-limit taker - wheat-balance taker - wheat-buying-liabilities taker

definition *offers-cross* ::

```

    sell-wheat-offer  $\Rightarrow$  sell-sheep-offer  $\Rightarrow$  bool
where
    offers-cross resting incoming  $\longleftrightarrow$  do {
        let sheep-per-wheat-resting = (sheep-per-wheat resting :: rat);
        let sheep-per-wheat-incoming-limit = 1 / (wheat-per-sheep incoming :: rat);
        if is-passive incoming
        then sheep-per-wheat-resting < sheep-per-wheat-incoming-limit
        else sheep-per-wheat-resting  $\leq$  sheep-per-wheat-incoming-limit
    }

definition release-sell-wheat-offer-liabilities ::
    sell-wheat-offer  $\Rightarrow$  maker-state  $\Rightarrow$  maker-state
where
    release-sell-wheat-offer-liabilities offer maker =
        (let amounts = max-unrestricted-exchange offer
         in maker(
             wheat-selling-liabilities :=
                 wheat-selling-liabilities maker - wheat-to-taker amounts,
             sheep-buying-liabilities :=
                 sheep-buying-liabilities maker - sheep-to-maker amounts))

record exchange-caps =
    maker-wheat-send-cap :: int
    taker-wheat-recv-cap :: int
    taker-sheep-send-cap :: int
    maker-sheep-recv-cap :: int

exchange-caps groups both parties' upper bounds on the two transferred
assets.

definition exchange-caps ::
    sell-wheat-offer  $\Rightarrow$  maker-state  $\Rightarrow$  sell-sheep-offer  $\Rightarrow$  taker-state  $\Rightarrow$  exchange-
    caps
where
    exchange-caps resting maker incoming taker =
        (let amounts = max-unrestricted-exchange resting
         in let released-maker =
             release-sell-wheat-offer-liabilities resting maker
         in (maker-wheat-send-cap =
             min (wheat-to-taker amounts)
                 (maker-available-wheat released-maker), — TODO: Isn't this
                 guaranteed to be the first arg since we just released the liabilities?
             taker-wheat-recv-cap = taker-wheat-headroom taker,
             taker-sheep-send-cap =
                 min (sheep-amount incoming) (taker-available-sheep taker),
             maker-sheep-recv-cap =
                 min (sheep-to-maker amounts) — NOTE: C++ just uses just the
                 headroom...
                 (maker-sheep-headroom released-maker)))

```

exchange-caps combines the resting offer's released reservation with the incoming offer's amount and the parties' current ledger capacities. By construction, it caps the resting side by its reservation even after releasing that reservation. Consequently, unrelated capacity acquired later cannot cause the crossing to consume more than the resting offer had reserved. The incoming offer's own price is deliberately absent: it decides whether the offers cross, while a selected crossing executes at the resting price.

definition *resting-capacity-in-wheat* ::

ledger-price $\Rightarrow$ *exchange-caps* $\Rightarrow$ *rat*

where

resting-capacity-in-wheat price caps =
 $\min$ (*maker-wheat-send-cap caps*)
 (*maker-sheep-receive-cap caps* / *price*)

definition *incoming-capacity-in-wheat* ::

ledger-price $\Rightarrow$ *exchange-caps* $\Rightarrow$ *rat*

where

incoming-capacity-in-wheat price caps =
 $\min$ (*taker-wheat-receive-cap caps*)
 (*taker-sheep-send-cap caps* / *price*)

resting-capacity-in-wheat and *incoming-capacity-in-wheat* express both sides' exact capacities in the common unit of wheat at the resting price. They are rational quantities so the comparison happens before whole-unit rounding.

definition *resting-offer-has-greater-capacity* ::

ledger-price $\Rightarrow$ *exchange-caps* $\Rightarrow$ *bool*

where

resting-offer-has-greater-capacity price caps $\longleftrightarrow$
resting-capacity-in-wheat price caps > *incoming-capacity-in-wheat price caps*

definition *resting-offer-stays-at-crossing* ::

sell-wheat-offer $\Rightarrow$ *maker-state* $\Rightarrow$ *sell-sheep-offer* $\Rightarrow$ *taker-state* $\Rightarrow$ *bool*

— TODO: does this actually implies it will stay after adjustment?

where

resting-offer-stays-at-crossing resting maker incoming taker $\longleftrightarrow$
 (*let caps* = *exchange-caps resting maker incoming taker*
 in *resting-offer-has-greater-capacity (sheep-per-wheat resting) caps*)

resting-offer-has-greater-capacity uses a strict comparison: the resting offer stays only when its exact wheat capacity is greater. Equal capacities therefore do not leave a remainder of the resting offer. *resting-offer-stays-at-crossing* instantiates that comparison with the live caps and the resting execution price.

The pre-threshold selector below mirrors the liability calculation: it describes every positive whole-unit crossing allowed by the four live caps and the

appropriate rounding direction, then chooses the greatest feasible wheat transfer. An empty feasible set denotes the absence of a trade.

definition *trade-before-error-threshold* ::

sell-wheat-offer $\Rightarrow$ *maker-state* $\Rightarrow$ *sell-sheep-offer* $\Rightarrow$ *taker-state* $\Rightarrow$ *trade option*

— TODO: shouldn't we just use the posted amount for maker capacity? It's supposed to be an invariant that liabilities are covered.

where

trade-before-error-threshold *resting maker incoming taker* =

(*let price* = *sheep-per-wheat* *resting*

in let caps =

exchange-caps *resting maker incoming taker*

in let stays =

resting-offer-has-greater-capacity price caps

in let feasible = {*wheat*. $\exists$ *sheep*.

$0 < \textit{wheat}$

$\wedge \textit{wheat} \leq \textit{maker-wheat-send-cap caps}$

$\wedge \textit{wheat} \leq \textit{taker-wheat-receive-cap caps}$

$\wedge \textit{wheat} \leq \textit{int64-max}$

$\wedge 0 < \textit{sheep}$

$\wedge \textit{sheep} \leq \textit{taker-sheep-send-cap caps}$

$\wedge \textit{sheep} \leq \textit{maker-sheep-receive-cap caps}$

$\wedge \textit{sheep} \leq \textit{int64-max}$

$\wedge \textit{wheat} * \textit{price} \leq \textit{int64-max}$

$\wedge \textit{sheep} / \textit{price} \leq \textit{int64-max}$

$\wedge$ (*if stays*

then sheep = $\lceil \textit{wheat} * \textit{price} \rceil$

$\wedge \textit{wheat} = \lfloor \textit{sheep} / \textit{price} \rfloor$

else sheep = $\lfloor \textit{wheat} * \textit{price} \rfloor$

$\wedge \textit{wheat} = \lceil \textit{sheep} / \textit{price} \rceil$))}

in if feasible = {} *then None*

else

(*let wheat* = *Max feasible*

in let sheep =

(*if stays then* $\lceil \textit{wheat} * \textit{price} \rceil$

else $\lfloor \textit{wheat} * \textit{price} \rfloor$)

in Some

(*wheat-to-taker* = *wheat*,

sheep-to-maker = *sheep*,

resting-offer-stays = *stays*)))

trade-before-error-threshold selects amounts before applying the price-error filter. When the resting offer stays, the rounding favors its seller; otherwise the opposite rounding favors the incoming seller. The mutually rounded equations rule out transfers made artificially favorable by more than whole-unit indivisibility requires.

definition *trade* ::

sell-wheat-offer $\Rightarrow$ *maker-state* $\Rightarrow$ *sell-sheep-offer* $\Rightarrow$ *taker-state* $\Rightarrow$ *trade option*

where

trade resting maker incoming taker =
(case trade-before-error-threshold
resting maker incoming taker of
None $\Rightarrow$ None
| Some selected $\Rightarrow$
if price-error-within-bound
(sheep-per-wheat resting) selected
then Some selected else None)

trade applies the one-percent price-error bound to the maximal pre-threshold trade. Rejection produces no trade.

definition *cross-sell-wheat-offer* ::

sell-wheat-offer $\Rightarrow$ maker-state $\Rightarrow$ sell-sheep-offer $\Rightarrow$ taker-state $\Rightarrow$ trade option

where

cross-sell-wheat-offer resting maker incoming taker =
(if offers-cross resting incoming
then trade resting maker incoming taker
else None)

cross-sell-wheat-offer first checks price compatibility and then delegates to the declarative selector. Well-formed offers and party states, together with coverage of the resting offer's reservation, are domain assumptions for later crossing theorems rather than additional result variants of this pure selection function.

definition *unrestricted-sell-sheep-offer* ::

sell-wheat-offer $\Rightarrow$ sell-sheep-offer

where

unrestricted-sell-sheep-offer resting =
(|is-passive = False,
sheep-amount = int64-max,
wheat-per-sheep =
Ledger-Price
(price-d (sheep-per-wheat resting))
(price-n (sheep-per-wheat resting)))

definition *unrestricted-taker-state* :: *taker-state*

where

unrestricted-taker-state =
(|wheat-balance = 0,
sheep-balance = int64-max,
sheep-selling-liabilities = 0,
wheat-limit = int64-max,
wheat-buying-liabilities = 0|)

unrestricted-sell-sheep-offer is an active incoming offer at the reciprocal resting price with the largest ledger amount. Together with *unrestricted-taker-state*, which has maximal sheep balance and wheat headroom and no liabilities, it provides the canonical counterparty used to state that a successfully

posted offer is executable.

Refinement note: *exchange-caps* caps the maker's sheep-receive capacity by both its live headroom and the buying liability reserved for the resting offer. In contrast, C++ `crossOfferV10` releases that liability and passes the full live headroom returned by `canBuyAtMost` to `exchangeV10`. Extra headroom can therefore change which offer stays and select a different rounding branch; for small amounts, an exchange admitted by this specification can become a zero exchange in C++. This mismatch remains when both exact-cap protocol choices are enabled because they do not change the initial stays comparison.

1.10 Properties

A resting offer successfully produced by *post-sell-wheat-offer* carries a full-fill pair given by *max-unrestricted-exchange*. The property below says that any price-compatible incoming offer whose net send and receive capacities cover that pair consumes the resting offer completely. The crossing-time maker state is otherwise arbitrary: well-formedness and coverage of the resting offer's two booked liabilities are the only maker requirements. In particular, unrelated extra maker balance or headroom must not change the transferred amounts or which offer stays.

lemma *max-unrestricted-exchange-all-zero-or-none:*

assumes *sell-wheat-offer-well-formed offer*

defines *amounts* $\equiv$ *max-unrestricted-exchange offer*

shows *wheat-to-taker amounts* = 0 $\longleftrightarrow$ *sheep-to-maker amounts* = 0

Proof sketch. The feasible wheat amounts form a finite set because they lie in an integer interval. If the set is empty, the definition assigns zero to both amounts. Otherwise its maximum belongs to the set; feasibility says that the maximum wheat amount and its corresponding rounded sheep amount are both positive. Hence neither amount is zero.

Proof sketch. Reaching the *Created* branch means that every earlier posting guard was false. The negated guards give price and amount validity, a nonzero unrestricted amount, acceptance of that exchange by the price-error threshold, and sufficient maker capacity. Injectivity of *Created* identifies the returned state with the state produced by reservation.

lemma *created-sell-wheat-offer-facts:*

assumes *posted:*

post-sell-wheat-offer offer maker = *Created offer posted-maker*

defines *amounts* $\equiv$ *max-unrestricted-exchange offer*

defines *unrestricted-trade* $\equiv$

$\langle$ *wheat-to-taker* = *wheat-to-taker amounts*,

sheep-to-maker = *sheep-to-maker amounts*,

$\text{resting-offer-stays} = \text{False})$
shows $\text{ledger-price-well-formed } (\text{sheep-per-wheat offer})$
 $\wedge 0 < \text{wheat-amount offer}$
 $\wedge \text{wheat-to-taker amounts} \neq 0$
 $\wedge \text{price-error-within-bound}$
 $\quad (\text{sheep-per-wheat offer}) \text{ unrestricted-trade}$
 $\wedge \text{sheep-to-maker amounts}$
 $\quad \leq \text{maker-sheep-headroom maker}$
 $\wedge \text{wheat-to-taker amounts}$
 $\quad \leq \text{maker-available-wheat maker}$
 $\wedge \text{posted-maker} =$
 $\quad \text{reserve-sell-wheat-offer-liabilities offer maker}$

Proof sketch. A nonzero wheat amount rules out the empty feasible set, because the empty branch assigns zero. The set is finite since all its wheat amounts lie in a finite integer interval. Its maximum therefore belongs to the set, and expanding the nonempty branch transfers every feasibility condition to the two stored amounts.

lemma *nonzero-max-unrestricted-exchange-is-feasible:*

assumes *nonzero:*

wheat-to-taker

$(\text{max-unrestricted-exchange offer}) \neq 0$

defines $\text{amounts} \equiv \text{max-unrestricted-exchange offer}$

shows

$\text{wheat-to-taker amounts} =$
 $\quad \lceil \text{sheep-to-maker amounts} /$
 $\quad \quad \text{sheep-per-wheat offer} \rceil$
 $\wedge \text{wheat-to-taker amounts} \leq \text{wheat-amount offer}$
 $\wedge \text{sheep-to-maker amounts} =$
 $\quad \lfloor \text{wheat-to-taker amounts} *$
 $\quad \quad \text{sheep-per-wheat offer} \rfloor$
 $\wedge 0 < \text{wheat-to-taker amounts}$
 $\wedge \text{wheat-to-taker amounts} \leq \text{int64-max}$
 $\wedge \text{sheep-to-maker amounts} /$
 $\quad \text{sheep-per-wheat offer} \leq \text{int64-max}$
 $\wedge 0 < \text{sheep-to-maker amounts}$
 $\wedge \text{sheep-to-maker amounts} \leq \text{int64-max}$
 $\wedge \text{wheat-to-taker amounts} * \text{sheep-per-wheat offer} \leq \text{int64-max}$

Proof sketch. Reserving and then releasing this offer cancels exactly the two liability additions. The successful posting guards say the original maker capacities cover the liabilities, so both maker-side minima select the liabilities. The canonical taker contributes *int64-max* on both of its sides.

lemma *unrestricted-caps-after-created:*

assumes *posted:*

post-sell-wheat-offer offer maker = Created offer posted-maker
defines *amounts* $\equiv$ *max-unrestricted-exchange offer*
shows
exchange-caps
offer posted-maker
(unrestricted-sell-sheep-offer offer)
unrestricted-taker-state =
(maker-wheat-send-cap =
wheat-to-taker amounts,
taker-wheat-receive-cap = int64-max,
taker-sheep-send-cap = int64-max,
maker-sheep-receive-cap =
sheep-to-maker amounts)

Proof sketch. The canonical incoming offer is active and stores the reciprocal ledger price. Taking its reciprocal recovers the resting price, so the non-strict active-offer crossing test holds with equality.

lemma *unrestricted-offer-crosses:*
offers-cross offer (unrestricted-sell-sheep-offer offer)

Proof sketch. The two resting caps are the unrestricted exchange amounts, bounds both by *int64-max*. Positivity of the posted price preserves the sheep-cap inequality when converting it to wheat. Monotonicity of *min* therefore makes resting capacity no greater than canonical incoming capacity, so the strict stays comparison is false.

lemma *unrestricted-resting-offer-does-not-stay-after-created:*
assumes *posted:*
post-sell-wheat-offer offer maker = Created offer posted-maker
defines *amounts* $\equiv$ *max-unrestricted-exchange offer*
shows
 $\neg$ *resting-offer-has-greater-capacity*
(sheep-per-wheat offer)
(exchange-caps
offer posted-maker
(unrestricted-sell-sheep-offer offer)
unrestricted-taker-state)

Proof sketch. The unrestricted exchange is feasible for the pre-threshold selector under the canonical caps and the non-stays rounding direction. Conversely, every selector-feasible wheat amount is bounded by the unrestricted wheat amount. Hence that amount is exactly the maximum. Its rounded sheep amount is the unrestricted sheep amount, so the selected record is that exchange.

lemma *unrestricted-prethreshold-exchange-after-created:*

assumes *posted*:
post-sell-wheat-offer offer maker = Created offer posted-maker
defines *amounts* $\equiv$ *max-unrestricted-exchange offer*
defines *unrestricted-trade* $\equiv$
 (*wheat-to-taker = wheat-to-taker amounts,*
sheep-to-maker = sheep-to-maker amounts,
resting-offer-stays = False)
shows
trade-before-error-threshold
offer posted-maker
(unrestricted-sell-sheep-offer offer)
unrestricted-taker-state =
Some unrestricted-trade

Proof sketch. The previous lemma identifies the pre-threshold selection. The additional posting guard records exactly that this unrestricted exchange satisfies the price-error bound, so the thresholded selector preserves it.

lemma *unrestricted-selected-exchange-after-created*:
assumes *posted*:
post-sell-wheat-offer offer maker = Created offer posted-maker
defines *amounts* $\equiv$ *max-unrestricted-exchange offer*
defines *unrestricted-trade* $\equiv$
 (*wheat-to-taker = wheat-to-taker amounts,*
sheep-to-maker = sheep-to-maker amounts,
resting-offer-stays = False)
shows
trade
offer posted-maker
(unrestricted-sell-sheep-offer offer)
unrestricted-taker-state =
Some unrestricted-trade

Proof sketch. The canonical incoming offer crosses at equality because it is active and reciprocal. The thresholded selector returns the unrestricted exchange by the preceding lemma, so unfolding the outer crossing check yields that same successful trade.

theorem *successfully-posted-offer-has-unrestricted-taker*:
assumes *posted*:
post-sell-wheat-offer offer maker = Created offer posted-maker
defines *amounts* $\equiv$ *max-unrestricted-exchange offer*
shows
cross-sell-wheat-offer
offer posted-maker
(unrestricted-sell-sheep-offer offer)
unrestricted-taker-state =
Some

```

    (|wheat-to-taker =
      wheat-to-taker amounts,
      sheep-to-maker =
        sheep-to-maker amounts,
      resting-offer-stays = False|)

end
theory Offer-Exchange-Arithmetic
  imports HOL-Library.Word HOL-Library.Monad-Syntax
begin

```

2 Exchange arithmetic

The export check of the Isabelle tooling audits, alongside the code equations of the exported model, every fact collected under *export-audit*: the theorems that connect the exported code-level definitions to what was proved about them must depend on no oracle and on no project axiom that is not a definition. Theories add their refinement and equivalence theorems to it.

```

named-theorems export-audit
  facts the export check audits together with the exported code equations

```

This theory gives an executable, bit-precise model of the arithmetic used by *exchangeV10*. It first presents the modeled operations and their main properties, then proves those properties in the same order so that the definitions and their verification can be read side by side.

2.1 The model and its main properties

This first section presents the model in full and, at its end, states the properties that the remaining sections prove. Nothing is proved here, so the model can be read on its own and compared line by line with *OfferExchange.cpp*. The definitions stay close to the C++ they model: the same branch structure, the same order of calls, and bit-precise arithmetic on 32-, 64-, and 128-bit words. The error monad below stands for the C++ control flow that a failed assertion, a detected overflow, or an explicit *throw* interrupts. Alongside each modeled function there is a specification phrased over unbounded integers; the proofs then show that the two agree.

2.1.1 Fixed-width C++ arithmetic

```

type-synonym int32 = 32 word
type-synonym int64 = 64 word
type-synonym uint128 = 128 word

abbreviation int64-max-int :: int

```

— C++: *INT64-MAX*
where *int64-max-int* $\equiv$ 9223372036854775807

int64-max-int is the value of the C++ constant *INT64-MAX* as an unbounded integer, that is $2^{63} - 1$. It is an abbreviation rather than a definition so that every arithmetic step still sees the plain numeral. The companion constant *int64-max* defined below is the signed 64-bit word with that bit pattern.

datatype *cxx-error* =
 Cxx-Assertion-Failed
 | *Cxx-Overflow*
 | *Cxx-Runtime-Error*

datatype 'a *cxx-result* =
 Cxx-Ok 'a
 | *Cxx-Err* *cxx-error*

fun *cxx-bind* ::
 'a *cxx-result* $\Rightarrow$ ('a $\Rightarrow$ 'b *cxx-result*) $\Rightarrow$ 'b *cxx-result*
where
 cxx-bind (*Cxx-Ok* x) f = f x
 | *cxx-bind* (*Cxx-Err* e) f = *Cxx-Err* e

adhoc-overloading
Monad-Syntax.bind $\Rightarrow$ *cxx-bind*

The operation ($\gg$) is an executable exception-style bind. A value built with *Cxx-Err* is propagated unchanged, while a value built with *Cxx-Ok* is passed to the next computation. The overload enables *do* notation in executable definitions below.

lemma *cxx-bind-assoc*:
 cxx-bind (*cxx-bind* m f) g = *cxx-bind* m (λx . *cxx-bind* (f x) g)
 — Sequencing is associative, so a computation may be repackaged as a helper that performs several steps without changing its result.

Proof sketch: an error in *m* is propagated by both sides, and an *Cxx-Ok* value is passed to the same continuation by both sides, so a case distinction on *m* closes the goal.

2.1.2 Wide multiplication

This models the C++ function *bigMultiply* from *util/numeric.cpp*. Its arguments have the bit patterns of C++ *int64-t* values, so they are interpreted with *sint*. The implementation asserts that both arguments are non-negative, widens them to unsigned 128-bit values, and multiplies them. Two non-negative signed 64-bit values have a product below 2^{126} , so the 128-bit multiplication cannot wrap.

definition *big-multiply* ::
 $\text{int64} \Rightarrow \text{int64} \Rightarrow \text{uint128} \text{ cxx-result}$
— C++: *bigMultiply* (*util/numeric.cpp*)
where
big-multiply *a b* =
(if *sint a* < 0 $\vee$ *sint b* < 0 then
Cxx-Err Cxx-Assertion-Failed
else
Cxx-Ok (*word-of-int* (*sint a* * *sint b*)))

2.1.3 Checked division

datatype *cxx-rounding* =
Cxx-Round-Down
| *Cxx-Round-Up*

fun *rounded-quotient* ::
 $\text{int} \Rightarrow \text{int} \Rightarrow \text{cxx-rounding} \Rightarrow \text{int}$
where
rounded-quotient *numerator divisor Cxx-Round-Down* =
numerator div divisor
| *rounded-quotient* *numerator divisor Cxx-Round-Up* =
(*numerator* + *divisor* - 1) *div divisor*

The two definitions below model the throwing public helpers *bigDivideOrThrow* and *bigDivideOrThrow128* from *util/numeric.cpp*. The ordinary helper widens a product of signed 64-bit operands. The 128-bit helper additionally preserves the explicit guard that prevents the numerator used for round-up from wrapping before division.

definition *big-divide-or-throw* ::
 $\text{int64} \Rightarrow \text{int64} \Rightarrow \text{int64} \Rightarrow \text{cxx-rounding} \Rightarrow$
 $\text{int64} \text{ cxx-result}$
— C++: *bigDivideOrThrow* (*util/numeric.cpp*)
where
big-divide-or-throw *a b c rounding* =
(if *sint a* < 0 $\vee$ *sint b* < 0 $\vee$ *sint c* $\leq$ 0 then
Cxx-Err Cxx-Assertion-Failed
else
let *quotient* =
rounded-quotient (*sint a* * *sint b*) (*sint c*) *rounding*
in if *quotient* > *int64-max-int* then
Cxx-Err Cxx-Overflow
else
Cxx-Ok (*word-of-int* *quotient*))

definition *big-divide-or-throw128* ::
 $\text{uint128} \Rightarrow \text{int64} \Rightarrow \text{cxx-rounding} \Rightarrow \text{int64} \text{ cxx-result}$
— C++: *bigDivideOrThrow128* (*util/numeric.cpp*)
where

```

big-divide-or-throw128 a b rounding =
  (if sint b ≤ 0 then
    Cxx-Err Cxx-Assertion-Failed
  else if rounding = Cxx-Round-Up ∧
    uint a > (2 :: int) ^ 128 - 1 - (sint b - 1) then
    Cxx-Err Cxx-Overflow
  else
    let quotient = rounded-quotient (uint a) (sint b) rounding
    in if quotient > int64-max-int then
      Cxx-Err Cxx-Overflow
    else
      Cxx-Ok (word-of-int quotient))

```

2.1.4 Price error bound

The definition below is the direct model of *checkPriceErrorBound* (*OfferExchange.cpp*). The C++ function first scales each signed 32-bit price component by one hundred in signed 64-bit arithmetic. It then compares two unsigned 128-bit products. If the caller permits unbounded error in favor of wheat, the favorable direction returns immediately; otherwise the absolute difference must be no greater than the unscaled wheat value.

definition *check-price-error-bound* ::
 $int32 \Rightarrow int32 \Rightarrow int64 \Rightarrow int64 \Rightarrow$
 $bool \Rightarrow bool$ *cxx-result*
 — C++: *checkPriceErrorBound* (*OfferExchange.cpp*)
where
check-price-error-bound price-n price-d wheat-receive sheep-send
can-favor-wheat = do {
lhs ← *big-multiply*
 (word-of-int (100 * sint price-n)) *wheat-receive*;
rhs ← *big-multiply*
 (word-of-int (100 * sint price-d)) *sheep-send*;
 if *can-favor-wheat* ∧ *rhs* > *lhs* then
 Cxx-Ok True
 else
 let *abs-diff* = (if *lhs* > *rhs* then *lhs* - *rhs* else *rhs* - *lhs*)
 in do {
cap ← *big-multiply* (scast price-n) *wheat-receive*;
 Cxx-Ok (*abs-diff* ≤ *cap*)
 }
 }

definition *price-error-bound-spec* ::
 $int32 \Rightarrow int32 \Rightarrow int64 \Rightarrow int64 \Rightarrow bool \Rightarrow bool$
where
price-error-bound-spec price-n price-d wheat-receive sheep-send
can-favor-wheat =
 (let *lhs* = 100 * sint price-n * sint wheat-receive;

$$\begin{aligned}
& rhs = 100 * sint\ price-d * sint\ sheep-send; \\
& cap = sint\ price-n * sint\ wheat-receive \\
& in\ (can-favor-wheat \wedge rhs > lhs) \vee abs\ (lhs - rhs) \leq cap)
\end{aligned}$$

price-error-bound-spec is the intended mathematical reading of the bit-precise test above, stated over the exact integers denoted by the arguments rather than over 128-bit words.

2.1.5 Price-error application

datatype *exchange-rounding* =
 Exchange-Normal
 | *Exchange-Strict-Send*
 | *Exchange-Strict-Receive*

record *exchange-options* =
 exact-receive-cap :: *bool*
 symmetric-exact-receive-cap :: *bool*

exchange-options groups protocol-gated choices passed through the exchange model. A record literal such as (*exact-receive-cap* = *True*, *symmetric-exact-receive-cap* = *False*) makes the selected behavior explicit at each call site and leaves room for additional options without adding more positional Boolean arguments. The *exact-receive-cap* field models the C++ parameter *exactReceiveCap*, which repairs the sheep-stays, wheat-more-valuable normal branch; the *symmetric-exact-receive-cap* field models the C++ parameter *symmetricExactReceiveCap*, which applies the mirror-image repair to the wheat-stays, sheep-more-valuable normal branch.

The three constructors of *exchange-rounding* correspond to *NORMAL*, *PATH-PAYMENT-STRICT-SEND*, and *PATH-PAYMENT-STRICT-RECEIVE*. The result record retains the two signed 64-bit amount bit patterns and the unchanged decision about which offer remains.

record *exchange-result-v10* =
 num-wheat-received :: *int64*
 num-sheep-send :: *int64*
 result-wheat-stays :: *bool*

definition *make-exchange-result* ::
 int64 $\Rightarrow$ *int64* $\Rightarrow$ *bool* $\Rightarrow$ *exchange-result-v10*
where
 make-exchange-result wheat-receive sheep-send wheat-stays =
 (*num-wheat-received* = *wheat-receive*,
 num-sheep-send = *sheep-send*,
 result-wheat-stays = *wheat-stays*)

definition *favored-seller-ok* ::
 int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *bool* $\Rightarrow$ *bool*

where

```

favored-seller-ok price-n price-d wheat-receive sheep-send wheat-stays =
  (let wheat-value = sint wheat-receive * sint price-n;
   sheep-value = sint sheep-send * sint price-d
   in if wheat-stays
     then wheat-value ≤ sheep-value
     else sheep-value ≤ wheat-value)

```

This definition follows the branch and mutation order of *applyPriceErrorThresholds*. In particular, arithmetic assertions are reached only when both amounts are positive, and the non-positive strict-send branch tests the raw sheep word for zero exactly as C++ does.

definition *apply-price-error-thresholds ::*

```

int32 ⇒ int32 ⇒ int64 ⇒ int64 ⇒ bool ⇒
  exchange-rounding ⇒ exchange-result-v10 cxx-result
— C++: applyPriceErrorThresholds (OfferExchange.cpp)

```

where

```

apply-price-error-thresholds price-n price-d wheat-receive sheep-send
  wheat-stays rounding =
    (if 0 < sint wheat-receive ∧ 0 < sint sheep-send then do {
      wheat-value ← big-multiply wheat-receive (scast price-n);
      sheep-value ← big-multiply sheep-send (scast price-d);
      if (wheat-stays ∧ sheep-value < wheat-value) ∨
        (¬ wheat-stays ∧ sheep-value > wheat-value)
      then Cxx-Err Cxx-Runtime-Error
      else if rounding = Exchange-Normal then do {
        within-bound ←
          check-price-error-bound price-n price-d wheat-receive sheep-send
          False;
        if within-bound
          then Cxx-Ok
          (make-exchange-result wheat-receive sheep-send wheat-stays)
        else Cxx-Ok (make-exchange-result 0 0 wheat-stays)
      } else do {
        within-bound ←
          check-price-error-bound price-n price-d wheat-receive sheep-send
          True;
        if within-bound
          then Cxx-Ok
          (make-exchange-result wheat-receive sheep-send wheat-stays)
        else Cxx-Err Cxx-Runtime-Error
      }
    )
  } else if rounding = Exchange-Strict-Send then
    if sheep-send = 0
      then Cxx-Err Cxx-Runtime-Error
    else Cxx-Ok
      (make-exchange-result wheat-receive sheep-send wheat-stays)
    else Cxx-Ok (make-exchange-result 0 0 wheat-stays)

```

definition *apply-price-error-thresholds-spec* ::
 $\text{int32} \Rightarrow \text{int32} \Rightarrow \text{int64} \Rightarrow \text{int64} \Rightarrow \text{bool} \Rightarrow$
 $\text{exchange-rounding} \Rightarrow \text{exchange-result-v10} \text{ cxx-result}$
where
apply-price-error-thresholds-spec *price-n price-d wheat-receive sheep-send*
wheat-stays rounding =
(let original =
make-exchange-result wheat-receive sheep-send wheat-stays;
zero = make-exchange-result 0 0 wheat-stays
in if $\neg (0 < \text{sint wheat-receive} \wedge 0 < \text{sint sheep-send})$ then
if rounding = Exchange-Strict-Send then
if sheep-send = 0 then Cxx-Err Cxx-Runtime-Error
else Cxx-Ok original
else Cxx-Ok zero
else if $\text{sint price-n} < 0 \vee \text{sint price-d} < 0$ then
Cxx-Err Cxx-Assertion-Failed
else if $\neg \text{favored-seller-ok price-n price-d wheat-receive sheep-send}$
wheat-stays
then Cxx-Err Cxx-Runtime-Error
else if rounding = Exchange-Normal then
Cxx-Ok
(if price-error-bound-spec price-n price-d wheat-receive
sheep-send False
then original else zero)
else if price-error-bound-spec price-n price-d wheat-receive
sheep-send True
then Cxx-Ok original
else Cxx-Err Cxx-Runtime-Error)

apply-price-error-thresholds-spec restates the same function as a single decision tree over exact integers, with no monadic sequencing left.

2.1.6 Offer value

This is the direct model of *calculateOfferValue*. The signed 32-bit prices are sign-extended to the signed 64-bit arguments expected by *big-multiply*. The final minimum uses the unsigned ordering on 128-bit words, matching C++ *std::min* on *uint128-t*.

definition *calculate-offer-value* ::
 $\text{int32} \Rightarrow \text{int32} \Rightarrow \text{int64} \Rightarrow$
 $\text{int64} \Rightarrow \text{uint128} \text{ cxx-result}$
— C++: *calculateOfferValue* (*OfferExchange.cpp*)
where
calculate-offer-value *price-n price-d max-send max-receive = do {*
send-value $\leftarrow$ big-multiply max-send (scast price-n);
receive-value $\leftarrow$ big-multiply max-receive (scast price-d);
Cxx-Ok (min send-value receive-value)
}

definition *calculate-offer-value-pre* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$
int64 $\Rightarrow$ *bool*

where

calculate-offer-value-pre *price-n price-d max-send max-receive* $\longleftrightarrow$
 $0 \leq \text{sint } \text{price-n} \wedge 0 \leq \text{sint } \text{price-d} \wedge$
 $0 \leq \text{sint } \text{max-send} \wedge 0 \leq \text{sint } \text{max-receive}$

definition *int64-max* :: *int64*

— C++: *INT64-MAX*

where *int64-max* = *word-of-int* ($2^{63} - 1$)

int64-max has the bit pattern of the C++ constant *INT64-MAX*.

definition *calculate-offer-value-with-exact-receive-cap* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$
int64 $\Rightarrow$ *uint128 cxx-result*

— proof-only: the value stage of *calculateOfferAmountFromValue* (*OfferExchange.cpp*)

where

calculate-offer-value-with-exact-receive-cap *price-n price-d max-send*
max-receive = *do* {
send-value $\leftarrow$ *big-multiply* *max-send* (*scast price-n*);
receive-product $\leftarrow$ *big-multiply* *max-receive* (*scast price-d*);
let *receive-value* = *receive-product* +
ucast (*scast* (*price-d* - 1) :: *int64*);
receive-cap $\leftarrow$ *big-multiply* *int64-max* (*scast price-d*);
Cxx-Ok (*min* *send-value* (*min* *receive-value* *receive-cap*))
}

This is the value stage of *calculateOfferAmountFromValue* (*OfferExchange.cpp*), split out for proofs. It is not itself a C++ function: the implementation-facing model is *calculate-offer-amount-from-value* below, which performs this calculation and the final round-down division in one helper. Compared with *calculate-offer-value*, the receive-side value is relaxed by *sint price-d* - 1 before the caller rounds down, so a receive cap that is not an exact multiple of the price still admits the last fractional lot. The addend models the C++ cast chain from *priceD* - 1 through *uint64-t* to *uint128-t*: 32-bit subtraction, sign extension to 64 bits, zero extension to 128 bits. The subsequent minimum with *big-multiply int64-max* (*scast price-d*) restores the plain cap when the receive cap is *INT64-MAX*, keeping such offers fixed points of adjustment.

definition *calculate-offer-amount-from-value* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$
int64 $\Rightarrow$ *int64 cxx-result*

— C++: *calculateOfferAmountFromValue* (*OfferExchange.cpp*)

where

calculate-offer-amount-from-value *price-n price-d max-send*

```

    max-receive = do {
    send-value ← big-multiply max-send (scast price-n);
    receive-product ← big-multiply max-receive (scast price-d);
    let receive-value = receive-product +
        ucast (scast (price-d - 1) :: int64);
    receive-cap ← big-multiply int64-max (scast price-d);
    let capped-receive-value = min receive-value receive-cap;
    let offer-value = min send-value capped-receive-value;
    big-divide-or-throw128 offer-value (scast price-n) Cxx-Round-Down
    }

```

This is the direct model of *calculateOfferAmountFromValue* (*OfferExchange.cpp*), the helper introduced for protocol 29. It keeps the C++ sequencing: the send-side product, the receive-side product relaxed by $priceD - 1$ through the cast chain *int32-t* to *uint64-t* to *uint128-t*, the saturation clamp by *big-multiply int64-max (scast price-d)*, the minimum of the send and capped receive values, and the final round-down division by *sint price-n*.

Returning the amount rather than the value is the one structural change protocol 29 makes to this part of the arithmetic. It exists because the receive cap a resting offer imposes on itself is its own buying liability $price-n * amount \div price-d$, and dividing that by the price again rounds down a second time, which can lose one unit. Relaxing the receive product by *sint price-d - 1* before the single round-down division compensates for exactly that lost unit, while the clamp keeps an *INT64-MAX* receive cap behaving as before.

lemma *calculate-offer-amount-from-value-eq-value-then-divide* [simp]:

$$\begin{aligned} & \text{calculate-offer-amount-from-value } price-n \text{ } price-d \text{ } max\text{-send } max\text{-receive} = \\ & (\text{calculate-offer-value-with-exact-receive-cap } price-n \text{ } price-d \text{ } max\text{-send} \\ & \quad max\text{-receive} \gg= \\ & \quad (\lambda offer\text{-value.} \\ & \quad \quad \text{big-divide-or-throw128 offer-value (scast price-n) Cxx-Round-Down})) \end{aligned}$$

— The protocol-29 helper is exactly the relaxed offer value followed by the round-down division the caller used to perform.

Proof sketch: both sides run the same three multiplications and form the same minimum. They differ only in where the division is sequenced, so unfolding the two definitions and reassociating the binds with *cxx-bind-assoc* makes them syntactically equal.

This equation is the bridge that carries the existing repaired-arithmetic results to the protocol-29 interface. Every proof phrased in terms of the relaxed value followed by a division applies verbatim to *calculate-offer-amount-from-value*, so the protocol-29 helper needs no separate development of the underlying arithmetic.

It is declared a simplification rule so that the existing repaired development

continues to apply unchanged. The helper remains the implementation-facing definition, and the one exported for code generation, because it is what protocol 29 actually computes; the decomposition into a relaxed value and a division is a proof device only.

2.1.7 Protocol-29 amount examples

The examples below are the boundary cases cited by the protocol-29 change (stellar-core commit *cb257d2810*), evaluated in the model. Each pairs the protocol-29 helper with the protocol-28 calculation it replaces, so the intended change of behavior is explicit and executable.

A resting offer of 2999 units at price $3/2$ books a buying liability of $(3::'a) * (2999::'a) \text{ div } (2::'a)$, that is 4498. Re-deriving the amount from that liability is exactly the round trip protocol 29 repairs: protocol 28 returns 2998, silently shrinking an offer that nothing had consumed, while protocol 29 returns the stored 2999.

lemma *p29-amount-recovers-offer-at-its-own-buying-liability:*
`calculate-offer-amount-from-value 3 2 2999 4498 = Cxx-Ok 2999`

lemma *p28-amount-loses-a-unit-at-its-own-buying-liability:*
`(calculate-offer-value 3 2 2999 4498 >=>`
`( $\lambda$ offer-value.`
`big-divide-or-throw128 offer-value 3 Cxx-Round-Down)) =`
`Cxx-Ok 2998`

The adjustment example uses price $7/3$ with a send cap of 428 and a receive cap of 998. The plain send and receive values are 2996 and 2994, so protocol 28 divides 2994 and returns 427; protocol 29 relaxes the receive product to 2996 and returns 428.

lemma *p29-adjustment-keeps-the-full-send-cap:*
`calculate-offer-amount-from-value 7 3 428 998 = Cxx-Ok 428`

lemma *p28-adjustment-reduces-the-send-cap:*
`(calculate-offer-value 7 3 428 998 >=>`
`( $\lambda$ offer-value.`
`big-divide-or-throw128 offer-value 7 Cxx-Round-Down)) =`
`Cxx-Ok 427`

At an unlimited receive cap the retained clamp makes the two protocols agree, which is the property the liability definitions rely on.

lemma *p29-amount-agrees-with-p28-at-unlimited-receive-cap:*
`calculate-offer-amount-from-value 3 2 2999 int64-max = Cxx-Ok 2999`
`(calculate-offer-value 3 2 2999 int64-max >=>`
`( $\lambda$ offer-value.`

big-divide-or-throw128 offer-value 3 Cxx-Round-Down)) =
Cxx-Ok 2999

2.1.8 Exchange before price-error thresholds

definition *signed-min64* :: *int64* $\Rightarrow$ *int64* $\Rightarrow$ *int64*

— C++: *std::min* on *int64-t*

where

signed-min64 a b = (if sint a $\leq$ sint b then a else b)

signed-min64 models C++ *std::min* on *int64-t* operands: the minimum of the two bit patterns in the signed order.

definition *exchange-v10-amounts* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *uint128* $\Rightarrow$ *uint128* $\Rightarrow$
int64 $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *bool* $\Rightarrow$
exchange-rounding $\Rightarrow$ *exchange-options* $\Rightarrow$
(int64 $\times$ *int64)* *cxx-result*

— C++: calculation block of *exchangeV10WithoutPriceErrorThresholds* (*Offer-Exchange.cpp*)

where

exchange-v10-amounts price-n price-d wheat-value sheep-value
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
wheat-stays rounding options =
(if wheat-stays then
if rounding = Exchange-Strict-Send then do {
wheat-receive $\leftarrow$ big-divide-or-throw128 sheep-value
(scast price-n) Cxx-Round-Down;
Cxx-Ok
(wheat-receive, signed-min64 max-sheep-send max-sheep-receive)
} else if sint price-n > sint price-d $\vee$
rounding = Exchange-Strict-Receive
then do {
wheat-receive $\leftarrow$ big-divide-or-throw128 sheep-value
(scast price-n) Cxx-Round-Down;
sheep-send $\leftarrow$ big-divide-or-throw wheat-receive
(scast price-n) (scast price-d) Cxx-Round-Up;
Cxx-Ok (wheat-receive, sheep-send)
} else do {
sheep-send $\leftarrow$
(if symmetric-exact-receive-cap options
then calculate-offer-amount-from-value price-d price-n
max-sheep-send max-wheat-receive
else big-divide-or-throw128 sheep-value
(scast price-d) Cxx-Round-Down);
wheat-receive $\leftarrow$ big-divide-or-throw sheep-send
(scast price-d) (scast price-n) Cxx-Round-Down;
Cxx-Ok (wheat-receive, sheep-send)
}

```

else if sint price-n > sint price-d then do {
  wheat-receive ←
    (if exact-receive-cap options
     then calculate-offer-amount-from-value price-n price-d
       max-wheat-send max-sheep-receive
     else big-divide-or-throw128 wheat-value
       (scast price-n) Cxx-Round-Down);
  sheep-send ← big-divide-or-throw wheat-receive
    (scast price-n) (scast price-d) Cxx-Round-Down;
  Cxx-Ok (wheat-receive, sheep-send)
} else do {
  sheep-send ← big-divide-or-throw128 wheat-value
    (scast price-d) Cxx-Round-Down;
  wheat-receive ← big-divide-or-throw sheep-send
    (scast price-d) (scast price-n) Cxx-Round-Up;
  Cxx-Ok (wheat-receive, sheep-send)
})

```

exchange-v10-amounts is the *wheatReceive/sheepSend* calculation block of *exchangeV10WithoutPriceErrorThresholds* (*OfferExchange.cpp*). Separating the tuple-valued block lets the top-level model retain the C++ sequencing while local proofs analyze each division branch independently. The final options argument carries the two C++ parameters *exactReceiveCap* and *symmetricExactReceiveCap*. When *exact-receive-cap* is set and the sheep offer stays with wheat the more valuable asset, the trade is valued by *calculate-offer-value-with-exact-receive-cap* applied to the resting wheat offer's send and receive caps instead of the plain wheat value, exactly as in the C++ conditional expression. When *symmetric-exact-receive-cap* is set and the wheat offer stays with sheep at least as valuable, the mirror call *calculate-offer-value-with-exact-receive-cap price-d price-n max-sheep-send max-wheat-receive* replaces the plain sheep value, so the taker's wheat-receive cap is treated as an exact unit cap before the round-down division by *sint price-d*. Both branches keep *wheatStays* decided on the two unadjusted offer values.

definition *exchange-v10-without-price-error-thresholds-with-options* ::

```

int32 ⇒ int32 ⇒ int64 ⇒ int64 ⇒ int64 ⇒
int64 ⇒ exchange-rounding ⇒ exchange-options ⇒
exchange-result-v10 cxx-result

```

— C++: *exchangeV10WithoutPriceErrorThresholds* (*OfferExchange.cpp*)

where

```

exchange-v10-without-price-error-thresholds-with-options price-n price-d
  max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
  rounding options = do {
  wheat-value ← calculate-offer-value price-n price-d
    max-wheat-send max-sheep-receive;
  sheep-value ← calculate-offer-value price-d price-n
    max-sheep-send max-wheat-receive;

```

```

let wheat-stays = wheat-value > sheep-value;
amounts ← exchange-v10-amounts price-n price-d wheat-value sheep-value
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
wheat-stays rounding options;
let wheat-receive = fst amounts;
let sheep-send = snd amounts;
if sint wheat-receive < 0 ∨
  sint wheat-receive >
    min (sint max-wheat-receive) (sint max-wheat-send)
then Cxx-Err Cxx-Runtime-Error
else if sint sheep-send < 0 ∨
  sint sheep-send >
    min (sint max-sheep-receive) (sint max-sheep-send)
then Cxx-Err Cxx-Runtime-Error
else Cxx-Ok
  (make-exchange-result wheat-receive sheep-send wheat-stays)
}

```

exchange-v10-without-price-error-thresholds-with-options is the direct model of *exchangeV10WithoutPriceErrorThresholds* (*OfferExchange.cpp*): the two offer values, the retained-offer decision, the branch block modeled by *exchange-v10-amounts*, and the final bounds checks on both amounts.

definition *check-exchange-v10-result* ::

```

int64 ⇒ int64 ⇒ int64 ⇒ int64 ⇒ bool ⇒
  (int64 × int64) ⇒ exchange-result-v10 cxx-result

```

— C++: bounds checks of *exchangeV10WithoutPriceErrorThresholds* (*OfferExchange.cpp*)

where

```

check-exchange-v10-result max-wheat-send max-wheat-receive max-sheep-send
max-sheep-receive wheat-stays amounts =
  (let wheat-receive = fst amounts;
    sheep-send = snd amounts
  in if sint wheat-receive < 0 ∨
    sint wheat-receive >
      min (sint max-wheat-receive) (sint max-wheat-send)
  then Cxx-Err Cxx-Runtime-Error
  else if sint sheep-send < 0 ∨
    sint sheep-send >
      min (sint max-sheep-receive) (sint max-sheep-send)
  then Cxx-Err Cxx-Runtime-Error
  else Cxx-Ok
    (make-exchange-result wheat-receive sheep-send wheat-stays))

```

check-exchange-v10-result restates the final bounds checks of *exchangeV10WithoutPriceErrorThresholds* (*OfferExchange.cpp*) as a separate function so that the specification below can reuse them verbatim.

definition *exchange-v10-without-price-error-thresholds-spec* ::

```

int32 ⇒ int32 ⇒ int64 ⇒ int64 ⇒ int64 ⇒

```

```

int64 ⇒ exchange-rounding ⇒ exchange-options ⇒
exchange-result-v10 cxx-result
where
exchange-v10-without-price-error-thresholds-spec price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding options =
(if calculate-offer-value-pre price-n price-d max-wheat-send
max-sheep-receive ∧
calculate-offer-value-pre price-d price-n max-sheep-send
max-wheat-receive
then
let wheat-value =
min
(word-of-int (sint max-wheat-send * sint price-n) ::
uint128)
(word-of-int (sint max-sheep-receive * sint price-d));
sheep-value =
min
(word-of-int (sint max-sheep-send * sint price-d) ::
uint128)
(word-of-int (sint max-wheat-receive * sint price-n));
wheat-stays = wheat-value > sheep-value
in exchange-v10-amounts price-n price-d wheat-value sheep-value
max-wheat-send max-wheat-receive max-sheep-send
max-sheep-receive wheat-stays rounding options ≫=
check-exchange-v10-result max-wheat-send max-wheat-receive
max-sheep-send max-sheep-receive wheat-stays
else Cxx-Err Cxx-Assertion-Failed)

```

The remaining definitions of this subsection describe the same calculation at the level of unbounded integers.

definition *exchange-v10-pre* ::
 $int32 \Rightarrow int32 \Rightarrow int64 \Rightarrow int64 \Rightarrow int64 \Rightarrow$
 $int64 \Rightarrow bool$
where
exchange-v10-pre price-n price-d max-wheat-send max-wheat-receive
max-sheep-send max-sheep-receive $\longleftrightarrow$
 $0 < sint\ price-n \wedge 0 < sint\ price-d \wedge$
 $0 \leq sint\ max-wheat-send \wedge 0 \leq sint\ max-wheat-receive \wedge$
 $0 \leq sint\ max-sheep-send \wedge 0 \leq sint\ max-sheep-receive$

definition *exchange-wheat-value-int* ::
 $int32 \Rightarrow int32 \Rightarrow int64 \Rightarrow int64 \Rightarrow int$
where
exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive =
min (sint max-wheat-send * sint price-n)
(sint max-sheep-receive * sint price-d)

definition *exchange-sheep-value-int* ::
 $\text{int32} \Rightarrow \text{int32} \Rightarrow \text{int64} \Rightarrow \text{int64} \Rightarrow \text{int}$
where
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive =
 $\min (\text{sint } \text{max-sheep-send} * \text{sint } \text{price-d})$
 $(\text{sint } \text{max-wheat-receive} * \text{sint } \text{price-n})$

definition *exchange-v10-amounts-int* ::
 $\text{int32} \Rightarrow \text{int32} \Rightarrow \text{int64} \Rightarrow \text{int64} \Rightarrow \text{int64} \Rightarrow$
 $\text{int64} \Rightarrow \text{exchange-rounding} \Rightarrow \text{int} \times \text{int}$
where
exchange-v10-amounts-int price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding =
 $(\text{let } \text{wheat-value} =$
 $\text{exchange-wheat-value-int price-n price-d max-wheat-send}$
 $\text{max-sheep-receive};$
 $\text{sheep-value} =$
 $\text{exchange-sheep-value-int price-n price-d max-sheep-send}$
 $\text{max-wheat-receive};$
 $\text{wheat-stays} = \text{wheat-value} > \text{sheep-value}$
in if wheat-stays then
if rounding = Exchange-Strict-Send then
 $(\text{sheep-value div sint price-n},$
 $\min (\text{sint max-sheep-send}) (\text{sint max-sheep-receive}))$
else if sint price-n > sint price-d $\vee$
 $\text{rounding} = \text{Exchange-Strict-Receive}$
then
 $\text{let wheat-receive} = \text{sheep-value div sint price-n}$
in (wheat-receive,
 $(\text{wheat-receive} * \text{sint price-n} + \text{sint price-d} - 1)$
 $\text{div sint price-d})$
else
 $\text{let sheep-send} = \text{sheep-value div sint price-d}$
 $\text{in } ((\text{sheep-send} * \text{sint price-d}) \text{ div sint price-n}, \text{sheep-send})$
else if sint price-n > sint price-d then
 $\text{let wheat-receive} = \text{wheat-value div sint price-n}$
in (wheat-receive,
 $(\text{wheat-receive} * \text{sint price-n}) \text{ div sint price-d})$
else
 $\text{let sheep-send} = \text{wheat-value div sint price-d}$
 $\text{in } ((\text{sheep-send} * \text{sint price-d} + \text{sint price-n} - 1)$
 $\text{div sint price-n},$
 $\text{sheep-send}))$

exchange-v10-pre is the precondition under which the model is claimed to be free of assertions and overflow, and *exchange-v10-amounts-int* is the pair of amounts that the calculation is then claimed to produce.

2.1.9 Full exchange

definition *exchange-v10-with-options* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$
int64 $\Rightarrow$ *exchange-rounding* $\Rightarrow$ *exchange-options* $\Rightarrow$
exchange-result-v10 cxx-result

— C++: *exchangeV10* (*OfferExchange.cpp*)

where

exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive
max-sheep-send max-sheep-receive rounding options = *do* {
before-thresholds $\leftarrow$
exchange-v10-without-price-error-thresholds-with-options price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding options;
apply-price-error-thresholds price-n price-d
(num-wheat-received before-thresholds)
(num-sheep-send before-thresholds)
(result-wheat-stays before-thresholds) rounding
}

exchange-v10-with-options is the direct model of *exchangeV10* (*OfferExchange.cpp*). Its monadic bind preserves the C++ call order and propagates every explicit error from the pre-threshold calculation before invoking *apply-price-error-thresholds*. The trailing options record carries the C++ parameter *exactReceiveCap* and is forwarded unchanged to the pre-threshold calculation; the threshold pass never inspects it.

2.1.10 Protocol-facing exchange interface

The definitions above are option-parametric proof kernels. They are useful because a proof about all four option settings covers both real protocols and two diagnostic mutants at once, but the settings are not themselves protocol versions. The layer below is the implementation-facing interface: it mirrors the C++ signatures, which take a ledger version, and maps that version onto exactly one of the two configurations the protocol can be in.

type-synonym *uint32* = *32 word*

definition *protocol-version-starts-from* :: *uint32* $\Rightarrow$ *uint32* $\Rightarrow$ *bool*

— C++: *protocolVersionStartsFrom* (*util/ProtocolVersion.cpp*)

where

protocol-version-starts-from protocol-version from-version $\longleftrightarrow$
uint from-version $\leq$ *uint protocol-version*

The C++ ledger version is a *uint32-t* and the comparison *protocolVersion* $\geq$ *static-cast<uint32-t>(fromVersion)* is therefore unsigned. The model uses *uint*, not *sint*, so a version with the high bit set compares as a large version rather than a negative one, exactly as in C++.

definition *protocol-version-v29* :: *uint32*
 — C++: *ProtocolVersion::V-29* (*util/ProtocolVersion.h*)
 where *protocol-version-v29* = 29

definition *legacy-exchange-options* :: *exchange-options*
 where
 legacy-exchange-options =
 (*exact-receive-cap* = *False*,
 symmetric-exact-receive-cap = *False*)

definition *repaired-exchange-options* :: *exchange-options*
 where
 repaired-exchange-options =
 (*exact-receive-cap* = *True*,
 symmetric-exact-receive-cap = *True*)

definition *exchange-options-at-version* :: *uint32* $\Rightarrow$ *exchange-options*
 where
 exchange-options-at-version ledger-version =
 (*if protocol-version-starts-from ledger-version protocol-version-v29*
 then repaired-exchange-options
 else legacy-exchange-options)

exchange-options-at-version is the whole protocol mapping. Ledger versions below 29 select the legacy consensus arithmetic; version 29 and later select the repaired arithmetic, in which both round-down branches use *calculate-offer-amount-from-value*. The two mixed configurations are proof mutants: they are reachable in the kernel but not through this mapping, as recorded below.

definition *exchange-v10-without-price-error-thresholds* ::
 uint32 $\Rightarrow$ *int32* $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$
 int64 $\Rightarrow$ *int64* $\Rightarrow$ *exchange-rounding* $\Rightarrow$
 exchange-result-v10 cxx-result
 — C++: *exchangeV10WithoutPriceErrorThresholds* (*OfferExchange.cpp*)

where
 exchange-v10-without-price-error-thresholds ledger-version price-n
 price-d max-wheat-send max-wheat-receive max-sheep-send
 max-sheep-receive rounding =
 exchange-v10-without-price-error-thresholds-with-options price-n price-d
 max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
 rounding (exchange-options-at-version ledger-version)

definition *exchange-v10* ::
 uint32 $\Rightarrow$ *int32* $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$
 int64 $\Rightarrow$ *int64* $\Rightarrow$ *exchange-rounding* $\Rightarrow$
 exchange-result-v10 cxx-result
 — C++: *exchangeV10* (*OfferExchange.cpp*)

where
 exchange-v10 ledger-version price-n price-d max-wheat-send

$\text{max-wheat-receive max-sheep-send max-sheep-receive rounding} =$
 $\text{exchange-v10-with-options price-n price-d max-wheat-send}$
 $\text{max-wheat-receive max-sheep-send max-sheep-receive rounding}$
 $(\text{exchange-options-at-version ledger-version})$

These two definitions have the argument lists of the C++ functions as protocol 29 leaves them: a leading *uint32-t* ledger version, then the price, the four caps, and the rounding mode. Everything below the interface is unchanged, so the reduction theorems that follow let every existing option-parametric result be specialized to a real protocol.

lemma *exchange-options-at-version-legacy* [simp]:

$\neg \text{protocol-version-starts-from ledger-version protocol-version-v29} \implies$
 $\text{exchange-options-at-version ledger-version} = \text{legacy-exchange-options}$

lemma *exchange-options-at-version-repaired* [simp]:

$\text{protocol-version-starts-from ledger-version protocol-version-v29} \implies$
 $\text{exchange-options-at-version ledger-version} = \text{repaired-exchange-options}$

lemma *exchange-v10-without-price-error-thresholds-legacy*:

$\neg \text{protocol-version-starts-from ledger-version protocol-version-v29} \implies$
 $\text{exchange-v10-without-price-error-thresholds ledger-version price-n price-d}$
 $\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive}$
 $\text{rounding} =$
 $\text{exchange-v10-without-price-error-thresholds-with-options price-n price-d}$
 $\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive}$
 $\text{rounding legacy-exchange-options}$

lemma *exchange-v10-without-price-error-thresholds-repaired*:

$\text{protocol-version-starts-from ledger-version protocol-version-v29} \implies$
 $\text{exchange-v10-without-price-error-thresholds ledger-version price-n price-d}$
 $\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive}$
 $\text{rounding} =$
 $\text{exchange-v10-without-price-error-thresholds-with-options price-n price-d}$
 $\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive}$
 $\text{rounding repaired-exchange-options}$

lemma *exchange-v10-legacy*:

$\neg \text{protocol-version-starts-from ledger-version protocol-version-v29} \implies$
 $\text{exchange-v10 ledger-version price-n price-d max-wheat-send}$
 $\text{max-wheat-receive max-sheep-send max-sheep-receive rounding} =$
 $\text{exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive}$
 $\text{max-sheep-send max-sheep-receive rounding legacy-exchange-options}$

lemma *exchange-v10-repaired*:

$\text{protocol-version-starts-from ledger-version protocol-version-v29} \implies$
 $\text{exchange-v10 ledger-version price-n price-d max-wheat-send}$
 $\text{max-wheat-receive max-sheep-send max-sheep-receive rounding} =$
 $\text{exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive}$
 $\text{max-sheep-send max-sheep-receive rounding repaired-exchange-options}$

The four lemmas above are the protocol reduction theorems. Protocol 28 and earlier reduce to the legacy kernel and protocol 29 and later to the repaired kernel, so protocol-28 preservation is a theorem about the public interface rather than an appeal to test evidence.

lemma *protocol-boundary-is-at-29*:

exchange-options-at-version 28 = legacy-exchange-options
exchange-options-at-version 29 = repaired-exchange-options
 — The boundary sits exactly between ledger versions 28 and 29.

lemma *mixed-exchange-options-are-unreachable*:

exchange-options-at-version ledger-version ≠
 (*exact-receive-cap = True, symmetric-exact-receive-cap = False*)
exchange-options-at-version ledger-version ≠
 (*exact-receive-cap = False, symmetric-exact-receive-cap = True*)
 — No ledger version selects a one-sided repair.

Proof sketch: the mapping returns one of only two records, and neither agrees with a mixed one on both fields, so unfolding the definitions and splitting on the version test closes both goals.

The two single-option configurations remain available to proofs as mutants and counterexample witnesses, but no ledger version reaches them. Any claim about protocol behavior must therefore be stated through *exchange-options-at-version*.

2.1.11 Main properties

This subsection collects the properties that the rest of the theory establishes about the model above. Each is stated here informally, together with the proposition that is actually proved; the proofs follow, one section per modeled function.

Faithfulness. Every modeled C++ function is shown equal, on all bit patterns, to a branch-free specification phrased in exact integers. The smallest instance is the offer value:

```
calculate-offer-value price-n price-d max-send max-receive =
(if calculate-offer-value-pre price-n price-d max-send max-receive
 then Cxx-Ok
  (min (word-of-int (sint max-send * sint price-n))
        (word-of-int (sint max-receive * sint price-d))))
 else Cxx-Err Cxx-Assertion-Failed)
```

The analogous statements for *big-divide-or-throw*, *big-divide-or-throw128*, *check-price-error-bound*, *apply-price-error-thresholds*, and *exchange-v10-without-price-error-thresholds-with-options* are proved in their own sections.

Absence of arithmetic failure. Under *exchange-v10-pre* the whole pre-threshold calculation is total: no assertion fails, no 128-bit numerator wraps, and no quotient escapes the signed 64-bit range. What it computes is exactly *exchange-v10-amounts-int*, evaluated in unbounded integers, so the model of *exchange-v10-with-options* reduces to

```

apply-price-error-thresholds price-n price-d
(word-of-int
  (fst (exchange-v10-amounts-int price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive rounding)))
(word-of-int
  (snd (exchange-v10-amounts-int price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive rounding)))
(exchange-sheep-value-int price-n price-d max-sheep-send max-wheat-receive
  < exchange-wheat-value-int price-n price-d max-wheat-send
    max-sheep-receive)
rounding

```

Result contract. Whenever *exchange-v10-with-options* succeeds with the plain receive cap, both returned amounts are non-negative and respect the four caps, and the retained-offer flag is decided by the two offer values:

```

exchange-v10-pre price-n price-d max-wheat-send max-wheat-receive
  max-sheep-send max-sheep-receive ∧
exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive
  max-sheep-send max-sheep-receive rounding
(⟦exact-receive-cap = False, symmetric-exact-receive-cap = False⟧ =
  Cxx-Ok exchange-result →
  0 ≤ sint (num-wheat-received exchange-result) ∧
  sint (num-wheat-received exchange-result)
  ≤ min (sint max-wheat-receive) (sint max-wheat-send) ∧
  0 ≤ sint (num-sheep-send exchange-result) ∧
  sint (num-sheep-send exchange-result)
  ≤ min (sint max-sheep-receive) (sint max-sheep-send)

```

Contract of a positive trade. When the returned amounts are both positive, the party whose offer is kept is never disadvantaged, and the trade respects the price error bound appropriate to the rounding mode:

```

favored-seller-ok price-n price-d (num-wheat-received exchange-result)
  (num-sheep-send exchange-result) (result-wheat-stays exchange-result) ∧
(if rounding = Exchange-Normal
  then price-error-bound-spec price-n price-d
    (num-wheat-received exchange-result) (num-sheep-send exchange-result)
    False
  else price-error-bound-spec price-n price-d
    (num-wheat-received exchange-result) (num-sheep-send exchange-result)
    True)

```

Outside strict-send mode the two amounts are moreover zero together, so a successful call either trades on both sides or trades nothing.

The properties of this subsection are stated and proved for the plain receive cap, matching the protocol versions the C++ has shipped with; their *exactReceiveCap* counterparts are future work.

2.2 Wide multiplication

The lemmas below characterize the outcomes of *big-multiply* and record that its successful result is the exact integer product.

lemma *big-multiply-nonnegative* [simp]:

assumes $0 \leq \text{sint } a$ **and** $0 \leq \text{sint } b$

shows *big-multiply* a $b =$

Cxx-Ok (*word-of-int* ($\text{sint } a * \text{sint } b$))

lemma *big-multiply-negative-left* [simp]:

assumes $\text{sint } a < 0$

shows *big-multiply* a $b = \text{Cxx-Err Cxx-Assertion-Failed}$

lemma *big-multiply-negative-right* [simp]:

assumes $\text{sint } b < 0$

shows *big-multiply* a $b = \text{Cxx-Err Cxx-Assertion-Failed}$

lemma *nonnegative-int64-product-fits-uint128*:

fixes a $b :: \text{int64}$

assumes *a-nonnegative*: $0 \leq \text{sint } a$

and *b-nonnegative*: $0 \leq \text{sint } b$

shows $0 \leq \text{sint } a * \text{sint } b$

and $\text{sint } a * \text{sint } b < (2 :: \text{int}) ^ 128$

lemma *big-multiply-uint-value*:

fixes a $b :: \text{int64}$

assumes $0 \leq \text{sint } a$ **and** $0 \leq \text{sint } b$

shows *uint* (*word-of-int* ($\text{sint } a * \text{sint } b$) :: *uint128*) =

$\text{sint } a * \text{sint } b$

lemma *sint-scast-int32-int64* [simp]:

$\text{sint } (\text{scast } x :: \text{int64}) = \text{sint } (x :: \text{int32})$

2.3 Checked division

The lemmas below determine, for every bit pattern, which of the three outcomes of *big-divide-or-throw* and *big-divide-or-throw128* occurs, and they identify the word returned on success with the exact rounded quotient.

lemma *rounded-quotient-nonnegative*:

assumes *numerator-nonnegative*: $0 \leq \text{numerator}$

and *divisor-positive*: $0 < \text{divisor}$
shows $0 \leq \text{rounded-quotient numerator divisor rounding}$

Proof sketch: split on the rounding constructor. The round-down numerator is non-negative directly; the round-up numerator remains non-negative because the positive divisor is at least one. Integer division by a positive value preserves non-negativity.

lemma *sint-word-of-int-nonnegative-int64*:
assumes *value-nonnegative*: $0 \leq \text{value}$
and *value-bounded*: $\text{value} \leq \text{int64-max-int}$
shows $\text{sint} (\text{word-of-int value} :: \text{int64}) = \text{value}$

Proof sketch: the assumptions place the value inside the non-negative half of the signed 64-bit range, so signed truncation is the identity.

lemma *nonnegative-int64-round-up-numerator-fits-wint128*:
fixes $a\ b\ c :: \text{int64}$
assumes *a-nonnegative*: $0 \leq \text{sint } a$
and *b-nonnegative*: $0 \leq \text{sint } b$
and *c-positive*: $0 < \text{sint } c$
shows $\text{sint } a * \text{sint } b + \text{sint } c - 1 < (2 :: \text{int}) ^ 128$

Proof sketch: each non-negative multiplicand is below two to the sixty-third power, so their product is below two to the one-hundred-twenty-sixth power. Adding a signed 64-bit divisor still leaves the round-up numerator strictly below the unsigned 128-bit limit.

lemma *uint128-round-up-numerator-fits*:
assumes *guard*:
 $\text{uint } a \leq (2 :: \text{int}) ^ 128 - 1 - (\text{sint } b - 1)$
shows $\text{uint } a + \text{sint } b - 1 < (2 :: \text{int}) ^ 128$

Proof sketch: rearranging the explicit C++ guard bounds the incremented numerator by the largest unsigned 128-bit integer, hence strictly below the modulus.

lemma *big-divide-or-throw-success*:
assumes *a-nonnegative*: $0 \leq \text{sint } a$
and *b-nonnegative*: $0 \leq \text{sint } b$
and *c-positive*: $0 < \text{sint } c$
and *quotient-bounded*:
 $\text{rounded-quotient} (\text{sint } a * \text{sint } b) (\text{sint } c) \text{ rounding}$

$\leq \text{int64-max-int}$
shows $\text{big-divide-or-throw } a \ b \ c \ \text{rounding} =$
 $\text{Cxx-Ok } (\text{word-of-int}$
 $\quad (\text{rounded-quotient } (\text{sint } a * \text{sint } b) (\text{sint } c) \ \text{rounding}))$
and $\text{sint } (\text{word-of-int}$
 $\quad (\text{rounded-quotient } (\text{sint } a * \text{sint } b) (\text{sint } c) \ \text{rounding})) ::$
 $\text{int64}) =$
 $\text{rounded-quotient } (\text{sint } a * \text{sint } b) (\text{sint } c) \ \text{rounding}$

Proof sketch: valid operands make the exact numerator and quotient non-negative. The quotient bound selects the successful branch, and the signed-word range lemma shows that the returned bit pattern denotes the same mathematical quotient.

lemma $\text{big-divide-or-throw-assertion-iff}$:
 $\text{big-divide-or-throw } a \ b \ c \ \text{rounding} =$
 $\text{Cxx-Err Cxx-Assertion-Failed}$
 $\longleftrightarrow \text{sint } a < 0 \vee \text{sint } b < 0 \vee \text{sint } c \leq 0$

Proof sketch: unfold the checked helper. Only its initial validity test constructs an assertion error; later branches construct either overflow or a successful word.

lemma $\text{big-divide-or-throw-overflow-iff}$:
assumes $0 \leq \text{sint } a$ **and** $0 \leq \text{sint } b$ **and** $0 < \text{sint } c$
shows $\text{big-divide-or-throw } a \ b \ c \ \text{rounding} = \text{Cxx-Err Cxx-Overflow}$
 $\longleftrightarrow \text{rounded-quotient } (\text{sint } a * \text{sint } b) (\text{sint } c) \ \text{rounding}$
 $\quad > \text{int64-max-int}$

Proof sketch: under the validity conditions the assertion branch is impossible, leaving overflow exactly when the mathematical quotient does not fit in a signed 64-bit result.

theorem $\text{big-divide-or-throw-success-iff}$:
 $\text{big-divide-or-throw } a \ b \ c \ \text{rounding} = \text{Cxx-Ok result}$
 $\longleftrightarrow 0 \leq \text{sint } a \wedge 0 \leq \text{sint } b \wedge 0 < \text{sint } c \wedge$
 $\quad \text{rounded-quotient } (\text{sint } a * \text{sint } b) (\text{sint } c) \ \text{rounding}$
 $\quad \leq \text{int64-max-int} \wedge$
 $\text{result} = \text{word-of-int}$
 $\quad (\text{rounded-quotient } (\text{sint } a * \text{sint } b) (\text{sint } c) \ \text{rounding})$

Proof sketch: unfold both checked branches. Equality with a successful result forces valid operands and a bounded quotient; conversely those conditions select the successful constructor with the exact word value.

lemma *big-divide-or-throw128-success*:
assumes *divisor-positive*: $0 < \text{ sint } b$
and *no-round-up-overflow*:
 $\neg (\text{rounding} = \text{Cxx-Round-Up} \wedge$
 $\text{ uint } a > (2 :: \text{ int}) ^{128} - 1 - (\text{ sint } b - 1))$
and *quotient-bounded*:
 $\text{rounded-quotient } (\text{ uint } a) (\text{ sint } b) \text{ rounding}$
 $\leq \text{ int64-max-int}$
shows *big-divide-or-throw128* $a \ b \text{ rounding} =$
 $\text{Cxx-Ok } (\text{word-of-int}$
 $\text{ (rounded-quotient } (\text{ uint } a) (\text{ sint } b) \text{ rounding}))$
and *sint* $(\text{word-of-int}$
 $\text{ (rounded-quotient } (\text{ uint } a) (\text{ sint } b) \text{ rounding}) ::$
 $\text{ int64}) =$
 $\text{rounded-quotient } (\text{ uint } a) (\text{ sint } b) \text{ rounding}$

Proof sketch: a word has a non-negative unsigned value and the divisor is positive, so the exact quotient is non-negative. The two remaining assumptions bypass both overflow checks, and the signed-word range lemma identifies the returned word with that quotient.

lemma *big-divide-or-throw128-assertion-iff*:
 $\text{big-divide-or-throw128 } a \ b \text{ rounding} =$
 $\text{Cxx-Err Cxx-Assertion-Failed}$
 $\longleftrightarrow \text{ sint } b \leq 0$

Proof sketch: the divisor check is the only assertion-producing branch; the explicit numerator guard and result-range check both report overflow.

lemma *big-divide-or-throw128-overflow-iff*:
assumes $0 < \text{ sint } b$
shows $\text{big-divide-or-throw128 } a \ b \text{ rounding} = \text{Cxx-Err Cxx-Overflow}$
 $\longleftrightarrow$
 $(\text{rounding} = \text{Cxx-Round-Up} \wedge$
 $\text{ uint } a > (2 :: \text{ int}) ^{128} - 1 - (\text{ sint } b - 1))$
 $\vee$
 $(\neg (\text{rounding} = \text{Cxx-Round-Up} \wedge$
 $\text{ uint } a > (2 :: \text{ int}) ^{128} - 1 - (\text{ sint } b - 1)) \wedge$
 $\text{rounded-quotient } (\text{ uint } a) (\text{ sint } b) \text{ rounding}$
 $> \text{ int64-max-int})$

Proof sketch: after the positive-divisor check, overflow arises either from the explicit round-up increment guard or, when that guard is clear, from narrowing a quotient above the signed 64-bit maximum.

theorem *big-divide-or-throw128-success-iff*:

big-divide-or-throw128 a b rounding = Cxx-Ok result
 $\iff 0 < \text{ sint } b \wedge$
 $\neg (\text{ rounding } = \text{ Cxx-Round-Up } \wedge$
 $\text{ uint } a > (2 :: \text{ int }) ^ 128 - 1 - (\text{ sint } b - 1)) \wedge$
 $\text{ rounded-quotient } (\text{ uint } a) (\text{ sint } b) \text{ rounding}$
 $\leq \text{ int64-max-int } \wedge$
 $\text{ result } = \text{ word-of-int }$
 $(\text{ rounded-quotient } (\text{ uint } a) (\text{ sint } b) \text{ rounding})$

Proof sketch: unfolding the definition shows that success is equivalent to passing the divisor assertion, the round-up increment guard, and the signed result bound, after which the returned word is fixed uniquely.

2.4 Price error bound

The lemmas below show that *check-price-error-bound* fails an assertion exactly when one of its four numeric arguments is negative, and that it otherwise returns the mathematical predicate *price-error-bound-spec*.

lemma *sint-price-scale*:

fixes *price* :: *int32*
shows *sint (word-of-int (100 * sint price)) :: int64* =
 $100 * \text{ sint } \text{ price}$

Proof sketch: the signed range of a 32-bit word, after multiplication by one hundred, is strictly inside the signed 64-bit range. Consequently signed truncation to 64 bits is the identity.

lemma *uint-word-abs-diff*:

fixes *a b* :: *uint128*
shows *uint (if a > b then a - b else b - a) =*
 $\text{ abs } (\text{ uint } a - \text{ uint } b)$

Proof sketch: split on the unsigned ordering of the words. In each branch the larger value is the minuend, so unsigned subtraction does not wrap and agrees with the corresponding non-negative integer difference.

lemma *nonnegative-price-error-product-fits-uint128*:

fixes *price* :: *int32* **and** *amount* :: *int64*
assumes *price-nonnegative*: $0 \leq \text{ sint } \text{ price}$
and *amount-nonnegative*: $0 \leq \text{ sint } \text{ amount}$
shows $0 \leq 100 * \text{ sint } \text{ price} * \text{ sint } \text{ amount}$
and $100 * \text{ sint } \text{ price} * \text{ sint } \text{ amount} < (2 :: \text{ int }) ^ 128$

Proof sketch: the scaled price is an exact, non-negative signed 64-bit value by *sint-price-scale*. The previously proved bound for multiplying two non-negative signed 64-bit words then supplies both claims.

lemma *price-error-product-uint-value*:
fixes *price* :: *int32* **and** *amount* :: *int64*
assumes *price-nonnegative*: $0 \leq \text{sint } \textit{price}$
and *amount-nonnegative*: $0 \leq \text{sint } \textit{amount}$
shows *uint* (*word-of-int* ($100 * \text{sint } \textit{price} * \text{sint } \textit{amount}$)) ::
uint128) = $100 * \text{sint } \textit{price} * \text{sint } \textit{amount}$

Proof sketch: interpret the scaled price as a signed 64-bit word and apply the exact unsigned value theorem for *big-multiply*; the scaling lemma then rewrites the result to mathematical integers.

lemma *check-price-error-bound-success-words*:
assumes *price-n-nonnegative*: $0 \leq \text{sint } \textit{price-n}$
and *price-d-nonnegative*: $0 \leq \text{sint } \textit{price-d}$
and *wheat-nonnegative*: $0 \leq \text{sint } \textit{wheat-receive}$
and *sheep-nonnegative*: $0 \leq \text{sint } \textit{sheep-send}$
shows *check-price-error-bound* *price-n price-d wheat-receive sheep-send*
can-favor-wheat =
Cxx-Ok
(*let* *lhs* = *word-of-int*
 $(100 * \text{sint } \textit{price-n} * \text{sint } \textit{wheat-receive}) :: \textit{uint128}$;
rhs = *word-of-int*
 $(100 * \text{sint } \textit{price-d} * \text{sint } \textit{sheep-send}) :: \textit{uint128}$;
cap = *word-of-int*
 $(\text{sint } \textit{price-n} * \text{sint } \textit{wheat-receive}) :: \textit{uint128}$;
abs-diff = (*if* *lhs* > *rhs* *then* *lhs* - *rhs* *else* *rhs* - *lhs*)
in (*can-favor-wheat* $\wedge$ *rhs* > *lhs*) $\vee$ *abs-diff* $\leq$ *cap*)

Proof sketch: non-negativity makes all three calls of *big-multiply* succeed. Substitute their exact word results into the monadic computation; the early return is equivalent to disjoining its condition with the ordinary bound check.

theorem *check-price-error-bound-integer-characterization*:
assumes *price-n-nonnegative*: $0 \leq \text{sint } \textit{price-n}$
and *price-d-nonnegative*: $0 \leq \text{sint } \textit{price-d}$
and *wheat-nonnegative*: $0 \leq \text{sint } \textit{wheat-receive}$
and *sheep-nonnegative*: $0 \leq \text{sint } \textit{sheep-send}$
shows *check-price-error-bound* *price-n price-d wheat-receive sheep-send*
can-favor-wheat =
Cxx-Ok

(*price-error-bound-spec price-n price-d wheat-receive sheep-send*
can-favor-wheat)

Proof sketch: use the word-level success lemma, then replace the unsigned values of the two scaled products and the cap by their exact integer values. The unsigned absolute difference is exact because the model always subtracts the smaller word from the larger one.

theorem *check-price-error-bound-negative:*

assumes *negative:*

sint price-n < 0 ∨ sint price-d < 0 ∨
sint wheat-receive < 0 ∨ sint sheep-send < 0

shows *check-price-error-bound price-n price-d wheat-receive sheep-send*
can-favor-wheat = Cxx-Err Cxx-Assertion-Failed

Proof sketch: exact signed scaling preserves the sign of each price component. Follow the monadic evaluation order: the first negative operand encountered by *big-multiply* produces the assertion error, which the bind propagates unchanged.

theorem *check-price-error-bound-characterization:*

check-price-error-bound price-n price-d wheat-receive sheep-send
can-favor-wheat =

(*if 0 ≤ sint price-n ∧ 0 ≤ sint price-d ∧*
0 ≤ sint wheat-receive ∧ 0 ≤ sint sheep-send

then Cxx-Ok

(*price-error-bound-spec price-n price-d wheat-receive sheep-send*
can-favor-wheat)

else Cxx-Err Cxx-Assertion-Failed)

Proof sketch: split on the weakest successful-input condition. The non-negative branch is the exact integer characterization; negating that conjunction exposes a negative operand, so the assertion theorem applies.

2.5 Price-error application

The lemmas below replace the monadic branch structure of *apply-price-error-thresholds* by the flat decision tree *apply-price-error-thresholds-spec*, and derive what a successful call guarantees about the record it returns.

lemma *apply-price-error-thresholds-nonpositive:*

assumes *nonpositive:*

$\neg (0 < \text{sint wheat-receive} \wedge 0 < \text{sint sheep-send})$

shows

apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays rounding =

```

(if rounding = Exchange-Strict-Send then
  if sheep-send = 0 then Cxx-Err Cxx-Runtime-Error
  else Cxx-Ok
  (make-exchange-result wheat-receive sheep-send wheat-stays)
else Cxx-Ok (make-exchange-result 0 0 wheat-stays))

```

Proof sketch: the failed positivity guard selects the final C++ branch; simplifying only that guard leaves the strict-send zero test unchanged.

lemma *apply-price-error-thresholds-negative-price:*

assumes *amounts-positive:*

$0 < \text{sint wheat-receive} \wedge 0 < \text{sint sheep-send}$

and *negative-price:* $\text{sint price-n} < 0 \vee \text{sint price-d} < 0$

shows

apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays rounding = Cxx-Err Cxx-Assertion-Failed

Proof sketch: positivity reaches both multiplications. Sign extension preserves each price sign, so the first negative price reached by *big-multiply* produces the modeled assertion and monadic bind propagates it.

theorem *apply-price-error-thresholds-positive-characterization:*

assumes *amounts-positive:*

$0 < \text{sint wheat-receive} \wedge 0 < \text{sint sheep-send}$

and *price-n-nonnegative:* $0 \leq \text{sint price-n}$

and *price-d-nonnegative:* $0 \leq \text{sint price-d}$

shows

apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays rounding =
 (let original =
 make-exchange-result wheat-receive sheep-send wheat-stays;
 zero = make-exchange-result 0 0 wheat-stays
 in if $\neg \text{favored-seller-ok price-n price-d wheat-receive sheep-send}$
 wheat-stays
 then Cxx-Err Cxx-Runtime-Error
 else if rounding = Exchange-Normal then
 Cxx-Ok
 (if price-error-bound-spec price-n price-d wheat-receive
 sheep-send False
 then original else zero)
 else if price-error-bound-spec price-n price-d wheat-receive
 sheep-send True
 then Cxx-Ok original
 else Cxx-Err Cxx-Runtime-Error)

Proof sketch: the non-negative multiplication contracts replace both 128-bit words by their exact integer products. Unsigned word comparisons then

become the favored-seller inequality, and the already-proved price-error contract replaces each checked bound call.

theorem *apply-price-error-thresholds-characterization:*

apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays rounding =
apply-price-error-thresholds-spec price-n price-d wheat-receive sheep-send
wheat-stays rounding

Proof sketch: partition all bit patterns first by the amount-positivity guard and then by price signs. The preceding three branch theorems cover the non-positive, assertion, and fully non-negative cases respectively.

theorem *apply-price-error-thresholds-preserves-wheat-stays:*

assumes *result:*

apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays rounding = Cxx-Ok exchange-result

shows *result-wheat-stays exchange-result = wheat-stays*

Proof sketch: rewrite the executable function to its total branch characterization. Every successful constructor in that specification uses the input stay flag.

theorem *apply-price-error-thresholds-normal-result:*

assumes *result:*

apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays Exchange-Normal = Cxx-Ok exchange-result

shows

exchange-result =
make-exchange-result wheat-receive sheep-send wheat-stays ∨
exchange-result = make-exchange-result 0 0 wheat-stays

Proof sketch: after rewriting with the total characterization, all normal success branches select either the original record or the explicit zero-amount record.

theorem *apply-price-error-thresholds-positive-path-result:*

assumes *amounts-positive:*

0 < sint wheat-receive ∧ 0 < sint sheep-send

and *path-mode: rounding ≠ Exchange-Normal*

and *result:*

apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays rounding = Cxx-Ok exchange-result

shows

*exchange-result =
make-exchange-result wheat-receive sheep-send wheat-stays*

Proof sketch: under the positive guard, every non-normal successful branch of the total characterization returns the original amount record; all alternatives are explicit errors.

theorem *apply-price-error-thresholds-positive-success-favored:*

assumes *amounts-positive:*

0 < sint wheat-receive $\wedge$ 0 < sint sheep-send

and *result:*

*apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays rounding = Cxx-Ok exchange-result*

shows

favored-seller-ok price-n price-d wheat-receive sheep-send wheat-stays

Proof sketch: a positive call can return successfully only after passing the exact favored-seller comparison in the total characterization. Negative prices and the opposite direction are error branches.

theorem *apply-price-error-thresholds-positive-trade-bound:*

assumes *amounts-positive:*

0 < sint wheat-receive $\wedge$ 0 < sint sheep-send

and *result:*

*apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays rounding = Cxx-Ok exchange-result*

and *result-positive:*

*0 < sint (num-wheat-received exchange-result) $\wedge$
0 < sint (num-sheep-send exchange-result)*

shows

*if rounding = Exchange-Normal
then price-error-bound-spec price-n price-d wheat-receive sheep-send False
else price-error-bound-spec price-n price-d wheat-receive sheep-send True*

Proof sketch: normal mode may evade its bound only by returning the zero-amount record, which the positive-result assumption excludes. Non-normal success already requires the one-sided bound.

2.6 Offer value

The lemmas below show that *calculate-offer-value* succeeds exactly under *calculate-offer-value-pre*, and that the unsigned 128-bit word it then returns denotes the exact integer *min (sint max-send * sint price-n) (sint max-receive * sint price-d)*.

lemma *calculate-offer-value-success*:
assumes *calculate-offer-value-pre price-n price-d max-send max-receive*
shows *calculate-offer-value price-n price-d max-send max-receive =*
Cxx-Ok
(min
*(word-of-int (sint max-send * sint price-n) :: uint128)*
*(word-of-int (sint max-receive * sint price-d) :: uint128))*

lemma *uint-min-word*:
uint (min a b) = min (uint a) (uint b)
for *a b :: 'a::len word*

theorem *calculate-offer-value-integer-characterization*:
assumes *pre*:
calculate-offer-value-pre price-n price-d max-send max-receive
obtains *v* **where**
calculate-offer-value price-n price-d max-send max-receive = Cxx-Ok v
and
uint v =
*min (sint max-send * sint price-n)*
*(sint max-receive * sint price-d)*

lemma *calculate-offer-value-example*:
calculate-offer-value
(3 :: int32) (2 :: int32)
(10 :: int64) (10 :: int64) =
Cxx-Ok (20 :: uint128)

theorem *calculate-offer-value-characterization*:
calculate-offer-value price-n price-d max-send max-receive =
(if calculate-offer-value-pre price-n price-d max-send max-receive
then Cxx-Ok
(min
*(word-of-int (sint max-send * sint price-n) :: uint128)*
*(word-of-int (sint max-receive * sint price-d) :: uint128))*
else Cxx-Err Cxx-Assertion-Failed)

Proof sketch: unfold both multiplications. The conjunction in *calculate-offer-value-pre* is exactly the condition under which neither sequential *big-multiply* call asserts; on that branch both exact product words reach the unsigned minimum.

2.7 Exchange before price-error thresholds

The lemmas below show that, under *exchange-v10-pre*, no assertion, overflow, or runtime error can arise in *exchange-v10-without-price-error-*

thresholds-with-options, and that the pair of amounts it returns is exactly *exchange-v10-amounts-int* evaluated in unbounded integer arithmetic. This is the longest development in the theory: each of the five division branches is bounded separately before the branches are recombined.

lemma *exchange-v10-without-as-checked-amounts:*

```

exchange-v10-without-price-error-thresholds-with-options price-n price-d
  max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
  rounding options = do {
    wheat-value ← calculate-offer-value price-n price-d
    max-wheat-send max-sheep-receive;
    sheep-value ← calculate-offer-value price-d price-n
    max-sheep-send max-wheat-receive;
    let wheat-stays = wheat-value > sheep-value;
    amounts ← exchange-v10-amounts price-n price-d wheat-value sheep-value
    max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
    wheat-stays rounding options;
    check-exchange-v10-result max-wheat-send max-wheat-receive
    max-sheep-send max-sheep-receive wheat-stays amounts
  }

```

Proof sketch: unfold the top-level model and the factored final checker. The tuple projections and two ordered bounds tests are definitionally the same as the explicit tail of the executable model.

theorem *exchange-v10-without-price-error-thresholds-characterization:*

```

exchange-v10-without-price-error-thresholds-with-options price-n price-d
  max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
  rounding options =
exchange-v10-without-price-error-thresholds-spec price-n price-d
  max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
  rounding options

```

Proof sketch: replace both sequential offer-value calls with their total characterizations. A failed precondition produces the first reached assertion; otherwise monadic simplification leaves precisely the two exact value words, the amount block, and the final checker in the specification.

lemma *int-div-upper-bound:*

```

fixes numerator divisor bound :: int
assumes divisor-positive: 0 < divisor
  and numerator-bound: numerator ≤ bound * divisor
shows numerator div divisor ≤ bound

```

Proof sketch: integer division by a positive divisor is monotone in the numerator. Compare with $bound * divisor$ and cancel the exact multiple.

lemma *int-ceiling-div-upper-bound*:
fixes *numerator divisor bound* :: *int*
assumes *divisor-positive*: $0 < divisor$
and *numerator-nonnegative*: $0 \leq numerator$
and *numerator-bound*: $numerator \leq bound * divisor$
shows $(numerator + divisor - 1) \text{ div } divisor \leq bound$

Proof sketch: the assumptions force a non-negative bound. Add $divisor - 1$ to the numerator inequality, use division monotonicity, and split the exact multiple from the remainder.

lemma *int-ceiling-div-nonnegative*:
fixes *numerator divisor* :: *int*
assumes $0 \leq numerator$ **and** $0 < divisor$
shows $0 \leq (numerator + divisor - 1) \text{ div } divisor$

Proof sketch: this expression is *rounded-quotient* in round-up mode, whose non-negativity theorem applies directly.

lemma *int-div-mult-le*:
fixes *numerator divisor* :: *int*
assumes *divisor-positive*: $0 < divisor$
shows $numerator \text{ div } divisor * divisor \leq numerator$

Proof sketch: decompose the numerator into its quotient multiple and remainder. A positive divisor makes the integer remainder non-negative.

lemma *sint64-upper-bound*:
 $sint\ (x :: int64) \leq int64\text{-max-int}$

Proof sketch: instantiate the generic strict upper bound for signed words at width 64 and convert the strict power-of-two bound to the largest signed 64-bit integer.

lemma *signed-min64-word*:
 $signed\text{-min64}\ a\ b =$
 $word\text{-of-int}\ (\min\ (sint\ a)\ (sint\ b))$

Proof sketch: split on the signed comparison used by the definition. Re-encoding the signed interpretation of either selected word returns the original

bit pattern.

lemma *big-divide-or-throw128-down-bounded*:
assumes *divisor-positive*: $0 < \text{sint divisor}$
and *value-bound*: $\text{uint value} \leq \text{bound} * \text{sint divisor}$
and *bound-bounded*: $\text{bound} \leq \text{int64-max-int}$
shows *big-divide-or-throw128* *value divisor Cxx-Round-Down* =
Cxx-Ok (*word-of-int* ($\text{uint value} \text{ div } \text{sint divisor}$))
and *sint*
 $(\text{word-of-int} (\text{uint value} \text{ div } \text{sint divisor}) :: \text{int64}) =$
 $\text{uint value} \text{ div } \text{sint divisor}$

Proof sketch: monotonic division turns the supplied product bound into a signed-64 quotient bound. Round-down makes the 128-bit increment guard false, so the existing checked-division success theorem yields both the exact returned word and its signed interpretation.

lemma *exchange-wheat-value-int-bounds*:
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
shows $0 \leq \text{exchange-wheat-value-int price-n price-d max-wheat-send}$
max-sheep-receive
and *exchange-wheat-value-int price-n price-d max-wheat-send*
 $\text{max-sheep-receive} \leq \text{sint max-wheat-send} * \text{sint price-n}$
and *exchange-wheat-value-int price-n price-d max-wheat-send*
 $\text{max-sheep-receive} \leq \text{sint max-sheep-receive} * \text{sint price-d}$

Proof sketch: well-formedness makes both exact products non-negative, and their mathematical minimum is non-negative and below each operand.

lemma *exchange-sheep-value-int-bounds*:
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
shows $0 \leq \text{exchange-sheep-value-int price-n price-d max-sheep-send}$
max-wheat-receive
and *exchange-sheep-value-int price-n price-d max-sheep-send*
 $\text{max-wheat-receive} \leq \text{sint max-sheep-send} * \text{sint price-d}$
and *exchange-sheep-value-int price-n price-d max-sheep-send*
 $\text{max-wheat-receive} \leq \text{sint max-wheat-receive} * \text{sint price-n}$

Proof sketch: this is the symmetric minimum argument for the reversed offer value.

lemma *exchange-wheat-value-word*:

```

assumes pre: exchange-v10-pre price-n price-d max-wheat-send
             max-wheat-receive max-sheep-send max-sheep-receive
defines value  $\equiv$ 
  min
    (word-of-int (sint max-wheat-send * sint price-n) :: uint128)
    (word-of-int (sint max-sheep-receive * sint price-d))
shows uint value =
  exchange-wheat-value-int price-n price-d max-wheat-send max-sheep-receive

```

Proof sketch: the established multiplication range theorem identifies each 128-bit word with its exact non-negative product; unsigned minimum then commutes with *uint*.

```

lemma exchange-sheep-value-word:
assumes pre: exchange-v10-pre price-n price-d max-wheat-send
             max-wheat-receive max-sheep-send max-sheep-receive
defines value  $\equiv$ 
  min
    (word-of-int (sint max-sheep-send * sint price-d) :: uint128)
    (word-of-int (sint max-wheat-receive * sint price-n))
shows uint value =
  exchange-sheep-value-int price-n price-d max-sheep-send max-wheat-receive

```

Proof sketch: apply the same exact-product and unsigned-minimum argument to the price and maxima in their reversed order.

```

lemma exchange-amounts-strict-send-bounds:
fixes price-n sheep-value wheat-value max-wheat-send max-wheat-receive
       max-sheep-send max-sheep-receive :: int
assumes price-positive:  $0 < \text{price-n}$ 
and sheep-nonnegative:  $0 \leq \text{sheep-value}$ 
and sheep-receive-bound:
  sheep-value  $\leq \text{max-wheat-receive} * \text{price-n}$ 
and wheat-stays: sheep-value < wheat-value
and wheat-send-bound: wheat-value  $\leq \text{max-wheat-send} * \text{price-n}$ 
and sheep-send-nonnegative:  $0 \leq \text{max-sheep-send}$ 
and sheep-receive-nonnegative:  $0 \leq \text{max-sheep-receive}$ 
shows  $0 \leq \text{sheep-value div price-n}$ 
and sheep-value div price-n  $\leq$ 
  min max-wheat-receive max-wheat-send
and  $0 \leq \text{min max-sheep-send max-sheep-receive}$ 
and min max-sheep-send max-sheep-receive  $\leq$ 
  min max-sheep-receive max-sheep-send

```

Proof sketch: floor division preserves non-negativity and each value bound yields one wheat maximum. The strict value ordering supplies the other; the sheep output is the signed minimum directly.

lemma *exchange-amounts-wheat-stays-up-bounds:*
fixes *price-n price-d sheep-value wheat-value max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive wheat-receive
sheep-send :: int
assumes *price-n-positive: 0 < price-n*
and *price-d-positive: 0 < price-d*
and *sheep-nonnegative: 0 ≤ sheep-value*
and *sheep-wheat-receive-bound:*
*sheep-value ≤ max-wheat-receive * price-n*
and *sheep-sheep-send-bound:*
*sheep-value ≤ max-sheep-send * price-d*
and *wheat-stays: sheep-value < wheat-value*
and *wheat-wheat-send-bound:*
*wheat-value ≤ max-wheat-send * price-n*
and *wheat-sheep-receive-bound:*
*wheat-value ≤ max-sheep-receive * price-d*
defines *wheat-receive ≡ sheep-value div price-n*
and *sheep-send ≡*
*(wheat-receive * price-n + price-d - 1) div price-d*
shows *0 ≤ wheat-receive*
and *wheat-receive ≤ min max-wheat-receive max-wheat-send*
and *0 ≤ sheep-send*
and *sheep-send ≤ min max-sheep-receive max-sheep-send*

Proof sketch: floor the sheep value to obtain wheat, then bound its product by the original value. The ceiling bound transfers both sheep maxima; the strict stay ordering transfers the two opposite-offer maxima.

lemma *exchange-amounts-wheat-stays-down-bounds:*
fixes *price-n price-d sheep-value wheat-value max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive wheat-receive
sheep-send :: int
assumes *price-n-positive: 0 < price-n*
and *price-d-positive: 0 < price-d*
and *sheep-nonnegative: 0 ≤ sheep-value*
and *sheep-wheat-receive-bound:*
*sheep-value ≤ max-wheat-receive * price-n*
and *sheep-sheep-send-bound:*
*sheep-value ≤ max-sheep-send * price-d*
and *wheat-stays: sheep-value < wheat-value*
and *wheat-wheat-send-bound:*
*wheat-value ≤ max-wheat-send * price-n*
and *wheat-sheep-receive-bound:*
*wheat-value ≤ max-sheep-receive * price-d*
defines *sheep-send ≡ sheep-value div price-d*
and *wheat-receive ≡ sheep-send * price-d div price-n*

shows $0 \leq \text{wheat-receive}$
and $\text{wheat-receive} \leq \min \text{max-wheat-receive max-wheat-send}$
and $0 \leq \text{sheep-send}$
and $\text{sheep-send} \leq \min \text{max-sheep-receive max-sheep-send}$

Proof sketch: this branch floors first in sheep units and then in wheat units. Each quotient product is below its numerator, so the two value minima and the strict stay ordering provide all four maxima.

lemma *exchange-amounts-sheep-stays-wheat-more-bounds:*
fixes $\text{price-n price-d wheat-value sheep-value max-wheat-send}$
 $\text{max-wheat-receive max-sheep-send max-sheep-receive wheat-receive}$
 $\text{sheep-send} :: \text{int}$
assumes $\text{price-n-positive: } 0 < \text{price-n}$
and $\text{price-d-positive: } 0 < \text{price-d}$
and $\text{wheat-nonnegative: } 0 \leq \text{wheat-value}$
and $\text{wheat-wheat-send-bound:}$
 $\text{wheat-value} \leq \text{max-wheat-send} * \text{price-n}$
and $\text{wheat-sheep-receive-bound:}$
 $\text{wheat-value} \leq \text{max-sheep-receive} * \text{price-d}$
and $\text{sheep-stays: wheat-value} \leq \text{sheep-value}$
and $\text{sheep-wheat-receive-bound:}$
 $\text{sheep-value} \leq \text{max-wheat-receive} * \text{price-n}$
and $\text{sheep-sheep-send-bound:}$
 $\text{sheep-value} \leq \text{max-sheep-send} * \text{price-d}$
defines $\text{wheat-receive} \equiv \text{wheat-value} \text{ div } \text{price-n}$
and $\text{sheep-send} \equiv \text{wheat-receive} * \text{price-n} \text{ div } \text{price-d}$
shows $0 \leq \text{wheat-receive}$
and $\text{wheat-receive} \leq \min \text{max-wheat-receive max-wheat-send}$
and $0 \leq \text{sheep-send}$
and $\text{sheep-send} \leq \min \text{max-sheep-receive max-sheep-send}$

Proof sketch: floor the smaller wheat value twice. Its own two product bounds and its ordering below the sheep value transfer the four maxima.

lemma *exchange-amounts-sheep-stays-sheep-more-bounds:*
fixes $\text{price-n price-d wheat-value sheep-value max-wheat-send}$
 $\text{max-wheat-receive max-sheep-send max-sheep-receive wheat-receive}$
 $\text{sheep-send} :: \text{int}$
assumes $\text{price-n-positive: } 0 < \text{price-n}$
and $\text{price-d-positive: } 0 < \text{price-d}$
and $\text{wheat-nonnegative: } 0 \leq \text{wheat-value}$
and $\text{wheat-wheat-send-bound:}$
 $\text{wheat-value} \leq \text{max-wheat-send} * \text{price-n}$
and $\text{wheat-sheep-receive-bound:}$
 $\text{wheat-value} \leq \text{max-sheep-receive} * \text{price-d}$
and $\text{sheep-stays: wheat-value} \leq \text{sheep-value}$

```

and sheep-wheat-receive-bound:
  sheep-value  $\leq$  max-wheat-receive * price-n
and sheep-sheep-send-bound:
  sheep-value  $\leq$  max-sheep-send * price-d
defines sheep-send  $\equiv$  wheat-value div price-d
and wheat-receive  $\equiv$ 
  (sheep-send * price-d + price-n - 1) div price-n
shows 0  $\leq$  wheat-receive
and wheat-receive  $\leq$  min max-wheat-receive max-wheat-send
and 0  $\leq$  sheep-send
and sheep-send  $\leq$  min max-sheep-receive max-sheep-send

```

Proof sketch: floor the smaller wheat value to sheep, then apply the ceiling bound to the remaining conversion. The value ordering supplies the maxima originating in the other offer.

```

theorem exchange-v10-amounts-int-bounds:
fixes amounts :: int  $\times$  int
assumes pre: exchange-v10-pre price-n price-d max-wheat-send
  max-wheat-receive max-sheep-send max-sheep-receive
and amounts:
  amounts = exchange-v10-amounts-int price-n price-d max-wheat-send
  max-wheat-receive max-sheep-send max-sheep-receive rounding
shows 0  $\leq$  fst amounts
and fst amounts  $\leq$ 
  min (sint max-wheat-receive) (sint max-wheat-send)
and 0  $\leq$  snd amounts
and snd amounts  $\leq$ 
  min (sint max-sheep-receive) (sint max-sheep-send)

```

Proof sketch: split on which exact offer value is smaller, the rounding constructor, and the price ordering. Each of the five C++ calculation branches is discharged by its preceding arithmetic-bound lemma.

```

lemma exchange-v10-amounts-wheat-stays-strict-send:
assumes pre: exchange-v10-pre price-n price-d max-wheat-send
  max-wheat-receive max-sheep-send max-sheep-receive
and sheep-value:
  uint sheep-word =
    exchange-sheep-value-int price-n price-d max-sheep-send
    max-wheat-receive
shows exchange-v10-amounts price-n price-d wheat-word sheep-word
  max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive True
  Exchange-Strict-Send options =
    Cxx-Ok
    (word-of-int
      (exchange-sheep-value-int price-n price-d max-sheep-send

```

$\text{max-wheat-receive} \text{ div } \text{sint price-n}),$
word-of-int
 $(\text{min } (\text{sint max-sheep-send}) (\text{sint max-sheep-receive})))$

Proof sketch: the reversed offer value is bounded by $\text{sint max-wheat-receive} * \text{sint price-n}$, so its round-down division fits signed 64 bits. The second component is exactly the signed minimum.

lemma *exchange-v10-amounts-wheat-stays-up:*

assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive

and *sheep-value:*

uint sheep-word =
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive

and *not-strict-send: rounding $\neq$ Exchange-Strict-Send*

and *up-branch:*

sint price-n > sint price-d $\vee$
rounding = Exchange-Strict-Receive

shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive True

rounding options =

(let wheat-receive =

exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive div sint price-n;

sheep-send =

*(wheat-receive * sint price-n + sint price-d - 1) div sint price-d*

in Cxx-Ok (word-of-int wheat-receive, word-of-int sheep-send))

Proof sketch: the first round-down quotient is bounded by the wheat-receive maximum. Its product remains below the sheep value, so the round-up quotient is bounded by the sheep-send maximum and both checked helpers return the exact integer formulas.

lemma *exchange-v10-amounts-wheat-stays-down:*

assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive

and *sheep-value:*

uint sheep-word =
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive

and *not-strict-send: rounding $\neq$ Exchange-Strict-Send*

and *down-branch:*

$\neg$ (sint price-n > sint price-d $\vee$
rounding = Exchange-Strict-Receive)

and *plain-cap: $\neg$ symmetric-exact-receive-cap options*

shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive True
rounding options =
 (let *sheep-send* =
 exchange-sheep-value-int price-n price-d max-sheep-send
 max-wheat-receive div sint price-d;
 *wheat-receive = sheep-send * sint price-d div sint price-n*
 in *Cxx-Ok (word-of-int wheat-receive, word-of-int sheep-send)*)

Proof sketch: with the symmetric receive cap disabled the branch divides the plain sheep value. The first floor is bounded by the sheep-send maximum. Its exact product stays below the sheep value, so the second floor is bounded by the wheat-receive maximum and both checked divisions succeed exactly.

lemma *exchange-v10-amounts-sheep-stays-wheat-more:*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *wheat-value:*
uint wheat-word =
 exchange-wheat-value-int price-n price-d max-wheat-send
 max-sheep-receive
and *wheat-more: sint price-n > sint price-d*
shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive False
rounding (⌊exact-receive-cap = False, symmetric-exact-receive-cap = False⌋) =
 (let *wheat-receive* =
 exchange-wheat-value-int price-n price-d max-wheat-send
 max-sheep-receive div sint price-n;
 *sheep-send = wheat-receive * sint price-n div sint price-d*
 in *Cxx-Ok (word-of-int wheat-receive, word-of-int sheep-send)*)

Proof sketch: divide the wheat value down by the numerator, then divide its exact product down by the denominator. The two defining product bounds of the wheat value keep both narrowing operations in signed-64 range.

lemma *exchange-v10-amounts-sheep-stays-sheep-more:*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *wheat-value:*
uint wheat-word =
 exchange-wheat-value-int price-n price-d max-wheat-send
 max-sheep-receive
and *sheep-more: ¬ sint price-n > sint price-d*
shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive False
rounding options =

```

(let sheep-send =
  exchange-wheat-value-int price-n price-d max-wheat-send
  max-sheep-receive div sint price-d;
wheat-receive =
  (sheep-send * sint price-d + sint price-n - 1) div sint price-n
  in Cxx-Ok (word-of-int wheat-receive, word-of-int sheep-send))

```

Proof sketch: the first floor is bounded by the sheep-receive maximum. Its product remains below the wheat value, so the final ceiling is bounded by the wheat-send maximum and both checked divisions return exactly.

theorem *exchange-v10-amounts-integer-characterization:*
fixes *rounding* :: *exchange-rounding*
and *wheat-word* *sheep-word* :: *uint128*
and *amounts* :: *int* × *int*
assumes *pre:* *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *wheat-word* ≡
min
 (*word-of-int* (*sint max-wheat-send* * *sint price-n*) :: *uint128*)
 (*word-of-int* (*sint max-sheep-receive* * *sint price-d*))
and *sheep-word* ≡
min
 (*word-of-int* (*sint max-sheep-send* * *sint price-d*) :: *uint128*)
 (*word-of-int* (*sint max-wheat-receive* * *sint price-n*))
and *amounts* ≡
exchange-v10-amounts-int price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding
shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
 (*wheat-word* > *sheep-word*) *rounding* (*exact-receive-cap* = *False*, *symmetric-*
exact-receive-cap = *False*) =
Cxx-Ok
 (*word-of-int* (*fst amounts*), *word-of-int* (*snd amounts*))
and *sint* (*word-of-int* (*fst amounts*) :: *int64*) = *fst amounts*
and *sint* (*word-of-int* (*snd amounts*) :: *int64*) = *snd amounts*

Proof sketch: identify both 128-bit values with their exact integers, then split on the value comparison, rounding mode, and price ordering. The five branch equations above give the exact pair; the global bounds theorem makes both re-encoded result words faithful signed-64 values.

theorem *exchange-v10-without-price-error-thresholds-integer-characterization:*
fixes *rounding* :: *exchange-rounding* **and** *amounts* :: *int* × *int*
assumes *pre:* *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive

```

defines amounts  $\equiv$ 
  exchange-v10-amounts-int price-n price-d max-wheat-send
  max-wheat-receive max-sheep-send max-sheep-receive rounding
shows exchange-v10-without-price-error-thresholds-with-options price-n price-d
  max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
  rounding ( $\text{exact-receive-cap} = \text{False}$ ,  $\text{symmetric-exact-receive-cap} = \text{False}$ ) =
  Cxx-Ok
  (make-exchange-result
    (word-of-int (fst amounts)) (word-of-int (snd amounts))
    (exchange-wheat-value-int price-n price-d max-wheat-send
      max-sheep-receive >
      exchange-sheep-value-int price-n price-d max-sheep-send
      max-wheat-receive))

```

Proof sketch: both offer-value calls succeed with their exact 128-bit minima under *exchange-v10-pre*. Substitute the exact amount-block theorem, then use its signed interpretations and four bounds to prove that neither final runtime check is reachable.

```

lemma int-ceiling-product-div-eq-zero-iff:
  fixes x multiplier divisor :: int
  assumes x-nonnegative:  $0 \leq x$ 
    and multiplier-positive:  $0 < \text{multiplier}$ 
    and divisor-positive:  $0 < \text{divisor}$ 
  shows  $((x * \text{multiplier} + \text{divisor} - 1) \text{ div } \text{divisor} = 0) =$ 
     $(x = 0)$ 

```

Proof sketch: the zero input simplifies directly. A positive integer input times a positive multiplier is at least one, so the augmented numerator is at least the divisor and monotonic division makes the quotient positive.

```

lemma int-floor-product-div-eq-zero-iff:
  fixes x multiplier divisor :: int
  assumes x-nonnegative:  $0 \leq x$ 
    and multiplier-at-least-divisor:  $\text{divisor} \leq \text{multiplier}$ 
    and divisor-positive:  $0 < \text{divisor}$ 
  shows  $(x * \text{multiplier} \text{ div } \text{divisor} = 0) = (x = 0)$ 

```

Proof sketch: a positive integer input is at least one. Multiplying it by a non-negative multiplier no smaller than the divisor makes the numerator at least the divisor, hence the floor quotient is positive.

```

theorem exchange-v10-amounts-zero-iff:
  assumes pre: exchange-v10-pre price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive

```

and *not-strict-send: rounding $\neq$ Exchange-Strict-Send*
shows $(fst (exchange-v10-amounts-int\ price-n\ price-d\ max-wheat-send$
 $max-wheat-receive\ max-sheep-send\ max-sheep-receive\ rounding) = 0) =$
 $(snd (exchange-v10-amounts-int\ price-n\ price-d\ max-wheat-send$
 $max-wheat-receive\ max-sheep-send\ max-sheep-receive\ rounding) = 0)$

Proof sketch: split along the executable branch tree. Every normal or strict-receive branch relates the two amounts by either a positive ceiling conversion or a floor conversion whose multiplier is at least its divisor; the two preceding zero lemmas apply.

theorem *exchange-v10-without-price-error-thresholds-zero-iff:*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
 $max-wheat-receive\ max-sheep-send\ max-sheep-receive$
and *not-strict-send: rounding $\neq$ Exchange-Strict-Send*
and *result:*
 $exchange-v10-without-price-error-thresholds-with-options\ price-n\ price-d$
 $max-wheat-send\ max-wheat-receive\ max-sheep-send\ max-sheep-receive$
 $rounding\ (\text{exact-receive-cap} = \text{False},\ \text{symmetric-exact-receive-cap} = \text{False})$
 $=\ Cxx-Ok\ exchange-result$
shows $(num-wheat-received\ exchange-result = 0) =$
 $(num-sheep-send\ exchange-result = 0)$

Proof sketch: the exact top-level characterization fixes the returned record to the integer amount pair. Its signed-word equalities convert word zero to integer zero, where $\llbracket exchange-v10-pre\ ?price-n\ ?price-d\ ?max-wheat-send\ ?max-wheat-receive\ ?max-sheep-send\ ?max-sheep-receive; ?rounding \neq Exchange-Strict-Send \rrbracket \implies (fst (exchange-v10-amounts-int\ ?price-n\ ?price-d\ ?max-wheat-send\ ?max-wheat-receive\ ?max-sheep-send\ ?max-sheep-receive\ ?rounding) = 0) = (snd (exchange-v10-amounts-int\ ?price-n\ ?price-d\ ?max-wheat-send\ ?max-wheat-receive\ ?max-sheep-send\ ?max-sheep-receive\ ?rounding) = 0)$ applies.

lemma *int-nested-floor-product-positive-iff:*
fixes $x\ multiplier\ divisor :: int$
assumes $x\text{-nonnegative}: 0 \leq x$
and $multiplier\text{-greater}: divisor < multiplier$
and $divisor\text{-positive}: 0 < divisor$
shows $(0 < (x\ div\ multiplier) * multiplier\ div\ divisor) =$
 $(multiplier \leq x)$

Proof sketch: if the first quotient is positive, it is at least one and its product is above the smaller divisor. Conversely the first quotient is zero exactly when the value lies below the positive multiplier.

theorem *exchange-v10-strict-send-sheep-positive-iff*:
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive*
shows $(0 < \text{snd}$
 $(\text{exchange-v10-amounts-int price-n price-d max-wheat-send}$
 $\text{max-wheat-receive max-sheep-send max-sheep-receive}$
 $\text{Exchange-Strict-Send})) =$
 $(\text{let wheat-value} =$
 $\text{exchange-wheat-value-int price-n price-d max-wheat-send}$
 $\text{max-sheep-receive};$
 $\text{sheep-value} =$
 $\text{exchange-sheep-value-int price-n price-d max-sheep-send}$
 max-wheat-receive
in if wheat-value > sheep-value
then $0 < \text{sint max-sheep-send} \wedge 0 < \text{sint max-sheep-receive}$
else if $\text{sint price-n} > \text{sint price-d}$
then $\text{sint price-n} \leq \text{wheat-value}$
else $\text{sint price-d} \leq \text{wheat-value}$)

Proof sketch: in the wheat-stays branch, sheep is the minimum of the two sheep maxima. Otherwise simplify the selected floor formula: when wheat is more valuable the nested quotient is positive exactly at one numerator unit, and in the other price order ordinary positive division has the same threshold.

theorem *exchange-v10-without-strict-send-sheep-positive-iff*:
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive*
and result:
 $\text{exchange-v10-without-price-error-thresholds-with-options price-n price-d}$
 $\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive}$
 $\text{Exchange-Strict-Send} (\text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap}$
 $= \text{False}) = \text{Cxx-Ok exchange-result}$
shows $(0 < \text{sint} (\text{num-sheep-send exchange-result})) =$
 $(\text{let wheat-value} =$
 $\text{exchange-wheat-value-int price-n price-d max-wheat-send}$
 $\text{max-sheep-receive};$
 $\text{sheep-value} =$
 $\text{exchange-sheep-value-int price-n price-d max-sheep-send}$
 max-wheat-receive
in if wheat-value > sheep-value
then $0 < \text{sint max-sheep-send} \wedge 0 < \text{sint max-sheep-receive}$
else if $\text{sint price-n} > \text{sint price-d}$
then $\text{sint price-n} \leq \text{wheat-value}$
else $\text{sint price-d} \leq \text{wheat-value}$)

Proof sketch: rewrite the successful result to the exact integer amount record

and use the signed interpretation of its sheep word. The preceding integer equivalence is then exactly the desired minimal input condition.

theorem *exchange-v10-without-price-error-thresholds-result-contract:*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *result:*
exchange-v10-without-price-error-thresholds-with-options price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding (exact-receive-cap = False, symmetric-exact-receive-cap = False)
= *Cxx-Ok exchange-result*
shows $0 \leq \text{sint } (\text{num-wheat-received } \text{exchange-result})$
and $\text{sint } (\text{num-wheat-received } \text{exchange-result}) \leq$
 $\text{min } (\text{sint } \text{max-wheat-receive}) (\text{sint } \text{max-wheat-send})$
and $0 \leq \text{sint } (\text{num-sheep-send } \text{exchange-result})$
and $\text{sint } (\text{num-sheep-send } \text{exchange-result}) \leq$
 $\text{min } (\text{sint } \text{max-sheep-receive}) (\text{sint } \text{max-sheep-send})$
and *result-wheat-stays exchange-result =*
(exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive >
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive)

Proof sketch: uniqueness of the successful constructor identifies the returned record with the exact integer characterization. Its signed-word equalities and the four amount bounds yield the range clauses, while the record constructor exposes the exact offer-value comparison.

2.8 Full exchange

This last section composes the results above into the contract of *exchange-v10-with-options* and then establishes the crossing property announced at the end of the model.

theorem *exchange-v10-characterization:*
exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive
max-sheep-send max-sheep-receive rounding options =
(case exchange-v10-without-price-error-thresholds-with-options price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding options of
Cxx-Err error $\Rightarrow$ Cxx-Err error
| *Cxx-Ok before-thresholds $\Rightarrow$*
apply-price-error-thresholds price-n price-d
(num-wheat-received before-thresholds)
(num-sheep-send before-thresholds)
(result-wheat-stays before-thresholds) rounding)

Proof sketch: unfold the single monadic bind. Its two constructor cases

are exactly propagation of the first error or the second C++ call with the successful record fields.

theorem *exchange-v10-integer-characterization:*

fixes *rounding* :: *exchange-rounding* **and** *amounts* :: *int* × *int*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *amounts* ≡
exchange-v10-amounts-int price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding
shows *exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive*
max-sheep-send max-sheep-receive rounding (exact-receive-cap = False, symmetric-exact-receive-cap = False) =
apply-price-error-thresholds price-n price-d
(word-of-int (fst amounts)) (word-of-int (snd amounts))
(exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive >
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive)
rounding

Proof sketch: *exchange-v10-without-price-error-thresholds-integer-characterization* replaces the first call by its exact amount record under *exchange-v10-pre*. Simplifying the record selectors leaves the threshold call shown here.

theorem *apply-price-error-thresholds-result-choices:*

assumes *result:*
apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays rounding = Cxx-Ok exchange-result
shows *exchange-result =*
make-exchange-result wheat-receive sheep-send wheat-stays ∨
exchange-result = make-exchange-result 0 0 wheat-stays

Proof sketch: every successful branch of the total threshold characterization returns either the unchanged record or its explicit zero-amount counterpart. All remaining branches are errors.

theorem *exchange-v10-result-contract:*

assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *result: exchange-v10-with-options price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive rounding (exact-receive-cap = False, symmetric-exact-receive-cap = False) =
Cxx-Ok exchange-result

shows $0 \leq \text{sint } (\text{num-wheat-received } \text{exchange-result})$
and $\text{sint } (\text{num-wheat-received } \text{exchange-result}) \leq$
 $\text{min } (\text{sint } \text{max-wheat-receive}) (\text{sint } \text{max-wheat-send})$
and $0 \leq \text{sint } (\text{num-sheep-send } \text{exchange-result})$
and $\text{sint } (\text{num-sheep-send } \text{exchange-result}) \leq$
 $\text{min } (\text{sint } \text{max-sheep-receive}) (\text{sint } \text{max-sheep-send})$
and $\text{result-wheat-stays } \text{exchange-result} =$
 $(\text{exchange-wheat-value-int } \text{price-n } \text{price-d } \text{max-wheat-send}$
 $\text{max-sheep-receive } >$
 $\text{exchange-sheep-value-int } \text{price-n } \text{price-d } \text{max-sheep-send}$
 $\text{max-wheat-receive})$

Proof sketch: the integer characterization fixes the input to the threshold caller. A successful threshold call returns either those bounded pre-threshold amounts or two zeros and always preserves the stay flag. The previously proved signed interpretations and four bounds therefore establish every clause.

lemma *int-le-ceiling-div-mult*:
fixes *numerator divisor :: int*
assumes *divisor-positive*: $0 < \text{divisor}$
shows *numerator* $\leq$
 $((\text{numerator} + \text{divisor} - 1) \text{div } \text{divisor}) * \text{divisor}$

Proof sketch: decompose the adjusted numerator into its quotient multiple and remainder. The remainder is strictly below the positive divisor, so cancellation of the common divisor increment gives the ceiling inequality.

lemma *exchange-strict-send-favored-int*:
fixes *wheat-value sheep-value price-n price-d max-sheep-send*
 $\text{max-sheep-receive} :: \text{int}$
assumes *pn*: $0 < \text{price-n}$ **and** *pd*: $0 < \text{price-d}$
and *stays*: $\text{sheep-value} < \text{wheat-value}$
and *wheat-receive-bound*:
 $\text{wheat-value} \leq \text{max-sheep-receive} * \text{price-d}$
and *sheep-send-bound*:
 $\text{sheep-value} \leq \text{max-sheep-send} * \text{price-d}$
shows $(\text{sheep-value div } \text{price-n}) * \text{price-n} \leq$
 $\text{min } \text{max-sheep-send } \text{max-sheep-receive} * \text{price-d}$

Proof sketch: the rounded-down received amount has value at most the sheep offer value. That value is bounded by the send maximum directly and, because wheat stays, strictly by the receive maximum; split which maximum is the minimum.

lemma *exchange-v10-amounts-int-favored*:
fixes *rounding* :: *exchange-rounding* **and** *amounts* :: *int* × *int*
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *amounts* ≡
exchange-v10-amounts-int price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding
shows *if exchange-wheat-value-int price-n price-d max-wheat-send*
max-sheep-receive >
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive
*then fst amounts * sint price-n ≤ snd amounts * sint price-d*
*else snd amounts * sint price-d ≤ fst amounts * sint price-n*

Proof sketch: follow the five amount branches. Every floor conversion favors the output side by $0 < ?divisor \implies ?numerator \text{ div } ?divisor * ?divisor \leq ?numerator$, every ceiling conversion favors it by $0 < ?divisor \implies ?numerator \leq (?numerator + ?divisor - 1) \text{ div } ?divisor * ?divisor$, and the special strict-send branch is $\llbracket 0 < ?price-n; 0 < ?price-d; ?sheep-value < ?wheat-value; ?wheat-value \leq ?max-sheep-receive * ?price-d; ?sheep-value \leq ?max-sheep-send * ?price-d \rrbracket \implies ?sheep-value \text{ div } ?price-n * ?price-n \leq \min ?max-sheep-send ?max-sheep-receive * ?price-d$.

lemma *exchange-v10-before-favored*:
fixes *rounding* :: *exchange-rounding*
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *amounts* ≡
exchange-v10-amounts-int price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding
defines *wheat-stays* ≡
exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive >
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive
shows *favored-seller-ok price-n price-d*
(word-of-int (fst amounts)) (word-of-int (snd amounts)) wheat-stays

Proof sketch: the amount words denote the exact mathematical amounts under *exchange-v10-pre*. Substitute those signed interpretations into *favored-seller-ok* and apply the preceding branch theorem.

theorem *exchange-v10-wellformed-characterization*:
fixes *rounding* :: *exchange-rounding* **and** *amounts* :: *int* × *int*
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*

```

    max-wheat-receive max-sheep-send max-sheep-receive
defines amounts  $\equiv$ 
    exchange-v10-amounts-int price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive rounding
defines wheat-stays  $\equiv$ 
    exchange-wheat-value-int price-n price-d max-wheat-send
    max-sheep-receive >
    exchange-sheep-value-int price-n price-d max-sheep-send
    max-wheat-receive
shows exchange-v10-with-options price-n price-d max-wheat-send max-wheat-
receive
    max-sheep-send max-sheep-receive rounding ( $\text{exact-receive-cap} = \text{False}$ , sym-
metric-exact-receive-cap =  $\text{False}$ ) =
    (let wheat-receive = (word-of-int (fst amounts) :: int64) in
    let sheep-send = (word-of-int (snd amounts) :: int64) in
    let original =
        make-exchange-result wheat-receive sheep-send wheat-stays in
    let zero = make-exchange-result 0 0 wheat-stays in
    if  $\neg (0 < \text{fst amounts} \wedge 0 < \text{snd amounts})$  then
        if rounding = Exchange-Strict-Send then
            if sheep-send = 0 then Cxx-Err Cxx-Runtime-Error
            else Cxx-Ok original
        else Cxx-Ok zero
    else if rounding = Exchange-Normal then
        Cxx-Ok
        (if price-error-bound-spec price-n price-d wheat-receive
        sheep-send False
        then original else zero)
    else if price-error-bound-spec price-n price-d wheat-receive
        sheep-send True
    then Cxx-Ok original
    else Cxx-Err Cxx-Runtime-Error)

```

Proof sketch: substitute the exact pre-threshold amount record. Its signed interpretations replace the positivity guard, positive prices eliminate assertion outcomes, and *exchange-v10-pre* *?price-n* *?price-d* *?max-wheat-send* *?max-wheat-receive* *?max-sheep-send* *?max-sheep-receive* $\implies$ *favorer-seller-ok* *?price-n* *?price-d* (word-of-int (fst (exchange-v10-amounts-int *?price-n* *?price-d* *?max-wheat-send* *?max-wheat-receive* *?max-sheep-send* *?max-sheep-receive* *?rounding*))) (word-of-int (snd (exchange-v10-amounts-int *?price-n* *?price-d* *?max-wheat-send* *?max-wheat-receive* *?max-sheep-send* *?max-sheep-receive* *?rounding*))) (exchange-sheep-value-int *?price-n* *?price-d* *?max-sheep-send* *?max-wheat-receive* < exchange-wheat-value-int *?price-n* *?price-d* *?max-wheat-send* *?max-sheep-receive*) eliminates both favored-seller runtime failures. The remaining branches are exactly the normal and path price-bound behavior plus the strict-send zero test.

corollary *exchange-v10-normal-characterization:*

fixes *amounts* :: *int* × *int*
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *amounts* ≡
exchange-v10-amounts-int price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive Exchange-Normal
shows *exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive*
max-sheep-send max-sheep-receive Exchange-Normal
 $(\text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False}) =$
 $(\text{let wheat-stays} =$
exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive >
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive in
let wheat-receive = (word-of-int (fst amounts) :: int64) in
let sheep-send = (word-of-int (snd amounts) :: int64) in
Cxx-Ok
 $(\text{if } 0 < \text{fst amounts} \wedge 0 < \text{snd amounts} \wedge$
price-error-bound-spec price-n price-d wheat-receive
sheep-send False
 $\text{then make-exchange-result wheat-receive sheep-send wheat-stays}$
 $\text{else make-exchange-result } 0 \ 0 \ \text{wheat-stays}))$

Proof sketch: specialize the well-formed characterization to normal mode. Every non-positive trade becomes the zero record; a positive trade retains its amounts exactly when the symmetric price-error predicate succeeds.

corollary *exchange-v10-path-characterization:*

fixes *rounding* :: *exchange-rounding* **and** *amounts* :: *int* × *int*
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *path-mode*: *rounding* ≠ *Exchange-Normal*
defines *amounts* ≡
exchange-v10-amounts-int price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding
shows *exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive*
max-sheep-send max-sheep-receive rounding $(\text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False}) =$
 $(\text{let wheat-stays} =$
exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive >
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive in
let wheat-receive = (word-of-int (fst amounts) :: int64) in
let sheep-send = (word-of-int (snd amounts) :: int64) in

```

let original =
  make-exchange-result wheat-receive sheep-send wheat-stays in
if  $\neg (0 < \text{fst amounts} \wedge 0 < \text{snd amounts})$  then
  if rounding = Exchange-Strict-Send then
    if sheep-send = 0 then Cxx-Err Cxx-Runtime-Error
    else Cxx-Ok original
  else Cxx-Ok (make-exchange-result 0 0 wheat-stays)
else if price-error-bound-spec price-n price-d wheat-receive
  sheep-send True
then Cxx-Ok original
else Cxx-Err Cxx-Runtime-Error)

```

Proof sketch: specialize the well-formed characterization to a non-normal constructor. Positive trades are preserved exactly when the one-sided price bound holds; the remaining non-positive branch distinguishes the raw strict-send sheep-zero check from strict-receive zeroing.

lemma *word-zero-iff-sint-zero*:

$((\text{word} :: \text{int64}) = 0) = (\text{sint word} = 0)$

Proof sketch: the forward direction is immediate. In the reverse direction $\text{uint } ?w = \text{take-bit LENGTH} (?a) (\text{sint } ?w)$ turns signed zero into unsigned zero, which uniquely identifies the zero word by $(\text{uint } ?x = 0) = (?x = 0)$.

theorem *exchange-v10-zero-iff*:

assumes pre: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
and *not-strict-send: rounding $\neq$ Exchange-Strict-Send*
and *result: exchange-v10-with-options price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive rounding ($\text{exact-receive-cap} = \text{False}$, symmetric-exact-receive-cap = False) =
Cxx-Ok exchange-result
shows $(\text{num-wheat-received exchange-result} = 0) =$
 $(\text{num-sheep-send exchange-result} = 0)$

Proof sketch: a successful threshold call returns either the exact pre-threshold record or two zeros. For the first choice, signed-word faithfulness and $\llbracket \text{exchange-v10-pre } ?\text{price-n } ?\text{price-d } ?\text{max-wheat-send } ?\text{max-wheat-receive } ?\text{max-sheep-send } ?\text{max-sheep-receive}; ?\text{rounding} \neq \text{Exchange-Strict-Send} \rrbracket \implies (\text{fst } (\text{exchange-v10-amounts-int } ?\text{price-n } ?\text{price-d } ?\text{max-wheat-send } ?\text{max-wheat-receive } ?\text{max-sheep-send } ?\text{max-sheep-receive } ?\text{rounding}) = 0) = (\text{snd } (\text{exchange-v10-amounts-int } ?\text{price-n } ?\text{price-d } ?\text{max-wheat-send } ?\text{max-wheat-receive } ?\text{max-sheep-send } ?\text{max-sheep-receive } ?\text{rounding}) = 0)$ establish simultaneous zero outside strict-send mode; the second choice is immediate.

theorem *apply-price-error-thresholds-strict-send-success:*

assumes *result:*

*apply-price-error-thresholds price-n price-d wheat-receive sheep-send
wheat-stays Exchange-Strict-Send = Cxx-Ok exchange-result*

shows *exchange-result =*

make-exchange-result wheat-receive sheep-send wheat-stays

and *sheep-send $\neq$ 0*

Proof sketch: a positive strict-send trade can only return the unchanged record, and positivity makes the sheep word nonzero. The non-positive strict-send branch also returns that record exactly when its explicit raw sheep-zero check is false.

theorem *exchange-v10-strict-send-sheep-positive:*

assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*

max-wheat-receive max-sheep-send max-sheep-receive

and result: *exchange-v10-with-options price-n price-d max-wheat-send*

max-wheat-receive max-sheep-send max-sheep-receive

Exchange-Strict-Send ($\neg$ exact-receive-cap = False, symmetric-exact-receive-cap = False) = Cxx-Ok exchange-result

shows *0 < sint (num-sheep-send exchange-result)*

Proof sketch: strict-send success preserves the pre-threshold record and proves its sheep word nonzero. That word denotes a non-negative exact integer amount, so nonzero strengthens to strictly positive.

theorem *exchange-v10-positive-trade-contract:*

assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*

max-wheat-receive max-sheep-send max-sheep-receive

and result: *exchange-v10-with-options price-n price-d max-wheat-send*

max-wheat-receive max-sheep-send max-sheep-receive rounding ($\neg$ exact-receive-cap = False, symmetric-exact-receive-cap = False) =

Cxx-Ok exchange-result

and positive:

0 < sint (num-wheat-received exchange-result) $\wedge$

0 < sint (num-sheep-send exchange-result)

shows *avored-seller-ok price-n price-d*

(num-wheat-received exchange-result)

(num-sheep-send exchange-result)

(result-wheat-stays exchange-result)

and if *rounding = Exchange-Normal*

then price-error-bound-spec price-n price-d

(num-wheat-received exchange-result)

(num-sheep-send exchange-result) False

```

else price-error-bound-spec price-n price-d
  (num-wheat-received exchange-result)
  (num-sheep-send exchange-result) True

```

Proof sketch: a positive returned trade cannot be the explicit zero record, so the threshold choice theorem identifies it with the exact pre-threshold record. Apply the threshold caller's favored-seller and price-bound contracts and substitute those unchanged fields.

```

export-code big-multiply big-divide-or-throw big-divide-or-throw128
  check-price-error-bound calculate-offer-value
  calculate-offer-value-with-exact-receive-cap apply-price-error-thresholds
  exchange-v10-without-price-error-thresholds-with-options exchange-v10-with-
  options
  Cxx-Round-Down Cxx-Round-Up checking SML

end
theory Offer-Exchange-Adjustment
  imports Offer-Exchange-Arithmetic
begin

```

3 Offer adjustment and liabilities

This theory isolates the maker-independent arithmetic surrounding *exchange-v10-with-options*. It models offer adjustment and liability projection, then defines the offer and raw-request filters used before ledger state is consulted. Party state, ordinary pre-flight, posting, liability mutation, and crossing belong to *Offer-Exchange-Lifecycle*.

3.1 Offer adjustment and offer liabilities

```

definition adjust-offer-with-options ::
  int32  $\Rightarrow$  int32  $\Rightarrow$  int64  $\Rightarrow$  int64  $\Rightarrow$ 
  exchange-options  $\Rightarrow$  int64 cxx-result
— C++: adjustOffer(Price, ...) (OfferExchange.cpp)
where
  adjust-offer-with-options price-n price-d max-wheat-send max-sheep-receive
  options = do {
    result  $\leftarrow$  exchange-v10-with-options price-n price-d max-wheat-send int64-max
    int64-max max-sheep-receive Exchange-Normal options;
    Cxx-Ok (num-wheat-received result)
  }

```

adjust-offer-with-options is the price-level overload of *adjustOffer* (*OfferExchange.cpp*): the wheat side of a hypothetical cross against a counterparty

without limits. The final argument is an *exchange-options* record. Its two exact-cap fields model the protocol gates threaded to ledger-level call sites; the lifecycle theory supplies the state-dependent caps at those sites.

definition *adjust-offer* ::

uint32 $\Rightarrow$ *int32* $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$
int64 cxx-result

— C++: *adjustOffer* with a ledger version (*OfferExchange.cpp*)

where

adjust-offer ledger-version price-n price-d max-wheat-send
max-sheep-receive =
adjust-offer-with-options price-n price-d max-wheat-send
max-sheep-receive (exchange-options-at-version ledger-version)

adjust-offer is the protocol-facing overload of *adjustOffer* as protocol 29 leaves it: a leading ledger version, the price, the send cap, and the receive cap. Threading the version here is what carries the repaired arithmetic into posting adjustment, preventative adjustment before a cross, and post-trade remainder adjustment, since all three go through this one helper.

lemma *adjust-offer-legacy*:

$\neg$ *protocol-version-starts-from ledger-version protocol-version-v29* $\Rightarrow$
adjust-offer ledger-version price-n price-d max-wheat-send
max-sheep-receive =
adjust-offer-with-options price-n price-d max-wheat-send max-sheep-receive
legacy-exchange-options

lemma *adjust-offer-repaired*:

protocol-version-starts-from ledger-version protocol-version-v29 $\Rightarrow$
adjust-offer ledger-version price-n price-d max-wheat-send
max-sheep-receive =
adjust-offer-with-options price-n price-d max-wheat-send max-sheep-receive
repaired-exchange-options

The C++ liability helpers carry no *exactReceiveCap* flag, and the fix did not touch *TransactionUtils.cpp*: liabilities are computed against a counterparty with a receive cap of *INT64-MAX*, where the clamp inside *calculateOfferAmountFromValue* restores the plain cap. The lemma records that agreement, so the flag-free liability definitions below match both protocol behaviors at once. Proof sketch: after the shared send-side multiplication, both sides multiply *int64-max* by the price denominator; that product is below 2^{126} and the relaxed cap adds less than 2^{64} , so the 128-bit addition cannot wrap, the relaxed receive value is at least the plain one, and the extra minimum collapses.

lemma *exact-receive-cap-collapses-at-unlimited-receive*:

calculate-offer-value-with-exact-receive-cap price-n price-d max-send
int64-max =
calculate-offer-value price-n price-d max-send int64-max

corollary *calculate-offer-amount-from-value-collapses-at-unlimited-receive:*
calculate-offer-amount-from-value price-n price-d max-send int64-max =
(calculate-offer-value price-n price-d max-send int64-max $\gg$
(λ offer-value.
big-divide-or-throw128 offer-value (scast price-n) Cxx-Round-Down))
 — At an unlimited receive cap the protocol-29 helper computes the same amount
 as the protocol-28 calculation it replaces.

Proof sketch: the helper is the relaxed value followed by the round-down
 division, and at *int64-max* the retained clamp collapses the relaxed value to
 the plain one, so the two divisions have equal numerators.

definition *offer-selling-liabilities ::*
int32 $\Rightarrow$ int32 $\Rightarrow$ int64 $\Rightarrow$ int64 cxx-result
 — C++: *getOfferSellingLiabilities (TransactionUtils.cpp)*
where
offer-selling-liabilities price-n price-d amount = do {
result $\leftarrow$ exchange-v10-without-price-error-thresholds-with-options price-n
price-d
amount int64-max int64-max int64-max Exchange-Normal
(λ exact-receive-cap = False,
symmetric-exact-receive-cap = False);
Cxx-Ok (num-wheat-received result)
}

definition *offer-buying-liabilities ::*
int32 $\Rightarrow$ int32 $\Rightarrow$ int64 $\Rightarrow$ int64 cxx-result
 — C++: *getOfferBuyingLiabilities (TransactionUtils.cpp)*
where
offer-buying-liabilities price-n price-d amount = do {
result $\leftarrow$ exchange-v10-without-price-error-thresholds-with-options price-n
price-d
amount int64-max int64-max int64-max Exchange-Normal
(λ exact-receive-cap = False,
symmetric-exact-receive-cap = False);
Cxx-Ok (num-sheep-send result)
}

calculate-offer-amount-from-value-collapses-at-unlimited-receive is the rea-
 son these two definitions need no protocol parameter. Both *offer-selling-*
liabilities and *offer-buying-liabilities* value a stored offer against a counter-
 party whose receive cap is *int64-max*, and there the two protocols agree.
 Protocol 29 therefore recomputes exactly the liabilities protocol 28 booked,
 which is what makes activation safe without rewriting any stored offer. Re-
 moving the *INT64-MAX * priceD* clamp would invalidate this, which is why
 protocol 29 retains it.

3.2 Protocol compatibility of the unlimited-cap projections

lemma *pre-thresholds-options-irrelevant-at-unlimited-caps:*

exchange-v10-without-price-error-thresholds-with-options price-n price-d
max-wheat-send int64-max int64-max int64-max rounding options =
exchange-v10-without-price-error-thresholds-with-options price-n price-d
max-wheat-send int64-max int64-max int64-max rounding
legacy-exchange-options

— With every counterparty cap at *int64-max* the exchange does not depend on the exact-cap options at all.

Proof sketch: the options are consulted in exactly two branches, and each consults them on a call whose receive cap is *int64-max*. After naming the two offer values, the retained clamp collapses each relaxed value to the plain one, so both branches compute what the legacy configuration computes and the two runs agree everywhere.

definition *offer-selling-liabilities-at-version ::*

uint32 ⇒ int32 ⇒ int32 ⇒ int64 ⇒
int64 cxx-result

— C++: *getOfferSellingLiabilities (TransactionUtils.cpp, invariant/LiabilitiesMatchOffers.cpp)*

where

offer-selling-liabilities-at-version ledger-version price-n price-d
amount = do {
result ← exchange-v10-without-price-error-thresholds ledger-version
price-n price-d amount int64-max int64-max int64-max Exchange-Normal;
Cxx-Ok (num-wheat-received result)
}

definition *offer-buying-liabilities-at-version ::*

uint32 ⇒ int32 ⇒ int32 ⇒ int64 ⇒
int64 cxx-result

— C++: *getOfferBuyingLiabilities (TransactionUtils.cpp, invariant/LiabilitiesMatchOffers.cpp)*

where

offer-buying-liabilities-at-version ledger-version price-n price-d
amount = do {
result ← exchange-v10-without-price-error-thresholds ledger-version
price-n price-d amount int64-max int64-max int64-max Exchange-Normal;
Cxx-Ok (num-sheep-send result)
}

Protocol 29 threads the ledger version through both liability helpers in *TransactionUtils.cpp* and through the copies the *LiabilitiesMatchOffers* invariant keeps. The two definitions above have those signatures. The theorems below show the version makes no difference, which is why the flag-free projections used throughout the lifecycle theory remain correct for both protocols.

theorem *offer-selling-liabilities-version-independent:*

offer-selling-liabilities-at-version ledger-version price-n price-d amount =
offer-selling-liabilities price-n price-d amount
— Protocol 29 recomputes exactly the selling liability protocol 28 booked.

Proof sketch: both project the same field out of a pre-threshold exchange whose counterparty caps are all *int64-max*, and there the exchange does not depend on the options the version selects.

theorem *offer-buying-liabilities-version-independent*:
offer-buying-liabilities-at-version ledger-version price-n price-d amount =
offer-buying-liabilities price-n price-d amount
— Protocol 29 recomputes exactly the buying liability protocol 28 booked.

Proof sketch: as for the selling liability, with the other projection.

theorem *persistent-liabilities-agree-across-activation*:
offer-selling-liabilities-at-version v28 price-n price-d amount =
offer-selling-liabilities-at-version v29 price-n price-d amount
offer-buying-liabilities-at-version v28 price-n price-d amount =
offer-buying-liabilities-at-version v29 price-n price-d amount
— A stored offer has the same booked liabilities before and after the protocol-29 upgrade, at any two ledger versions.

Proof sketch: each side is version-independent, so both reduce to the same flag-free projection.

persistent-liabilities-agree-across-activation is the hinge of migration safety. Liabilities acquired under protocol 28 are exactly the liabilities protocol 29 recomputes when it releases them, so activating protocol 29 needs no liability rewrite and no stored-offer migration pass. It depends on the retained *INT64-MAX * priceD* clamp: without that clamp the relaxed receive value would exceed the plain one even at an unlimited cap and the two protocols would disagree here.

definition *manage-buy-selling-liabilities* ::
int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64 cxx-result*
— C++: *ManageBuyOfferOpFrame::getOfferSellingLiabilities* (*ManageBuyOfferOpFrame.cpp*)

where

manage-buy-selling-liabilities price-n price-d buy-amount = do {
result $\leftarrow$ *exchange-v10-without-price-error-thresholds-with-options*
price-d price-n int64-max int64-max int64-max buy-amount
Exchange-Normal
(|*exact-receive-cap* = *False*,
symmetric-exact-receive-cap = *False*);
Cxx-Ok (*num-wheat-received result*)
}

definition *manage-buy-buying-liabilities* ::
 $\text{int32} \Rightarrow \text{int32} \Rightarrow \text{int64} \Rightarrow \text{int64} \text{ cxx-result}$
— C++: *ManageBuyOfferOpFrame::getOfferBuyingLiabilities* (*ManageBuyOfferOpFrame.cpp*)
where
manage-buy-buying-liabilities *price-n price-d buy-amount* = do {
 result $\leftarrow$ *exchange-v10-without-price-error-thresholds-with-options*
 price-d price-n int64-max int64-max int64-max buy-amount
 Exchange-Normal
 (*exact-receive-cap* = *False*,
 symmetric-exact-receive-cap = *False*);
 Cxx-Ok (*num-sheep-send result*)
}

definition *manage-buy-selling-liabilities-at-version* ::
 $\text{uint32} \Rightarrow \text{int32} \Rightarrow \text{int32} \Rightarrow \text{int64} \Rightarrow$
 int64 cxx-result
— C++: *ManageBuyOfferOpFrame::getOfferSellingLiabilities* (*ManageBuyOfferOpFrame.cpp*)
where
manage-buy-selling-liabilities-at-version *ledger-version price-n price-d*
buy-amount = do {
 result $\leftarrow$ *exchange-v10-without-price-error-thresholds ledger-version*
 price-d price-n int64-max int64-max int64-max buy-amount
 Exchange-Normal;
 Cxx-Ok (*num-wheat-received result*)
}

definition *manage-buy-buying-liabilities-at-version* ::
 $\text{uint32} \Rightarrow \text{int32} \Rightarrow \text{int32} \Rightarrow \text{int64} \Rightarrow$
 int64 cxx-result
— C++: *ManageBuyOfferOpFrame::getOfferBuyingLiabilities* (*ManageBuyOfferOpFrame.cpp*)
where
manage-buy-buying-liabilities-at-version *ledger-version price-n price-d*
buy-amount = do {
 result $\leftarrow$ *exchange-v10-without-price-error-thresholds ledger-version*
 price-d price-n int64-max int64-max int64-max buy-amount
 Exchange-Normal;
 Cxx-Ok (*num-sheep-send result*)
}

Protocol 29 threads the ledger version through both manage-buy request helpers, exactly as it does for the stored-offer projections. Here, however, the version genuinely matters. A stored offer is valued against a counterparty whose every cap is *int64-max*, where the retained clamp makes the two protocols agree. A manage-buy request instead places the finite submitted buy amount on the receive cap, which is precisely the cap the re-

paired non-staying branch consults, so *pre-thresholds-options-irrelevant-at-unlimited-caps* does not apply and the computed liabilities can move at the boundary.

lemma *manage-buy-liabilities-at-version-legacy:*

- ¬ *protocol-version-starts-from ledger-version protocol-version-v29* $\implies$
manage-buy-selling-liabilities-at-version ledger-version price-n price-d
buy-amount = manage-buy-selling-liabilities price-n price-d buy-amount
- ¬ *protocol-version-starts-from ledger-version protocol-version-v29* $\implies$
manage-buy-buying-liabilities-at-version ledger-version price-n price-d
buy-amount = manage-buy-buying-liabilities price-n price-d buy-amount
- Before protocol 29 the versioned helpers are the existing flag-free ones, so every protocol-28 result about them still applies.

lemma *manage-buy-liabilities-at-version-repaired:*

- protocol-version-starts-from ledger-version protocol-version-v29* $\implies$
manage-buy-selling-liabilities-at-version ledger-version price-n price-d
buy-amount =
(exchange-v10-without-price-error-thresholds-with-options price-d price-n
int64-max int64-max int64-max buy-amount Exchange-Normal
repaired-exchange-options $\gg$
(λresult. Cxx-Ok (num-wheat-received result)))
- protocol-version-starts-from ledger-version protocol-version-v29* $\implies$
manage-buy-buying-liabilities-at-version ledger-version price-n price-d
buy-amount =
(exchange-v10-without-price-error-thresholds-with-options price-d price-n
int64-max int64-max int64-max buy-amount Exchange-Normal
repaired-exchange-options $\gg$
(λresult. Cxx-Ok (num-sheep-send result)))
- From protocol 29 the versioned helpers use the repaired kernel.

lemma *manage-buy-request-liabilities-change-at-activation:*

- manage-buy-selling-liabilities-at-version 28 100 101 4 = Cxx-Ok 3*
- manage-buy-selling-liabilities-at-version 29 100 101 4 = Cxx-Ok 4*
- manage-buy-buying-liabilities-at-version 28 100 101 4 = Cxx-Ok 3*
- manage-buy-buying-liabilities-at-version 29 100 101 4 = Cxx-Ok 4*
- A manage-buy request for four units at price 100/101 books three units of each liability before protocol 29 and four from protocol 29 on.

manage-buy-request-liabilities-change-at-activation is the counterpart of *persistant-liabilities-agree-across-activation*, and it goes the other way. Stored-offer liabilities are the same at every ledger version, which is what makes activation safe without rewriting any offer. Manage-buy request-time liabilities are not: the same submitted request books different amounts before and after the boundary. This is an ordinary protocol-behavior change at request time, not a migration hazard, because nothing persists a request's liabilities – only the resulting stored offer is persisted, and that is valued by the version-independent projections. The equality theorem for stored offers

must therefore not be reused for this path.

definition *manage-buy-normalized-amount* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64*

— the canonical resting selling amount of a manage-buy request; no standalone C++ entry point

where

manage-buy-normalized-amount price-n price-d buy-amount =

(let *offer-n* = *sint price-d*;

offer-d = *sint price-n*;

wheat-value =

min (*int64-max-int* * *offer-n*)

(*sint buy-amount* * *offer-d*);

selling =

(if *offer-n* > *offer-d*

then *wheat-value* div *offer-n*

else

(let *sheep* = *wheat-value* div *offer-d*

in (*sheep* * *offer-d* + *offer-n* - 1) div *offer-n*))

in word-of-int *selling*)

offer-selling-liabilities and *offer-buying-liabilities* are *getOfferSellingLiabilities* and *getOfferBuyingLiabilities* (*TransactionUtils.cpp*): the reservation demanded by an offer is defined as the outcome of its own unlimited cross, before the price-error thresholds.

The two manage-buy liability functions model *ManageBuyOfferOpFrame::getOfferSellingLiabilities* and *getOfferBuyingLiabilities* (*ManageBuyOfferOpFrame.cpp*). They invert the submitted price, leave the selling side unlimited, and put the submitted buy amount on the receive cap. *manage-buy-normalized-amount* exposes the canonical resting selling amount computed by that exchange for the swapped price. stellar-core has no standalone entry point for it. It is deliberately a direct integer formula so request normalization remains executable without repeating it in the stateful lifecycle or the differential transport layer.

3.3 Reusable normalization and replay results

lemma *adjustment-above-floor-cap-replays-unlimited*:

assumes *pn*: $0 < \text{sint } \text{price-n}$

and *pd*: $0 < \text{sint } \text{price-d}$

and *amount-positive*: $0 < \text{sint } \text{amount}$

and *unsaturated*:

$\text{sint } \text{amount} * \text{sint } \text{price-n} \leq$

$\text{int64-max-int} * \text{sint } \text{price-d}$

and *cap-nonnegative*: $0 \leq \text{sint } \text{max-sheep-receive}$

and *above-floor*:

$(\text{sint } \text{amount} * \text{sint } \text{price-n}) \text{ div } \text{sint } \text{price-d} <$

$\text{sint } \text{max-sheep-receive}$

shows *adjust-offer-with-options price-n price-d amount max-sheep-receive*
 (*exact-receive-cap = False, symmetric-exact-receive-cap = False*) =
adjust-offer-with-options price-n price-d amount int64-max
 (*exact-receive-cap = False, symmetric-exact-receive-cap = False*)

Proof sketch: Euclidean division places the product strictly below the next multiple of the positive denominator. An integral cap strictly above its floor is at least that next integer, so neither this cap nor the unlimited cap clips the wheat value. The normal-exchange characterizations then have identical integer amounts, stays flag, and threshold result.

lemma *manage-sell-request-liabilities-explicit:*

fixes *wheat-value selling buying :: int*
assumes *pn: 0 < sint price-n*
and *pd: 0 < sint price-d*
and *amount-positive: 0 < sint amount*
defines *wheat-value* $\equiv$
*min (sint amount * sint price-n)*
*(int64-max-int * sint price-d)*
and *selling* $\equiv$
(if sint price-n > sint price-d
then wheat-value div sint price-n
else
(let sheep = wheat-value div sint price-d
*in (sheep * sint price-d + sint price-n - 1) div sint price-n))*
and *buying* $\equiv$
(if sint price-n > sint price-d
*then (selling * sint price-n) div sint price-d*
else wheat-value div sint price-d)
shows
offer-selling-liabilities price-n price-d amount =
Cxx-Ok (word-of-int selling)
offer-buying-liabilities price-n price-d amount =
Cxx-Ok (word-of-int buying)
sint (word-of-int selling :: int64) = selling
sint (word-of-int buying :: int64) = buying
0 ≤ selling
selling ≤ sint amount
0 ≤ buying

Proof sketch: characterize the request's unlimited, pre-threshold liability exchange in unbounded integers. Its wheat value is the stated minimum, the counteroffer is never the side that stays, and the normal rounding formulas are exactly the stated selling and buying amounts. The two liability helpers project those fields from the same exchange result.

```

lemma manage-buy-request-liabilities-explicit:
  fixes wheat-value selling buying :: int
  assumes pn: 0 < sint price-n
    and pd: 0 < sint price-d
    and amount-positive: 0 < sint buy-amount
  defines wheat-value  $\equiv$ 
    min (int64-max-int * sint price-d)
    (sint buy-amount * sint price-n)
  and selling  $\equiv$ 
    (if sint price-d > sint price-n
     then wheat-value div sint price-d
     else
       (let sheep = wheat-value div sint price-n
        in (sheep * sint price-n + sint price-d - 1) div sint price-d))
  and buying  $\equiv$ 
    (if sint price-d > sint price-n
     then (selling * sint price-d) div sint price-n
     else wheat-value div sint price-n)
shows
  manage-buy-selling-liabilities price-n price-d buy-amount =
    Cxx-Ok (word-of-int selling)
  manage-buy-buying-liabilities price-n price-d buy-amount =
    Cxx-Ok (word-of-int buying)
  manage-buy-normalized-amount price-n price-d buy-amount =
    word-of-int selling
  sint (word-of-int selling :: int64) = selling
  sint (word-of-int buying :: int64) = buying
  0  $\leq$  selling
  0  $\leq$  buying
  buying  $\leq$  sint buy-amount

```

Proof sketch: characterize the unlimited-send exchange at the inverted raw price. The submitted buy amount is its receive cap, so its wheat value is the stated minimum. The opposing unlimited offer cannot be the smaller side, and NORMAL rounding therefore yields exactly the stated canonical selling and buying liabilities. The generated-word bounds follow from the general integer characterization.

```

lemma adjustment-at-exact-receive-value-replays-unlimited:
  assumes pn: 0 < sint price-n
    and pd: 0 < sint price-d
    and amount-nonnegative: 0  $\leq$  sint amount
    and cap-nonnegative: 0  $\leq$  sint max-sheep-receive
    and unsaturated:
      sint amount * sint price-n  $\leq$ 
      int64-max-int * sint price-d
    and exact-value:
      sint amount * sint price-n =

```

```

    sint max-sheep-receive * sint price-d
shows adjust-offer-with-options price-n price-d amount max-sheep-receive
    (exact-receive-cap = False,
     symmetric-exact-receive-cap = False) =
    adjust-offer-with-options price-n price-d amount int64-max
    (exact-receive-cap = False,
     symmetric-exact-receive-cap = False)

```

Proof sketch: at the finite cap, both arguments of the wheat-value minimum are the same exact product. Unsaturation makes the unlimited cap select that product as well. The normal exchange consequently has identical integer amounts and stay flag at the two receive caps.

```

lemma adjustment-int-le-div-from-product:
  fixes lower numerator divisor :: int
  assumes divisor-positive: 0 < divisor
    and product-bound: lower * divisor ≤ numerator
  shows lower ≤ numerator div divisor

```

Proof sketch: the lower value is its own product's quotient, and integer division by a positive divisor is monotone in the numerator.

```

lemma capped-normal-result-candidates:
  fixes wheat-value selling buying :: int
    and price-n price-d :: int32
    and request-send request-receive max-send max-receive posted :: int64
  defines wheat-value ≡
    min (sint request-send * sint price-n)
      (sint request-receive * sint price-d)
    and selling ≡ wheat-value div sint price-n
    and buying ≡
      (selling * sint price-n) div sint price-d
  assumes pn: 0 < sint price-n
    and pd: 0 < sint price-d
    and request-send-nonnegative: 0 ≤ sint request-send
    and request-receive-nonnegative: 0 ≤ sint request-receive
    and wheat-more: sint price-n > sint price-d
    and send-nonnegative: 0 ≤ sint max-send
    and receive-nonnegative: 0 ≤ sint max-receive
    and send-le-request: sint max-send ≤ sint request-send
    and receive-le-request: sint max-receive ≤ sint request-receive
    and selling-fits: selling ≤ sint max-send
    and buying-fits: buying ≤ sint max-receive
  and adjustment:
    adjust-offer-with-options price-n price-d max-send max-receive
    (exact-receive-cap = False,

```

$\text{symmetric-exact-receive-cap} = \text{False}) = \text{Cxx-Ok posted}$
and $\text{posted-positive}: 0 < \text{sint posted}$
shows
 $\text{sint posted} = \text{selling} \vee$
 $(\text{sint posted} = \text{selling} - 1 \wedge$
 $(\text{selling} * \text{sint price-n}) \bmod \text{sint price-d} \neq 0)$

Proof sketch: capacities covering the normalized selling and buying amounts sandwich the capped wheat value between the buying multiple and the raw request caps. Dividing by the more valuable wheat price confines a positive successful NORMAL result to the normalized selling amount or its immediate predecessor. Exact divisibility rules out the predecessor.

lemma *capped-normal-manage-sell-result-candidates:*
fixes $\text{wheat-value selling buying} :: \text{int}$
and $\text{price-n price-d} :: \text{int32}$
and $\text{amount max-send max-receive posted} :: \text{int64}$
defines $\text{wheat-value} \equiv$
 $\min (\text{sint amount} * \text{sint price-n})$
 $(\text{int64-max-int} * \text{sint price-d})$
and $\text{selling} \equiv \text{wheat-value} \text{ div } \text{sint price-n}$
and $\text{buying} \equiv$
 $(\text{selling} * \text{sint price-n}) \text{ div } \text{sint price-d}$
assumes $\text{pn}: 0 < \text{sint price-n}$
and $\text{pd}: 0 < \text{sint price-d}$
and $\text{amount-positive}: 0 < \text{sint amount}$
and $\text{wheat-more}: \text{sint price-n} > \text{sint price-d}$
and $\text{send-nonnegative}: 0 \leq \text{sint max-send}$
and $\text{receive-nonnegative}: 0 \leq \text{sint max-receive}$
and $\text{send-le-amount}: \text{sint max-send} \leq \text{sint amount}$
and $\text{selling-fits}: \text{selling} \leq \text{sint max-send}$
and $\text{buying-fits}: \text{buying} \leq \text{sint max-receive}$
and $\text{adjustment}:$
 $\text{adjust-offer-with-options price-n price-d max-send max-receive}$
 $(\text{exact-receive-cap} = \text{False},$
 $\text{symmetric-exact-receive-cap} = \text{False}) = \text{Cxx-Ok posted}$
and $\text{posted-positive}: 0 < \text{sint posted}$
shows
 $\text{sint posted} = \text{selling} \vee$
 $(\text{sint posted} = \text{selling} - 1 \wedge$
 $(\text{selling} * \text{sint price-n}) \bmod \text{sint price-d} \neq 0)$

Proof sketch: specialize the general two-request-cap confinement theorem to a ManageSell request, whose receive cap is int64-max .

lemma *capped-normal-manage-buy-result-candidates:*

```

fixes wheat-value selling buying :: int
  and price-n price-d :: int32
  and buy-amount max-send max-receive posted :: int64
defines wheat-value  $\equiv$ 
  min (int64-max-int * sint price-d)
    (sint buy-amount * sint price-n)
  and selling  $\equiv$  wheat-value div sint price-d
  and buying  $\equiv$ 
    (selling * sint price-d) div sint price-n
assumes pn:  $0 < \text{sint price-n}$ 
  and pd:  $0 < \text{sint price-d}$ 
  and amount-positive:  $0 < \text{sint buy-amount}$ 
  and wheat-more:  $\text{sint price-d} > \text{sint price-n}$ 
  and send-nonnegative:  $0 \leq \text{sint max-send}$ 
  and receive-nonnegative:  $0 \leq \text{sint max-receive}$ 
  and receive-le-amount:  $\text{sint max-receive} \leq \text{sint buy-amount}$ 
  and selling-fits:  $\text{selling} \leq \text{sint max-send}$ 
  and buying-fits:  $\text{buying} \leq \text{sint max-receive}$ 
  and adjustment:
    adjust-offer-with-options price-d price-n max-send max-receive
      (exact-receive-cap = False,
        symmetric-exact-receive-cap = False) = Cxx-Ok posted
  and posted-positive:  $0 < \text{sint posted}$ 
shows
  sint posted = selling  $\vee$ 
    (sint posted = selling - 1  $\wedge$ 
      (selling * sint price-d) mod sint price-n  $\neq 0$ )

```

Proof sketch: specialize the general confinement theorem to ManageBuy's unlimited send cap and submitted receive cap, after swapping the raw price components into canonical resting orientation.

```

lemma capped-normal-result-replays-unlimited:
  fixes wheat-value selling buying :: int
    and price-n price-d :: int32
    and request-send request-receive max-send max-receive posted :: int64
  defines wheat-value  $\equiv$ 
    min (sint request-send * sint price-n)
      (sint request-receive * sint price-d)
  and selling  $\equiv$  wheat-value div sint price-n
  and buying  $\equiv$ 
    (selling * sint price-n) div sint price-d
assumes pn:  $0 < \text{sint price-n}$ 
  and pd:  $0 < \text{sint price-d}$ 
  and request-send-nonnegative:  $0 \leq \text{sint request-send}$ 
  and request-receive-nonnegative:  $0 \leq \text{sint request-receive}$ 
  and wheat-more:  $\text{sint price-n} > \text{sint price-d}$ 
  and send-nonnegative:  $0 \leq \text{sint max-send}$ 

```

```

and receive-nonnegative:  $0 \leq \text{sint max-receive}$ 
and send-le-request:  $\text{sint max-send} \leq \text{sint request-send}$ 
and receive-le-request:  $\text{sint max-receive} \leq \text{sint request-receive}$ 
and selling-fits:  $\text{selling} \leq \text{sint max-send}$ 
and buying-fits:  $\text{buying} \leq \text{sint max-receive}$ 
and adjustment:
  adjust-offer-with-options price-n price-d max-send max-receive
  ( $\text{exact-receive-cap} = \text{False}$ ,
    $\text{symmetric-exact-receive-cap} = \text{False}$ ) = Cxx-Ok posted
and posted-positive:  $0 < \text{sint posted}$ 
and fixed:
  adjust-offer-with-options price-n price-d posted max-receive
  ( $\text{exact-receive-cap} = \text{False}$ ,
    $\text{symmetric-exact-receive-cap} = \text{False}$ ) = Cxx-Ok posted
and posted-unsaturated:
   $\text{sint posted} * \text{sint price-n} \leq$ 
   $\text{int64-max-int} * \text{sint price-d}$ 
shows
  adjust-offer-with-options price-n price-d posted int64-max
  ( $\text{exact-receive-cap} = \text{False}$ ,
    $\text{symmetric-exact-receive-cap} = \text{False}$ ) = Cxx-Ok posted

```

Proof sketch: use the confinement theorem to split on the normalized amount and its fractional predecessor. A predecessor's buying floor lies strictly below the covered receive cap, so raising that cap to unlimited replays the result. The full candidate either has the same strict-floor argument or the receive cap is exactly the normalized liability; in the latter case the successful capped exchange forces an exact price product and the exact-value replay theorem applies.

```

lemma capped-normal-manage-sell-result-replays-unlimited:
fixes wheat-value selling buying :: int
and price-n price-d :: int32
and amount max-send max-receive posted :: int64
defines wheat-value  $\equiv$ 
   $\min (\text{sint amount} * \text{sint price-n})$ 
   $(\text{int64-max-int} * \text{sint price-d})$ 
and selling  $\equiv \text{wheat-value} \text{ div } \text{sint price-n}$ 
and buying  $\equiv$ 
   $(\text{selling} * \text{sint price-n}) \text{ div } \text{sint price-d}$ 
assumes pn:  $0 < \text{sint price-n}$ 
and pd:  $0 < \text{sint price-d}$ 
and amount-positive:  $0 < \text{sint amount}$ 
and wheat-more:  $\text{sint price-n} > \text{sint price-d}$ 
and send-nonnegative:  $0 \leq \text{sint max-send}$ 
and receive-nonnegative:  $0 \leq \text{sint max-receive}$ 
and send-le-amount:  $\text{sint max-send} \leq \text{sint amount}$ 

```

and *selling-fits*: $\text{selling} \leq \text{sint max-send}$
and *buying-fits*: $\text{buying} \leq \text{sint max-receive}$
and *adjustment*:
 adjust-offer-with-options price-n price-d max-send max-receive
 ($\text{exact-receive-cap} = \text{False}$,
 $\text{symmetric-exact-receive-cap} = \text{False}$) = *Cxx-Ok posted*
and *posted-positive*: $0 < \text{sint posted}$
and *fixed*:
 adjust-offer-with-options price-n price-d posted max-receive
 ($\text{exact-receive-cap} = \text{False}$,
 $\text{symmetric-exact-receive-cap} = \text{False}$) = *Cxx-Ok posted*
and *posted-unsaturated*:
 $\text{sint posted} * \text{sint price-n} \leq$
 $\text{int64-max-int} * \text{sint price-d}$
shows
 adjust-offer-with-options price-n price-d posted int64-max
 ($\text{exact-receive-cap} = \text{False}$,
 $\text{symmetric-exact-receive-cap} = \text{False}$) = *Cxx-Ok posted*

Proof sketch: specialize the two-request-cap replay theorem to an unlimited ManageSell receive cap.

lemma *capped-normal-manage-buy-result-replays-unlimited*:
fixes *wheat-value selling buying :: int*
 and *price-n price-d :: int32*
 and *buy-amount max-send max-receive posted :: int64*
defines *wheat-value* $\equiv$
 $\min (\text{int64-max-int} * \text{sint price-d})$
 $(\text{sint buy-amount} * \text{sint price-n})$
and *selling* $\equiv \text{wheat-value} \text{ div } \text{sint price-d}$
and *buying* $\equiv$
 $(\text{selling} * \text{sint price-d}) \text{ div } \text{sint price-n}$
assumes *pn*: $0 < \text{sint price-n}$
 and *pd*: $0 < \text{sint price-d}$
and *amount-positive*: $0 < \text{sint buy-amount}$
and *wheat-more*: $\text{sint price-d} > \text{sint price-n}$
and *send-nonnegative*: $0 \leq \text{sint max-send}$
and *receive-nonnegative*: $0 \leq \text{sint max-receive}$
and *receive-le-amount*: $\text{sint max-receive} \leq \text{sint buy-amount}$
and *selling-fits*: $\text{selling} \leq \text{sint max-send}$
and *buying-fits*: $\text{buying} \leq \text{sint max-receive}$
and *adjustment*:
 adjust-offer-with-options price-d price-n max-send max-receive
 ($\text{exact-receive-cap} = \text{False}$,
 $\text{symmetric-exact-receive-cap} = \text{False}$) = *Cxx-Ok posted*
and *posted-positive*: $0 < \text{sint posted}$
and *fixed*:
 adjust-offer-with-options price-d price-n posted max-receive

```

    (exact-receive-cap = False,
     symmetric-exact-receive-cap = False) = Cxx-Ok posted
  and posted-unsaturated:
    sint posted * sint price-d ≤
    int64-max-int * sint price-n
  shows
    adjust-offer-with-options price-d price-n posted int64-max
    (exact-receive-cap = False,
     symmetric-exact-receive-cap = False) = Cxx-Ok posted

```

Proof sketch: specialize the two-request-cap replay theorem to ManageBuy's unlimited send cap and submitted receive cap, in canonical price order.

```

end
theory Offer-Exchange-Posting-Refinement
  imports Offer-Exchange-Specification Offer-Exchange-Adjustment HOL.Real
begin

hide-const (open)
  Offer-Exchange-Specification.price-n
  Offer-Exchange-Specification.price-d

```

4 Minimal implementation posting slice

This theory repeats the smallest implementation-facing slice needed to state and prove the posting refinement without importing the larger lifecycle development. The record, invariants, capacities, liability reservation, pre-flight outcomes, and posting core below are copied definition-for-definition from *Offer-Exchange-Lifecycle*. They model the corresponding protocol-10 path through *ManageOfferOpFrameBase::doApply*, the *canSellAtMost* and *canBuyAtMost* helpers in *OfferExchange.cpp*, and liability acquisition in *TransactionUtils.cpp*. Keeping the definitions unchanged preserves their executable, bit-precise correspondence with the C++ code.

```

record party-state =
  sell-balance :: int64
  sell-liabilities :: int64 — Committed to sell
  buy-limit :: int64
  buy-balance :: int64
  buy-liabilities :: int64 — Committed to buy

```

A party is described by five quantities. On the side of the asset it sells, *sell-balance* is the spendable balance (net of any reserve) and *sell-liabilities* the selling liabilities already reserved for open offers. On the side of the asset it buys, *buy-limit* is the trustline limit, *buy-balance* the current balance, and *buy-liabilities* the buying liabilities already reserved. The maker sells wheat

and buys sheep; the taker sells sheep and buys wheat. The selling side carries no limit because nothing in the modeled code paths ever receives into it, and the buying side carries no selling liabilities because nothing ever spends from it.

definition *party-state-wf* :: *party-state* $\Rightarrow$ *bool*

where

party-state-wf party $\longleftrightarrow$
 $0 \leq \text{sint} (\text{sell-liabilities } \text{party}) \wedge$
 $\text{sint} (\text{sell-liabilities } \text{party}) \leq \text{sint} (\text{sell-balance } \text{party}) \wedge$
 $0 \leq \text{sint} (\text{buy-balance } \text{party}) \wedge$
 $0 \leq \text{sint} (\text{buy-liabilities } \text{party}) \wedge$
 $\text{sint} (\text{buy-balance } \text{party}) + \text{sint} (\text{buy-liabilities } \text{party}) \leq$
 $\text{sint} (\text{buy-limit } \text{party})$

party-state-wf is the ledger invariant that operations preserve: liabilities are non-negative, selling liabilities fit inside the balance, and the balance together with the buying liabilities fits inside the limit.

definition *can-sell-at-most* :: *party-state* $\Rightarrow$ *int64*

— C++: *canSellAtMost* (*OfferExchange.cpp*)

where

can-sell-at-most party =
 (let available =
 $\text{sint} (\text{sell-balance } \text{party}) - \text{sint} (\text{sell-liabilities } \text{party})$
 in if available ≤ 0 then 0 else word-of-int available)

definition *can-buy-at-most* :: *party-state* $\Rightarrow$ *int64*

— C++: *canBuyAtMost* (*OfferExchange.cpp*)

where

can-buy-at-most party =
 (let available =
 $\text{sint} (\text{buy-limit } \text{party}) - \text{sint} (\text{buy-balance } \text{party}) -$
 $\text{sint} (\text{buy-liabilities } \text{party})$
 in if available ≤ 0 then 0 else word-of-int available)

can-sell-at-most and *can-buy-at-most* model the C++ functions of the same names (*OfferExchange.cpp*): the available balance, respectively the available limit, clamped to zero from below.

definition *add-liability-checked* ::

int $\Rightarrow$ *int64* $\Rightarrow$ *int* $\Rightarrow$ *int64 cxx-result*

— C++: *addBuyingLiabilities* and *addSellingLiabilities* (*TransactionUtils.cpp*)

where

add-liability-checked cap current delta =
 (let new-total = $\text{sint } \text{current} + \text{delta}$
 in if new-total $< 0 \vee \text{cap} < \text{new-total}$
 then *Cxx-Err Cxx-Runtime-Error*
 else *Cxx-Ok* (word-of-int new-total))

add-liability-checked models the checked liability updates *addBuyingLiabili-*

ties and *addSellingLiabilities* (*TransactionUtils.cpp*): adding the delta must keep the liability total between zero and the capacity supplied by the caller, and a violation is the runtime error that the C++ call sites raise.

definition *acquire-offer-liabilities* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *party-state* $\Rightarrow$
party-state *cxx-result*

— C++: *acquireLiabilities* (*TransactionUtils.cpp*)

where

acquire-offer-liabilities *price-n* *price-d* *amount* *maker* = do {
buying $\leftarrow$ *offer-buying-liabilities* *price-n* *price-d* *amount*;
new-buying $\leftarrow$ *add-liability-checked*
(*sint* (*buy-limit* *maker*) – *sint* (*buy-balance* *maker*))
(*buy-liabilities* *maker*) (*sint* *buying*);
selling $\leftarrow$ *offer-selling-liabilities* *price-n* *price-d* *amount*;
new-selling $\leftarrow$ *add-liability-checked* (*sint* (*sell-balance* *maker*))
(*sell-liabilities* *maker*) (*sint* *selling*);
Cxx-Ok (*maker*(*buy-liabilities* := *new-buying*,
sell-liabilities := *new-selling*))
}

acquire-offer-liabilities models *acquireLiabilities* (*TransactionUtils.cpp*), the acquiring direction of *acquireOrReleaseLiabilities*. Following the C++, the buying side is updated before the selling side, each against the maker's trustline capacity.

datatype *offer-preflight-outcome* =

Preflight-Line-Full
| *Preflight-Underfunded*
| *Preflight-Ready* *int64* *int64*

datatype *post-outcome* =

Post-Malformed
| *Post-Line-Full*
| *Post-Underfunded*
| *Post-No-Offer*
| *Post-Created* *int64* *party-state*

definition *preflight-offer-core* ::

int64 *cxx-result* $\Rightarrow$ *int64* *cxx-result* $\Rightarrow$ *int64* $\Rightarrow$
int64 $\Rightarrow$ *party-state* $\Rightarrow$ *offer-preflight-outcome* *cxx-result*

— C++: *ManageOfferOpFrameBase::computeOfferExchangeParameters* (*ManageOfferOpFrameBase.cpp*)

where

preflight-offer-core *buying-liability* *selling-liability* *max-send-cap*
max-receive-cap *maker* = do {
buying $\leftarrow$ *buying-liability*;
if *sint* (*buy-limit* *maker*) – *sint* (*buy-balance* *maker*) –
sint (*buy-liabilities* *maker*) < *sint* *buying*
then *Cxx-Ok* *Preflight-Line-Full*

```

else do {
  selling ← selling-liability;
  if sint (sell-balance maker) - sint (sell-liabilities maker) <
    sint selling
  then Cxx-Ok Preflight-Underfunded
  else
    (let max-sheep-send =
      signed-min64 max-send-cap (can-sell-at-most maker);
      max-wheat-receive =
        (if max-receive-cap = int64-max
          then can-buy-at-most maker
          else signed-min64 max-receive-cap (can-buy-at-most maker))
      in if max-wheat-receive = 0
        then Cxx-Ok Preflight-Line-Full
        else Cxx-Ok
        (Preflight-Ready max-sheep-send max-wheat-receive))
}
}

```

preflight-offer-core models the protocol-10 pre-flight tests of *ManageOfferOpFrameBase::computeOfferExchangeParameters* (*ManageOfferOpFrameBase.cpp*) together with the immediate check in *doApply* that reports *LINE-FULL* when *maxWheatReceive* is zero. The request-specific liabilities and operation caps arrive as parameters; the cap minima model the sell and buy instantiations of *applyOperationSpecificLimits*.

definition *post-offer-core* ::

```

int32 ⇒ int32 ⇒ bool ⇒ int64 cxx-result ⇒
int64 cxx-result ⇒ int64 ⇒ int64 ⇒ party-state ⇒
exchange-options ⇒ post-outcome cxx-result

```

— C++: *ManageOfferOpFrameBase::doApply* (*ManageOfferOpFrameBase.cpp*)

where

```

post-offer-core price-n price-d request-valid buying-liability
  selling-liability max-send-cap max-receive-cap maker options =
  (if sint price-n ≤ 0 ∨ sint price-d ≤ 0 ∨ ¬ request-valid
   then Cxx-Ok Post-Malformed
   else do {
     preflight ← preflight-offer-core buying-liability
       selling-liability max-send-cap max-receive-cap maker;
     case preflight of
       Preflight-Line-Full ⇒ Cxx-Ok Post-Line-Full
     | Preflight-Underfunded ⇒ Cxx-Ok Post-Underfunded
     | Preflight-Ready max-sheep-send max-wheat-receive ⇒ do {
       adjusted ← adjust-offer-with-options price-n price-d
         max-sheep-send max-wheat-receive options;
       if 0 < sint adjusted
       then do {
         maker-after ← acquire-offer-liabilities price-n price-d
           adjusted maker;
         Cxx-Ok (Post-Created adjusted maker-after)
       }
     }
   }

```

```

    }
    else Cxx-Ok Post-No-Offer
  }
})

```

post-offer-core models the protocol-10 posting path of *ManageOfferOpFrameBase::doApply* (*ManageOfferOpFrameBase.cpp*) for an offer that crosses nothing on entry: malformed-request rejection, pre-flight, preventative adjustment through *adjust-offer-with-options*, and liability acquisition for the created offer.

definition *post-sell-offer* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *party-state* $\Rightarrow$
exchange-options $\Rightarrow$ *post-outcome* *cxx-result*
 — C++: *doApply* for *ManageSellOfferOpFrame*

where

post-sell-offer *price-n* *price-d* *amount* *maker* *options* =
post-offer-core *price-n* *price-d* (*0* < *sint* *amount*)
 (*offer-buying-liabilities* *price-n* *price-d* *amount*)
 (*offer-selling-liabilities* *price-n* *price-d* *amount*)
amount *int64-max* *maker* *options*

post-sell-offer instantiates the core for a *ManageSellOffer*: the liability arguments are *ManageSellOfferOpFrame::getOfferBuyingLiabilities* and *getOfferSellingLiabilities* (*ManageSellOfferOpFrame.cpp*), and the submitted amount caps the send side exactly as *ManageSellOfferOpFrame::applyOperationSpecificLimits* does.

The copied posting slice deliberately omits balance transfer, liability release, crossing, and lifecycle predicates: none is used by the refinement theorem. The arithmetic lemmas below are included only where the repaired adjustment calculation needs them.

4.1 Posting-state invariant facts

lemma *can-sell-at-most-nonnegative*:

assumes *wf*: *party-state-wf* *party*
shows $0 \leq \text{sint}$ (*can-sell-at-most* *party*)

Proof sketch: the well-formedness invariant makes the available selling balance non-negative and at most the signed-word maximum. The clamping definition therefore returns either zero or a word with that same non-negative signed value.

lemma *can-buy-at-most-nonnegative*:

assumes *wf*: *party-state-wf* *party*
shows $0 \leq \text{sint}$ (*can-buy-at-most* *party*)

Proof sketch: well-formedness makes the unused buying capacity non-negative and bounds it by the trustline limit. The clamping definition therefore preserves its non-negative signed value.

4.2 Repaired adjustment arithmetic

The refinement uses the two repaired receive caps that are absent from the older plain-cap integer characterization. The following word facts and repaired amount characterization are the exact dependency closure copied from *Offer-Exchange-Lifecycle*; later lifecycle maximality, crossing, and reachability results are intentionally not included.

lemma *exact-value-word-characterization*:

assumes *pn*: $0 < \text{sint } \text{price-n}$ **and** *pd*: $0 < \text{sint } \text{price-d}$ **and** *send*: $0 \leq \text{sint } \text{max-send}$ **and** *receive*: $0 \leq \text{sint } \text{max-receive}$

shows *calculate-offer-value-with-exact-receive-cap price-n price-d max-send max-receive* = *Cxx-Ok* $(\min (\text{word-of-int } (\text{sint } \text{max-send} * \text{sint } \text{price-n}) :: \text{uint128}) (\min ((\text{word-of-int } (\text{sint } \text{max-receive} * \text{sint } \text{price-d}) :: \text{uint128}) + \text{ucast } (\text{sint } (\text{price-d} - 1) :: \text{int64})) (\text{word-of-int } (\text{sint } \text{int64-max} * \text{sint } \text{price-d}) :: \text{uint128})))$

Proof sketch: all operands are non-negative signed words, so the multiplications succeed with their integer products. Unfolding the exact cap leaves the nested minimum displayed in the conclusion.

lemma *positive-price-denominator-minus-one-cast*:

fixes *price-d* :: *int32*

assumes $0 < \text{sint } \text{price-d}$

shows $\text{sint } (\text{sint } (\text{price-d} - 1) :: \text{int64}) = \text{sint } \text{price-d} - 1$

Proof sketch: subtracting one from a positive signed 32-bit denominator does not wrap; widening that non-negative result to 64 bits preserves its signed integer value.

lemma *uint-eq-sint-nonnegative-int64*:

fixes *word* :: *int64*

assumes $0 \leq \text{sint } \text{word}$

shows $\text{uint } \text{word} = \text{sint } \text{word}$

Proof sketch: a non-negative signed 64-bit value lies in the unsigned 64-bit range, so the unsigned truncation in $\text{uint } ?w = \text{take-bit } \text{LENGTH}(?'a) (\text{sint } ?w)$ is the identity.

lemma *int-le-div-from-product*:

fixes *lower numerator divisor* :: *int*

assumes *divisor-positive*: $0 < \text{divisor}$

and *product-bound*: $lower * divisor \leq numerator$
shows $lower \leq numerator \text{ div } divisor$

Proof sketch: the lower value is its own product's quotient, and integer division by a positive divisor is monotone in the numerator.

lemma *int-div-below-next-multiple*:
fixes *numerator divisor bound* :: *int*
assumes *divisor-positive*: $0 < divisor$
and *numerator-bound*: $numerator \leq bound * divisor + divisor - 1$
shows $numerator \text{ div } divisor \leq bound$

Proof sketch: division monotonicity compares the numerator against the largest value below the next multiple of the divisor, whose quotient is exactly the bound.

lemma *int-div-le-imp-below-next-multiple*:
fixes *numerator divisor bound* :: *int*
assumes *divisor-positive*: $0 < divisor$
and *quotient-bound*: $numerator \text{ div } divisor \leq bound$
shows $numerator \leq bound * divisor + divisor - 1$

Proof sketch: decompose the numerator into quotient multiple and remainder; the remainder is below the divisor and the quotient multiple is below the bound multiple.

lemma *exact-receive-cap-value-int*:
assumes *pn*: $0 < \text{sint } price\text{-}n$ **and** *pd*: $0 < \text{sint } price\text{-}d$
and *send-nonnegative*: $0 \leq \text{sint } max\text{-}send$
and *receive-nonnegative*: $0 \leq \text{sint } max\text{-}receive$
obtains *trade-word* :: *uint128* **where**
calculate-offer-value-with-exact-receive-cap price-n price-d max-send
 $max\text{-}receive = Cxx\text{-}Ok \text{ trade-word}$
 $\text{uint } trade\text{-}word =$
 $\min (\text{sint } max\text{-}send * \text{sint } price\text{-}n)$
 $(\min (\text{sint } max\text{-}receive * \text{sint } price\text{-}d + \text{sint } price\text{-}d - 1)$
 $(\text{sint } int64\text{-}max * \text{sint } price\text{-}d))$

Proof sketch: the word-level characterization gives a nested minimum of three 128-bit values. The two products are exact by the multiplication range theorem, the relaxation addend is the denominator minus one, and the receive product is far below the 128-bit modulus, so the relaxed receive value is an exact sum and the unsigned minimum commutes with *uint*.

lemma *exchange-v10-amounts-wheat-stays-down-repaired:*
fixes *wheat-word sheep-word* :: *uint128*
and *trade sheep-send wheat-receive* :: *int*
assumes *pn*: $0 < \text{sint price-n}$ **and** *pd*: $0 < \text{sint price-d}$
and *sheep-more*: $\neg \text{sint price-n} > \text{sint price-d}$
and *receive-nonnegative*: $0 \leq \text{sint max-wheat-receive}$
and *send-nonnegative*: $0 \leq \text{sint max-sheep-send}$
and *symmetric*: *symmetric-exact-receive-cap options*
defines *trade* $\equiv$
 $\text{min} (\text{sint max-sheep-send} * \text{sint price-d})$
 $(\text{min} (\text{sint max-wheat-receive} * \text{sint price-n} + \text{sint price-n} - 1)$
 $(\text{sint int64-max} * \text{sint price-n}))$
and *sheep-send* $\equiv \text{trade div sint price-d}$
and *wheat-receive* $\equiv \text{sheep-send} * \text{sint price-d div sint price-n}$
shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive True
Exchange-Normal options =
Cxx-Ok (word-of-int wheat-receive, word-of-int sheep-send)
and *sint (word-of-int wheat-receive :: int64) = wheat-receive*
and *sint (word-of-int sheep-send :: int64) = sheep-send*

Proof sketch: the symmetric exact cap values the trade by the reversed exact receive-cap calculation. Its value is at most the sheep-send product, so the first round-down division fits signed 64 bits; the resulting sheep multiple is at most the relaxed wheat-receive component, so the second round-down division is bounded by the wheat-receive cap.

lemma *exchange-v10-amounts-sheep-stays-wheat-more-repaired:*
fixes *wheat-word sheep-word* :: *uint128*
and *trade wheat-receive sheep-send* :: *int*
assumes *pn*: $0 < \text{sint price-n}$ **and** *pd*: $0 < \text{sint price-d}$
and *wheat-more*: $\text{sint price-n} > \text{sint price-d}$
and *send-nonnegative*: $0 \leq \text{sint max-wheat-send}$
and *receive-nonnegative*: $0 \leq \text{sint max-sheep-receive}$
and *exact*: *exact-receive-cap options*
defines *trade* $\equiv$
 $\text{min} (\text{sint max-wheat-send} * \text{sint price-n})$
 $(\text{min} (\text{sint max-sheep-receive} * \text{sint price-d} + \text{sint price-d} - 1)$
 $(\text{sint int64-max} * \text{sint price-d}))$
and *wheat-receive* $\equiv \text{trade div sint price-n}$
and *sheep-send* $\equiv \text{wheat-receive} * \text{sint price-n div sint price-d}$
shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive False
rounding options =
Cxx-Ok (word-of-int wheat-receive, word-of-int sheep-send)
and *sint (word-of-int wheat-receive :: int64) = wheat-receive*
and *sint (word-of-int sheep-send :: int64) = sheep-send*

Proof sketch: the exact receive cap values the trade by the exact calculation on the wheat offer's caps. Its value is at most the wheat-send product, so the first round-down division fits signed 64 bits; the resulting wheat multiple is at most the relaxed sheep-receive component, so the second round-down division is bounded by the sheep-receive cap.

definition *exchange-v10-amounts-int-repaired* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$
int64 $\Rightarrow$ *int* $\times$ *int*

where

exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive =
 (let wheat-value =
exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive;
sheep-value =
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive;
wheat-stays = *wheat-value* > *sheep-value*
 in if *wheat-stays* then
 if *sint price-n* > *sint price-d* then
 let wheat-receive = *sheep-value div sint price-n*
 in (wheat-receive,
 (*wheat-receive* * *sint price-n* + *sint price-d* - 1)
div sint price-d)
 else
 let trade =
 min (*sint max-sheep-send* * *sint price-d*)
 (min (*sint max-wheat-receive* * *sint price-n* +
sint price-n - 1)
 (*sint int64-max* * *sint price-n*));
sheep-send = *trade div sint price-d*
 in (*sheep-send* * *sint price-d div sint price-n*, *sheep-send*)
 else if *sint price-n* > *sint price-d* then
 let trade =
 min (*sint max-wheat-send* * *sint price-n*)
 (min (*sint max-sheep-receive* * *sint price-d* +
sint price-d - 1)
 (*sint int64-max* * *sint price-d*));
wheat-receive = *trade div sint price-n*
 in (wheat-receive,
wheat-receive * *sint price-n div sint price-d*)
 else
 let *sheep-send* = *wheat-value div sint price-d*
 in ((*sheep-send* * *sint price-d* + *sint price-n* - 1)
div sint price-n,
sheep-send))

exchange-v10-amounts-int-repaired is the normal-mode amount pair computed with both receive-cap repairs enabled, phrased in unbounded integers. The two branches selected when the staying offer is the more valuable side coincide with *exchange-v10-amounts-int*; the two corrected branches replace the plain offer value by the exact receive-cap minimum of the send-side product, the receive cap relaxed by one divisor unit, and the *Offer-Exchange-Arithmetic.int64-max* saturation ceiling.

lemma *repaired-wheat-stays-down-bounds*:

fixes *price-n price-d imax max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive trade sheep-send wheat-receive :: int*
assumes *pn: 0 < price-n and pd: 0 < price-d*
and *sheep-more: price-n ≤ price-d*
and *imax-nonnegative: 0 ≤ imax*
and *receive-nonnegative: 0 ≤ max-wheat-receive*
and *send-nonnegative: 0 ≤ max-sheep-send*
and *wheat-stays:*
 $\min (\max\text{-sheep-send} * \text{price-d}) (\max\text{-wheat-receive} * \text{price-n}) <$
 $\min (\max\text{-wheat-send} * \text{price-n}) (\max\text{-sheep-receive} * \text{price-d})$
defines *trade* $\equiv$
 $\min (\max\text{-sheep-send} * \text{price-d})$
 $(\min (\max\text{-wheat-receive} * \text{price-n} + \text{price-n} - 1) (\text{imax} * \text{price-n}))$
and *sheep-send* $\equiv \text{trade} \text{ div } \text{price-d}$
and *wheat-receive* $\equiv \text{sheep-send} * \text{price-d} \text{ div } \text{price-n}$
shows $0 \leq \text{wheat-receive}$
and *wheat-receive* $\leq \min \max\text{-wheat-receive} \max\text{-wheat-send}$
and $0 \leq \text{sheep-send}$
and *sheep-send* $\leq \min \max\text{-sheep-receive} \max\text{-sheep-send}$

Proof sketch: the exact trade value exceeds the plain sheep value by at most $\text{price-n} - 1$, and the plain sheep value is strictly below the wheat value, which both opposite-side caps bound. Since the price numerator is at most the denominator, each floor quotient stays within the two caps of its output asset.

lemma *repaired-sheep-stays-wheat-more-bounds*:

fixes *price-n price-d imax max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive trade wheat-receive sheep-send :: int*
assumes *pn: 0 < price-n and pd: 0 < price-d*
and *wheat-more: price-d < price-n*
and *imax-nonnegative: 0 ≤ imax*
and *send-nonnegative: 0 ≤ max-wheat-send*
and *receive-nonnegative: 0 ≤ max-sheep-receive*
and *sheep-stays:*
 $\min (\max\text{-wheat-send} * \text{price-n}) (\max\text{-sheep-receive} * \text{price-d}) \leq$
 $\min (\max\text{-sheep-send} * \text{price-d}) (\max\text{-wheat-receive} * \text{price-n})$
defines *trade* $\equiv$
 $\min (\max\text{-wheat-send} * \text{price-n})$

$(\min (\max\text{-sheep-receive} * \text{price-d} + \text{price-d} - 1) (\text{imax} * \text{price-d}))$
and $\text{wheat-receive} \equiv \text{trade} \text{ div } \text{price-n}$
and $\text{sheep-send} \equiv \text{wheat-receive} * \text{price-n} \text{ div } \text{price-d}$
shows $0 \leq \text{wheat-receive}$
and $\text{wheat-receive} \leq \min \max\text{-wheat-receive} \max\text{-wheat-send}$
and $0 \leq \text{sheep-send}$
and $\text{sheep-send} \leq \min \max\text{-sheep-receive} \max\text{-sheep-send}$

Proof sketch: this is the mirror of the wheat-stays argument. The exact trade value exceeds the plain wheat value by at most $\text{price-d} - 1$, the plain wheat value is at most the sheep value, and the price denominator is strictly below the numerator, so each floor quotient stays within the two caps of its output asset.

lemma *exchange-v10-amounts-int-repaired-bounds:*

assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
 $\max\text{-wheat-receive} \max\text{-sheep-send} \max\text{-sheep-receive}$
shows $0 \leq \text{fst} (\text{exchange-v10-amounts-int-repaired price-n price-d}$
 $\max\text{-wheat-send} \max\text{-wheat-receive} \max\text{-sheep-send} \max\text{-sheep-receive})$
and $\text{fst} (\text{exchange-v10-amounts-int-repaired price-n price-d}$
 $\max\text{-wheat-send} \max\text{-wheat-receive} \max\text{-sheep-send} \max\text{-sheep-receive}) \leq$
 $\min (\text{sint} \max\text{-wheat-receive}) (\text{sint} \max\text{-wheat-send})$
and $0 \leq \text{snd} (\text{exchange-v10-amounts-int-repaired price-n price-d}$
 $\max\text{-wheat-send} \max\text{-wheat-receive} \max\text{-sheep-send} \max\text{-sheep-receive})$
and $\text{snd} (\text{exchange-v10-amounts-int-repaired price-n price-d}$
 $\max\text{-wheat-send} \max\text{-wheat-receive} \max\text{-sheep-send} \max\text{-sheep-receive}) \leq$
 $\min (\text{sint} \max\text{-sheep-receive}) (\text{sint} \max\text{-sheep-send})$

Proof sketch: split on the offer-value comparison and the price order. The two branches whose staying offer is the more valuable side reuse the plain-cap branch bound lemmas; the two corrected branches are discharged by the repaired bound lemmas above.

theorem *exchange-v10-amounts-repaired-integer-characterization:*

fixes *wheat-word sheep-word :: uint128 and amounts :: int $\times$ int*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
 $\max\text{-wheat-receive} \max\text{-sheep-send} \max\text{-sheep-receive}$
defines $\text{wheat-word} \equiv$
 $\min$
 $(\text{word-of-int} (\text{sint} \max\text{-wheat-send} * \text{sint} \text{price-n}) :: \text{uint128})$
 $(\text{word-of-int} (\text{sint} \max\text{-sheep-receive} * \text{sint} \text{price-d}))$
and $\text{sheep-word} \equiv$
 $\min$
 $(\text{word-of-int} (\text{sint} \max\text{-sheep-send} * \text{sint} \text{price-d}) :: \text{uint128})$

```

    (word-of-int (sint max-wheat-receive * sint price-n))
and amounts  $\equiv$ 
    exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive
shows exchange-v10-amounts price-n price-d wheat-word sheep-word
    max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
    (wheat-word > sheep-word) Exchange-Normal
    (exact-receive-cap = True, symmetric-exact-receive-cap = True) =
    Cxx-Ok
    (word-of-int (fst amounts), word-of-int (snd amounts))
and sint (word-of-int (fst amounts) :: int64) = fst amounts
and sint (word-of-int (snd amounts) :: int64) = snd amounts

```

Proof sketch: identify both 128-bit values with their exact integers, then split on the value comparison and the price ordering. The unaffected branches use the existing plain-cap branch equations; the two corrected branches use the repaired branch equations. The repaired bounds theorem makes both re-encoded result words faithful signed-64 values.

```

theorem exchange-v10-without-repaired-integer-characterization:
  fixes amounts :: int  $\times$  int
  assumes pre: exchange-v10-pre price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive
  defines amounts  $\equiv$ 
    exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive
  shows exchange-v10-without-price-error-thresholds-with-options price-n price-d
    max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
    Exchange-Normal
    (exact-receive-cap = True, symmetric-exact-receive-cap = True) =
    Cxx-Ok
    (make-exchange-result
      (word-of-int (fst amounts)) (word-of-int (snd amounts))
      (exchange-wheat-value-int price-n price-d max-wheat-send
        max-sheep-receive >
        exchange-sheep-value-int price-n price-d max-sheep-send
        max-wheat-receive))

```

Proof sketch: both offer-value calls succeed with their exact 128-bit minima under *exchange-v10-pre*. Substitute the repaired amount characterization, then use its signed interpretations and the repaired bounds to prove that neither final runtime check is reachable.

```

theorem exchange-v10-repaired-integer-characterization:
  fixes amounts :: int  $\times$  int
  assumes pre: exchange-v10-pre price-n price-d max-wheat-send

```

```

    max-wheat-receive max-sheep-send max-sheep-receive
defines amounts  $\equiv$ 
    exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive
shows exchange-v10-with-options price-n price-d max-wheat-send max-wheat-
receive
    max-sheep-send max-sheep-receive Exchange-Normal
    (exact-receive-cap = True, symmetric-exact-receive-cap = True) =
    apply-price-error-thresholds price-n price-d
    (word-of-int (fst amounts)) (word-of-int (snd amounts))
    (exchange-wheat-value-int price-n price-d max-wheat-send
    max-sheep-receive >
    exchange-sheep-value-int price-n price-d max-sheep-send
    max-wheat-receive)
    Exchange-Normal

```

Proof sketch: the repaired pre-threshold characterization replaces the first call by its exact amount record; simplifying the record selectors leaves the threshold call shown here.

4.3 Posting refinement against the specification

For a positive integer denominator, rational floor and ceiling agree with the integer division formulas used by the executable exchange model. Proof sketch: multiply the defining floor or ceiling interval by the positive denominator. Euclidean division supplies the lower and upper endpoint inequalities.

lemma *floor-of-int-ratio*:

```

fixes numerator denominator :: int
assumes denominator-positive: 0 < denominator
shows
    [(of-int numerator :: rat) / of-int denominator] =
    numerator div denominator

```

lemma *ceiling-of-nonnegative-int-ratio*:

```

fixes numerator denominator :: int
assumes numerator-nonnegative: 0 ≤ numerator
and denominator-positive: 0 < denominator
shows
    [(of-int numerator :: rat) / of-int denominator] =
    (numerator + denominator - 1) div denominator

```

Specializing the preceding identities to a positive ledger price rewrites the specification's two rational round-trip equations into the canonical integer rounding equations of the implementation. Proof sketch: unfold the ledger-price coercion, cancel the nonzero price component, and invoke the corresponding ratio identity.

lemma *floor-times-ledger-price*:
fixes *numerator price-n price-d :: int*
assumes *price-d-positive*: $0 < \text{price-d}$
shows

$$\lfloor (\text{of-int } \text{numerator} :: \text{rat}) * \text{Offer-Exchange-Specification.Ledger-Price } \text{price-n } \text{price-d} \rfloor = \text{numerator} * \text{price-n} \text{ div } \text{price-d}$$

lemma *ceiling-div-ledger-price*:
fixes *numerator price-n price-d :: int*
assumes *numerator-nonnegative*: $0 \leq \text{numerator}$
and *price-n-positive*: $0 < \text{price-n}$
and *price-d-positive*: $0 < \text{price-d}$
shows

$$\lceil (\text{of-int } \text{numerator} :: \text{rat}) / \text{Offer-Exchange-Specification.Ledger-Price } \text{price-n } \text{price-d} \rceil = (\text{numerator} * \text{price-d} + \text{price-n} - 1) \text{ div } \text{price-n}$$

The specification's declarative maximum unrestricted exchange is the same normalized pair computed by an unlimited implementation exchange. Proof sketch: make the feasible wheat set explicit and split on the price ordering. If wheat is more valuable, the implementation takes the largest wheat amount permitted by the request and the signed saturation bound; its rounded sheep amount round-trips and every feasible wheat amount is no larger. If sheep is at least as valuable, first take the largest sheep amount below the requested wheat value and round it back up to wheat. A zero sheep amount makes the feasible set empty; otherwise this pair is feasible and bounds every other round-tripping pair. Finiteness then identifies the specification's *Max* with the implementation amount.

lemma *specification-max-unrestricted-exchange-explicit*:
fixes *offer :: Offer-Exchange-Specification.sell-wheat-offer*
and *price-n price-d amount wheat-value selling buying :: int*
assumes *pn*: $0 < \text{price-n}$
and *pd*: $0 < \text{price-d}$
and *amount-positive*: $0 < \text{amount}$
and *price-n-bound*: $\text{price-n} \leq \text{int64-max-int}$
and *price-d-bound*: $\text{price-d} \leq \text{int64-max-int}$
and *amount-bound*: $\text{amount} \leq \text{int64-max-int}$
and *offer-amount*: $\text{wheat-amount } \text{offer} = \text{amount}$
and *offer-price*:

$$\text{sheep-per-wheat } \text{offer} = \text{Offer-Exchange-Specification.Ledger-Price } \text{price-n } \text{price-d}$$

defines *wheat-value* $\equiv$

$$\min (\text{amount} * \text{price-n}) (\text{int64-max-int} * \text{price-d})$$

and *selling* $\equiv$

$$(\text{if } \text{price-n} > \text{price-d} \text{ then } \text{wheat-value} \text{ div } \text{price-n})$$

```

    else
      (let sheep = wheat-value div price-d
        in (sheep * price-d + price-n - 1) div price-n))
  and buying ≡
    (if price-n > price-d
      then (selling * price-n) div price-d
      else wheat-value div price-d)
shows
  Offer-Exchange-Specification.max-unrestricted-exchange offer =
    (wheat-to-taker = selling,
     sheep-to-maker = buying)

```

Once pre-flight capacities cover the maximum unrestricted exchange, the repaired adjustment replays that exchange at the maker's actual caps. Proof sketch: the two unrestricted counterparty caps make the wheat offer the non-staying side. In the wheat-more branch, the exact receive slack covers the exchange's final division remainder; in the opposite branch, the rounded-up wheat amount covers the primary sheep amount. Thus the repaired integer amount pair is exactly the maximum unrestricted exchange. The normal price-error filter then either preserves it or returns the zero record.

lemma *repaired-adjust-offer-replays-max-unrestricted-exchange:*
fixes price-n price-d :: int32
and amount max-send max-receive :: int64
and wheat-value selling buying :: int
assumes pn: 0 < sint price-n
and pd: 0 < sint price-d
and amount-positive: 0 < sint amount
and wheat-value-defn: wheat-value =
 min (sint amount * sint price-n)
 (int64-max-int * sint price-d)
and selling-defn: selling =
 (if sint price-n > sint price-d
 then wheat-value div sint price-n
 else
 (let sheep = wheat-value div sint price-d
 in (sheep * sint price-d + sint price-n - 1) div sint price-n))
and buying-defn: buying =
 (if sint price-n > sint price-d
 then selling * sint price-n div sint price-d
 else wheat-value div sint price-d)
and selling-nonnegative: 0 ≤ selling
and buying-nonnegative: 0 ≤ buying
and selling-word:
 sint (word-of-int selling :: int64) = selling
and buying-word:
 sint (word-of-int buying :: int64) = buying
and zeroes: selling = 0 ↔ buying = 0

and *max-send-nonnegative*: $0 \leq \text{sint } \text{max-send}$
and *max-receive-nonnegative*: $0 \leq \text{sint } \text{max-receive}$
and *max-send-le-amount*: $\text{sint } \text{max-send} \leq \text{sint } \text{amount}$
and *selling-fits*: $\text{selling} \leq \text{sint } \text{max-send}$
and *buying-fits*: $\text{buying} \leq \text{sint } \text{max-receive}$
shows
adjust-offer-with-options price-n price-d max-send max-receive
 $(\text{exact-receive-cap} = \text{True},$
 $\text{symmetric-exact-receive-cap} = \text{True}) =$
Cxx-Ok
 $(\text{if } 0 < \text{selling} \wedge$
 $\text{price-error-bound-spec price-n price-d}$
 $(\text{word-of-int selling}) (\text{word-of-int buying}) \text{ False}$
 $\text{then word-of-int selling else } 0)$

On a positive non-staying unrestricted exchange, the specification's rational one-percent price-error bound is exactly the implementation's scaled integer bound. Proof sketch: put the two rational prices over the common positive denominator, multiply the inequality by the positive wheat, price-denominator, and percentage factors, and use preservation of absolute value by multiplication with the positive factor.

lemma *specification-price-error-iff-implementation*:
fixes *price-n price-d* :: *int32* **and** *wheat sheep* :: *int*
assumes *pn*: $0 < \text{sint } \text{price-n}$
and *pd*: $0 < \text{sint } \text{price-d}$
and *wheat-positive*: $0 < \text{wheat}$
and *wheat-word*:
 $\text{sint } (\text{word-of-int wheat} :: \text{int64}) = \text{wheat}$
and *sheep-word*:
 $\text{sint } (\text{word-of-int sheep} :: \text{int64}) = \text{sheep}$
shows

Offer-Exchange-Specification.price-error-within-bound
 $(\text{Offer-Exchange-Specification.Ledger-Price}$
 $(\text{sint } \text{price-n}) (\text{sint } \text{price-d}))$
 $(\text{wheat-to-taker} = \text{wheat},$
 $\text{sheep-to-maker} = \text{sheep}) =$
 $\text{price-error-bound-spec price-n price-d}$
 $(\text{word-of-int wheat}) (\text{word-of-int sheep}) \text{ False}$

Recomputing liabilities for the positive normalized wheat amount returns the same pair. Proof sketch: for a wheat-more price, the normalized wheat product is already under the saturation bound and division reproduces its rounded sheep amount. For the other price order, the defining ceiling and floor round-trip equations reproduce both fields. The explicit liability characterization then yields the two executable helper results.

lemma *explicit-sell-wheat-liabilities-recompute*:

```

fixes price-n price-d :: int32 and amount :: int64
and wheat-value selling buying :: int
assumes pn: 0 < sint price-n
and pd: 0 < sint price-d
and amount-positive: 0 < sint amount
and wheat-value-defn: wheat-value =
  min (sint amount * sint price-n)
    (int64-max-int * sint price-d)
and selling-defn: selling =
  (if sint price-n > sint price-d
   then wheat-value div sint price-n
   else
    (let sheep = wheat-value div sint price-d
     in (sheep * sint price-d + sint price-n - 1) div sint price-n))
and buying-defn: buying =
  (if sint price-n > sint price-d
   then selling * sint price-n div sint price-d
   else wheat-value div sint price-d)
and selling-positive: 0 < selling
and selling-word:
  sint (word-of-int selling :: int64) = selling
shows
  offer-selling-liabilities price-n price-d (word-of-int selling) =
    Cxx-Ok (word-of-int selling)
  offer-buying-liabilities price-n price-d (word-of-int selling) =
    Cxx-Ok (word-of-int buying)

```

The specification also recomputes the same maximum unrestricted exchange after replacing the requested amount by its positive normalized wheat amount. Proof sketch: instantiate the executable characterization at that normalized amount. The preceding fixed-point lemma identifies its two returned words with the original pair, while the characterization's signed round-trip clauses identify the underlying integers. The declarative characterization at the updated offer then yields the same record.

```

lemma specification-max-unrestricted-exchange-recompute:
fixes price-n price-d :: int32 and amount :: int64
and offer :: Offer-Exchange-Specification.sell-wheat-offer
and wheat-value selling buying :: int
assumes pn: 0 < sint price-n
and pd: 0 < sint price-d
and amount-positive: 0 < sint amount
and wheat-value-defn: wheat-value =
  min (sint amount * sint price-n)
    (int64-max-int * sint price-d)
and selling-defn: selling =
  (if sint price-n > sint price-d
   then wheat-value div sint price-n

```

```

    else
      (let sheep = wheat-value div sint price-d
        in (sheep * sint price-d + sint price-n - 1) div sint price-n))
and buying-defn: buying =
  (if sint price-n > sint price-d
    then selling * sint price-n div sint price-d
    else wheat-value div sint price-d)
and selling-positive: 0 < selling
and selling-word:
  sint (word-of-int selling :: int64) = selling
and buying-word:
  sint (word-of-int buying :: int64) = buying
and offer-amount: wheat-amount offer = selling
and offer-price:
  sheep-per-wheat offer =
    Offer-Exchange-Specification.Ledger-Price
    (sint price-n) (sint price-d)
shows
  Offer-Exchange-Specification.max-unrestricted-exchange offer =
    (wheat-to-taker = selling,
     sheep-to-maker = buying)

```

lemma *post-sell-offer-refines-post-sell-wheat-offer* [export-audit]:

```

fixes price-n price-d :: int32
and amount :: int64
and passive-flag :: bool
and maker :: party-state
and offer :: Offer-Exchange-Specification.sell-wheat-offer
and abstract-maker :: Offer-Exchange-Specification.maker-state

```

defines

```

offer ≡
  (is-passive = passive-flag,
   wheat-amount = sint amount,
   sheep-per-wheat =
     Offer-Exchange-Specification.Ledger-Price
     (sint price-n) (sint price-d))

```

and

```

abstract-maker ≡
  (wheat-balance = sint (sell-balance maker),
   sheep-balance = sint (buy-balance maker),
   wheat-selling-liabilities = sint (sell-liabilities maker),
   sheep-limit = sint (buy-limit maker),
   sheep-buying-liabilities = sint (buy-liabilities maker))

```

assumes

```

party-state-wf maker
Offer-Exchange-Specification.sell-wheat-offer-well-formed offer

```

shows

```

(case post-sell-offer price-n price-d amount maker
  (exact-receive-cap = True, symmetric-exact-receive-cap = True) of

```

```

    Cxx-Err -  $\Rightarrow$  False
  | Cxx-Ok Post-Malformed  $\Rightarrow$ 
    Offer-Exchange-Specification.post-sell-wheat-offer
    offer abstract-maker =
    Offer-Exchange-Specification.Malformed
  | Cxx-Ok Post-Line-Full  $\Rightarrow$ 
    Offer-Exchange-Specification.post-sell-wheat-offer
    offer abstract-maker =
    Offer-Exchange-Specification.Line-Full
  | Cxx-Ok Post-Underfunded  $\Rightarrow$ 
    Offer-Exchange-Specification.post-sell-wheat-offer
    offer abstract-maker =
    Offer-Exchange-Specification.Underfunded
  | Cxx-Ok Post-No-Offer  $\Rightarrow$ 
    Offer-Exchange-Specification.post-sell-wheat-offer
    offer abstract-maker =
    Offer-Exchange-Specification.No-Offer
  | Cxx-Ok (Post-Created posted concrete-maker)  $\Rightarrow$ 
    Offer-Exchange-Specification.post-sell-wheat-offer
    offer abstract-maker =
    Offer-Exchange-Specification.Created
    (offer(|wheat-amount := sint posted|))
    (|wheat-balance = sint (sell-balance concrete-maker),
    sheep-balance = sint (buy-balance concrete-maker),
    wheat-selling-liabilities =
    sint (sell-liabilities concrete-maker),
    sheep-limit = sint (buy-limit concrete-maker),
    sheep-buying-liabilities =
    sint (buy-liabilities concrete-maker)|))

```

Proof sketch: first identify both models' maximum unrestricted exchange with the same explicit integer formulas, and identify the implementation's clamped capacities with the maker's raw specification headroom. The first two failed capacity tests then give the same line-full and underfunded outcomes. Zero receive capacity forces both exchanged amounts to zero, giving the remaining line-full case. Otherwise the repaired adjustment replays the covered exchange; its integer price test is the specification's rational test. A zero or rejected pair gives no offer. For an accepted positive exchange, recomputation is a fixed point and both models reserve its two amounts as liabilities in the maker state, so their created offers and mapped states coincide bit-precisely.

```

end
theory Offer-Exchange-Divide-Layered
  imports Offer-Exchange-Arithmetic
begin

```

5 Layered model of the wide arithmetic helpers

This theory models the wide arithmetic helpers of *util/numeric.cpp* with one definition per C++ function, computing on 128-bit words exactly as the C++ does: *bigDivideUnsigned*, *bigDivide*, and *bigDivideOrThrow*; their 128-bit counterparts *bigDivideUnsigned128*, *bigDivide128*, and *bigDivideOrThrow128*; and *bigMultiplyUnsigned* and *bigMultiply*. Each layer is characterized over exact integers, and each top-level wrapper is proved equal to the collapsed constant of the same name in *OfferExchange.Offer-Exchange-Arithmetic*, so the integer characterizations already proved there transfer unchanged. The differential harness exercises every layer separately through the transport definitions in *Offer-Exchange-Test-Interface*.

5.1 Definitions

type-synonym *uint64* = 64 word

definition *uint64-max128* :: *uint128*
 — C++: *UINT64-MAX* after conversion to *uint128-t*
 where *uint64-max128* = $2^{64} - 1$

The definition *big-divide-unsigned* below models *bigDivideUnsigned*. The C++ function asserts that the divisor is non-zero, widens the three unsigned 64-bit operands to 128 bits, forms the rounded quotient in 128-bit arithmetic, stores its low 64 bits through the out-parameter, and returns whether the quotient fitted in 64 bits. The out-parameter and the Boolean return are modelled together as a pair.

definition *big-divide-unsigned* ::
 $uint64 \Rightarrow uint64 \Rightarrow uint64 \Rightarrow cxx\text{-rounding} \Rightarrow$
 $(bool \times uint64) \text{ cxx-result}$
 — C++: *bigDivideUnsigned* (*util/numeric.cpp*)
where
big-divide-unsigned *A B C rounding* =
 (if $\neg (0 < uint\ C)$ then *Cxx-Err Cxx-Assertion-Failed*
 else
 let *a* = (*ucast A* :: *uint128*); *b* = (*ucast B* :: *uint128*);
 c = (*ucast C* :: *uint128*);
 x = (if *rounding* = *Cxx-Round-Down* then (*a* * *b*) div *c*
 else (*a* * *b* + *c* - 1) div *c*)
 in *Cxx-Ok* (*x* ≤ *uint64-max128*, *ucast x*)

The definition *big-divide-layered* below models *bigDivide*. The C++ function asserts that the two multiplicands are non-negative and the divisor positive, casts all three to *uint64-t*, calls the unsigned helper, and on success additionally checks the result against *INT64-MAX*. Casting a signed 64-bit word to an unsigned one is the identity on bit patterns, so the arguments

are passed through unchanged; only their interpretation switches from *sint* to *uint*. The comparison with *INT64-MAX* is unsigned in C++ because the constant is promoted, hence *uint*. On the failure path the C++ leaves the out-parameter unassigned; the model returns zero there, and its only caller never reads that value.

definition *big-divide-layered* ::
 $int64 \Rightarrow int64 \Rightarrow int64 \Rightarrow cxx\text{-rounding} \Rightarrow$
 $(bool \times int64) \text{ cxx-result}$
 — C++: *bigDivide* (*util/numeric.cpp*)
where
 $big\text{-}divide\text{-}layered\ A\ B\ C\ rounding =$
 $(if\ \neg\ (0 \leq sint\ A \wedge 0 \leq sint\ B \wedge 0 < sint\ C)$
 $\text{ then } Cxx\text{-Err } Cxx\text{-Assertion-Failed}$
 $\text{ else do } \{$
 $\quad (res, r2) \leftarrow big\text{-}divide\text{-}unsigned\ A\ B\ C\ rounding;$
 $\quad if\ res\ \text{ then } Cxx\text{-Ok } (uint\ r2 \leq int64\text{-max-int}, r2)$
 $\quad \text{ else } Cxx\text{-Ok } (False, 0)$
 $\quad \})$

The definition *big-divide-or-throw-layered* below models *bigDivideOrThrow*, which turns a false return from *bigDivide* into an overflow exception.

definition *big-divide-or-throw-layered* ::
 $int64 \Rightarrow int64 \Rightarrow int64 \Rightarrow cxx\text{-rounding} \Rightarrow$
 $int64 \text{ cxx-result}$
 — C++: *bigDivideOrThrow* (*util/numeric.cpp*)
where
 $big\text{-}divide\text{-}or\text{-}throw\text{-}layered\ A\ B\ C\ rounding = do \{$
 $\quad (ok, res) \leftarrow big\text{-}divide\text{-}layered\ A\ B\ C\ rounding;$
 $\quad if\ ok\ \text{ then } Cxx\text{-Ok } res\ \text{ else } Cxx\text{-Err } Cxx\text{-Overflow}$
 $\quad \}$

5.1.1 Examples

lemma *big-divide-unsigned 10 1 3 Cxx-Round-Down = Cxx-Ok (True, 3)*

lemma *big-divide-unsigned 10 1 3 Cxx-Round-Up = Cxx-Ok (True, 4)*

lemma *big-divide-or-throw-layered 10 1 3 Cxx-Round-Up = Cxx-Ok 4*

lemma *big-divide-or-throw-layered (-1) 1 3 Cxx-Round-Up =*
Cxx-Err Cxx-Assertion-Failed

lemma *big-divide-or-throw-layered int64-max int64-max 1 Cxx-Round-Down =*
Cxx-Err Cxx-Overflow

5.2 The 128-bit family

definition *uint128-max* :: *uint128*

— C++: *uint128-max*()
where *uint128-max* = *word-of-int* ($2^{128} - 1$)

The definition *big-divide-unsigned128* below models *bigDivideUnsigned128*. The C++ function asserts that the divisor is non-zero, widens it to 128 bits, and, when rounding up, returns false early if the incremented numerator would wrap. Otherwise it forms the rounded quotient, stores its low 64 bits through the out-parameter, and returns whether the quotient fitted in 64 bits. On the early exit the out-parameter is left unassigned; the model returns zero there.

definition *big-divide-unsigned128* ::
uint128 $\Rightarrow$ *uint64* $\Rightarrow$ *cxx-rounding* $\Rightarrow$ (*bool* $\times$ *uint64*) *cxx-result*
— C++: *bigDivideUnsigned128* (*util/numeric.cpp*)
where
big-divide-unsigned128 *a B rounding* =
(if *B* = 0 then *Cxx-Err Cxx-Assertion-Failed*
else
let *b* = (*ucast B* :: *uint128*)
in if *rounding* = *Cxx-Round-Up* $\wedge$ *a* > *uint128-max* - (*b* - 1)
then *Cxx-Ok* (*False*, 0)
else
let *x* = (if *rounding* = *Cxx-Round-Down* then *a div b*
else (*a* + *b* - 1) *div b*)
in *Cxx-Ok* (*x* $\leq$ *uint64-max128*, *ucast x*))

The definitions *big-divide128-layered* and *big-divide-or-throw128-layered* below model *bigDivide128* and *bigDivideOrThrow128*. They mirror the signed wrapper and the throwing wrapper of the 64-bit family exactly.

definition *big-divide128-layered* ::
uint128 $\Rightarrow$ *int64* $\Rightarrow$ *cxx-rounding* $\Rightarrow$ (*bool* $\times$ *int64*) *cxx-result*
— C++: *bigDivide128* (*util/numeric.cpp*)
where
big-divide128-layered *a B rounding* =
(if $\neg 0 < \text{sint } B$ then *Cxx-Err Cxx-Assertion-Failed*
else do {
(*res*, *r2*) $\leftarrow$ *big-divide-unsigned128 a B rounding*;
if *res* then *Cxx-Ok* (*uint r2* $\leq$ *int64-max-int*, *r2*)
else *Cxx-Ok* (*False*, 0)
})

definition *big-divide-or-throw128-layered* ::
uint128 $\Rightarrow$ *int64* $\Rightarrow$ *cxx-rounding* $\Rightarrow$ *int64* *cxx-result*
— C++: *bigDivideOrThrow128* (*util/numeric.cpp*)
where
big-divide-or-throw128-layered *a B rounding* = do {
(*ok*, *res*) $\leftarrow$ *big-divide128-layered a B rounding*;
if *ok* then *Cxx-Ok* *res* else *Cxx-Err Cxx-Overflow*
}

5.2.1 Examples

lemma *big-divide-unsigned128 10 3 Cxx-Round-Up = Cxx-Ok (True, 4)*

lemma *big-divide-unsigned128 uint128-max 2 Cxx-Round-Up = Cxx-Ok (False, 0)*

With divisor one the guard cannot fire, exactly as the C++ comment argues: the incremented numerator wraps to the same value it started from, and the overflow is caught by the 64-bit range check instead.

lemma *big-divide-unsigned128 uint128-max 1 Cxx-Round-Up =
Cxx-Ok (False, ucast uint128-max)*

lemma *big-divide-unsigned128 uint128-max 1 Cxx-Round-Down =
Cxx-Ok (False, ucast uint128-max)*

lemma *big-divide-or-throw128-layered 10 3 Cxx-Round-Down = Cxx-Ok 3*

lemma *big-divide-or-throw128-layered 10 0 Cxx-Round-Down =
Cxx-Err Cxx-Assertion-Failed*

lemma *big-divide-or-throw128-layered uint128-max 1 Cxx-Round-Up =
Cxx-Err Cxx-Overflow*

5.3 Wide multiplication

The definitions *big-multiply-unsigned* and *big-multiply-layered* below model *bigMultiplyUnsigned* and *bigMultiply*. The unsigned helper zero-extends both operands to 128 bits and multiplies; the signed wrapper asserts that both operands are non-negative and passes them through, the cast to *uint64-t* being the identity on bit patterns.

definition *big-multiply-unsigned :: uint64 $\Rightarrow$ uint64 $\Rightarrow$ uint128*
 — C++: *bigMultiplyUnsigned (util/numeric.cpp)*
where *big-multiply-unsigned a b = (ucast a :: uint128) * ucast b*

definition *big-multiply-layered :: int64 $\Rightarrow$ int64 $\Rightarrow$ uint128 cxx-result*
 — C++: *bigMultiply (util/numeric.cpp)*
where
big-multiply-layered a b =
(if $\neg (0 \leq \text{ sint } a \wedge 0 \leq \text{ sint } b)$ then Cxx-Err Cxx-Assertion-Failed
else Cxx-Ok (big-multiply-unsigned a b))

5.3.1 Examples

lemma *big-multiply-unsigned (- 1) (- 1) = (2 $\wedge$ 64 - 1) * (2 $\wedge$ 64 - 1)*

lemma *big-multiply-layered 6 7 = Cxx-Ok 42*

lemma *big-multiply-layered* $(- 1) \ 7 = Cxx-Err \ Cxx-Assertion-Failed$

5.4 Reading the word arithmetic as integer arithmetic

The lemmas below let the 128-bit word operations of *big-divide-unsigned* be read as operations on unbounded integers. Zero extension preserves the unsigned value, the product of two unsigned 64-bit values fits in 128 bits, and so does the round-up numerator.

lemma *uint-ucast-uint64-uint128* [simp]:
 $uint \ (ucast \ (w :: uint64) :: uint128) = uint \ w$

Proof sketch: zero extension from 64 to 128 bits is an up-cast.

lemma *uint-uint64-max128* [simp]:
 $uint \ uint64-max128 = 2 \wedge 64 - 1$

Proof sketch: the constant is a numeral below the 128-bit modulus.

lemma *uint64-product-fits-uint128*:
fixes $A \ B :: uint64$
shows $uint \ A * uint \ B < (2 :: int) \wedge 128$

Proof sketch: both factors are non-negative and below two to the sixty-fourth power, so their product is below two to the one-hundred-twenty-eighth power.

lemma *uint64-round-up-numerator-fits-uint128*:
fixes $A \ B \ C :: uint64$
shows $uint \ A * uint \ B + uint \ C < (2 :: int) \wedge 128$

Proof sketch: each unsigned 64-bit value is at most two to the sixty-fourth power minus one. The product of two such bounds plus a third bound is two to the one-hundred-twenty-eighth power minus two to the sixty-fourth power, which is below the modulus.

lemma *uint-product-uint64-uint128*:
fixes $A \ B :: uint64$
shows $uint \ ((ucast \ A :: uint128) * ucast \ B) = uint \ A * uint \ B$

Proof sketch: the product fits, so 128-bit multiplication is exact.

lemma *uint-round-up-numerator-uint64-uint128*:
fixes $A\ B\ C :: \text{uint64}$
assumes *divisor-positive*: $0 < \text{uint } C$
shows $\text{uint } ((\text{ucast } A :: \text{uint128}) * \text{ucast } B + \text{ucast } C - 1) =$
 $\text{uint } A * \text{uint } B + \text{uint } C - 1$

Proof sketch: the sum fits, so the 128-bit addition is exact, and it is at least one because the divisor is, so subtracting one is exact too.

lemma *ucast-uint128-uint64*:
 $(\text{ucast } (x :: \text{uint128}) :: \text{uint64}) = \text{word-of-int } (\text{uint } x)$

Proof sketch: this is the general cast law at one type instance.

lemma *sint-nonnegative-eq-uint*:
fixes $w :: \text{int64}$
assumes $0 \leq \text{sint } w$
shows $\text{sint } w = \text{uint } w$

Proof sketch: a non-negative signed value lies below two to the sixty-third power, so taking its low 64 bits is the identity, and the unsigned value is exactly those low 64 bits.

5.5 Characterizations

Each layer is now characterized over exact integers. The unsigned helper is characterized for all inputs, not only for the non-negative signed ones its caller supplies.

theorem *big-divide-unsigned-characterization*:
 $\text{big-divide-unsigned } A\ B\ C\ \text{rounding} =$
 $(\text{if } C = 0 \text{ then } \text{Cxx-Err } \text{Cxx-Assertion-Failed}$
 $\text{else } \text{Cxx-Ok}$
 $(\text{rounded-quotient } (\text{uint } A * \text{uint } B) (\text{uint } C) \text{ rounding} \leq 2^{64} - 1,$
 $\text{word-of-int } (\text{rounded-quotient } (\text{uint } A * \text{uint } B) (\text{uint } C) \text{ rounding})))$

Proof sketch: a zero divisor fails the assertion. Otherwise split on the rounding mode; in each case the 128-bit numerator is exact by the fitting lemmas, word division is exact, the comparison against the 64-bit maximum is the unsigned comparison of exact values, and the final narrowing cast is *word-of-int* of the exact quotient.

lemma *uint-word-of-int-uint64*:

assumes $0 \leq q$ **and** $q \leq (2 :: \text{int}) \wedge 64 - 1$
shows $\text{uint } (\text{word-of-int } q :: \text{uint64}) = q$

Proof sketch: the value is inside the unsigned 64-bit range.

theorem *big-divide-layered-characterization:*

big-divide-layered A B C rounding =
(if $\neg (0 \leq \text{sint } A \wedge 0 \leq \text{sint } B \wedge 0 < \text{sint } C)$
then Cxx-Err Cxx-Assertion-Failed
else Cxx-Ok
*(rounded-quotient (sint A * sint B) (sint C) rounding $\leq \text{int64-max-int}$,*
*if rounded-quotient (sint A * sint B) (sint C) rounding $\leq 2 \wedge 64 - 1$*
*then word-of-int (rounded-quotient (sint A * sint B) (sint C) rounding)*
else 0))

Proof sketch: under the assertion the three signed values equal their unsigned readings, so the unsigned characterization applies with the same exact quotient. If that quotient fits in 64 bits the wrapper reads it back exactly and compares it against the signed maximum; otherwise the unsigned flag is already false and so is the signed comparison.

5.6 Agreement with the collapsed model

theorem *big-divide-or-throw-layered-eq [export-audit]:*

big-divide-or-throw-layered A B C rounding =
big-divide-or-throw A B C rounding

Proof sketch: rewrite the layered wrapper with the characterization of its inner layer. The assertion conditions coincide, and a quotient passes the signed range check exactly when the collapsed model does not report overflow, in which case both return the same word.

corollary *big-divide-or-throw-layered-success-iff:*

big-divide-or-throw-layered A B C rounding = Cxx-Ok result
 $\longleftrightarrow 0 \leq \text{sint } A \wedge 0 \leq \text{sint } B \wedge 0 < \text{sint } C \wedge$
 $\text{rounded-quotient (sint A * sint B) (sint C) rounding}$
 $\leq \text{int64-max-int} \wedge$
 $\text{result} = \text{word-of-int}$
 $(\text{rounded-quotient (sint A * sint B) (sint C) rounding})$

Proof sketch: transfer the existing theorem along the equality.

5.7 Wide multiplication agrees with the collapsed model

lemma *uint-big-multiply-unsigned:*

$\text{uint } (\text{big-multiply-unsigned } a \ b) = \text{uint } a * \text{uint } b$

Proof sketch: the product of two 64-bit values fits in 128 bits.

lemma *big-multiply-unsigned-word*:

fixes $a\ b :: \text{int64}$

assumes $0 \leq \text{sint } a$ **and** $0 \leq \text{sint } b$

shows *big-multiply-unsigned* $a\ b = \text{word-of-int } (\text{sint } a * \text{sint } b)$

Proof sketch: both words have the unsigned value $\text{sint } a * \text{sint } b$, the left one by the exact product lemma and the reading of non-negative signed values as unsigned ones, so they are the same word.

theorem *big-multiply-layered-eq* [*export-audit*]:

big-multiply-layered $a\ b = \text{big-multiply } a\ b$

Proof sketch: the assertions coincide, and on the success path both sides return the word with unsigned value $\text{sint } a * \text{sint } b$.

5.8 The 128-bit family: word arithmetic as integer arithmetic

lemma *uint-uint128-max* [*simp*]:

uint *uint128-max* $= 2^{128} - 1$

Proof sketch: the constant is the largest 128-bit numeral.

lemma *uint128-guard-value*:

fixes $B :: \text{uint64}$

assumes $B \neq 0$

shows *uint* (*uint128-max* $- ((\text{ucast } B :: \text{uint128}) - 1)$) $= 2^{128} - 1 - (\text{uint } B - 1)$

Proof sketch: the divisor is at least one, so subtracting one from its zero extension is exact, and subtracting that from the all-ones word is exact as well because it is at most the all-ones value.

lemma *uint128-guard-word-iff*:

fixes $a :: \text{uint128}$ **and** $B :: \text{uint64}$

assumes $B \neq 0$

shows ($a > \text{uint128-max} - ((\text{ucast } B :: \text{uint128}) - 1)$) $= (\text{uint } a > 2^{128} - 1 - (\text{uint } B - 1))$

Proof sketch: unsigned word comparison compares the exact values.

lemma *uint128-round-up-numerator*:

fixes $a :: \text{uint128}$ **and** $B :: \text{uint64}$
assumes $B \neq 0$ **and** $\text{uint } a \leq 2^{128} - 1 - (\text{uint } B - 1)$
shows $\text{uint } (a + \text{ucast } B - 1) = \text{uint } a + \text{uint } B - 1$

Proof sketch: the two word operations compute the exact result modulo the 128-bit modulus, whatever the intermediate sum does. The guard says the exact result is at most the largest 128-bit value, and it is non-negative because the divisor is at least one, so the reduction is the identity. This is precisely the argument in the C++ comment.

5.9 The 128-bit family: characterizations

theorem *big-divide-unsigned128-characterization*:

big-divide-unsigned128 a B rounding =
(if B = 0 then Cxx-Err Cxx-Assertion-Failed
else if rounding = Cxx-Round-Up $\wedge$ $\text{uint } a > 2^{128} - 1 - (\text{uint } B - 1)$
then Cxx-Ok (False, 0)
else Cxx-Ok (rounded-quotient (uint a) (uint B) rounding $\leq 2^{64} - 1$,
word-of-int (rounded-quotient (uint a) (uint B) rounding)))

Proof sketch: a zero divisor fails the assertion. Otherwise the word guard is the exact integer guard. When it fires the function returns false without touching the out-parameter. When it does not, split on the rounding mode: word division is exact, the round-up numerator is exact by the previous lemma, the 64-bit comparison compares exact values, and the narrowing cast is *word-of-int* of the exact quotient.

theorem *big-divide128-layered-characterization*:

big-divide128-layered a B rounding =
(if $\neg 0 < \text{sint } B$ then Cxx-Err Cxx-Assertion-Failed
else if rounding = Cxx-Round-Up $\wedge$ $\text{uint } a > 2^{128} - 1 - (\text{sint } B - 1)$
then Cxx-Ok (False, 0)
else Cxx-Ok
(rounded-quotient (uint a) (sint B) rounding $\leq \text{int64-max-int}$,
if rounded-quotient (uint a) (sint B) rounding $\leq 2^{64} - 1$
then word-of-int (rounded-quotient (uint a) (sint B) rounding)
else 0))

Proof sketch: under the assertion the signed divisor equals its unsigned reading, so the unsigned characterization applies. If the guard fires, the false flag is passed through. Otherwise, if the quotient fits in 64 bits the wrapper reads it back exactly and compares it against the signed maximum, and if it does not, both flags are false.

5.10 The 128-bit family agrees with the collapsed model

theorem *big-divide-or-throw128-layered-eq* [*export-audit*]:

$$\begin{aligned} & \text{big-divide-or-throw128-layered } a \text{ } B \text{ rounding} = \\ & \text{big-divide-or-throw128 } a \text{ } B \text{ rounding} \end{aligned}$$

Proof sketch: rewrite the layered wrapper with the characterization of its inner layer. The assertion and the guard coincide with the collapsed model's first two branches, and a quotient passes the signed range check exactly when the collapsed model does not report overflow.

corollary *big-divide-or-throw128-layered-success-iff*:

$$\begin{aligned} & \text{big-divide-or-throw128-layered } a \text{ } B \text{ rounding} = \text{Cxx-Ok result} \\ & \iff 0 < \text{sint } B \wedge \\ & \quad \neg (\text{rounding} = \text{Cxx-Round-Up} \wedge \\ & \quad \quad \text{uint } a > (2 :: \text{int}) \wedge 128 - 1 - (\text{sint } B - 1)) \wedge \\ & \quad \text{rounded-quotient } (\text{uint } a) (\text{sint } B) \text{ rounding} \\ & \quad \leq \text{int64-max-int} \wedge \\ & \quad \text{result} = \text{word-of-int} \\ & \quad (\text{rounded-quotient } (\text{uint } a) (\text{sint } B) \text{ rounding}) \end{aligned}$$

Proof sketch: transfer the existing theorem along the equality.

end

theory *Offer-Exchange-Lifecycle*

imports *Offer-Exchange-Adjustment HOL.Real*

begin

6 Posting an offer and then crossing it

This theory present a bit-precise model of the ledger-level life of a single resting offer around *exchange-v10-with-options*: a maker posts a wheat-for-sheep offer, which enters the order book, and a taker later crosses it. Posting follows the protocol-10 branch of *ManageOfferOpFrameBase::doApply* for an offer that crosses nothing on entry; crossing follows *crossOfferV10* (*Offer-Exchange.cpp*).

The balances, trustline limits, and existing liabilities of the parties are parameters at each step. In particular the maker's state at posting time and at crossing time are independent inputs, so any intervening ledger activity by third parties—for example a payment that consumes the maker's trustline headroom—can be expressed by choosing the two states independently. A predicate near the end of the theory captures the part of the reachability constraint that connects the two steps.

Assets are named from the resting offer's perspective: the posted offer sells

wheat and buys sheep, and its price quotes *sint price-n* sheep for *sint price-d* wheat. Native-asset reserves, trustline authorization, and sponsorship are abstracted away: a party is reduced to the signed 64-bit quantities that the C++ helpers *canSellAtMost* and *canBuyAtMost* consume.

The imported *Offer-Exchange-Adjustment* theory supplies the maker-independent offer adjustment, liability projection, and executable filtering arithmetic. This theory begins where ledger state first matters.

6.1 Parties

```
record party-state =
  sell-balance :: int64
  sell-liabilities :: int64 — Committed to sell
  buy-limit :: int64
  buy-balance :: int64
  buy-liabilities :: int64 — Committed to buy
```

A party is described by five quantities. On the side of the asset it sells, *sell-balance* is the spendable balance (net of any reserve) and *sell-liabilities* the selling liabilities already reserved for open offers. On the side of the asset it buys, *buy-limit* is the trustline limit, *buy-balance* the current balance, and *buy-liabilities* the buying liabilities already reserved. The maker sells wheat and buys sheep; the taker sells sheep and buys wheat. The selling side carries no limit because nothing in the modeled code paths ever receives into it, and the buying side carries no selling liabilities because nothing ever spends from it.

definition *party-state-wf* :: *party-state* $\Rightarrow$ *bool*

where

```
party-state-wf party  $\longleftrightarrow$ 
  0  $\leq$  sint (sell-liabilities party)  $\wedge$ 
  sint (sell-liabilities party)  $\leq$  sint (sell-balance party)  $\wedge$ 
  0  $\leq$  sint (buy-balance party)  $\wedge$ 
  0  $\leq$  sint (buy-liabilities party)  $\wedge$ 
  sint (buy-balance party) + sint (buy-liabilities party)  $\leq$ 
    sint (buy-limit party)
```

party-state-wf is the ledger invariant that operations preserve: liabilities are non-negative, selling liabilities fit inside the balance, and the balance together with the buying liabilities fits inside the limit.

definition *can-sell-at-most* :: *party-state* $\Rightarrow$ *int64*

— C++: *canSellAtMost* (*OfferExchange.cpp*)

where

```
can-sell-at-most party =
  (let available =
    sint (sell-balance party) - sint (sell-liabilities party)
  in if available  $\leq$  0 then 0 else word-of-int available)
```

definition *can-buy-at-most* :: *party-state* $\Rightarrow$ *int64*

— C++: *canBuyAtMost* (*OfferExchange.cpp*)

where

can-buy-at-most *party* =
 (let *available* =
 sint (*buy-limit* *party*) – *sint* (*buy-balance* *party*) –
 sint (*buy-liabilities* *party*)
 in if *available* $\leq$ 0 then 0 else *word-of-int* *available*)

can-sell-at-most and *can-buy-at-most* model the C++ functions of the same names (*OfferExchange.cpp*): the available balance, respectively the available limit, clamped to zero from below. The subtractions here are exact; under *party-state-wf* they stay within the signed 64-bit range, matching the ledger invariants that protect the C++ subtractions from overflowing.

definition *party-receive-buy-asset* ::

party-state $\Rightarrow$ *int64* $\Rightarrow$ *party-state* *cxx-result*

— C++: *TrustLineWrapper::addBalance* (*ledger/TrustLineWrapper.cpp*)

where

party-receive-buy-asset *party* *delta* =
 (if *delta* = 0 then *Cxx-Ok* *party*
 else if *sint* (*buy-limit* *party*) – *sint* (*buy-balance* *party*) –
 sint (*buy-liabilities* *party*) < *sint* *delta*
 then *Cxx-Err* *Cxx-Runtime-Error*
 else *Cxx-Ok*
 (*party* \ *buy-balance* :=
 word-of-int (*sint* (*buy-balance* *party*) + *sint* *delta*)))

definition *party-spend-sell-asset* ::

party-state $\Rightarrow$ *int64* $\Rightarrow$ *party-state* *cxx-result*

— C++: *TrustLineWrapper::addBalance* (*ledger/TrustLineWrapper.cpp*)

where

party-spend-sell-asset *party* *delta* =
 (if *delta* = 0 then *Cxx-Ok* *party*
 else if *sint* (*sell-balance* *party*) – *sint* (*sell-liabilities* *party*) <
 sint *delta*
 then *Cxx-Err* *Cxx-Runtime-Error*
 else *Cxx-Ok*
 (*party* \ *sell-balance* :=
 word-of-int (*sint* (*sell-balance* *party*) – *sint* *delta*)))

party-receive-buy-asset and *party-spend-sell-asset* model the trustline *addBalance* calls that move the traded amounts: receiving is bounded by the limit minus the balance and the buying liabilities, spending by the balance minus the selling liabilities, and a violation is the runtime error that *crossOfferV10* and *ManageOfferOpFrameBase::doApply* raise when a computed amount does not fit. The C++ sites skip the call when the amount is zero, so neither function can fail on a zero amount.

6.2 State-dependent adjustment and offer liabilities

definition *adjust-stable* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *party-state* $\Rightarrow$
exchange-options $\Rightarrow$ *bool*

where

adjust-stable price-n price-d amount maker options $\longleftrightarrow$
adjust-offer-with-options price-n price-d
 (*signed-min64 amount (can-sell-at-most maker)*)
 (*can-buy-at-most maker*) *options* = *Cxx-Ok amount*

adjust-stable names the write-time invariant of the order book: an amount is *adjust-stable* at a party state when the adjustment computed from that state’s capacities returns it unchanged. Both ledger call sites of *adjustOffer* feed it exactly these arguments—posting against the maker’s bare state, crossing against the state left by releasing the offer’s liabilities—so a stable amount passes the preventative adjustment untouched while an unstable one is clipped. Every write to an offer entry goes through *adjust-offer-with-options*, so the book only ever records amounts that were stable at the moment of writing; but stability relates the amount to a state the entry does not record, and later balance or limit changes can destroy it while every ledger invariant still holds. The lemmas closing the reservation-anomaly subsection record both halves on the walkthrough instance.

definition *add-liability-checked* ::

int $\Rightarrow$ *int64* $\Rightarrow$ *int* $\Rightarrow$ *int64 cxx-result*
 — C++: *addBuyingLiabilities* and *addSellingLiabilities* (*TransactionUtils.cpp*)

where

add-liability-checked cap current delta =
 (*let new-total* = *sint current* + *delta*
 in if *new-total* < 0 $\vee$ *cap* < *new-total*
 then *Cxx-Err Cxx-Runtime-Error*
 else *Cxx-Ok (word-of-int new-total)*)

definition *acquire-offer-liabilities* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *party-state* $\Rightarrow$
party-state cxx-result
 — C++: *acquireLiabilities* (*TransactionUtils.cpp*)

where

acquire-offer-liabilities price-n price-d amount maker = *do* {
buying $\leftarrow$ *offer-buying-liabilities price-n price-d amount*;
new-buying $\leftarrow$ *add-liability-checked*
 (*sint (buy-limit maker)* – *sint (buy-balance maker)*)
 (*buy-liabilities maker*) (*sint buying*);
selling $\leftarrow$ *offer-selling-liabilities price-n price-d amount*;
new-selling $\leftarrow$ *add-liability-checked (sint (sell-balance maker))*
 (*sell-liabilities maker*) (*sint selling*);
Cxx-Ok (maker (buy-liabilities := new-buying,
sell-liabilities := new-selling))

}

definition *release-offer-liabilities* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *party-state* $\Rightarrow$
party-state *cxx-result*

— C++: *releaseLiabilities* (*TransactionUtils.cpp*)

where

release-offer-liabilities price-n price-d amount maker = *do* {
buying $\leftarrow$ *offer-buying-liabilities price-n price-d amount*;
new-buying $\leftarrow$ *add-liability-checked*
 (*sint* (*buy-limit maker*) – *sint* (*buy-balance maker*))
 (*buy-liabilities maker*) (– *sint buying*);
selling $\leftarrow$ *offer-selling-liabilities price-n price-d amount*;
new-selling $\leftarrow$ *add-liability-checked* (*sint* (*sell-balance maker*))
 (*sell-liabilities maker*) (– *sint selling*);
Cxx-Ok (*maker*(*buy-liabilities* := *new-buying*,
 sell-liabilities := *new-selling*))
}

Acquiring and releasing an offer’s liabilities brackets every offer mutation (*acquireLiabilities* and *releaseLiabilities*, *TransactionUtils.cpp*). Following the C++, the buying side is updated before the selling side, and a liability total that would leave its valid interval—from zero to the limit minus the balance on the buying side, from zero to the balance on the selling side—is the runtime error of the C++ helpers.

6.3 Posting an offer

The pre-flight phase either rejects the requested offer or returns the two exchange limits computed for it. The limits use the C++ names: the maker sells sheep and receives wheat in *ManageOfferOpFrameBase*; in this theory those assets are the resting offer’s wheat and sheep, respectively.

datatype *offer-preflight-outcome* =

Preflight-Line-Full
| *Preflight-Underfunded*
| *Preflight-Ready int64 int64*

definition *preflight-offer* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *party-state* $\Rightarrow$
offer-preflight-outcome *cxx-result*

— C++: *ManageOfferOpFrameBase::computeOfferExchangeParameters* (*ManageOfferOpFrameBase.cpp*)

where

preflight-offer price-n price-d amount maker = *do* {
buying $\leftarrow$ *offer-buying-liabilities price-n price-d amount*;
if *sint* (*buy-limit maker*) – *sint* (*buy-balance maker*) –
 sint (*buy-liabilities maker*) < *sint buying*
then *Cxx-Ok Preflight-Line-Full*

```

else do {
  selling ← offer-selling-liabilities price-n price-d amount;
  if sint (sell-balance maker) – sint (sell-liabilities maker) <
    sint selling
  then Cxx-Ok Preflight-Underfunded
  else
    (let max-sheep-send =
      signed-min64 amount (can-sell-at-most maker);
      max-wheat-receive = can-buy-at-most maker
    in if max-wheat-receive = 0
      then Cxx-Ok Preflight-Line-Full
      else Cxx-Ok
      (Preflight-Ready max-sheep-send max-wheat-receive))
}
}

```

preflight-offer models the protocol-10 pre-flight filtering for a *ManageSellOffer*. It first performs the two unclamped tests in *ManageOfferOpFrameBase::computeOfferExchangeParameters* (*ManageOfferOpFrameBase.cpp*): the requested offer’s buying liability must fit the available limit and its selling liability must fit the available balance. It then applies the sell-offer-specific amount cap and models the immediate check in *doApply* that reports *LINE-FULL* when *maxWheatReceive* is zero. This last check is distinct from adjustment: in particular, a positive request whose rounded buying liability is zero is rejected when the maker has no receive capacity, rather than succeeding without creating an offer. Native-asset reserve bookkeeping, trustline authorization, and sponsorship remain abstracted away as described above.

datatype *post-outcome* =
 | *Post-Malformed*
 | *Post-Line-Full*
 | *Post-Underfunded*
 | *Post-No-Offer*
 | *Post-Created* int64 party-state

definition *post-offer* ::
 int32 ⇒ int32 ⇒ int64 ⇒ party-state ⇒
 exchange-options ⇒ post-outcome cxx-result
 — C++: *ManageOfferOpFrameBase::doApply* (*ManageOfferOpFrameBase.cpp*)

where

post-offer price-n price-d amount maker options =
 (if sint price-n ≤ 0 ∨ sint price-d ≤ 0 ∨ sint amount ≤ 0
 then Cxx-Ok Post-Malformed
 else do {
 preflight ← preflight-offer price-n price-d amount maker;
 case preflight of
 Preflight-Line-Full ⇒ Cxx-Ok Post-Line-Full
 | Preflight-Underfunded ⇒ Cxx-Ok Post-Underfunded

```

| Preflight-Ready max-sheep-send max-wheat-receive  $\Rightarrow$  do {
  adjusted  $\leftarrow$  adjust-offer-with-options price-n price-d
  max-sheep-send max-wheat-receive options;
  if 0 < sint adjusted
  then do {
    maker-after  $\leftarrow$  acquire-offer-liabilities price-n price-d
    adjusted maker;
    Cxx-Ok (Post-Created adjusted maker-after)
  }
  else Cxx-Ok Post-No-Offer
}
})

```

definition *preflight-offer-core* ::

int64 cxx-result $\Rightarrow$ int64 cxx-result $\Rightarrow$ int64 $\Rightarrow$
int64 $\Rightarrow$ party-state $\Rightarrow$ offer-preflight-outcome cxx-result
 — C++: *ManageOfferOpFrameBase::computeOfferExchangeParameters* (*ManageOfferOpFrameBase.cpp*)

where

```

preflight-offer-core buying-liability selling-liability max-send-cap
max-receive-cap maker = do {
  buying  $\leftarrow$  buying-liability;
  if sint (buy-limit maker) - sint (buy-balance maker) -
    sint (buy-liabilities maker) < sint buying
  then Cxx-Ok Preflight-Line-Full
  else do {
    selling  $\leftarrow$  selling-liability;
    if sint (sell-balance maker) - sint (sell-liabilities maker) <
      sint selling
    then Cxx-Ok Preflight-Underfunded
    else
      (let max-sheep-send =
        signed-min64 max-send-cap (can-sell-at-most maker);
        max-wheat-receive =
          (if max-receive-cap = int64-max
            then can-buy-at-most maker
            else signed-min64 max-receive-cap (can-buy-at-most maker))
        in if max-wheat-receive = 0
          then Cxx-Ok Preflight-Line-Full
          else Cxx-Ok
          (Preflight-Ready max-sheep-send max-wheat-receive))
      )
  }
}

```

definition *post-offer-core* ::

int32 $\Rightarrow$ int32 $\Rightarrow$ bool $\Rightarrow$ int64 cxx-result $\Rightarrow$
int64 cxx-result $\Rightarrow$ int64 $\Rightarrow$ int64 $\Rightarrow$ party-state $\Rightarrow$
exchange-options $\Rightarrow$ post-outcome cxx-result
 — C++: *ManageOfferOpFrameBase::doApply* (*ManageOfferOpFrameBase.cpp*)

where

```

post-offer-core price-n price-d request-valid buying-liability
  selling-liability max-send-cap max-receive-cap maker options =
(if sint price-n ≤ 0 ∨ sint price-d ≤ 0 ∨ ¬ request-valid
then Cxx-Ok Post-Malformed
else do {
  preflight ← preflight-offer-core buying-liability
    selling-liability max-send-cap max-receive-cap maker;
  case preflight of
    Preflight-Line-Full ⇒ Cxx-Ok Post-Line-Full
  | Preflight-Underfunded ⇒ Cxx-Ok Post-Underfunded
  | Preflight-Ready max-sheep-send max-wheat-receive ⇒ do {
    adjusted ← adjust-offer-with-options price-n price-d
      max-sheep-send max-wheat-receive options;
    if 0 < sint adjusted
    then do {
      maker-after ← acquire-offer-liabilities price-n price-d
        adjusted maker;
      Cxx-Ok (Post-Created adjusted maker-after)
    }
    else Cxx-Ok Post-No-Offer
  }
})

```

definition *post-sell-offer* ::

```

int32 ⇒ int32 ⇒ int64 ⇒ party-state ⇒
  exchange-options ⇒ post-outcome cxx-result
— C++: doApply for ManageSellOfferOpFrame

```

where

```

post-sell-offer price-n price-d amount maker options =
  post-offer-core price-n price-d (0 < sint amount)
    (offer-buying-liabilities price-n price-d amount)
    (offer-selling-liabilities price-n price-d amount)
    amount int64-max maker options

```

definition *post-buy-offer* ::

```

uint32 ⇒ int32 ⇒ int32 ⇒ int64 ⇒
  party-state ⇒ post-outcome cxx-result
— C++: doApply for ManageBuyOfferOpFrame

```

where

```

post-buy-offer ledger-version price-n price-d buy-amount maker =
  post-offer-core price-d price-n (0 < sint buy-amount)
    (manage-buy-buying-liabilities-at-version ledger-version price-n
      price-d buy-amount)
    (manage-buy-selling-liabilities-at-version ledger-version price-n
      price-d buy-amount)
    int64-max buy-amount maker
    (exchange-options-at-version ledger-version)

```

preflight-offer-core and *post-offer-core* factor the common no-crossing post-

ing path. They receive the request-specific liabilities and operation caps but always adjust and acquire liabilities at the canonical resting price. *post-sell-offer* supplies the submitted sell amount as its send cap. *post-buy-offer* swaps the submitted price, supplies the buy request's unlimited-send liabilities, and caps the receive side by the requested buy amount, exactly matching *ManageBuyOfferOpFrame::applyOperationSpecificLimits*.

post-buy-offer takes a ledger version rather than an options record because it needs one for two distinct purposes, exactly as *computeOfferExchangeParameters* does: the version selects the exchange arithmetic, and it is also passed to *ManageBuyOfferOpFrame::getOfferSellingLiabilities* and *getOfferBuyingLiabilities*. Those request-time liabilities really do move at the protocol boundary, so deriving them from a fixed configuration while adjusting with a versioned one would be unfaithful. *post-sell-offer* has no such obligation: its liabilities are the stored-offer projections, which are version-independent.

The older *post-offer* remains the established *ManageSell* model. The refinement theorem below checks extensionally that the factored sell wrapper did not change any result, including malformed, ordinary rejection, no-offer, created-offer, or modeled C++ error outcomes.

lemma *signed-min64-int64-max-left [simp]:*

signed-min64 int64-max value = value

theorem *post-sell-offer-eq-post-offer:*

*post-sell-offer price-n price-d amount maker options =
post-offer price-n price-d amount maker options*

Proof sketch: unfold both wrappers. The common pre-flight specializes to the original liability calls and capacities; taking the signed minimum with *int64-max* is the identity. Every following adjustment, acquisition, and outcome branch is consequently identical.

lemma *post-offer-core-created-facts:*

assumes *post:*

*post-offer-core price-n price-d request-valid buying-liability
selling-liability max-send-cap max-receive-cap maker options =
Cxx-Ok (Post-Created posted maker-after)*

shows

*0 < sint price-n
0 < sint price-d
request-valid
0 < sint posted
acquire-offer-liabilities price-n price-d posted maker =
Cxx-Ok maker-after*

Proof sketch: invert the only created branch of the common posting core.

Reaching it excludes every malformed and ordinary pre-flight branch, the adjustment is positive, and the final bind is precisely liability acquisition for that adjusted canonical amount.

lemma *post-offer-core-created-adjustment:*

assumes *post:*

*post-offer-core price-n price-d request-valid buying-liability
selling-liability max-send-cap max-receive-cap maker options =
Cxx-Ok (Post-Created posted maker-after)*

shows

*adjust-offer-with-options price-n price-d
(signed-min64 max-send-cap (can-sell-at-most maker))
(if max-receive-cap = int64-max then can-buy-at-most maker
else signed-min64 max-receive-cap (can-buy-at-most maker))
options = Cxx-Ok posted*

Proof sketch: invert the created branch of the common posting core once more, this time keeping the adjustment call itself rather than discarding it. Reaching the created outcome forces the ready pre-flight branch, whose two capacities are exactly the send cap clipped by the maker's selling capacity and the receive cap clipped by the maker's buying capacity, with the unlimited receive cap left uncapped. The adjusted amount returned there is the amount written to the book.

Together with *post-offer-core-created-facts* this is the whole created-branch inversion: the two lemmas cover positivity, liability acquisition, and the adjustment that produced the posted amount, for both the ManageSell and the ManageBuy caps.

post-offer models a *ManageSellOffer* that creates a fresh offer crossing nothing on entry, so the requested offer goes straight into the book. The operation is malformed unless the price components and the amount are positive (*doCheckValid*; a zero amount, an offer deletion in C++, is folded into *Post-Malformed* here). It then delegates the liability and capacity filters to *pre-flight-offer*. For a ready request, the amount actually written to the book is the returned selling limit clipped by *adjust-offer-with-options* (the *adjustOffer* call in *doApply*); an amount clipped to zero creates no offer; and a created offer reserves its liabilities. The final *exchange-options* argument carries the protocol choices that *doApply* passes to that *adjustOffer* call; the liability computations take no options, matching the C++ helpers.

In the comfortably funded walkthrough state, pre-flight preserves Alice's one-unit request and reports her receive capacity of two. Proof sketch: evaluate the liability checks and the two derived capacities.

lemma *preflight-offer-ready-example:*

preflight-offer 101 100 1

$\langle \text{sell-balance} = 1, \text{sell-liabilities} = 0, \\ \text{buy-limit} = 2, \text{buy-balance} = 0, \text{buy-liabilities} = 0 \rangle = \\ \text{Cxx-Ok (Preflight-Ready 1 2)}$

At price one half, a one-unit request has zero rounded buying liability. The unclamped liability test therefore passes at zero headroom, but the following `zero-maxWheatReceive` filter reports line full; posting propagates that result instead of treating adjustment to zero as a successful no-offer. Proof sketch: unfold the executable pre-flight and posting definitions and evaluate both results.

lemma *preflight-zero-receive-capacity-is-line-full:*

$\text{preflight-offer 1 2 1} \\ \langle \text{sell-balance} = 1, \text{sell-liabilities} = 0, \\ \text{buy-limit} = 0, \text{buy-balance} = 0, \text{buy-liabilities} = 0 \rangle = \\ \text{Cxx-Ok Preflight-Line-Full}$

lemma *post-zero-receive-capacity-is-line-full:*

$\text{post-offer 1 2 1} \\ \langle \text{sell-balance} = 1, \text{sell-liabilities} = 0, \\ \text{buy-limit} = 0, \text{buy-balance} = 0, \text{buy-liabilities} = 0 \rangle \\ \langle \text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False} \rangle = \\ \text{Cxx-Ok Post-Line-Full}$

6.4 Crossing the posted offer

record *cross-result-v10* =

$\text{cross-wheat-received} :: \text{int64} \\ \text{cross-sheep-send} :: \text{int64} \\ \text{cross-wheat-stays} :: \text{bool} \\ \text{cross-offer-amount} :: \text{int64} \\ \text{cross-maker} :: \text{party-state} \\ \text{cross-taker} :: \text{party-state}$

definition *make-cross-result* ::

$\text{int64} \Rightarrow \text{int64} \Rightarrow \text{bool} \Rightarrow \text{int64} \Rightarrow \text{party-state} \Rightarrow \\ \text{party-state} \Rightarrow \text{cross-result-v10}$

where

$\text{make-cross-result wheat-received sheep-send wheat-stays remaining maker} \\ \text{taker} = \\ \langle \text{cross-wheat-received} = \text{wheat-received}, \\ \text{cross-sheep-send} = \text{sheep-send}, \\ \text{cross-wheat-stays} = \text{wheat-stays}, \\ \text{cross-offer-amount} = \text{remaining}, \\ \text{cross-maker} = \text{maker}, \\ \text{cross-taker} = \text{taker} \rangle$

definition *cross-offer-v10* ::

```

int32 ⇒ int32 ⇒ int64 ⇒ party-state ⇒ party-state ⇒
int64 ⇒ exchange-rounding ⇒ exchange-options ⇒
cross-result-v10 cxx-result
— C++: crossOfferV10 (OfferExchange.cpp)
where
cross-offer-v10 price-n price-d offer-amount maker taker taker-amount
rounding options =
  (let max-wheat-receive = can-buy-at-most taker;
   max-sheep-send = signed-min64 taker-amount (can-sell-at-most taker)
   in if sint max-wheat-receive ≤ 0 ∨ sint max-sheep-send ≤ 0
   then Cxx-Err Cxx-Assertion-Failed
   else do {
     maker-released ← release-offer-liabilities price-n price-d
       offer-amount maker;
     adjusted ← adjust-offer-with-options price-n price-d
       (signed-min64 offer-amount (can-sell-at-most maker-released))
       (can-buy-at-most maker-released) options;
     let max-wheat-send =
       signed-min64 adjusted (can-sell-at-most maker-released);
     let max-sheep-receive = can-buy-at-most maker-released;
     result ← exchange-v10-with-options price-n price-d max-wheat-send
       max-wheat-receive max-sheep-send max-sheep-receive rounding
       options;
     let wheat-received = num-wheat-received result;
     let sheep-send = num-sheep-send result;
     let wheat-stays = result-wheat-stays result;
     maker-credited ← party-receive-buy-asset maker-released
       sheep-send;
     maker-moved ← party-spend-sell-asset maker-credited
       wheat-received;
     taker-credited ← party-receive-buy-asset taker wheat-received;
     taker-after ← party-spend-sell-asset taker-credited sheep-send;
     if wheat-stays then do {
       adjusted-after ← adjust-offer-with-options price-n price-d
         (signed-min64 (adjusted - wheat-received)
          (can-sell-at-most maker-moved))
         (can-buy-at-most maker-moved) options;
       if adjusted-after = 0
       then Cxx-Ok (make-cross-result wheat-received sheep-send
         wheat-stays 0 maker-moved taker-after)
       else do {
         maker-final ← acquire-offer-liabilities price-n price-d
           adjusted-after maker-moved;
         Cxx-Ok (make-cross-result wheat-received sheep-send
           wheat-stays adjusted-after maker-final taker-after)
       }
     } else
     Cxx-Ok (make-cross-result wheat-received sheep-send wheat-stays
       0 maker-moved taker-after)
  )

```

})

cross-offer-v10 is the ledger bookkeeping of *crossOfferV10* (*OfferExchange.cpp*) around one call to *exchange-v10-with-options*, with the taker's limits derived from its party state the way *ManageOfferOpFrameBase::computeOfferExchangeParameters* derives them: the taker sends at most the smaller of its remaining operation amount *taker-amount* and its available sheep balance, and receives at most its available wheat limit. The steps mirror the C++ order: assert the taker limits positive; release the resting offer's liabilities; apply the preventative *adjust-offer-with-options* that the C++ comment on the pre-exchange *adjustOffer* call claims "should have no effect"; exchange; move the traded balances; and either erase the offer or shrink it, adjust it once more against the moved balances, and re-acquire liabilities for what remains. The taker's balance updates, which the C++ performs in the operation frame after all crossings, are folded in here so that the result describes both parties. The final *exchange-options* argument carries the protocol choices: the C++ passes the exact-receive-cap choice to *exchangeV10* inside *crossOfferV10*, and the two bracketing adjustments derive the same choice from the ledger header in the ledger-level *adjustOffer* overload.

6.5 Executable offer lifecycle

```

datatype offer-request =
  Manage-Sell int32 int32 int64
  | Manage-Buy int32 int32 int64

fun request-canonical-price-n :: offer-request ⇒ int32
  where
    request-canonical-price-n (Manage-Sell price-n price-d amount) = price-n
    | request-canonical-price-n (Manage-Buy price-n price-d buy-amount) = price-d

fun request-canonical-price-d :: offer-request ⇒ int32
  where
    request-canonical-price-d (Manage-Sell price-n price-d amount) = price-d
    | request-canonical-price-d (Manage-Buy price-n price-d buy-amount) = price-n

fun post-offer-request ::
  uint32 ⇒ offer-request ⇒ party-state ⇒
  post-outcome cxx-result
  where
    post-offer-request ledger-version (Manage-Sell price-n price-d amount)
      maker =
      post-sell-offer price-n price-d amount maker
      (exchange-options-at-version ledger-version)
    | post-offer-request ledger-version (Manage-Buy price-n price-d buy-amount)
      maker =

```

post-buy-offer ledger-version price-n price-d buy-amount maker

definition *maximum-capacity-taker* :: *party-state*

where

maximum-capacity-taker =
 (sell-balance = int64-max, sell-liabilities = 0,
 buy-limit = int64-max, buy-balance = 0, buy-liabilities = 0)

record *lifecycle-trace* =

lifecycle-request :: *offer-request*
lifecycle-price-n :: *int32*
lifecycle-price-d :: *int32*
lifecycle-posted-amount :: *int64*
lifecycle-maker-after-post :: *party-state*
lifecycle-maker-after-limit :: *party-state*
lifecycle-taker-before :: *party-state*
lifecycle-cross-result :: *cross-result-v10*

record *lifecycle-prefix* =

lifecycle-prefix-request :: *offer-request*
lifecycle-prefix-price-n :: *int32*
lifecycle-prefix-price-d :: *int32*
lifecycle-prefix-posted-amount :: *int64*
lifecycle-prefix-maker-after-post :: *party-state*
lifecycle-prefix-maker-after-limit :: *party-state*
lifecycle-prefix-taker-before :: *party-state*

datatype *lifecycle-outcome* =

Post-Failed post-outcome
 | *Post-Cxx-Error cxx-error*
 | *Limit-Change-Invalid lifecycle-prefix*
 | *Cross-Cxx-Error cxx-error lifecycle-prefix*
 | *Crossed lifecycle-trace*

definition *run-offer-lifecycle* ::

uint32 ⇒ *offer-request* ⇒ *party-state* ⇒
int64 ⇒ *lifecycle-outcome*

where

run-offer-lifecycle ledger-version request maker-at-post new-buy-limit =
 (case *post-offer-request ledger-version request maker-at-post* of
Cxx-Err error ⇒ *Post-Cxx-Error error*
 | *Cxx-Ok post* ⇒
 (case *post* of
 Post-Created posted maker-after ⇒
 (let *maker-at-cross* =
 maker-after(buy-limit := *new-buy-limit*);
 price-n = *request-canonical-price-n request*;
 price-d = *request-canonical-price-d request*;
 prefix =

```

    (lifecycle-prefix-request = request,
     lifecycle-prefix-price-n = price-n,
     lifecycle-prefix-price-d = price-d,
     lifecycle-prefix-posted-amount = posted,
     lifecycle-prefix-maker-after-post = maker-after,
     lifecycle-prefix-maker-after-limit = maker-at-cross,
     lifecycle-prefix-taker-before = maximum-capacity-taker)
  in if ¬ party-state-wf maker-at-cross
    then Limit-Change-Invalid prefix
    else
      (case cross-offer-v10 price-n price-d posted
        maker-at-cross maximum-capacity-taker int64-max
        Exchange-Normal
        (exchange-options-at-version ledger-version) of
        Cxx-Err error ⇒ Cross-Cxx-Error error prefix
      | Cxx-Ok crossed ⇒
        Crossed
        (lifecycle-request = request,
         lifecycle-price-n = price-n,
         lifecycle-price-d = price-d,
         lifecycle-posted-amount = posted,
         lifecycle-maker-after-post = maker-after,
         lifecycle-maker-after-limit = maker-at-cross,
         lifecycle-taker-before = maximum-capacity-taker,
         lifecycle-cross-result = crossed)))
  | failed ⇒ Post-Failed failed))

```

run-offer-lifecycle is the complete executable path in this phase. The request retains its raw operation orientation; manage-buy swaps the price only in the posting wrapper and trace. Overlay rejection, ordinary operation outcomes, a ledger-invalid limit change, and modeled C++ failures are distinct constructors. A successful value embeds the actual *cross-result-v10*, so its positive transfer and final balances are executable observations rather than a Boolean claim.

The named taker has maximum signed capacity on both relevant sides. The transaction-level C++ oracle materializes the same one-shot witness with a *ManageBuyOffer* that buys exactly the full posted wheat amount. Its raw price is the canonical resting price, which the buy wrapper inverts into the reciprocal counteroffer price. If the maker is taken, the exact-receive request is fully consumed and leaves no taker offer or liabilities, matching the abstract direct crossing. A ceil-sized reciprocal *ManageSellOffer* is not equivalent: fractional prices can leave a one-unit taker offer after an otherwise successful crossing. The abstract crossing call states the stronger maximum-capacity witness directly and therefore needs no redundant counter-price input.

lemma *maximum-capacity-taker-equations:*

```

party-state-wf maximum-capacity-taker
can-sell-at-most maximum-capacity-taker = int64-max

```

can-buy-at-most maximum-capacity-taker = int64-max

Proof sketch: evaluate the record invariant and both capacity projections at zero liabilities and zero buying balance.

lemma *legacy-sell-positive-crossing-example:*

(case run-offer-lifecycle 28 (Manage-Sell 101 100 3)
 (sell-balance = 3, sell-liabilities = 0,
 buy-limit = 4, buy-balance = 0, buy-liabilities = 0) 3 of
 Crossed trace $\Rightarrow$
 0 < sint (cross-wheat-received (lifecycle-cross-result trace)) $\wedge$
 0 < sint (cross-sheep-send (lifecycle-cross-result trace))
 | - $\Rightarrow$ False)

lemma *legacy-buy-positive-crossing-example:*

(case run-offer-lifecycle 28 (Manage-Buy 100 101 4)
 (sell-balance = 3, sell-liabilities = 0,
 buy-limit = 4, buy-balance = 0, buy-liabilities = 0) 3 of
 Crossed trace $\Rightarrow$
 0 < sint (cross-wheat-received (lifecycle-cross-result trace)) $\wedge$
 0 < sint (cross-sheep-send (lifecycle-cross-result trace))
 | - $\Rightarrow$ False)

lemma *post-offer-request-created-facts:*

assumes *post:*

post-offer-request ledger-version request maker =
 Cxx-Ok (Post-Created posted maker-after)

shows

0 < sint (request-canonical-price-n request)
 0 < sint (request-canonical-price-d request)
 0 < sint posted
 acquire-offer-liabilities (request-canonical-price-n request)
 (request-canonical-price-d request) posted maker = Cxx-Ok maker-after

Proof sketch: split on the raw request constructor and apply the common-core created-branch inversion theorem. The buy case swaps the two raw price fields, exactly as the canonical price projections do.

6.6 The whole process

definition *maker-covers-offer-liabilities ::*

int32 $\Rightarrow$ int32 $\Rightarrow$ int64 $\Rightarrow$ party-state $\Rightarrow$ bool

where

maker-covers-offer-liabilities price-n price-d offer-amount maker $\longleftrightarrow$
 (case (offer-selling-liabilities price-n price-d offer-amount,
 offer-buying-liabilities price-n price-d offer-amount) of
 (Cxx-Ok selling, Cxx-Ok buying) $\Rightarrow$
 sint selling $\leq$ sint (sell-liabilities maker) $\wedge$

sint *buying* $\leq$ *sint* (*buy-liabilities* *maker*)
 | - $\Rightarrow$ *False*)

definition *post-then-cross* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *party-state* $\Rightarrow$ *party-state* $\Rightarrow$
party-state $\Rightarrow$ *int64* $\Rightarrow$ *exchange-rounding* $\Rightarrow$ *exchange-options* $\Rightarrow$
 (*post-outcome* $\times$ *cross-result-v10* *option*) *cxx-result*

where

post-then-cross *price-n* *price-d* *amount* *maker-at-post* *maker-at-cross*
taker *taker-amount* *rounding* *options* = *do* {
outcome $\leftarrow$ *post-offer* *price-n* *price-d* *amount* *maker-at-post*
options;
 (*case outcome of*
Post-Created *posted* - $\Rightarrow$ *do* {
crossed $\leftarrow$ *cross-offer-v10* *price-n* *price-d* *posted* *maker-at-cross*
taker *taker-amount* *rounding* *options*;
Cxx-Ok (*outcome*, *Some* *crossed*)
 }
 | - $\Rightarrow$ *Cxx-Ok* (*outcome*, *None*))
 }

post-then-cross chains the two steps. The maker's state at the crossing step is a fresh parameter rather than the state returned by posting: between the two steps, third parties may move the maker's balances and limits. *maker-covers-offer-liabilities* states the part of the ledger invariant that connects the two: whatever else changed, the maker's liability totals at crossing time still include the posted offer's own reservation. A single *exchange-options* value serves both steps: it models a post and a cross under the same protocol version, so an offer posted before a protocol upgrade and crossed after it is out of scope here. *maker-covers-offer-liabilities* needs no flag because the liability functions carry none.

6.7 Intended properties

The principal properties below capture what the exchange lifecycle should guarantee. The first assigns the rounding benefit to the offer that stays, and the second strengthens that direction guarantee to a maximal positive normal fill. The third is local to the preventative adjustment at crossing time. Several increasingly strong liveness properties say that a successfully posted offer has a positive crossing when its maker is unchanged, after any admissible change to only the maker's sheep limit, and finally after any admissible maker-state change when the exact receive cap is enabled. This subsection only defines and explains the predicates; their supporting proofs and counterexamples appear in later top-level subsections.

6.7.1 Rounding favors the offer that stays

definition *rounding-favors-offer-that-stays* :: *exchange-options* $\Rightarrow$ *bool*

where

```

rounding-favors-offer-that-stays options  $\longleftrightarrow$ 
  ( $\forall$  price-n price-d max-wheat-send max-wheat-receive max-sheep-send
    max-sheep-receive rounding result.
    exchange-v10-pre price-n price-d max-wheat-send max-wheat-receive
      max-sheep-send max-sheep-receive  $\wedge$ 
    exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive
      max-sheep-send max-sheep-receive rounding options =
      Cxx-Ok result  $\wedge$ 
    0 < sint (num-wheat-received result)  $\wedge$ 
    0 < sint (num-sheep-send result)  $\longrightarrow$ 
    favored-seller-ok price-n price-d
      (num-wheat-received result) (num-sheep-send result)
      (result-wheat-stays result))

```

rounding-favors-offer-that-stays applies only to a successful positive exchange, because a zero exchange has no effective price or rounding beneficiary. When the resting wheat offer stays, the predicate requires the effective price to favor its maker; when that offer is removed, it requires the effective price to favor the taker's sheep offer instead. Equivalently, it requires $\text{sint } (\text{num-wheat-received result}) * \text{sint price-n} \leq \text{sint } (\text{num-sheep-send result}) * \text{sint price-d}$ when *result-wheat-stays result*, and the reverse inequality otherwise.

This property constrains the direction of rounding after the staying offer has been chosen; it does not itself specify how that choice is made. The options argument is explicit so the shipped and exact-receive-cap paths can be verified separately. Its proof is deferred to a later subsection.

6.7.2 Maximal positive normal crossing

definition *canonical-favored-rounding* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *bool* $\Rightarrow$ *int* $\Rightarrow$ *int* $\Rightarrow$ *bool*

where

```

canonical-favored-rounding price-n price-d wheat-stays
  wheat-receive sheep-send  $\longleftrightarrow$ 
  (if sint price-n > sint price-d
    then sheep-send =
      (if wheat-stays
        then (wheat-receive * sint price-n + sint price-d - 1)
          div sint price-d
        else wheat-receive * sint price-n div sint price-d)
    else wheat-receive =
      (if wheat-stays
        then sheep-send * sint price-d div sint price-n
        else (sheep-send * sint price-d + sint price-n - 1)

```

$\text{div sint price-n}))$

definition *normal-favored-candidate* ::

$\text{int32} \Rightarrow \text{int32} \Rightarrow \text{int64} \Rightarrow \text{int64} \Rightarrow \text{int64} \Rightarrow$
 $\text{int64} \Rightarrow \text{bool} \Rightarrow \text{int} \Rightarrow \text{int} \Rightarrow \text{bool}$

where

normal-favored-candidate price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive wheat-stays
wheat-receive sheep-send $\longleftrightarrow$
 $0 \leq \text{wheat-receive} \wedge$
 $\text{wheat-receive} \leq$
 $\min (\text{sint max-wheat-send}) (\text{sint max-wheat-receive}) \wedge$
 $0 \leq \text{sheep-send} \wedge$
 $\text{sheep-send} \leq$
 $\min (\text{sint max-sheep-send}) (\text{sint max-sheep-receive}) \wedge$
canonical-favored-rounding price-n price-d wheat-stays
wheat-receive sheep-send $\wedge$
(if sint price-n > sint price-d
*then wheat-receive * sint price-n ≤*
*sint int64-max * sint price-d*
*else sheep-send * sint price-d ≤*
*sint int64-max * sint price-n)*

definition *maximal-normal-favored-result* ::

$\text{int32} \Rightarrow \text{int32} \Rightarrow \text{int64} \Rightarrow \text{int64} \Rightarrow \text{int64} \Rightarrow$
 $\text{int64} \Rightarrow \text{exchange-result-v10} \Rightarrow \text{bool}$

where

maximal-normal-favored-result price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive result $\longleftrightarrow$
normal-favored-candidate price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive
(result-wheat-stays result)
(sint (num-wheat-received result))
(sint (num-sheep-send result)) $\wedge$
 $(\forall \text{wheat-receive sheep-send.}$
normal-favored-candidate price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive
(result-wheat-stays result) wheat-receive sheep-send $\longrightarrow$
(if sint price-n > sint price-d
then wheat-receive ≤ sint (num-wheat-received result)
else sheep-send ≤ sint (num-sheep-send result)))

definition *positive-normal-crosses-are-maximal* :: *exchange-options* $\Rightarrow$ *bool*

where

positive-normal-crosses-are-maximal options $\longleftrightarrow$
 $(\forall \text{price-n price-d max-wheat-send max-wheat-receive max-sheep-send}$
 $\text{max-sheep-receive result.}$

```

exchange-v10-pre price-n price-d max-wheat-send max-wheat-receive
max-sheep-send max-sheep-receive  $\wedge$ 
exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive
max-sheep-send max-sheep-receive Exchange-Normal
options = Cxx-Ok result  $\wedge$ 
0 < sint (num-wheat-received result)  $\wedge$ 
0 < sint (num-sheep-send result)  $\longrightarrow$ 
maximal-normal-favored-result price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive result)

```

canonical-favored-rounding uses units of the more valuable asset as the primary measure of exchange size. When wheat is more valuable, a wheat amount uniquely determines the sheep amount by rounding up if the wheat offer stays and down if the sheep offer stays. When sheep is at least as valuable, the symmetric formulas derive wheat from a primary sheep amount. These are the four normal-mode formulas that round by the least whole unit in favor of the offer marked as staying.

normal-favored-candidate additionally requires both transferred amounts to lie within all four exchange caps. Its final conjunct retains the implementation's saturation ceiling: even an exact receive cap does not admit a scaled value above what *int64-max* units of the other asset can represent. *maximal-normal-favored-result* requires the returned pair itself to be such a candidate and says that no candidate has a larger primary amount. Thus it maximizes wheat when wheat is more valuable and sheep otherwise, avoiding an arbitrary ordering between unlike assets.

positive-normal-crosses-are-maximal lifts this specification to every successful positive normal exchange. Zero results remain outside its scope because the current one-percent threshold may reject the initially selected pair without searching downward for a smaller positive pair. Strict-send and strict-receive need mode-specific objectives and are likewise outside this predicate.

The model now carries two independently selectable receive-cap gates. The *exact-receive-cap* field repairs the sheep-stays, wheat-more-valuable normal branch by applying *calculate-offer-value-with-exact-receive-cap* to the resting wheat offer's sheep-receive cap. The *symmetric-exact-receive-cap* field repairs the mirror wheat-stays, sheep-more-valuable normal branch with the symmetric call *calculate-offer-value-with-exact-receive-cap price-d price-n max-sheep-send max-wheat-receive* before dividing the resulting value by *sint price-d*, treating the taker's wheat-receive cap as an exact unit cap. Both repairs are implemented in *OfferExchange.cpp* behind their protocol gates and mirrored in *exchange-v10-amounts*, and neither changes the decision about which offer stays: *wheatStays* is still compared on the two unadjusted offer values.

Without the symmetric gate the exact-receive-cap path does not satisfy the intended flag-on property. For example, at the price with numerator 99 and

denominator *100*, with wheat caps *102* and *101* and sheep caps *100* and *101*, it returns *100* wheat and *99* sheep with wheat staying. The pair *101* wheat and *100* sheep is within every cap, uses the canonical favoring round, and is larger. With both gates enabled the same inputs produce that maximal *101*-wheat, *100*-sheep cross while *wheatStays* remains true.

With both repairs enabled the property holds: the theorem *positive-normal-crosses-are-maximal-repaired*, proved in the maximality subsection below, establishes *positive-normal-crosses-are-maximal* ($\text{exact-receive-cap} = \text{True}$, *symmetric-exact-receive-cap* = *True*).

6.7.3 Coverage implies adjustment stability

definition *cover-implies-adjust-stable* :: *exchange-options* $\Rightarrow$ *bool*

where

cover-implies-adjust-stable options $\longleftrightarrow$
 $(\forall \text{price-n price-d amount maker-at-post posted maker-after}$
 $\text{maker-at-cross released.}$
 $\text{party-state-wf maker-at-post} \wedge$
 $\text{party-state-wf maker-at-cross} \wedge$
 $\text{post-offer price-n price-d amount maker-at-post options} =$
 $\text{Cxx-Ok (Post-Created posted maker-after)} \wedge$
 $\text{maker-covers-offer-liabilities price-n price-d posted}$
 $\text{maker-at-cross} \wedge$
 $\text{release-offer-liabilities price-n price-d posted maker-at-cross} =$
 $\text{Cxx-Ok released} \longrightarrow$
 $\text{adjust-stable price-n price-d posted released options})$

cover-implies-adjust-stable ranges only over amounts produced by a successful *post-offer*; this excludes arbitrary amounts that never entered the order book. *maker-covers-offer-liabilities* then speaks about the maker's booked state at crossing time, in which the posted offer's selling and buying reservations are still present. Crossing first releases those reservations, so the property explicitly relates that booked state to the released state seen by *adjust-offer-with-options*.

The intended implication says that every successfully posted offer, in every well-formed crossing state which still covers its liabilities, must remain stable after release. If it holds, the preventative adjustment is the identity at every admissible crossing state: a balance or limit change that leaves the reservation covered cannot clip the offer. The shipped behavior violates this property, as the reservation-anomaly walkthrough below demonstrates.

6.7.4 Takeability after adjusting the maker's sheep limit

definition *limit-adjustment-stable* :: *exchange-options* $\Rightarrow$ *bool*

where

limit-adjustment-stable options $\longleftrightarrow$

$$\begin{aligned}
& (\forall \text{ price-n price-d amount maker-at-post posted maker-after} \\
& \quad \text{new-sheep-limit.} \\
& \text{party-state-wf maker-at-post} \wedge \\
& \text{post-offer price-n price-d amount maker-at-post options} = \\
& \quad \text{Cxx-Ok (Post-Created posted maker-after)} \longrightarrow \\
& (\text{let maker-at-cross} = \\
& \quad \text{maker-after}(\text{buy-limit} := \text{new-sheep-limit}) \\
& \text{in party-state-wf maker-at-cross} \longrightarrow \\
& (\exists \text{ taker taker-amount crossed.} \\
& \quad \text{party-state-wf taker} \wedge \\
& \quad 0 < \text{sint (can-buy-at-most taker)} \wedge \\
& \quad 0 < \text{sint} \\
& \quad \quad (\text{signed-min64 taker-amount (can-sell-at-most taker)}) \wedge \\
& \quad \text{cross-offer-v10 price-n price-d posted maker-at-cross taker} \\
& \quad \text{taker-amount Exchange-Normal options} = \\
& \quad \quad \text{Cxx-Ok crossed} \wedge \\
& \quad 0 < \text{sint (cross-wheat-received crossed)} \wedge \\
& \quad 0 < \text{sint (cross-sheep-send crossed)})))))
\end{aligned}$$

limit-adjustment-stable isolates changes to the maker's sheep trustline limit. It starts with the booked maker state returned by a successful *post-offer*, changes only *buy-limit*, and requires every resulting well-formed state to retain some successful positive normal-mode crossing. Thus no ledger-admissible sheep-limit adjustment may make the posted offer untakeable.

Requiring *party-state-wf* after the update excludes limits below the maker's current sheep balance plus buying liabilities, which the ledger would not admit. Every other field—including the offer's selling and buying liabilities—is held fixed. As in the surrounding liveness properties, takeability is existential in the taker: the property does not promise a nonzero fill to every counterparty.

definition *buy-limit-adjustment-stable* :: *uint32* $\Rightarrow$ *bool*

where

$$\begin{aligned}
& \text{buy-limit-adjustment-stable ledger-version} \longleftrightarrow \\
& (\forall \text{ price-n price-d buy-amount maker-at-post posted maker-after} \\
& \quad \text{new-sheep-limit.} \\
& \text{party-state-wf maker-at-post} \wedge \\
& \text{post-buy-offer ledger-version price-n price-d buy-amount} \\
& \quad \text{maker-at-post} = \text{Cxx-Ok (Post-Created posted maker-after)} \longrightarrow \\
& (\text{let maker-at-cross} = \\
& \quad \text{maker-after}(\text{buy-limit} := \text{new-sheep-limit}) \\
& \text{in party-state-wf maker-at-cross} \longrightarrow \\
& (\exists \text{ taker taker-amount crossed.} \\
& \quad \text{party-state-wf taker} \wedge \\
& \quad 0 < \text{sint (can-buy-at-most taker)} \wedge \\
& \quad 0 < \text{sint} \\
& \quad \quad (\text{signed-min64 taker-amount (can-sell-at-most taker)}) \wedge \\
& \quad \text{cross-offer-v10 price-d price-n posted maker-at-cross taker} \\
& \quad \text{taker-amount Exchange-Normal}
\end{aligned}$$

$$\begin{aligned}
& (\text{exchange-options-at-version ledger-version}) = \\
& \quad \text{Cxx-Ok crossed} \wedge \\
& \quad 0 < \text{sint} (\text{cross-wheat-received crossed}) \wedge \\
& \quad 0 < \text{sint} (\text{cross-sheep-send crossed})))))
\end{aligned}$$

buy-limit-adjustment-stable is *limit-adjustment-stable* for the ManageBuy route, with the same inversion of the canonical price.

definition *offer-limit-adjustment-stable* ::

$$\text{int32} \Rightarrow \text{int32} \Rightarrow \text{int64} \Rightarrow \text{bool}$$

where

$$\text{offer-limit-adjustment-stable price-n price-d amount} \longleftrightarrow$$

$$0 < \text{sint price-n} \wedge$$

$$0 < \text{sint price-d} \wedge$$

$$0 < \text{sint amount} \wedge$$

$$\text{adjust-offer-with-options price-n price-d amount int64-max}$$

$$(\text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False}) =$$

$$\text{Cxx-Ok amount} \wedge$$

$$(\exists \text{buying.}$$

$$\text{offer-buying-liabilities price-n price-d amount} = \text{Cxx-Ok buying} \wedge$$

$$0 < \text{sint buying} \wedge$$

$$(\forall \text{max-sheep-receive.}$$

$$0 \leq \text{sint max-sheep-receive} \wedge$$

$$\text{sint buying} \leq \text{sint max-sheep-receive} \longrightarrow$$

$$(\exists \text{adjusted.}$$

$$\text{adjust-offer-with-options price-n price-d amount max-sheep-receive}$$

$$(\text{exact-receive-cap} = \text{False},$$

$$\text{symmetric-exact-receive-cap} = \text{False}) =$$

$$\text{Cxx-Ok adjusted} \wedge$$

$$0 < \text{sint adjusted})))))$$

offer-limit-adjustment-stable is the non-vacuous, offer-local stability notion used to state completeness of the maker-independent filter. It requires positive inputs and a positive buying liability, requires the unlimited receive cap to preserve the offered amount, and requires every nonnegative receive cap covering that liability to leave a strictly positive adjustment.

This differs deliberately from *limit-adjustment-stable*, which is a global lifecycle property over all successfully posted offers and their maker states. A direct per-offer converse cannot be stated using that global predicate. The local notion also excludes malformed or unpostable offers that would otherwise satisfy a universally quantified stability condition vacuously.

6.7.5 Takeability after maker-state changes

definition *posted-offers-remain-takeable* :: *exchange-options* $\Rightarrow$ *bool*

where

$$\text{posted-offers-remain-takeable options} \longleftrightarrow$$

$$(\forall \text{price-n price-d amount maker-at-post posted maker-after maker-at-cross.}$$

party-state-wf maker-at-post $\wedge$
party-state-wf maker-at-cross $\wedge$
post-offer price-n price-d amount maker-at-post options =
Cxx-Ok (Post-Created posted maker-after) $\wedge$
maker-covers-offer-liabilities price-n price-d posted
maker-at-cross $\longrightarrow$
 $(\exists \text{ taker taker-amount crossed.}$
party-state-wf taker $\wedge$
 $0 < \text{sint (can-buy-at-most taker)} \wedge$
 $0 < \text{sint}$
 $(\text{signed-min64 taker-amount (can-sell-at-most taker)}) \wedge$
cross-offer-v10 price-n price-d posted maker-at-cross taker
taker-amount Exchange-Normal options =
Cxx-Ok crossed $\wedge$
 $0 < \text{sint (cross-wheat-received crossed)} \wedge$
 $0 < \text{sint (cross-sheep-send crossed)}))$

posted-offers-remain-takeable is existential in the taker: it says that every successfully posted offer has some well-formed normal-mode counterparty for which crossing succeeds and transfers strictly positive amounts of both assets. It does not say that every taker must receive a nonzero fill.

The crossing-time maker state is universally quantified independently of the state returned by posting. It may therefore reflect arbitrary intervening balance or limit changes, provided it remains well formed and still covers the posted offer's liabilities. The successful fill may depend on the changed maker state; only existence and positivity are required. The stored-offer theory proves the property at *repaired-exchange-options* as *posted-offers-remain-takeable-repaired* and restates it through the protocol mapping as *posted-offers-remain-takeable-p29*.

definition *posted-buy-offers-remain-takeable* :: *uint32* $\Rightarrow$ *bool*

where

posted-buy-offers-remain-takeable ledger-version $\longleftrightarrow$
 $(\forall \text{ price-n price-d buy-amount maker-at-post posted maker-after}$
maker-at-cross.
party-state-wf maker-at-post $\wedge$
party-state-wf maker-at-cross $\wedge$
post-buy-offer ledger-version price-n price-d buy-amount
maker-at-post = *Cxx-Ok (Post-Created posted maker-after)* $\wedge$
maker-covers-offer-liabilities price-d price-n posted
maker-at-cross $\longrightarrow$
 $(\exists \text{ taker taker-amount crossed.}$
party-state-wf taker $\wedge$
 $0 < \text{sint (can-buy-at-most taker)} \wedge$
 $0 < \text{sint}$
 $(\text{signed-min64 taker-amount (can-sell-at-most taker)}) \wedge$
cross-offer-v10 price-d price-n posted maker-at-cross taker
taker-amount Exchange-Normal
 $(\text{exchange-options-at-version ledger-version}) = \text{Cxx-Ok crossed} \wedge$

$$0 < \text{sint } (\text{cross-wheat-received crossed}) \wedge \\ 0 < \text{sint } (\text{cross-sheep-send crossed}))$$

posted-buy-offers-remain-takeable is *posted-offers-remain-takeable* for the ManageBuy route. It is indexed by a ledger version rather than an option record because *post-buy-offer* is: the same version selects the request liabilities, the posting adjustment, and the crossing arithmetic. The offer's canonical price is the submitted price inverted, so both the coverage hypothesis and the crossing call use (*price-d*, *price-n*).

6.7.6 Exact amount when a posted offer is fully taken

definition *fully-taken-posted-offer-exchanges-posted-amount* ::

exchange-options $\Rightarrow$ bool

where

fully-taken-posted-offer-exchanges-posted-amount options $\longleftrightarrow$
 $(\forall \text{price-n price-d amount maker-at-post posted maker-after}$
 $\text{maker-at-cross taker taker-amount rounding crossed.}$
 $\text{party-state-wf maker-at-post} \wedge$
 $\text{party-state-wf maker-at-cross} \wedge$
 $\text{post-offer price-n price-d amount maker-at-post options} =$
 $\text{Cxx-Ok (Post-Created posted maker-after)} \wedge$
 $\text{maker-covers-offer-liabilities price-n price-d posted}$
 $\text{maker-at-cross} \wedge$
 $\text{cross-offer-v10 price-n price-d posted maker-at-cross taker}$
 $\text{taker-amount rounding options} = \text{Cxx-Ok crossed} \wedge$
 $\neg \text{cross-wheat-stays crossed} \longrightarrow$
 $\text{cross-wheat-received crossed} = \text{posted})$

fully-taken-posted-offer-exchanges-posted-amount formalizes the resting-offer branch that *crossOfferV10* treats as fully taken. The requested amount may have been clipped while posting, so the conclusion uses the actual *posted* amount returned by *post-offer*. The maker state at crossing is independent of the state returned by posting and is constrained only by well-formedness and continued coverage of the posted liabilities. The taker, its operation amount, and the rounding mode are all universally quantified; successful crossing supplies the operational admissibility checks needed for those otherwise arbitrary inputs.

The premise $\neg \text{cross-wheat-stays crossed}$ is the branch condition that erases the resting offer and therefore also forces *cross-offer-amount crossed* = 0. The converse is deliberately not used: when wheat nominally stays, the post-trade dust adjustment can still erase the remainder. The exact receive cap is sufficient: the proof subsection establishes the property for either value of the symmetric cap, then packages the requested record with both flags true. Its proof also explains why the final normal-mode threshold cannot turn this case into a zero trade: successful posting and liability coverage make the positive preventative adjustment replay exactly before the non-staying

exchange. The legacy record with both flags false violates the property through the Alice-and-Carol reservation anomaly proved below.

6.7.7 Exact amount when an incoming offer is fully taken

definition *fully-taken-incoming-offer-exchanges-incoming-amount* ::
exchange-options $\Rightarrow$ bool
where
fully-taken-incoming-offer-exchanges-incoming-amount options $\longleftrightarrow$
 $(\forall \text{ price-n price-d resting-request maker-at-post posted maker-after}$
 $\text{ maker-at-cross taker incoming-amount incoming-receive-cap rounding}$
 crossed.
 $0 < \text{sint price-n} \wedge$
 $0 < \text{sint price-d} \wedge$
 $0 < \text{sint incoming-amount} \wedge$
 $\text{party-state-wf maker-at-post} \wedge$
 $\text{party-state-wf maker-at-cross} \wedge$
 $\text{post-offer price-n price-d resting-request maker-at-post options} =$
 $\text{Cxx-Ok (Post-Created posted maker-after)} \wedge$
 $\text{maker-covers-offer-liabilities price-n price-d posted}$
 $\text{maker-at-cross} \wedge$
 $\text{party-state-wf taker} \wedge$
 $\text{preflight-offer price-d price-n incoming-amount taker} =$
 Cxx-Ok
 $(\text{Preflight-Ready incoming-amount incoming-receive-cap}) \wedge$
 $\text{cross-offer-v10 price-n price-d posted maker-at-cross taker}$
 $\text{incoming-amount rounding options} = \text{Cxx-Ok crossed} \wedge$
 $\text{cross-wheat-stays crossed} \longrightarrow$
 $\text{cross-sheep-send crossed} = \text{incoming-amount})$

fully-taken-incoming-offer-exchanges-incoming-amount is the incoming sheep-selling mirror of the preceding resting-offer property. A valid *ManageSellOffer* request is pre-flighted at the reciprocal price because the incoming party sells sheep and buys wheat. Requiring *preflight-offer price-d price-n incoming-amount taker = Cxx-Ok (Preflight-Ready incoming-amount incoming-receive-cap)* does two important jobs: the ready send cap is the requested amount itself, rather than a request clipped by the taker's available balance, and the request's rounded buying liability has passed the taker's receive-headroom check. Thus *incoming-amount* is the actual operative sheep-send cap supplied to *cross-offer-v10*.

The resting side has the same lifecycle premises as *fully-taken-posted-offer-exchanges-posted-amount*: a well-formed maker successfully posts an actual *posted* amount, and an arbitrary later well-formed maker state still covers that amount's liabilities. The successful cross uses this actual posted amount rather than the raw resting request, so posting-time clipping cannot be mistaken for a later fill.

The exposed flag *cross-wheat-stays crossed* is the crossing branch in which

the resting wheat offer stays and the incoming sheep offer is the limiting side. Nevertheless, the both-repairs claim is false even for a positive exact-price exchange: integer granularity can leave part of the incoming send cap too small to buy another whole unit of wheat while the true wheat-stays flag is preserved. The counterexample at the end of this theory records that distinction explicitly within the full posting and crossing lifecycle. It models the active incoming request before the caller makes any later decision about posting its leftover; unlike the resting side, its raw request has not already passed a stored-offer adjustment.

6.8 Proofs and counterexamples

6.8.1 Rounding favors the offer that stays

The rounding direction is enforced by the final price-error-threshold pass, independently of how the pre-threshold exchange amounts were calculated. Every successful threshold result is either the original result record or an explicit zero-amount record. Consequently, a positive successful exchange must be the original record, and threshold success proves that its value inequality favors the offer marked as staying. In particular, the exact receive cap may change the trade amount but cannot reverse this inequality.

lemma *successful-exchange-favors-offer-that-stays:*

assumes *result:*

*exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive
max-sheep-send max-sheep-receive rounding options =
Cxx-Ok exchange-result*

and *positive:*

$0 < \text{sint } (\text{num-wheat-received } \text{exchange-result}) \wedge$
 $0 < \text{sint } (\text{num-sheep-send } \text{exchange-result})$

shows *avored-seller-ok price-n price-d*

(num-wheat-received exchange-result)
(num-sheep-send exchange-result)
(result-wheat-stays exchange-result)

Proof sketch: expose the successful pre-threshold result and its subsequent threshold call. The threshold result-choice theorem, together with the positive-output premise, rules out the zero record and identifies the returned record with the threshold input. The threshold pass's favored-seller theorem then supplies the required inequality.

6.8.2 Unchanged-maker adjustment stability

The arithmetic fact needed here is that a positive amount returned by adjustment is a fixed point of the same adjustment and remains within its original selling cap. At the lifecycle level, when the state presented at crossing

is exactly the state returned by posting, releasing the offer's own liabilities reverses their acquisition and restores the maker's original capacities. Successful posting and the unchanged-state premise actually make the explicit coverage premise conservative, but retaining it gives the lemma the same premise shape as the intended property.

lemma *adjust-offer-positive-idempotent-false:*

assumes *pn*: $0 < \text{sint price-n}$ **and** *pd*: $0 < \text{sint price-d}$ **and** *wheat-nonnegative*: $0 \leq \text{sint max-wheat}$ **and** *sheep-nonnegative*: $0 \leq \text{sint max-sheep}$ **and** *adjusted*: *adjust-offer-with-options price-n price-d max-wheat max-sheep* (*exact-receive-cap* = *False*, *symmetric-exact-receive-cap* = *False*) = *Cxx-Ok result and result-positive*: $0 < \text{sint result}$

shows *adjust-offer-with-options price-n price-d result max-sheep* (*exact-receive-cap* = *False*, *symmetric-exact-receive-cap* = *False*) = *Cxx-Ok result* $\wedge$ *sint result* $\leq$ *sint max-wheat*

Proof sketch: characterize both adjustments in unbounded integers. A positive first result respects the send cap. In the wheat-more branch it saturates the product used by the second adjustment; in the other branch, floor and ceiling inequalities show that the same sheep units are selected. The threshold inputs are therefore unchanged.

lemma *exchange-v10-amounts-exact-irrelevant-if-not-wheat-more:*

assumes $\neg \text{sint price-n} > \text{sint price-d}$

shows *exchange-v10-amounts price-n price-d wheat-value sheep-value max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive wheat-stays rounding* (*exact-receive-cap* = *True*, *symmetric-exact-receive-cap* = *False*) = *exchange-v10-amounts price-n price-d wheat-value sheep-value max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive wheat-stays rounding* (*exact-receive-cap* = *False*, *symmetric-exact-receive-cap* = *False*)

Proof sketch: when wheat is not the more valuable side, the exact-receive branch is not selected; unfolding the amount calculation makes the two flags identical.

lemma *exchange-v10-exact-irrelevant-if-not-wheat-more:*

assumes $\neg \text{sint price-n} > \text{sint price-d}$

shows *exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive rounding* (*exact-receive-cap* = *True*, *symmetric-exact-receive-cap* = *False*) = *exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive rounding* (*exact-receive-cap* = *False*, *symmetric-exact-receive-cap* = *False*)

Proof sketch: unfold the outer exchange composition and replace its amount calculation with the preceding equality.

lemma *adjust-offer-exact-irrelevant-if-not-wheat-more:*

assumes $\neg \text{ sint price-n} > \text{ sint price-d}$

shows *adjust-offer-with-options price-n price-d max-wheat max-sheep* ($\text{exact-receive-cap} = \text{True}$, $\text{symmetric-exact-receive-cap} = \text{False}$) = *adjust-offer-with-options price-n price-d max-wheat max-sheep* ($\text{exact-receive-cap} = \text{False}$, $\text{symmetric-exact-receive-cap} = \text{False}$)

Proof sketch: unfold adjustment and apply the preceding exchange equality.

lemma *exact-value-word-characterization:*

assumes $\text{pn}: 0 < \text{ sint price-n}$ **and** $\text{pd}: 0 < \text{ sint price-d}$ **and** $\text{send}: 0 \leq \text{ sint max-send}$ **and** $\text{receive}: 0 \leq \text{ sint max-receive}$

shows *calculate-offer-value-with-exact-receive-cap price-n price-d max-send max-receive* = *Cxx-Ok* ($\min (\text{word-of-int} (\text{ sint max-send} * \text{ sint price-n}) :: \text{ uint128}) (\min ((\text{word-of-int} (\text{ sint max-receive} * \text{ sint price-d}) :: \text{ uint128}) + \text{ ucast} (\text{ scast} (\text{ price-d} - 1) :: \text{ int64})) (\text{word-of-int} (\text{ sint int64-max} * \text{ sint price-d}) :: \text{ uint128}))$)

Proof sketch: all operands are non-negative signed words, so the multiplications succeed with their integer products. Unfolding the exact cap leaves the nested minimum displayed in the conclusion.

lemma *adjust-offer-positive-idempotent-true-wheat-more:*

assumes $\text{pn}: 0 < \text{ sint price-n}$ **and** $\text{pd}: 0 < \text{ sint price-d}$ **and** *wheat-more: sint price-n > sint price-d* **and** *wheat-nonnegative: 0 ≤ sint max-wheat* **and** *sheep-nonnegative: 0 ≤ sint max-sheep* **and** *adjusted: adjust-offer-with-options price-n price-d max-wheat max-sheep* ($\text{exact-receive-cap} = \text{True}$, $\text{symmetric-exact-receive-cap} = \text{False}$) = *Cxx-Ok result* **and** *result-positive: 0 < sint result*

shows *adjust-offer-with-options price-n price-d result max-sheep* ($\text{exact-receive-cap} = \text{True}$, $\text{symmetric-exact-receive-cap} = \text{False}$) = *Cxx-Ok result* $\wedge$ *sint result ≤ sint max-wheat*

Proof sketch: in the wheat-more exact-cap branch, divide the clamped trade value by the price numerator. Reapplying adjustment to the positive quotient makes the send-side product the least cap, so both divisions and the price-threshold call repeat unchanged. The runtime checks also yield the send bound.

lemma *exchange-v10-amounts-symmetric-irrelevant-if-not-wheat-stays:*

exchange-v10-amounts price-n price-d wheat-value sheep-value

max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive False

rounding ($\text{exact-receive-cap} = \text{cap}$, $\text{symmetric-exact-receive-cap} = \text{sym-cap}$) =

exchange-v10-amounts price-n price-d wheat-value sheep-value

max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive False
rounding (exact-receive-cap = cap, symmetric-exact-receive-cap = False)

Proof sketch: when the sheep offer stays, the only branch that consults an option field is the wheat-more-valuable one, and it reads *exact-receive-cap*; unfolding the amount calculation makes the two symmetric settings identical.

lemma *adjust-offer-symmetric-irrelevant:*

assumes *pn: 0 < sint price-n and pd: 0 < sint price-d*
and *wheat-nonnegative: 0 ≤ sint max-wheat*
and *sheep-nonnegative: 0 ≤ sint max-sheep*
shows *adjust-offer-with-options price-n price-d max-wheat max-sheep*
(exact-receive-cap = cap, symmetric-exact-receive-cap = sym-cap) =
adjust-offer-with-options price-n price-d max-wheat max-sheep
(exact-receive-cap = cap, symmetric-exact-receive-cap = False)

Proof sketch: adjustment crosses against an unlimited counteroffer, whose sheep value is at least any well-formed wheat value, so the wheat offer never stays and the wheat-stays branch guarded by the symmetric field is unreachable.

lemma *adjust-offer-positive-idempotent:*

assumes *pn: 0 < sint price-n*
and *pd: 0 < sint price-d*
and *wheat-nonnegative: 0 ≤ sint max-wheat*
and *sheep-nonnegative: 0 ≤ sint max-sheep*
and *adjusted:*
adjust-offer-with-options price-n price-d max-wheat max-sheep options =
Cxx-Ok result
and *result-positive: 0 < sint result*
shows
adjust-offer-with-options price-n price-d result max-sheep options =
Cxx-Ok result ∧
sint result ≤ sint max-wheat

Proof sketch: adjustment never reaches the wheat-stays branch, so the symmetric field is first normalized to *False*. Then split on the exact flag and the relative prices: the plain branch uses its fixed-point lemma, and the exact branch either uses the wheat-more lemma or reduces to the plain calculation.

lemma *can-sell-at-most-nonnegative:*

assumes *wf: party-state-wf party*
shows *0 ≤ sint (can-sell-at-most party)*

Proof sketch: the well-formedness invariant makes the available selling balance non-negative and at most the signed-word maximum. The clamping definition therefore returns either zero or a word with that same non-negative signed value.

lemma *can-buy-at-most-nonnegative:*
assumes *wf: party-state-wf party*
shows $0 \leq \text{sint } (\text{can-buy-at-most party})$

Proof sketch: well-formedness makes the unused buying capacity non-negative and bounds it by the trustline limit. The clamping definition therefore preserves its non-negative signed value.

lemma *unchanged-maker-cover-implies-adjust-stable:*
assumes *post-wf: party-state-wf maker-at-post*
and *cross-wf: party-state-wf maker-at-cross*
and *post:*
post-offer price-n price-d amount maker-at-post options =
Cxx-Ok (Post-Created posted maker-after)
and *unchanged: maker-at-cross = maker-after*
and *cover:*
maker-covers-offer-liabilities price-n price-d posted maker-at-cross
and *release:*
release-offer-liabilities price-n price-d posted maker-at-cross =
Cxx-Ok released
shows *adjust-stable price-n price-d posted released options*

Proof sketch: a successful post supplies positive prices and a positive adjusted amount. The fixed-point result above shows that this amount remains unchanged when adjusted against the original capacities. Posting acquires the offer's liabilities; because the maker state at crossing is exactly the returned post state, releasing those liabilities reverses the checked additions and restores the original state. The crossing-time adjustment is consequently the same fixed-point calculation.

6.8.3 Release, replay and capacity lemmas for posted offers

These lemmas support the changed-state properties: the coverage and fully-taken results below, and the protocol-29 takeability results of the stored-offer theory. They recover the positive facts and the liabilities of a successful post, show that liability coverage exposes enough capacity after release for the posted offer and for all guarded balance movements, and show that a positive adjustment result replays unchanged against unlimited caps and against an exact receive cap that covers its booked buying liability.

lemma *post-created-positive-facts*:

assumes *post*:

post-offer price-n price-d amount maker options =

Cxx-Ok (Post-Created posted maker-after)

shows $0 < \text{sint price-n}$ $0 < \text{sint price-d}$ $0 < \text{sint amount}$

$0 < \text{sint posted}$

Proof sketch: a created outcome bypasses the malformed branch, so both price components are positive. It also bypasses the no-offer branch reached when the adjusted amount is non-positive, so the amount written to the book is strictly positive.

lemma *offer-liabilities-result*:

assumes *pn*: $0 < \text{sint price-n}$

and *pd*: $0 < \text{sint price-d}$

and *amount-nonnegative*: $0 \leq \text{sint amount}$

obtains *result* **where**

exchange-v10-without-price-error-thresholds-with-options price-n price-d amount

int64-max int64-max int64-max Exchange-Normal (exact-receive-cap = False,

symmetric-exact-receive-cap = False) = Cxx-Ok result

offer-selling-liabilities price-n price-d amount =

Cxx-Ok (num-wheat-received result)

offer-buying-liabilities price-n price-d amount =

Cxx-Ok (num-sheep-send result)

$0 \leq \text{sint (num-wheat-received result)}$

$\text{sint (num-wheat-received result)} \leq \text{sint amount}$

$0 \leq \text{sint (num-sheep-send result)}$

Proof sketch: with positive prices and a non-negative offer amount, the unlimited liability exchange satisfies the arithmetic precondition and has its canonical successful result. The generic pre-threshold result contract makes both liability amounts non-negative and bounds the selling liability by the offered amount.

lemma *covered-offer-release*:

assumes *pn*: $0 < \text{sint price-n}$

and *pd*: $0 < \text{sint price-d}$

and *amount-nonnegative*: $0 \leq \text{sint amount}$

and *wf*: *party-state-wf maker*

and *cover*:

maker-covers-offer-liabilities price-n price-d amount maker

obtains *selling buying released* **where**

offer-selling-liabilities price-n price-d amount = Cxx-Ok selling

offer-buying-liabilities price-n price-d amount = Cxx-Ok buying

release-offer-liabilities price-n price-d amount maker = Cxx-Ok released

party-state-wf released

$\text{sint selling} \leq \text{sint (can-sell-at-most released)}$
 $\text{sint buying} \leq \text{sint (can-buy-at-most released)}$

Proof sketch: the liability exchange supplies non-negative selling and buying reservations. Coverage says each is contained in the maker's corresponding booked total, while well-formedness bounds those totals by the balance and trustline headroom. Subtracting the reservations therefore passes both checked updates, preserves well-formedness, and exposes at least the released reservation on each capacity side.

lemma *exchange-normal-positive-wheat-has-positive-sheep:*

assumes *exchange:*

$\text{exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive}$
 $\text{max-sheep-send max-sheep-receive Exchange-Normal options =}$
 Cxx-Ok result

and *wheat-positive:* $0 < \text{sint (num-wheat-received result)}$

shows $0 < \text{sint (num-sheep-send result)}$

Proof sketch: the full exchange ends in the common price-threshold function. In normal mode every branch whose input pair is not strictly positive returns the explicit two-zero record. A successful result with positive wheat must therefore be the unchanged positive input record, whose sheep field is also strictly positive. This argument is independent of the receive-cap flag.

lemma *exchange-without-thresholds-bounds-any-cap:*

assumes *result:*

$\text{exchange-v10-without-price-error-thresholds-with-options price-n price-d}$
 $\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive}$
 $\text{rounding options = Cxx-Ok exchange-result}$

shows $0 \leq \text{sint (num-wheat-received exchange-result)}$

$\text{sint (num-wheat-received exchange-result)} \leq$

$\min (\text{sint max-wheat-receive}) (\text{sint max-wheat-send})$

$0 \leq \text{sint (num-sheep-send exchange-result)}$

$\text{sint (num-sheep-send exchange-result)} \leq$

$\min (\text{sint max-sheep-receive}) (\text{sint max-sheep-send})$

Proof sketch: independently of the receive-cap calculation, the pre-threshold exchange passes its amount pair through one common checked constructor. A successful constructor call is exactly the branch in which both signed amounts are non-negative and no larger than either party's cap.

lemma *exchange-normal-positive-result-bounds-any-cap:*

assumes *exchange:*

$\text{exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive}$

$\text{max-sheep-send max-sheep-receive Exchange-Normal options} =$
 Cxx-Ok result
and $\text{wheat-positive: } 0 < \text{sint (num-wheat-received result)}$
shows $\text{sint (num-wheat-received result)} \leq$
 $\text{min (sint max-wheat-receive) (sint max-wheat-send)}$
 $\text{sint (num-sheep-send result)} \leq$
 $\text{min (sint max-sheep-receive) (sint max-sheep-send)}$

Proof sketch: decompose the full exchange into its checked pre-threshold result and the common threshold call. Positive final wheat excludes the threshold function's zero record, so the final record is the original one and inherits both checked cap bounds.

lemma *exchange-against-unlimited-counterparty-does-not-leave-wheat:*
assumes $\text{pn: } 0 < \text{sint price-n}$
and $\text{pd: } 0 < \text{sint price-d}$
and $\text{wheat-nonnegative: } 0 \leq \text{sint max-wheat-send}$
and $\text{sheep-nonnegative: } 0 \leq \text{sint max-sheep-receive}$
and exchange:
 $\text{exchange-v10-with-options price-n price-d max-wheat-send int64-max int64-max}$
 $\text{max-sheep-receive rounding options} = \text{Cxx-Ok result}$
shows $\neg \text{result-wheat-stays result}$

Proof sketch: both maker-side caps are bounded by the signed maximum, so the maker's wheat value is no larger than the value offered by a counterparty with both caps at that maximum. The pre-threshold exchange records this comparison, and the price-threshold step preserves its wheat-stays flag.

lemma *party-receive-within-capacity:*
assumes $\text{wf: party-state-wf party}$
and $\text{delta-positive: } 0 < \text{sint delta}$
and $\text{within: sint delta} \leq \text{sint (can-buy-at-most party)}$
obtains after where
 $\text{party-receive-buy-asset party delta} = \text{Cxx-Ok after}$
 $\text{party-state-wf after}$
 $\text{can-sell-at-most after} = \text{can-sell-at-most party}$

Proof sketch: well-formedness makes the raw unused trustline headroom a non-negative signed value represented exactly by *can-buy-at-most*. The assumed bound therefore passes the guarded balance update. Adding the delta stays below the limit, preserves well-formedness, and does not touch the selling fields or their capacity.

lemma *party-spend-within-capacity*:
assumes *wf*: *party-state-wf party*
and *delta-positive*: $0 < \text{sint } \text{delta}$
and *within*: $\text{sint } \text{delta} \leq \text{sint } (\text{can-sell-at-most } \text{party})$
obtains after where
party-spend-sell-asset party delta = Cxx-Ok after
party-state-wf after

Proof sketch: well-formedness makes the uncommitted selling balance a non-negative signed value represented exactly by *can-sell-at-most*. Spending within it leaves at least the booked selling liabilities, so the guarded subtraction succeeds and the updated party remains well formed.

lemma *positive-price-denominator-minus-one-cast*:
fixes *price-d* :: *int32*
assumes $0 < \text{sint } \text{price-d}$
shows $\text{sint } (\text{scast } (\text{price-d} - 1) :: \text{int64}) = \text{sint } \text{price-d} - 1$

Proof sketch: subtracting one from a positive signed 32-bit denominator does not wrap; widening that non-negative result to 64 bits preserves its signed integer value.

lemma *uint-eq-sint-nonnegative-int64*:
fixes *word* :: *int64*
assumes $0 \leq \text{sint } \text{word}$
shows $\text{uint } \text{word} = \text{sint } \text{word}$

Proof sketch: a non-negative signed 64-bit value lies in the unsigned 64-bit range, so the unsigned truncation in $\text{uint } ?w = \text{take-bit } \text{LENGTH}(?'a) (\text{sint } ?w)$ is the identity.

lemma *exact-cap-replays-unlimited-positive-adjustment*:
assumes *pn*: $0 < \text{sint } \text{price-n}$
and *pd*: $0 < \text{sint } \text{price-d}$
and *amount-positive*: $0 < \text{sint } \text{amount}$
and *receive-nonnegative*: $0 \leq \text{sint } \text{max-sheep-receive}$
and *unlimited*:
 $\text{adjust-offer-with-options } \text{price-n } \text{price-d } \text{amount } \text{int64-max } (\text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False}) = \text{Cxx-Ok } \text{amount}$
and *buying*:
 $\text{offer-buying-liabilities } \text{price-n } \text{price-d } \text{amount} = \text{Cxx-Ok } \text{buying}$
and *buying-fits*: $\text{sint } \text{buying} \leq \text{sint } \text{max-sheep-receive}$
shows $\text{adjust-offer-with-options } \text{price-n } \text{price-d } \text{amount } \text{max-sheep-receive } (\text{exact-receive-cap} = \text{True}, \text{symmetric-exact-receive-cap} = \text{False}) = \text{Cxx-Ok } \text{amount}$

Proof sketch: the unlimited successful adjustment applies the normal price threshold to the offer-liability amount pair and, because its wheat result is positive, preserves that pair. Thus *buying* is exactly the sheep amount paired with *amount*. If wheat is the more valuable side, the exact receive cap adds the denominator-minus-one slack that makes this floor payment sufficient for all of *amount*; otherwise the exact flag is inert and the ceiling conversion already replays the same pair. The common threshold call then returns the original positive result.

lemma *positive-adjustment-replays-unlimited:*

assumes *pn*: $0 < \text{sint price-}n$
and *pd*: $0 < \text{sint price-}d$
and *wheat-nonnegative*: $0 \leq \text{sint max-wheat}$
and *sheep-nonnegative*: $0 \leq \text{sint max-sheep}$
and *adjusted*:
 adjust-offer-with-options price-n price-d max-wheat max-sheep (*exact-receive-cap* = *True*, *symmetric-exact-receive-cap* = *False*) =
 Cxx-Ok result
and *result-positive*: $0 < \text{sint result}$
shows *adjust-offer-with-options price-n price-d result int64-max* (*exact-receive-cap* = *False*, *symmetric-exact-receive-cap* = *False*) =
 Cxx-Ok result

Proof sketch: first use positive idempotence to replay the adjusted amount against its original receive cap. When wheat is not more valuable, the exact flag is inert; the ceiling formula uniquely determines the same sheep units even after raising the receive cap to the signed maximum. When wheat is more valuable, the successful exact calculation proves that the full result-price product fits the global cap; the unlimited plain calculation therefore repeats the same two divisions and the same successful threshold call.

lemma *unlimited-positive-adjustment-identifies-liabilities:*

assumes *amount-positive*: $0 < \text{sint amount}$
and *unlimited*:
 adjust-offer-with-options price-n price-d amount int64-max (*exact-receive-cap* = *False*, *symmetric-exact-receive-cap* = *False*) = *Cxx-Ok amount*
and *selling*:
 offer-selling-liabilities price-n price-d amount = *Cxx-Ok selling*
and *buying*:
 offer-buying-liabilities price-n price-d amount = *Cxx-Ok buying*
shows *selling* = *amount* $0 < \text{sint buying}$

Proof sketch: the unlimited adjustment and both liability helpers share the same pre-threshold exchange call. Positive final wheat excludes normal mode's explicit two-zero threshold result, so the threshold preserves the

pre-threshold record. Its wheat field is therefore the adjusted amount and its sheep field is strictly positive, exactly identifying both liabilities.

lemma *adjust-offer-congruent-sint-receive-cap:*
assumes *pn: 0 < sint price-n*
and *pd: 0 < sint price-d*
and *amount-nonnegative: 0 ≤ sint amount*
and *first-nonnegative: 0 ≤ sint first-cap*
and *second-nonnegative: 0 ≤ sint second-cap*
and *cap-sint: sint first-cap = sint second-cap*
shows *adjust-offer-with-options price-n price-d amount first-cap*
(exact-receive-cap = False, symmetric-exact-receive-cap = False) =
adjust-offer-with-options price-n price-d amount second-cap
(exact-receive-cap = False, symmetric-exact-receive-cap = False)

Proof sketch: the exchange reads receive caps only through their signed integer values. Equal signed caps therefore give equal clipped wheat values, integer amount pairs, stays flags, and normal-mode adjustment results.

lemma *unlimited-adjustment-explicit-filter-conditions:*
assumes *pn: 0 < sint price-n*
and *pd: 0 < sint price-d*
and *amount-positive: 0 < sint amount*
and *unlimited:*
adjust-offer-with-options price-n price-d amount int64-max
(exact-receive-cap = False, symmetric-exact-receive-cap = False) =
Cxx-Ok amount
shows
*sint amount * sint price-n ≤*
*int64-max-int * sint price-d ∧*
(let buying =
*(sint amount * sint price-n) div sint price-d*
in let wheat =
(if sint price-n > sint price-d
*then (buying * sint price-d) div sint price-n*
*else (buying * sint price-d + sint price-n - 1) div sint price-n)*
in (sint price-n > sint price-d ∨ wheat = sint amount) ∧
0 < buying ∧
*abs (100 * sint price-n * sint amount -*
*100 * sint price-d * buying) ≤*
*sint price-n * sint amount)*

Proof sketch: invert the positive unlimited adjustment through the normal exchange characterization. Its wheat field equals the input amount, which forces the price product below the signed receive cap and identifies the in-

teger amount pair with the full offer and its floor-rounded buying amount. Positivity and the successful threshold branch then give the remaining explicit arithmetic conjuncts.

lemma *minimum-adjustment-explicit-filter-conditions:*
assumes *pn: 0 < sint price-n*
and *pd: 0 < sint price-d*
and *amount-positive: 0 < sint amount*
and *unsaturated:*
 $\text{sint amount} * \text{sint price-n} \leq$
 $\text{int64-max-int} * \text{sint price-d}$
and *minimum:*
 $\text{adjust-offer-with-options price-n price-d amount}$
 $(\text{word-of-int}$
 $((\text{sint amount} * \text{sint price-n}) \text{div sint price-d}))$
 $(\text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False}) =$
 Cxx-Ok adjusted
and *adjusted-positive: 0 < sint adjusted*
shows
 $(\text{let buying} =$
 $(\text{sint amount} * \text{sint price-n}) \text{div sint price-d}$
 $\text{in let wheat} =$
 $(\text{if sint price-n} > \text{sint price-d}$
 $\text{then } (\text{buying} * \text{sint price-d}) \text{div sint price-n}$
 $\text{else } (\text{buying} * \text{sint price-d} + \text{sint price-n} - 1) \text{div sint price-n})$
 $\text{in let sheep} =$
 $(\text{if sint price-n} > \text{sint price-d}$
 $\text{then } (\text{wheat} * \text{sint price-n}) \text{div sint price-d}$
 $\text{else buying})$
 $\text{in } 0 < \text{wheat} \wedge$
 $0 < \text{sheep} \wedge$
 $\text{abs } (100 * \text{sint price-n} * \text{wheat} -$
 $100 * \text{sint price-d} * \text{sheep}) \leq \text{sint price-n} * \text{wheat})$

Proof sketch: the cap is the explicit floor-rounded buying liability. Its signed representation is in range, and Euclidean division makes the clipped wheat value exactly the floor times the denominator. The normal exchange therefore computes exactly the filter's *wheat* and *sheep* pair. Inverting the positive successful adjustment gives positivity of both components and its successful one-percent threshold check.

lemma *unlimited-adjustment-identifies-buying-liability:*
assumes *pn: 0 < sint price-n*
and *pd: 0 < sint price-d*
and *amount-positive: 0 < sint amount*
and *unlimited:*

adjust-offer-with-options price-n price-d amount int64-max
(exact-receive-cap = False, symmetric-exact-receive-cap = False) =
Cxx-Ok amount
shows *offer-buying-liabilities price-n price-d amount =*
Cxx-Ok
(word-of-int
*((sint amount * sint price-n) div sint price-d))*

Proof sketch: the explicit unlimited conditions identify the pre-threshold normal amount pair as the full offered wheat amount and the floor-rounded buying amount. The liability helper selects the sheep field of exactly that pre-threshold exchange.

lemma *acquired-offer-is-covered:*
assumes *wf: party-state-wf maker*
and *acquire:*
acquire-offer-liabilities price-n price-d amount maker =
Cxx-Ok maker-after
shows *maker-covers-offer-liabilities price-n price-d amount maker-after*

Proof sketch: successful acquisition adds the offer's two nonnegative liabilities to the maker's existing nonnegative totals under checked caps. Decoding the checked sums therefore shows that each new total covers the corresponding offer liability.

lemma *post-created-acquires-posted-liabilities:*
assumes *post:*
post-offer price-n price-d amount maker options =
Cxx-Ok (Post-Created posted maker-after)
shows *acquire-offer-liabilities price-n price-d posted maker =*
Cxx-Ok maker-after

Proof sketch: invert the successful-created branch of *post-offer*. Its adjusted amount is exactly *posted*, and the party state paired with that amount is exactly the result of acquiring its liabilities.

lemma *post-created-replays-unlimited:*
assumes *post-wf: party-state-wf maker-at-post*
and *post:*
post-offer price-n price-d amount maker-at-post (exact-receive-cap = True,
symmetric-exact-receive-cap = False) =
Cxx-Ok (Post-Created posted maker-after)
shows *adjust-offer-with-options price-n price-d posted int64-max (exact-receive-*
cap = False, symmetric-exact-receive-cap = False) =
Cxx-Ok posted

Proof sketch: unfold the successful post to recover the exact positive adjustment result that produced the posted amount. Party well-formedness makes both posting capacities non-negative. The general replay lemma then shows that the positive result is unchanged when both capacities are made unlimited; at unlimited receive capacity the exact flag is inert.

6.8.4 Maximality of positive normal crossings with both repairs

This subsection proves the intended property *positive-normal-crosses-are-maximal* for the options record with both receive-cap repairs enabled. The development mirrors the plain-cap integer characterization: the two corrected normal branches are characterized exactly at the word level, the resulting integer amount pair is bounded by all four caps, the characterization is lifted through *exchange-v10-with-options*, and finally every returned positive pair is shown to be a *normal-favored-candidate* that dominates all candidates.

lemma *int-le-div-from-product*:

fixes *lower numerator divisor* :: *int*
assumes *divisor-positive*: $0 < \text{divisor}$
and *product-bound*: $\text{lower} * \text{divisor} \leq \text{numerator}$
shows $\text{lower} \leq \text{numerator} \text{ div } \text{divisor}$

Proof sketch: the lower value is its own product's quotient, and integer division by a positive divisor is monotone in the numerator.

lemma *int-div-below-next-multiple*:

fixes *numerator divisor bound* :: *int*
assumes *divisor-positive*: $0 < \text{divisor}$
and *numerator-bound*: $\text{numerator} \leq \text{bound} * \text{divisor} + \text{divisor} - 1$
shows $\text{numerator} \text{ div } \text{divisor} \leq \text{bound}$

Proof sketch: division monotonicity compares the numerator against the largest value below the next multiple of the divisor, whose quotient is exactly the bound.

lemma *int-div-le-imp-below-next-multiple*:

fixes *numerator divisor bound* :: *int*
assumes *divisor-positive*: $0 < \text{divisor}$
and *quotient-bound*: $\text{numerator} \text{ div } \text{divisor} \leq \text{bound}$
shows $\text{numerator} \leq \text{bound} * \text{divisor} + \text{divisor} - 1$

Proof sketch: decompose the numerator into quotient multiple and remainder; the remainder is below the divisor and the quotient multiple is below the bound multiple.

lemma *exact-receive-cap-value-int*:
assumes *pn*: $0 < \text{sint price-n}$ **and** *pd*: $0 < \text{sint price-d}$
and *send-nonnegative*: $0 \leq \text{sint max-send}$
and *receive-nonnegative*: $0 \leq \text{sint max-receive}$
obtains *trade-word* :: *uint128* **where**
calculate-offer-value-with-exact-receive-cap price-n price-d max-send
max-receive = *Cxx-Ok trade-word*
uint trade-word =
 $\min (\text{sint max-send} * \text{sint price-n})$
 $(\min (\text{sint max-receive} * \text{sint price-d} + \text{sint price-d} - 1)$
 $(\text{sint int64-max} * \text{sint price-d}))$

Proof sketch: the word-level characterization gives a nested minimum of three 128-bit values. The two products are exact by the multiplication range theorem, the relaxation addend is the denominator minus one, and the receive product is far below the 128-bit modulus, so the relaxed receive value is an exact sum and the unsigned minimum commutes with *uint*.

lemma *exchange-v10-amounts-wheat-stays-down-repaired*:
fixes *wheat-word sheep-word* :: *uint128*
and *trade sheep-send wheat-receive* :: *int*
assumes *pn*: $0 < \text{sint price-n}$ **and** *pd*: $0 < \text{sint price-d}$
and *sheep-more*: $\neg \text{sint price-n} > \text{sint price-d}$
and *receive-nonnegative*: $0 \leq \text{sint max-wheat-receive}$
and *send-nonnegative*: $0 \leq \text{sint max-sheep-send}$
and *symmetric*: *symmetric-exact-receive-cap options*
defines *trade* $\equiv$
 $\min (\text{sint max-sheep-send} * \text{sint price-d})$
 $(\min (\text{sint max-wheat-receive} * \text{sint price-n} + \text{sint price-n} - 1)$
 $(\text{sint int64-max} * \text{sint price-n}))$
and *sheep-send* $\equiv \text{trade div sint price-d}$
and *wheat-receive* $\equiv \text{sheep-send} * \text{sint price-d div sint price-n}$
shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive True
Exchange-Normal options =
Cxx-Ok (word-of-int wheat-receive, word-of-int sheep-send)
and *sint (word-of-int wheat-receive :: int64)* = *wheat-receive*
and *sint (word-of-int sheep-send :: int64)* = *sheep-send*

Proof sketch: the symmetric exact cap values the trade by the reversed exact receive-cap calculation. Its value is at most the sheep-send product, so the first round-down division fits signed 64 bits; the resulting sheep multiple is at most the relaxed wheat-receive component, so the second round-down division is bounded by the wheat-receive cap.

```

lemma exchange-v10-amounts-sheep-stays-wheat-more-repaired:
  fixes wheat-word sheep-word :: uint128
  and trade wheat-receive sheep-send :: int
assumes pn:  $0 < \text{sint } \text{price-n}$  and pd:  $0 < \text{sint } \text{price-d}$ 
  and wheat-more:  $\text{sint } \text{price-n} > \text{sint } \text{price-d}$ 
  and send-nonnegative:  $0 \leq \text{sint } \text{max-wheat-send}$ 
  and receive-nonnegative:  $0 \leq \text{sint } \text{max-sheep-receive}$ 
  and exact: exact-receive-cap options
defines trade  $\equiv$ 
   $\min (\text{sint } \text{max-wheat-send} * \text{sint } \text{price-n})$ 
   $(\min (\text{sint } \text{max-sheep-receive} * \text{sint } \text{price-d} + \text{sint } \text{price-d} - 1)$ 
   $(\text{sint } \text{int64-max} * \text{sint } \text{price-d}))$ 
and wheat-receive  $\equiv \text{trade div sint } \text{price-n}$ 
and sheep-send  $\equiv \text{wheat-receive} * \text{sint } \text{price-n div sint } \text{price-d}$ 
shows exchange-v10-amounts price-n price-d wheat-word sheep-word
  max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive False
  rounding options =
  Cxx-Ok (word-of-int wheat-receive, word-of-int sheep-send)
and sint (word-of-int wheat-receive :: int64) = wheat-receive
and sint (word-of-int sheep-send :: int64) = sheep-send

```

Proof sketch: the exact receive cap values the trade by the exact calculation on the wheat offer's caps. Its value is at most the wheat-send product, so the first round-down division fits signed 64 bits; the resulting wheat multiple is at most the relaxed sheep-receive component, so the second round-down division is bounded by the sheep-receive cap.

```

definition exchange-v10-amounts-int-repaired ::
  int32  $\Rightarrow$  int32  $\Rightarrow$  int64  $\Rightarrow$  int64  $\Rightarrow$  int64  $\Rightarrow$ 
  int64  $\Rightarrow$  int  $\times$  int
where
  exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
  max-wheat-receive max-sheep-send max-sheep-receive =
  (let wheat-value =
    exchange-wheat-value-int price-n price-d max-wheat-send
    max-sheep-receive;
    sheep-value =
    exchange-sheep-value-int price-n price-d max-sheep-send
    max-wheat-receive;
    wheat-stays = wheat-value > sheep-value
  in if wheat-stays then
    if sint price-n > sint price-d then
      let wheat-receive = sheep-value div sint price-n
      in (wheat-receive,
        (wheat-receive * sint price-n + sint price-d - 1)
        div sint price-d)
    else

```

```

let trade =
  min (sint max-sheep-send * sint price-d)
    (min (sint max-wheat-receive * sint price-n +
          sint price-n - 1)
        (sint int64-max * sint price-n));
  sheep-send = trade div sint price-d
  in (sheep-send * sint price-d div sint price-n, sheep-send)
else if sint price-n > sint price-d then
  let trade =
    min (sint max-wheat-send * sint price-n)
      (min (sint max-sheep-receive * sint price-d +
            sint price-d - 1)
          (sint int64-max * sint price-d));
    wheat-receive = trade div sint price-n
  in (wheat-receive,
      wheat-receive * sint price-n div sint price-d)
else
  let sheep-send = wheat-value div sint price-d
  in ((sheep-send * sint price-d + sint price-n - 1)
      div sint price-n,
      sheep-send))

```

exchange-v10-amounts-int-repaired is the normal-mode amount pair computed with both receive-cap repairs enabled, phrased in unbounded integers. The two branches selected when the staying offer is the more valuable side coincide with *exchange-v10-amounts-int*; the two corrected branches replace the plain offer value by the exact receive-cap minimum of the send-side product, the receive cap relaxed by one divisor unit, and the *int64-max* saturation ceiling.

lemma *repaired-wheat-stays-down-bounds*:

fixes *price-n price-d imax max-wheat-send max-wheat-receive max-sheep-send*
max-sheep-receive trade sheep-send wheat-receive :: int

assumes *pn*: $0 < \text{price-n}$ **and** *pd*: $0 < \text{price-d}$

and *sheep-more*: $\text{price-n} \leq \text{price-d}$

and *imax-nonnegative*: $0 \leq \text{imax}$

and *receive-nonnegative*: $0 \leq \text{max-wheat-receive}$

and *send-nonnegative*: $0 \leq \text{max-sheep-send}$

and *wheat-stays*:

$\min (\text{max-sheep-send} * \text{price-d}) (\text{max-wheat-receive} * \text{price-n}) <$
 $\min (\text{max-wheat-send} * \text{price-n}) (\text{max-sheep-receive} * \text{price-d})$

defines *trade* $\equiv$

$\min (\text{max-sheep-send} * \text{price-d})$
 $(\min (\text{max-wheat-receive} * \text{price-n} + \text{price-n} - 1) (\text{imax} * \text{price-n}))$

and *sheep-send* $\equiv \text{trade div price-d}$

and *wheat-receive* $\equiv \text{sheep-send} * \text{price-d div price-n}$

shows $0 \leq \text{wheat-receive}$

and *wheat-receive* $\leq \min \text{max-wheat-receive max-wheat-send}$

and $0 \leq \text{sheep-send}$

and $\text{sheep-send} \leq \min \text{max-sheep-receive max-sheep-send}$

Proof sketch: the exact trade value exceeds the plain sheep value by at most $\text{price-n} - 1$, and the plain sheep value is strictly below the wheat value, which both opposite-side caps bound. Since the price numerator is at most the denominator, each floor quotient stays within the two caps of its output asset.

lemma *repaired-sheep-stays-wheat-more-bounds:*

fixes $\text{price-n price-d imax max-wheat-send max-wheat-receive max-sheep-send}$
 $\text{max-sheep-receive trade wheat-receive sheep-send} :: \text{int}$

assumes $\text{pn: } 0 < \text{price-n}$ **and** $\text{pd: } 0 < \text{price-d}$

and $\text{wheat-more: price-d} < \text{price-n}$

and $\text{imax-nonnegative: } 0 \leq \text{imax}$

and $\text{send-nonnegative: } 0 \leq \text{max-wheat-send}$

and $\text{receive-nonnegative: } 0 \leq \text{max-sheep-receive}$

and sheep-stays:

$\min (\text{max-wheat-send} * \text{price-n}) (\text{max-sheep-receive} * \text{price-d}) \leq$

$\min (\text{max-sheep-send} * \text{price-d}) (\text{max-wheat-receive} * \text{price-n})$

defines $\text{trade} \equiv$

$\min (\text{max-wheat-send} * \text{price-n})$

$(\min (\text{max-sheep-receive} * \text{price-d} + \text{price-d} - 1) (\text{imax} * \text{price-d}))$

and $\text{wheat-receive} \equiv \text{trade} \text{ div } \text{price-n}$

and $\text{sheep-send} \equiv \text{wheat-receive} * \text{price-n} \text{ div } \text{price-d}$

shows $0 \leq \text{wheat-receive}$

and $\text{wheat-receive} \leq \min \text{max-wheat-receive max-wheat-send}$

and $0 \leq \text{sheep-send}$

and $\text{sheep-send} \leq \min \text{max-sheep-receive max-sheep-send}$

Proof sketch: this is the mirror of the wheat-stays argument. The exact trade value exceeds the plain wheat value by at most $\text{price-d} - 1$, the plain wheat value is at most the sheep value, and the price denominator is strictly below the numerator, so each floor quotient stays within the two caps of its output asset.

lemma *exchange-v10-amounts-int-repaired-bounds:*

assumes $\text{pre: exchange-v10-pre price-n price-d max-wheat-send}$

$\text{max-wheat-receive max-sheep-send max-sheep-receive}$

shows $0 \leq \text{fst} (\text{exchange-v10-amounts-int-repaired price-n price-d}$

$\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive})$

and $\text{fst} (\text{exchange-v10-amounts-int-repaired price-n price-d}$

$\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive}) \leq$

$\min (\text{sint max-wheat-receive}) (\text{sint max-wheat-send})$

and $0 \leq \text{snd} (\text{exchange-v10-amounts-int-repaired price-n price-d}$

$\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive})$

and $\text{snd} (\text{exchange-v10-amounts-int-repaired price-n price-d}$

$$\text{max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive} \leq \min (\text{sint max-sheep-receive}) (\text{sint max-sheep-send})$$

Proof sketch: split on the offer-value comparison and the price order. The two branches whose staying offer is the more valuable side reuse the plain-cap branch bound lemmas; the two corrected branches are discharged by the repaired bound lemmas above.

theorem *exchange-v10-amounts-repaired-integer-characterization:*
fixes *wheat-word sheep-word :: uint128 and amounts :: int × int*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *wheat-word* $\equiv$
 $\min$
 (*word-of-int (sint max-wheat-send * sint price-n) :: uint128*)
 (*word-of-int (sint max-sheep-receive * sint price-d)*)
and *sheep-word* $\equiv$
 $\min$
 (*word-of-int (sint max-sheep-send * sint price-d) :: uint128*)
 (*word-of-int (sint max-wheat-receive * sint price-n)*)
and *amounts* $\equiv$
exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive
shows *exchange-v10-amounts price-n price-d wheat-word sheep-word*
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
(wheat-word > sheep-word) Exchange-Normal
(exact-receive-cap = True, symmetric-exact-receive-cap = True) =
Cxx-Ok
(word-of-int (fst amounts), word-of-int (snd amounts))
and *sint (word-of-int (fst amounts) :: int64) = fst amounts*
and *sint (word-of-int (snd amounts) :: int64) = snd amounts*

Proof sketch: identify both 128-bit values with their exact integers, then split on the value comparison and the price ordering. The unaffected branches use the existing plain-cap branch equations; the two corrected branches use the repaired branch equations. The repaired bounds theorem makes both re-encoded result words faithful signed-64 values.

theorem *exchange-v10-without-repaired-integer-characterization:*
fixes *amounts :: int × int*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *amounts* $\equiv$
exchange-v10-amounts-int-repaired price-n price-d max-wheat-send

```

    max-wheat-receive max-sheep-send max-sheep-receive
shows exchange-v10-without-price-error-thresholds-with-options price-n price-d
    max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
    Exchange-Normal
    (exact-receive-cap = True, symmetric-exact-receive-cap = True) =
    Cxx-Ok
    (make-exchange-result
      (word-of-int (fst amounts)) (word-of-int (snd amounts))
      (exchange-wheat-value-int price-n price-d max-wheat-send
        max-sheep-receive >
        exchange-sheep-value-int price-n price-d max-sheep-send
        max-wheat-receive))

```

Proof sketch: both offer-value calls succeed with their exact 128-bit minima under *exchange-v10-pre*. Substitute the repaired amount characterization, then use its signed interpretations and the repaired bounds to prove that neither final runtime check is reachable.

```

theorem exchange-v10-repaired-integer-characterization:
  fixes amounts :: int × int
  assumes pre: exchange-v10-pre price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive
  defines amounts ≡
    exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive
  shows exchange-v10-with-options price-n price-d max-wheat-send max-wheat-
    receive
    max-sheep-send max-sheep-receive Exchange-Normal
    (exact-receive-cap = True, symmetric-exact-receive-cap = True) =
    apply-price-error-thresholds price-n price-d
    (word-of-int (fst amounts)) (word-of-int (snd amounts))
    (exchange-wheat-value-int price-n price-d max-wheat-send
      max-sheep-receive >
      exchange-sheep-value-int price-n price-d max-sheep-send
      max-wheat-receive)
    Exchange-Normal

```

Proof sketch: the repaired pre-threshold characterization replaces the first call by its exact amount record; simplifying the record selectors leaves the threshold call shown here.

6.9 An options-parametric exact-integer abstraction

exchange-v10-amounts-int hard-codes the legacy branch formulas and *exchange-v10-amounts-int-repaired* the repaired ones in normal mode only. Neither covers a strict-mode call at the repaired record, which is what the path-payment routes issue from protocol 29 on. This subsection closes the

gap with one abstraction carrying both the rounding mode and the options record, so that the whole five-branch calculation has a single exact integer image and each existing abstraction is one of its instances.

Only two branches are option-sensitive, and they are the two the repair changes: the round-down sheep-send branch taken when the wheat offer stays and sheep is at least as valuable (*symmetric-exact-receive-cap*), and the round-down wheat-receive branch taken when the wheat offer does not stay and wheat is more valuable (*exact-receive-cap*). Both replace a plain offer value by the protocol-29 helper *calculate-offer-amount-from-value*, whose exact image is defined first.

definition *calculate-offer-amount-from-value-int* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *int*

where

calculate-offer-amount-from-value-int price-n price-d max-send
max-receive =
 $\min$ (*sint max-send* * *sint price-n*)
 $(\min$ (*sint max-receive* * *sint price-d* + *sint price-d* - 1)
 $(\text{sint int64-max} * \text{sint price-d}))$
 div *sint price-n*

calculate-offer-amount-from-value-int is the exact-integer image of *calculate-offer-amount-from-value*: the send-side product, the receive-side product relaxed by one divisor unit, the *int64-max* saturation ceiling, and the single round-down division that the protocol-29 helper performs in place of the caller's second rounding.

lemma *calculate-offer-amount-from-value-int-correspondence*:

assumes *pn*: $0 < \text{sint price-n}$ **and** *pd*: $0 < \text{sint price-d}$

and *send-nonnegative*: $0 \leq \text{sint max-send}$

and *receive-nonnegative*: $0 \leq \text{sint max-receive}$

shows *calculate-offer-amount-from-value price-n price-d max-send*

max-receive =

Cxx-Ok

(*word-of-int*

(*calculate-offer-amount-from-value-int price-n price-d max-send*
max-receive))

and *sint*

(*word-of-int*

(*calculate-offer-amount-from-value-int price-n price-d max-send*
max-receive) :: *int64*) =

calculate-offer-amount-from-value-int price-n price-d max-send max-receive

Proof sketch: the relaxed offer value is the exact nested minimum of the three products. That minimum is at most the send-side product, so the round-down division by the numerator has a signed-64 quotient and the checked helper returns exactly the integer formula and its faithful signed interpretation.

definition *exchange-v10-amounts-int-with-options* ::
 $int32 \Rightarrow int32 \Rightarrow int64 \Rightarrow int64 \Rightarrow int64 \Rightarrow$
 $int64 \Rightarrow exchange-rounding \Rightarrow exchange-options \Rightarrow int \times int$
where
exchange-v10-amounts-int-with-options price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding options =
(let wheat-value =
exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive;
sheep-value =
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive;
wheat-stays = wheat-value > sheep-value
in if wheat-stays then
if rounding = Exchange-Strict-Send then
(sheep-value div sint price-n,
min (sint max-sheep-send) (sint max-sheep-receive))
else if sint price-n > sint price-d $\vee$
rounding = Exchange-Strict-Receive
then
let wheat-receive = sheep-value div sint price-n
in (wheat-receive,
(wheat-receive * sint price-n + sint price-d - 1)
div sint price-d)
else
let sheep-send =
(if symmetric-exact-receive-cap options
then calculate-offer-amount-from-value-int price-d price-n
max-sheep-send max-wheat-receive
else sheep-value div sint price-d)
in ((sheep-send * sint price-d) div sint price-n, sheep-send)
else if sint price-n > sint price-d then
let wheat-receive =
(if exact-receive-cap options
then calculate-offer-amount-from-value-int price-n price-d
max-wheat-send max-sheep-receive
else wheat-value div sint price-n)
in (wheat-receive,
(wheat-receive * sint price-n) div sint price-d)
else
let sheep-send = wheat-value div sint price-d
in ((sheep-send * sint price-d + sint price-n - 1)
div sint price-n,
sheep-send))

exchange-v10-amounts-int-with-options is *exchange-v10-amounts-int* with the two option-sensitive branches switched on the corresponding flag. Every

other branch is copied verbatim, which is what makes the two collapse equations below hold by unfolding alone.

lemma *exchange-v10-amounts-int-with-options-legacy* [simp]:

*exchange-v10-amounts-int-with-options price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding
legacy-exchange-options =
exchange-v10-amounts-int price-n price-d max-wheat-send max-wheat-receive
max-sheep-send max-sheep-receive rounding*

— At the legacy record the new abstraction is the established one, so no existing result changes meaning.

lemma *exchange-v10-amounts-int-with-options-repaired-normal* [simp]:

*exchange-v10-amounts-int-with-options price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive Exchange-Normal
repaired-exchange-options =
exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive*

— At the repaired record in normal mode it is the established repaired abstraction.

lemma *exchange-v10-amounts-int-with-options-bounds*:

assumes pre: *exchange-v10-pre price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive*

shows $0 \leq fst$ (*exchange-v10-amounts-int-with-options price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding options*)

and fst (*exchange-v10-amounts-int-with-options price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding options*) $\leq$
 min (*sint max-wheat-receive*) (*sint max-wheat-send*)

and $0 \leq snd$ (*exchange-v10-amounts-int-with-options price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding options*)

and snd (*exchange-v10-amounts-int-with-options price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding options*) $\leq$
 min (*sint max-sheep-receive*) (*sint max-sheep-send*)

Proof sketch: split on the offer-value comparison, the rounding mode, the price order, and the relevant flag. The five plain branches reuse the established plain-cap bound lemmas; the two corrected branches reuse the repaired ones, whose trade expressions are exactly the exact-image helper unfolded.

lemma *exchange-v10-amounts-sheep-stays-wheat-more-plain*:

assumes pre: *exchange-v10-pre price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive*

and *wheat-value*:

```

uint wheat-word =
  exchange-wheat-value-int price-n price-d max-wheat-send
  max-sheep-receive
and wheat-more: sint price-n > sint price-d
and plain-cap:  $\neg$  exact-receive-cap options
shows exchange-v10-amounts price-n price-d wheat-word sheep-word
  max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive False
  rounding options =
  (let wheat-receive =
    exchange-wheat-value-int price-n price-d max-wheat-send
    max-sheep-receive div sint price-n;
    sheep-send = wheat-receive * sint price-n div sint price-d
    in Cxx-Ok (word-of-int wheat-receive, word-of-int sheep-send))
— The plain-cap form of the non-staying wheat-more branch, generic in the
options record rather than pinned to the legacy one.

```

Proof sketch: with the exact receive cap disabled this branch consults no other field of the options record, so the call agrees with the legacy call by unfolding, and the established legacy branch equation applies.

```

theorem exchange-v10-amounts-with-options-integer-characterization:
fixes wheat-word sheep-word :: uint128 and amounts :: int  $\times$  int
and rounding :: exchange-rounding and options :: exchange-options
assumes pre: exchange-v10-pre price-n price-d max-wheat-send
  max-wheat-receive max-sheep-send max-sheep-receive
defines wheat-word  $\equiv$ 
  min
    (word-of-int (sint max-wheat-send * sint price-n) :: uint128)
    (word-of-int (sint max-sheep-receive * sint price-d))
and sheep-word  $\equiv$ 
  min
    (word-of-int (sint max-sheep-send * sint price-d) :: uint128)
    (word-of-int (sint max-wheat-receive * sint price-n))
and amounts  $\equiv$ 
  exchange-v10-amounts-int-with-options price-n price-d max-wheat-send
  max-wheat-receive max-sheep-send max-sheep-receive rounding options
shows exchange-v10-amounts price-n price-d wheat-word sheep-word
  max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
  (wheat-word > sheep-word) rounding options =
  Cxx-Ok
    (word-of-int (fst amounts), word-of-int (snd amounts))
and sint (word-of-int (fst amounts) :: int64) = fst amounts
and sint (word-of-int (snd amounts) :: int64) = snd amounts

```

Proof sketch: identify both 128-bit values with their exact integers, then split exactly as the abstraction does. Each of the seven cases is one branch equation — five plain, two repaired — and the bounds theorem above makes both re-encoded result words faithful signed-64 values.

theorem *exchange-v10-without-thresholds-options-integer-characterization:*
fixes *amounts* :: *int* × *int*
and *rounding* :: *exchange-rounding* **and** *options* :: *exchange-options*
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *amounts* ≡
exchange-v10-amounts-int-with-options price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding options
shows *exchange-v10-without-price-error-thresholds-with-options price-n*
price-d max-wheat-send max-wheat-receive max-sheep-send
max-sheep-receive rounding options =
Cxx-Ok
(make-exchange-result
(word-of-int (fst amounts)) (word-of-int (snd amounts))
(exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive >
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive))

Proof sketch: both offer-value calls succeed with their exact 128-bit minima under *exchange-v10-pre*. Substitute the options-parametric amount characterization, then use its signed interpretations and four bounds to prove that neither final runtime check is reachable.

theorem *exchange-v10-options-integer-characterization:*
fixes *amounts* :: *int* × *int*
and *rounding* :: *exchange-rounding* **and** *options* :: *exchange-options*
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *amounts* ≡
exchange-v10-amounts-int-with-options price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding options
shows *exchange-v10-with-options price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive rounding options =
apply-price-error-thresholds price-n price-d
(word-of-int (fst amounts)) (word-of-int (snd amounts))
(exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive >
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive)
rounding

— The whole kernel, in every rounding mode and at every exact-cap configuration, is the exact-integer calculation followed by the threshold pass.

Proof sketch: the options-parametric pre-threshold characterization replaces

the first call by its exact amount record; simplifying the record selectors leaves the threshold call shown here.

exchange-v10-options-integer-characterization generalizes both *exchange-v10-integer-characterization* and *exchange-v10-repaired-integer-characterization*: the first is its instance at *legacy-exchange-options*, the second its instance at *repaired-exchange-options* in normal mode, and the two collapse equations above make each instance syntactically the established one. What is new is the repaired record in the two strict modes, which is what the path-payment routes issue from protocol 29 on.

6.10 Strict-send positivity at the repaired record

With the options-parametric abstraction available, the legacy strict-send positivity *iff* can be redone at the repaired record. It does not survive verbatim. In the branch where the wheat offer does not stay and wheat is the more valuable asset, the repaired calculation divides the relaxed exact trade value rather than the plain wheat value, so the exact positivity threshold moves: the legacy condition *sint price-n ≤ exchange-wheat-value-int price-n price-d max-wheat-send max-sheep-receive* becomes the same comparison against *calculate-offer-amount-from-value-int*'s numerator. The two other strict-send branches are option-free and their conditions are unchanged.

definition *exact-trade-value-int* ::

int32 ⇒ int32 ⇒ int64 ⇒ int64 ⇒ int

where

exact-trade-value-int price-n price-d max-send max-receive =
 $\min (\text{sint max-send} * \text{sint price-n})$
 $(\min (\text{sint max-receive} * \text{sint price-d} + \text{sint price-d} - 1)$
 $(\text{sint int64-max} * \text{sint price-d}))$

lemma *calculate-offer-amount-from-value-int-as-trade*:

calculate-offer-amount-from-value-int price-n price-d max-send
max-receive =
exact-trade-value-int price-n price-d max-send max-receive
div sint price-n

— Naming the numerator of the protocol-29 helper.

lemma *exact-trade-value-int-nonnegative*:

assumes *pn*: $0 < \text{sint price-n}$ **and** *pd*: $0 < \text{sint price-d}$

and *send-nonnegative*: $0 \leq \text{sint max-send}$

and *receive-nonnegative*: $0 \leq \text{sint max-receive}$

shows $0 \leq \text{exact-trade-value-int price-n price-d max-send max-receive}$

— All three components of the relaxed trade value are non-negative.

Proof sketch: each of the three products is a product of two non-negative factors, and the relaxation addend is non-negative because the denominator is positive.

theorem *exchange-v10-strict-send-sheep-positive-iff-with-options:*

assumes *pre: exchange-v10-pre price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive*

shows $(0 < \text{snd}$

$(\text{exchange-v10-amounts-int-with-options price-n price-d max-wheat-send}$
 $\text{max-wheat-receive max-sheep-send max-sheep-receive}$
 $\text{Exchange-Strict-Send options})) =$

$(\text{let wheat-value} =$

$\text{exchange-wheat-value-int price-n price-d max-wheat-send}$
 $\text{max-sheep-receive};$

$\text{sheep-value} =$

$\text{exchange-sheep-value-int price-n price-d max-sheep-send}$
 max-wheat-receive

$\text{in if wheat-value} > \text{sheep-value}$

$\text{then } 0 < \text{sint max-sheep-send} \wedge 0 < \text{sint max-sheep-receive}$

$\text{else if sint price-n} > \text{sint price-d}$

$\text{then if exact-receive-cap options}$

$\text{then sint price-n} \leq$

$\text{exact-trade-value-int price-n price-d max-wheat-send}$
 max-sheep-receive

$\text{else sint price-n} \leq \text{wheat-value}$

$\text{else sint price-d} \leq \text{wheat-value})$

— The exact strict-send positivity threshold, at every exact-cap configuration.

Only the non-staying wheat-more branch differs from the legacy statement, and it differs by replacing the plain wheat value with the relaxed exact trade value.

Proof sketch: in the wheat-stays branch the sheep amount is the signed minimum of the two sheep maxima, exactly as before. Otherwise the selected floor formula is a nested quotient of a non-negative numerator, positive exactly at one numerator unit when wheat is more valuable, and an ordinary positive division otherwise. The exact-cap flag changes only which non-negative numerator is divided.

corollary *exchange-v10-strict-send-sheep-positive-iff-repaired:*

assumes *pre: exchange-v10-pre price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive*

shows $(0 < \text{snd}$

$(\text{exchange-v10-amounts-int-with-options price-n price-d max-wheat-send}$
 $\text{max-wheat-receive max-sheep-send max-sheep-receive}$
 $\text{Exchange-Strict-Send repaired-exchange-options})) =$

$(\text{let wheat-value} =$

$\text{exchange-wheat-value-int price-n price-d max-wheat-send}$
 $\text{max-sheep-receive};$

$\text{sheep-value} =$

$\text{exchange-sheep-value-int price-n price-d max-sheep-send}$
 max-wheat-receive

in if wheat-value > sheep-value
then $0 < \text{sint max-sheep-send} \wedge 0 < \text{sint max-sheep-receive}$
else if $\text{sint price-n} > \text{sint price-d}$
then $\text{sint price-n} \leq$
$\text{exact-trade-value-int price-n price-d max-wheat-send}$
max-sheep-receive
else $\text{sint price-d} \leq \text{wheat-value}$
 — The protocol-29 strict-send positivity threshold.

lemma *exchange-wheat-value-int-le-exact-trade-value-int:*
assumes *pd: $0 < \text{sint price-d}$*
shows *exchange-wheat-value-int price-n price-d max-wheat-send*
$\text{max-sheep-receive} \leq$
$\text{exact-trade-value-int price-n price-d max-wheat-send max-sheep-receive}$
 — The relaxed exact trade value is never below the plain wheat value.

Proof sketch: the receive component is relaxed upwards by one divisor unit, and the saturation ceiling cannot bind below it, because the sheep-receive cap is a signed 64-bit amount and so at most *int64-max*. Both minima take the same send component, so the relaxed minimum dominates.

corollary *legacy-strict-send-positivity-implies-repaired:*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
$\text{max-wheat-receive max-sheep-send max-sheep-receive}$
and *legacy-positive:*
$0 < \text{snd}$
($\text{exchange-v10-amounts-int price-n price-d max-wheat-send}$
$\text{max-wheat-receive max-sheep-send max-sheep-receive}$
$\text{Exchange-Strict-Send}$)
shows *$0 < \text{snd}$*
($\text{exchange-v10-amounts-int-with-options price-n price-d max-wheat-send}$
$\text{max-wheat-receive max-sheep-send max-sheep-receive}$
$\text{Exchange-Strict-Send repaired-exchange-options}$)
 — Protocol 29 makes more strict-send trades positive, never fewer: every positive legacy strict-send sheep amount is still positive at the repaired record.

Proof sketch: the two thresholds differ only in the non-staying wheat-more branch, where the legacy one compares the price numerator against the plain wheat value and the repaired one against the relaxed exact trade value. The preceding lemma orders those two numerators, so the legacy condition implies the repaired one; the other two branch conditions are identical.

Comparing *exchange-v10-strict-send-sheep-positive-iff-repaired* with *exchange-v10-strict-send-sheep-positive-iff* makes the divergence explicit and localizes it: the two outer conditions are word for word the legacy ones, and only the non-staying wheat-more condition changes, from *sint price-n ≤ exchange-wheat-value-int price-n price-d max-wheat-send max-sheep-receive* to

the same comparison against *exact-trade-value-int*. *exchange-wheat-value-int-le-exact-trade-value-int* orders those two numerators, so the repaired condition is the weaker one and *legacy-strict-send-positivity-implies-repaired* follows: protocol 29 makes more strict-send trades positive and never fewer, which is the intended direction of the repair.

6.11 Counterexample-first probes for the repaired strict modes

The results above quantify over inputs whose branches could in principle be unreachable, so each is pinned down by an evaluation. The first two probes show that the repaired strict-send threshold really does differ from the legacy one, the third that the difference is visible at the protocol-facing interface, and the last two that the strict-mode contracts are not vacuous at protocol 29 — including on the branch the exact receive cap changes.

lemma *repaired-strict-send-amounts-differ-probe:*

exchange-v10-amounts-int 3 2 1 1 1 Exchange-Strict-Send = (0, 0)
exchange-v10-amounts-int-with-options 3 2 1 1 1 Exchange-Strict-Send
repaired-exchange-options = (1, 1)

— At price three halves with unit caps the legacy strict-send calculation yields no trade and the repaired one yields a unit trade, so the two positivity thresholds are genuinely different predicates.

Proof sketch: evaluate both exact-integer calculations at the witness.

lemma *repaired-strict-send-relaxes-threshold-probe:*

exact-trade-value-int 3 2 1 1 = 3
exchange-wheat-value-int 3 2 1 1 = 2

— The same witness seen through the two numerators the thresholds compare against the price numerator: the relaxed exact trade value reaches it and the plain wheat value does not.

Proof sketch: evaluate both integer values at the witness.

lemma *repaired-strict-send-threshold-rejection-probe:*

exchange-v10-without-price-error-thresholds-with-options 3 2 3 3 4 4
Exchange-Strict-Send legacy-exchange-options =
Cxx-Ok (make-exchange-result 2 3 False)
exchange-v10-without-price-error-thresholds-with-options 3 2 3 3 4 4
Exchange-Strict-Send repaired-exchange-options =
Cxx-Ok (make-exchange-result 3 4 False)
exchange-v10 28 3 2 3 3 4 4 Exchange-Strict-Send =
Cxx-Ok (make-exchange-result 2 3 False)
exchange-v10 29 3 2 3 3 4 4 Exchange-Strict-Send = Cxx-Err Cxx-Runtime-Error

— A strict-send call that succeeds at protocol 28 and raises a modeled runtime error at protocol 29.

Proof sketch: evaluate the pre-threshold calculation at both records and the protocol-facing call at both versions.

What the last probe means. It is not a failure of any theorem above — every strict-mode contract is conditional on a *successful* exchange — and it is not a live protocol-29 regression either. On this input the exact receive cap enlarges the pre-threshold pair from 2 wheat for 3 sheep to 3 for 4. Three wheat at price three halves is worth four and a half sheep, so the enlarged pair underpays by half a sheep unit, and the tight one-sided price-error bound that strict modes apply rejects it. The legacy pair was exact and passed.

What the probe actually shows is that the standing precondition of the exchange — that the resting wheat offer has been adjusted *at the version the exchange is running at* — became load-bearing at protocol 29 in a way it was not at protocol 28. The witness violates it: the same offer state is adjusted to 2 before the boundary and to 0 from it, so at protocol 29 the offer is deleted rather than rested and the rejected call is one no crossing can issue.

lemma *repaired-strict-send-rejection-needs-unadjusted-offer*:

adjust-offer 28 3 2 3 4 = Cxx-Ok 2

adjust-offer 29 3 2 3 4 = Cxx-Ok 0

— The rejected witness is not a protocol-29 adjustment fixed point: at that version the offer is adjusted away entirely.

Proof sketch: evaluate the protocol-facing adjustment at both versions.

definition *p29-adjusted-strict-case* ::

int32 ⇒ int32 ⇒ int64 ⇒ int64 ⇒ int64 ⇒

int64 ⇒ bool

where

p29-adjusted-strict-case price-n price-d max-wheat-send max-wheat-receive

max-sheep-send max-sheep-receive $\longleftrightarrow$

(if *adjust-offer 29 price-n price-d max-wheat-send max-sheep-receive =*

Cxx-Ok max-wheat-send

then *list-all*

(λ *rounding.*

case *exchange-v10 29 price-n price-d max-wheat-send*

max-wheat-receive max-sheep-send max-sheep-receive rounding of

Cxx-Ok - $\Rightarrow$ *True*

| *Cxx-Err* - $\Rightarrow$ *False*)

[*Exchange-Strict-Send*, *Exchange-Strict-Receive*]

else *True*)

p29-adjusted-strict-case restricts attention to the calls a crossing can actually

issue. *cross-offer-v10* adjusts the resting offer at the crossing version and only then exchanges, and its send cap is the adjusted amount, so the pair it hands to the exchange is always a fixed point of *adjust-offer 29* at the receive cap in force. The predicate passes vacuously on every other pair and, on those, requires both strict modes to return successfully.

lemma *bounded-p29-adjusted-strict-search:*

```
list-all (λprice-n.
  list-all (λprice-d.
    list-all (λmax-wheat-send.
      list-all (λmax-wheat-receive.
        list-all (λmax-sheep-send.
          list-all (λmax-sheep-receive.
            p29-adjusted-strict-case price-n price-d max-wheat-send
              max-wheat-receive max-sheep-send max-sheep-receive)
            ([1, 2, 3, 4, 5] :: int64 list))
            ([1, 2, 3, 4, 5] :: int64 list))
            ([1, 2, 3, 4, 5] :: int64 list))
            ([1, 2, 3, 4, 5] :: int64 list))
            ([1, 2, 3, 4, 5] :: int32 list))
            ([1, 2, 3, 4, 5] :: int32 list))
```

— No adjusted protocol-29 offer reaches the rejection in either strict mode, across the whole small grid.

Proof sketch: execute the fifteen-thousand-six-hundred-and-twenty-five-case grid. One hundred and fifty of its price and cap combinations are protocol-29 adjustment fixed points; on each, both strict-mode exchanges succeed.

lemma *p29-adjusted-strict-case-not-vacuous:*

```
adjust-offer 29 2 1 1 2 = Cxx-Ok 1
exchange-v10 29 2 1 1 1 2 2 Exchange-Strict-Send =
  Cxx-Ok (make-exchange-result 1 2 False)
```

— The search is not vacuous where it matters: this offer is a protocol-29 adjustment fixed point, its cross has a positive numerator and a non-staying wheat offer — so it is decided by the very branch the exact receive cap changes — and it succeeds.

Proof sketch: evaluate the adjustment and the strict-send exchange.

So the enlarged pair is unreachable through the adjustment discipline that *cross-offer-v10* maintains, which is what the C++ comment at the corresponding throw site asserts without proof. The bounded search above is evidence for that claim, not a proof of it; a proof would have to show that a protocol-29 adjustment fixed point always satisfies the tight bound in both strict modes, which is a genuine open item rather than a restatement. What remains genuinely unmodelled is only whether some *other* caller reaches *exchange-v10* without the adjustment step.

lemma *repaired-strict-mode-positive-success-probes:*

exchange-v10 29 1 1 1 1 1 Exchange-Strict-Send =
Cxx-Ok (make-exchange-result 1 1 False)
exchange-v10 29 1 1 1 1 1 Exchange-Strict-Receive =
Cxx-Ok (make-exchange-result 1 1 False)

— Both strict modes have positive protocol-29 successes, so the strict-send positivity theorem and the positive case of the strict-receive contract are not vacuous.

Proof sketch: evaluate the protocol-facing call at version 29 in both strict modes.

lemma *repaired-strict-send-exact-branch-success-probe:*

exchange-v10 29 2 1 1 1 2 Exchange-Strict-Send =
Cxx-Ok (make-exchange-result 1 2 False)
exchange-v10 28 3 2 1 1 1 Exchange-Strict-Receive =
Cxx-Ok (make-exchange-result 0 0 False)

— The first call has a positive numerator and a non-staying wheat offer, so it is decided by the branch the exact receive cap changes, and it succeeds. The second is a strict-receive success returning the zero record, so both sides of the simultaneous-zero contract are witnessed.

Proof sketch: evaluate the two protocol-facing calls.

lemma *exchange-v10-amounts-int-repaired-candidate:*

assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
shows *normal-favored-candidate price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
(exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive >
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive)
(fst (exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive))
(snd (exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive))

Proof sketch: the repaired bounds supply non-negativity and the four cap clauses. In each of the four branches the pair satisfies its canonical favored rounding equation by construction. The primary scaled amount is bounded by the saturation ceiling: the two corrected branches divide a value already clamped by the *int64-max* product, and the two plain branches divide an offer value bounded by a signed-64 cap times the ceiling's price component.

lemma *exchange-v10-amounts-int-repaired-maximal:*

assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*

```

    max-wheat-receive max-sheep-send max-sheep-receive
and candidate: normal-favored-candidate price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive
    (exchange-wheat-value-int price-n price-d max-wheat-send
     max-sheep-receive >
     exchange-sheep-value-int price-n price-d max-sheep-send
     max-wheat-receive)
    wheat-receive sheep-send
shows if sint price-n > sint price-d
    then wheat-receive ≤
      fst (exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
          max-wheat-receive max-sheep-send max-sheep-receive)
    else sheep-send ≤
      snd (exchange-v10-amounts-int-repaired price-n price-d max-wheat-send
          max-wheat-receive max-sheep-send max-sheep-receive)

```

Proof sketch: split on the price order and the staying offer. In each branch the candidate's canonical rounding equation converts its cap on the derived amount into a bound on the primary scaled amount; together with the candidate's own caps and saturation clause, the primary scaled amount is at most the value divided by the implementation, so division monotonicity bounds the candidate's primary amount by the returned one.

```

lemma exchange-v10-repaired-positive-normal-maximal:
assumes pre: exchange-v10-pre price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive
and result: exchange-v10-with-options price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive Exchange-Normal
    (exact-receive-cap = True, symmetric-exact-receive-cap = True) =
    Cxx-Ok result
and positive:
    0 < sint (num-wheat-received result)
    0 < sint (num-sheep-send result)
shows maximal-normal-favored-result price-n price-d max-wheat-send
    max-wheat-receive max-sheep-send max-sheep-receive result

```

Proof sketch: the repaired integer characterization identifies the exchange with a threshold call on the repaired amount pair. Positivity rules out the threshold's explicit zero record, so the returned fields are exactly the repaired amounts and the exact stay flag. The candidate lemma makes that pair a favored candidate and the dominance lemma bounds every competing candidate's primary amount.

theorem positive-normal-crosses-are-maximal-repaired:

positive-normal-crosses-are-maximal
 $\langle \text{exact-receive-cap} = \text{True}, \text{symmetric-exact-receive-cap} = \text{True} \rangle$

Proof sketch: unfold the intended property and apply the pointwise maximality result to every successful positive normal exchange with both receive-cap repairs enabled.

6.11.1 The reservation anomaly, replayed end to end

The definitions are executable, so the walkthrough in Section 5 of *formal/docs/offer-lifecycle.md* can be replayed. Alice holds one TOKEN and a USD trustline with limit two and balance zero; she posts an offer selling one TOKEN at the price of 101 USD units per 100 TOKEN units. The offer is created at its full amount and reserves one TOKEN of selling liabilities and one USD of buying liabilities. All three lemmas of this subsection are proved by evaluating both sides.

lemma *alice-posts-her-offer*:

post-offer 101 100 1
 $\langle \text{sell-balance} = 1, \text{sell-liabilities} = 0,$
 $\text{buy-limit} = 2, \text{buy-balance} = 0, \text{buy-liabilities} = 0 \rangle \langle \text{exact-receive-cap} = \text{False},$
 $\text{symmetric-exact-receive-cap} = \text{False} \rangle =$
Cxx-Ok (Post-Created 1
 $\langle \text{sell-balance} = 1, \text{sell-liabilities} = 1,$
 $\text{buy-limit} = 2, \text{buy-balance} = 0, \text{buy-liabilities} = 1 \rangle)$

If nothing disturbs Alice's account, Bob—holding two USD, ample TOKEN headroom, and willing to sell up to two USD—crosses her offer at its full amount: one TOKEN moves against one USD and the offer leaves the book fully consumed. The maker state at the crossing step is exactly the state in which posting left it.

lemma *bob-takes-the-offer*:

post-then-cross 101 100 1
 $\langle \text{sell-balance} = 1, \text{sell-liabilities} = 0,$
 $\text{buy-limit} = 2, \text{buy-balance} = 0, \text{buy-liabilities} = 0 \rangle$
 $\langle \text{sell-balance} = 1, \text{sell-liabilities} = 1,$
 $\text{buy-limit} = 2, \text{buy-balance} = 0, \text{buy-liabilities} = 1 \rangle$
 $\langle \text{sell-balance} = 2, \text{sell-liabilities} = 0,$
 $\text{buy-limit} = 1000, \text{buy-balance} = 0, \text{buy-liabilities} = 0 \rangle$
 $2 \text{ Exchange-Normal } \langle \text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} =$
 $\text{False} \rangle =$
Cxx-Ok
(Post-Created 1
 $\langle \text{sell-balance} = 1, \text{sell-liabilities} = 1,$
 $\text{buy-limit} = 2, \text{buy-balance} = 0, \text{buy-liabilities} = 1 \rangle,$
Some
 $\langle \text{cross-wheat-received} = 1,$

cross-sheep-send = 1,
cross-wheat-stays = False,
cross-offer-amount = 0,
cross-maker =
 (*sell-balance* = 0, *sell-liabilities* = 0,
 buy-limit = 2, *buy-balance* = 1, *buy-liabilities* = 0),
cross-taker =
 (*sell-balance* = 1, *sell-liabilities* = 0,
 buy-limit = 1000, *buy-balance* = 1, *buy-liabilities* = 0))

But if Carol first pays Alice one USD—a payment Alice cannot refuse and that touches no offer, visible below as the maker’s buying balance rising from zero to one under an unchanged limit—Alice’s headroom drops to exactly her reservation. The same crossing now erases the offer for a zero fill: the preventative adjustment clips the offer to nothing, because its own reservation, fed back in as a limit, makes the offer look smaller than itself. This is property P9 of the survey document, violated at the lifecycle level.

lemma *carol-griefts-alice*:

post-then-cross 101 100 1
 (*sell-balance* = 1, *sell-liabilities* = 0,
 buy-limit = 2, *buy-balance* = 0, *buy-liabilities* = 0)
 (*sell-balance* = 1, *sell-liabilities* = 1,
 buy-limit = 2, *buy-balance* = 1, *buy-liabilities* = 1)
 (*sell-balance* = 2, *sell-liabilities* = 0,
 buy-limit = 1000, *buy-balance* = 0, *buy-liabilities* = 0)
 2 *Exchange-Normal* (*exact-receive-cap* = False, *symmetric-exact-receive-cap* =
False) =
 Cxx-Ok
 (*Post-Created 1*
 (*sell-balance* = 1, *sell-liabilities* = 1,
 buy-limit = 2, *buy-balance* = 0, *buy-liabilities* = 1),
 Some
 (*cross-wheat-received* = 0,
 cross-sheep-send = 0,
 cross-wheat-stays = False,
 cross-offer-amount = 0,
 cross-maker =
 (*sell-balance* = 1, *sell-liabilities* = 0,
 buy-limit = 2, *buy-balance* = 1, *buy-liabilities* = 0),
 cross-taker =
 (*sell-balance* = 2, *sell-liabilities* = 0,
 buy-limit = 1000, *buy-balance* = 0, *buy-liabilities* = 0))
)

Carol’s payment does more than deprive Bob of one fill: it leaves a dead offer. The lemma below quantifies over every taker satisfying the positivity contract of *crossOfferV10* and shows each of them receives nothing, pays

nothing, and erases the offer from the book. Together with *bob-takes-the-offer* this separates the failure cleanly: the same posted offer was takeable before the payment, and only the interference killed it. Proof sketch: releasing the offer's liabilities restores the state Carol left behind, where the preventative adjustment is a closed computation returning zero, because the reservation fed back in as a limit values the offer at 100 sheep units, one short of a lot of wheat at this price. A zero wheat cap then forces a zero wheat value, whose unsigned comparison against any taker's sheep value keeps the trade on the fully-consumed branch, where both divisions return zero and the thresholds pass the empty trade through. Every step around the symbolic taker is a ground evaluation, and the taker's own balance moves are the identity on a zero amount.

lemma *no-taker-can-take-the-griefed-offer*:

assumes $0 < \text{sint}$ (*can-buy-at-most taker*)

and $0 < \text{sint}$ (*signed-min64 taker-amount* (*can-sell-at-most taker*))

shows *cross-offer-v10 101 100 1*

$\langle \text{sell-balance} = 1, \text{sell-liabilities} = 1,$

$\text{buy-limit} = 2, \text{buy-balance} = 1, \text{buy-liabilities} = 1 \rangle$

taker taker-amount Exchange-Normal ($\langle \text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False} \rangle =$

Cxx-Ok

$\langle \text{cross-wheat-received} = 0, \text{cross-sheep-send} = 0,$

$\text{cross-wheat-stays} = \text{False}, \text{cross-offer-amount} = 0,$

$\text{cross-maker} =$

$\langle \text{sell-balance} = 1, \text{sell-liabilities} = 0,$

$\text{buy-limit} = 2, \text{buy-balance} = 1, \text{buy-liabilities} = 0 \rangle,$

$\text{cross-taker} = \text{taker} \rangle$

The stability vocabulary of *adjust-stable* separates the two halves of the anomaly. Posting writes only adjust-stable amounts: the amount admitted by Alice's post (*alice-posts-her-offer*) is stable at the state it was validated against. The universal form of this certificate—every successful post leaves an adjust-stable amount, which is what makes a maker-unchanged crossing replay the posted trade—is *unchanged-maker-cover-implies-adjust-stable*. Ledger interference, by contrast, preserves the specification's covering premise but not stability: after Carol's payment the maker still covers the offer's booked liabilities, yet the amount is no longer stable at the released state that the crossing adjustment actually sees. This is *no-taker-can-take-the-griefed-offer* in vocabulary form, and the repair goal is the missing implication from covering to stability: under either candidate fix the preventative adjustment must become the identity at every admissible crossing state. Both lemmas are proved by evaluation.

lemma *posting-certifies-adjust-stability*:

adjust-stable 101 100 1

$\langle \text{sell-balance} = 1, \text{sell-liabilities} = 0,$

$\text{buy-limit} = 2, \text{buy-balance} = 0, \text{buy-liabilities} = 0 \rangle \langle \text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False} \rangle$

lemma *covering-does-not-imply-adjust-stability:*

maker-covers-offer-liabilities 101 100 1
 $\langle \text{sell-balance} = 1, \text{sell-liabilities} = 1, \text{buy-limit} = 2, \text{buy-balance} = 1, \text{buy-liabilities} = 1 \rangle \wedge$
release-offer-liabilities 101 100 1
 $\langle \text{sell-balance} = 1, \text{sell-liabilities} = 1, \text{buy-limit} = 2, \text{buy-balance} = 1, \text{buy-liabilities} = 1 \rangle =$
Cxx-Ok $\langle \text{sell-balance} = 1, \text{sell-liabilities} = 0, \text{buy-limit} = 2, \text{buy-balance} = 1, \text{buy-liabilities} = 0 \rangle \wedge$
 $\neg \text{adjust-stable } 101 \ 100 \ 1$
 $\langle \text{sell-balance} = 1, \text{sell-liabilities} = 0, \text{buy-limit} = 2, \text{buy-balance} = 1, \text{buy-liabilities} = 0 \rangle \langle \text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False} \rangle$

6.11.2 A sheep-limit adjustment can make an offer untakeable

lemma *no-taker-can-take-the-limit-adjusted-offer:*

assumes $0 < \text{sint} \ (\text{can-buy-at-most taker})$
and $0 < \text{sint} \ (\text{signed-min64 taker-amount} \ (\text{can-sell-at-most taker}))$
shows *cross-offer-v10 101 100 1*
 $\langle \text{sell-balance} = 1, \text{sell-liabilities} = 1, \text{buy-limit} = 1, \text{buy-balance} = 0, \text{buy-liabilities} = 1 \rangle$
taker taker-amount Exchange-Normal
 $\langle \text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False} \rangle =$
Cxx-Ok
 $\langle \text{cross-wheat-received} = 0, \text{cross-sheep-send} = 0, \text{cross-wheat-stays} = \text{False}, \text{cross-offer-amount} = 0, \text{cross-maker} =$
 $\langle \text{sell-balance} = 1, \text{sell-liabilities} = 0, \text{buy-limit} = 1, \text{buy-balance} = 0, \text{buy-liabilities} = 0 \rangle,$
 $\text{cross-taker} = \text{taker} \rangle$

Proof sketch: releasing the posted offer's liabilities leaves the maker with one unit of wheat and one unit of sheep headroom. With both receive-cap flags disabled, the preventative adjustment of a one-unit offer at price *real-of-int 101 / 100* evaluates to zero. The remaining exchange calculation therefore transfers zero on both sides for every taker satisfying the crossing function's positive-capacity precondition. All state updates by zero are identities.

theorem *limit-adjustment-stable-counterexample:*

$\neg \text{limit-adjustment-stable}$
 $\langle \text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False} \rangle$

Proof sketch: Alice posts one unit of wheat at price *real-of-int 101 / 100*

while her sheep limit is two. Posting books one unit of buying liability. Lowering only *buy-limit* to one is still well formed because the new limit exactly covers that liability. If *limit-adjustment-stable* held, some positive-capacity taker would then complete a positive cross. The preceding lemma shows that every such cross instead returns zero wheat and zero sheep, a contradiction.

6.12 Continuous specification and fixed-width refinement

This subsection presents the refinement story in one place. It first gives a continuous normal-exchange model over the reals, then identifies that model with the unbounded-integer quantities used to reason about the executable arithmetic. Finally it proves that the fixed-width implementation with both receive-cap repairs returns the canonical whole-unit approximation for every successful positive normal exchange.

6.12.1 Continuous normal exchange

definition *ideal-exchange-price* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *real*

where

ideal-exchange-price price-n price-d =
of-int (*sint price-n*) / *of-int* (*sint price-d*)

definition *ideal-wheat-offer-capacity* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *real*

where

ideal-wheat-offer-capacity price-n price-d max-wheat-send
max-sheep-receive =
min (*of-int* (*sint max-wheat-send*))
(*of-int* (*sint max-sheep-receive*) /
ideal-exchange-price price-n price-d)

definition *ideal-sheep-offer-capacity* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *real*

where

ideal-sheep-offer-capacity price-n price-d max-sheep-send
max-wheat-receive =
min (*of-int* (*sint max-wheat-receive*))
(*of-int* (*sint max-sheep-send*) /
ideal-exchange-price price-n price-d)

definition *ideal-normal-wheat-receive* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$ *int64* $\Rightarrow$
int64 $\Rightarrow$ *real*

where

ideal-normal-wheat-receive price-n price-d max-wheat-send

$$\begin{aligned}
& \text{max-wheat-receive max-sheep-send max-sheep-receive} = \\
& \min \\
& \quad (\text{ideal-wheat-offer-capacity price-n price-d max-wheat-send} \\
& \quad \quad \text{max-sheep-receive}) \\
& \quad (\text{ideal-sheep-offer-capacity price-n price-d max-sheep-send} \\
& \quad \quad \text{max-wheat-receive})
\end{aligned}$$

definition *ideal-normal-sheep-send* ::

$$\begin{aligned}
& \text{int32} \Rightarrow \text{int32} \Rightarrow \text{int64} \Rightarrow \text{int64} \Rightarrow \text{int64} \Rightarrow \\
& \text{int64} \Rightarrow \text{real}
\end{aligned}$$

where

$$\begin{aligned}
& \text{ideal-normal-sheep-send price-n price-d max-wheat-send} \\
& \quad \text{max-wheat-receive max-sheep-send max-sheep-receive} = \\
& \quad \text{ideal-normal-wheat-receive price-n price-d max-wheat-send} \\
& \quad \text{max-wheat-receive max-sheep-send max-sheep-receive} * \\
& \quad \text{ideal-exchange-price price-n price-d}
\end{aligned}$$

definition *ideal-normal-wheat-stays* ::

$$\begin{aligned}
& \text{int32} \Rightarrow \text{int32} \Rightarrow \text{int64} \Rightarrow \text{int64} \Rightarrow \text{int64} \Rightarrow \\
& \text{int64} \Rightarrow \text{bool}
\end{aligned}$$

where

$$\begin{aligned}
& \text{ideal-normal-wheat-stays price-n price-d max-wheat-send} \\
& \quad \text{max-wheat-receive max-sheep-send max-sheep-receive} \longleftrightarrow \\
& \quad \text{ideal-wheat-offer-capacity price-n price-d max-wheat-send} \\
& \quad \text{max-sheep-receive} > \\
& \quad \text{ideal-sheep-offer-capacity price-n price-d max-sheep-send} \\
& \quad \text{max-wheat-receive}
\end{aligned}$$

ideal-exchange-price embeds the ledger price as an exact real quotient. The two capacity definitions measure, in wheat units, how much each offer could exchange at that price: each is the smaller of its wheat cap and its sheep cap divided by the price. The ideal normal exchange takes the smaller capacity, transfers that much wheat and its exact-price amount of sheep, and keeps precisely the offer with the larger capacity.

These definitions intentionally contain neither integer rounding nor fixed-width bounds. They describe the continuous economic transaction; the discrete specification below states how an executable result projects that transaction onto whole ledger units.

6.12.2 Refinement relation

definition *normal-result-refines-ideal* ::

$$\begin{aligned}
& \text{int32} \Rightarrow \text{int32} \Rightarrow \text{int64} \Rightarrow \text{int64} \Rightarrow \text{int64} \Rightarrow \\
& \text{int64} \Rightarrow \text{exchange-result-v10} \Rightarrow \text{bool}
\end{aligned}$$

where

$$\begin{aligned}
& \text{normal-result-refines-ideal price-n price-d max-wheat-send} \\
& \quad \text{max-wheat-receive max-sheep-send max-sheep-receive result} \longleftrightarrow \\
& \quad \text{result-wheat-stays result} =
\end{aligned}$$

```

ideal-normal-wheat-stays price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive ∧
(if sint price-n > sint price-d
then abs
  (of-int (sint (num-wheat-received result)) –
   ideal-normal-wheat-receive price-n price-d max-wheat-send
   max-wheat-receive max-sheep-send max-sheep-receive) < 1
else abs
  (of-int (sint (num-sheep-send result)) –
   ideal-normal-sheep-send price-n price-d max-wheat-send
   max-wheat-receive max-sheep-send max-sheep-receive) < 1) ∧
favored-seller-ok price-n price-d
(num-wheat-received result) (num-sheep-send result)
(result-wheat-stays result) ∧
maximal-normal-favored-result price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive result

```

normal-result-refines-ideal connects the continuous and discrete layers. Its first clause requires the real capacity comparison to choose the surviving offer. Its second places the primary whole-unit amount less than one unit from the corresponding continuous amount. Its third requires the effective-price inequality to favor the offer that stays. The final clause requires the pair to be the maximal cap-respecting canonical round of the exact price, including the signed-result saturation ceiling. The relation is stated only for the normal-mode policy; threshold zeroing is excluded later by requiring a positive returned trade.

6.12.3 Real and integer bridge

lemma *real-min-divide-by-positive*:

fixes $a\ b\ c :: \text{real}$

assumes *denominator-positive*: $0 < c$

shows $\min a\ (b / c) = \min (a * c)\ b / c$

Proof sketch: multiplying both candidates by the same positive denominator preserves their order, so taking the minimum commutes with division by it.

lemma *ideal-wheat-offer-capacity-integer*:

assumes *numerator-positive*: $0 < \text{sint price-n}$

shows *ideal-wheat-offer-capacity* price-n price-d max-wheat-send
max-sheep-receive =

of-int (exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive) / of-int (sint price-n)

Proof sketch: express the receive cap in wheat units by dividing it by the exact real price. Clearing the positive numerator scales both alternatives to the common integer value used by the executable model.

lemma *ideal-sheep-offer-capacity-integer*:
assumes *numerator-positive*: $0 < \text{sint } \text{price-n}$
shows *ideal-sheep-offer-capacity* $\text{price-n price-d max-sheep-send}$
 $\text{max-wheat-receive} =$
 $\text{of-int } (\text{exchange-sheep-value-int } \text{price-n price-d max-sheep-send}$
 $\text{max-wheat-receive}) / \text{of-int } (\text{sint } \text{price-n})$

Proof sketch: express the sheep offer's send cap in wheat units and clear the same positive price numerator. The resulting minimum is the exact common-unit sheep value, up to commutativity of *min*.

lemma *ideal-normal-wheat-stays-integer*:
assumes *pre*: $\text{exchange-v10-pre } \text{price-n price-d max-wheat-send}$
 $\text{max-wheat-receive max-sheep-send max-sheep-receive}$
shows *ideal-normal-wheat-stays* $\text{price-n price-d max-wheat-send}$
 $\text{max-wheat-receive max-sheep-send max-sheep-receive} =$
 $(\text{exchange-wheat-value-int } \text{price-n price-d max-wheat-send}$
 $\text{max-sheep-receive} >$
 $\text{exchange-sheep-value-int } \text{price-n price-d max-sheep-send}$
 $\text{max-wheat-receive})$

Proof sketch: both real capacities are their exact common-unit integer values divided by the same positive price numerator. Cancelling that denominator leaves precisely the executable model's stay comparison.

lemma *ideal-normal-wheat-receive-integer*:
assumes *pre*: $\text{exchange-v10-pre } \text{price-n price-d max-wheat-send}$
 $\text{max-wheat-receive max-sheep-send max-sheep-receive}$
shows *ideal-normal-wheat-receive* $\text{price-n price-d max-wheat-send}$
 $\text{max-wheat-receive max-sheep-send max-sheep-receive} =$
 of-int
 $(\text{min}$
 $(\text{exchange-wheat-value-int } \text{price-n price-d max-wheat-send}$
 $\text{max-sheep-receive})$
 $(\text{exchange-sheep-value-int } \text{price-n price-d max-sheep-send}$
 $\text{max-wheat-receive})) /$
 $\text{of-int } (\text{sint } \text{price-n})$

Proof sketch: the ideal wheat transfer is the smaller offer capacity. Replace both capacities by their common-unit integer forms and commute a shared positive division with *min*.

lemma *ideal-normal-sheep-send-integer*:

assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
shows *ideal-normal-sheep-send price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive =
of-int
(min
(exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive)
(exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive)) /
of-int (sint price-d)

Proof sketch: the ideal sheep transfer is the ideal wheat transfer times the exact real price. Substitute the common trade value and cancel the positive price numerator.

lemma *real-rounded-division-with-slack:*
fixes *base adjusted divisor :: int*
assumes *divisor-positive: 0 < divisor*
and *base-le-adjusted: base ≤ adjusted*
and *adjusted-le: adjusted ≤ base + divisor - 1*
shows *abs (of-int (adjusted div divisor) -*
of-int base / of-int divisor :: real) < 1

Proof sketch: the adjusted numerator is less than one denominator above the ideal numerator. The defining lower and upper bounds of integer division therefore put its quotient less than one real unit on either side of the exact quotient.

6.12.4 Primary-unit proximity

lemma *exchange-v10-amounts-int-repaired-within-ideal:*
assumes *pre: exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
shows *if sint price-n > sint price-d*
then abs
(of-int
(fst (exchange-v10-amounts-int-repaired price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive)) -
ideal-normal-wheat-receive price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive) < 1
else abs
(of-int
(snd (exchange-v10-amounts-int-repaired price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive)) -
ideal-normal-sheep-send price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive) < 1

Proof sketch: split on the more valuable asset and the offer that stays. The two unchanged branches floor the exact common trade value. Each repaired branch can raise that value by less than the primary divisor, and never lowers it; the generic slack lemma therefore bounds the primary whole-unit result within one unit of the continuous ideal.

6.12.5 End-to-end refinement

theorem *exchange-v10-repaired-positive-normal-refines-ideal:*

assumes *pre: exchange-v10-pre price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive*

and result: *exchange-v10-with-options price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive Exchange-Normal
(exact-receive-cap = True, symmetric-exact-receive-cap = True) =
Cxx-Ok exchange-result*

and *wheat-positive:*

0 < sint (num-wheat-received exchange-result)

and *sheep-positive:*

0 < sint (num-sheep-send exchange-result)

shows *normal-result-refines-ideal price-n price-d max-wheat-send*

max-wheat-receive max-sheep-send max-sheep-receive exchange-result

Proof sketch: the repaired integer characterization exposes the exact pre-threshold amount record. Positivity excludes the threshold's zero alternative, fixing the returned stay flag to the integer offer-value comparison, which is the real ideal capacity comparison. The amount characterization and continuous proximity lemma establish the strict one-primary-unit error bound. The positive-success theorem makes the staying offer's favorable effective-price inequality explicit, and the repaired maximality theorem supplies the final canonical lattice round.

6.13 An unconstrained-maker benchmark, kept for reference

The following definitions formalize one possible notion of maker non-interference. They are retained as diagnostic material, not asserted as intended lifecycle properties: it is not yet clear that the unconstrained-maker result is the behavior the protocol should require.

The benchmark function below computes the fill from the posted amount and the taker's limits alone by replacing the maker's receive limit with *int64-max*. This removes maker-state dependence from the exchange arithmetic, including from the decision about which offer stays.

definition *exchange-unconstrained-maker ::*

int32 ⇒ int32 ⇒ int64 ⇒ party-state ⇒ int64 ⇒

exchange-rounding ⇒ exchange-options ⇒ exchange-result-v10 cxx-result

where

exchange-unconstrained-maker price-n price-d amount taker taker-amount

rounding options =
exchange-v10-with-options price-n price-d amount (can-buy-at-most taker)
(signed-min64 taker-amount (can-sell-at-most taker)) int64-max
rounding options

definition *maker-non-interference-at* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *party-state* $\Rightarrow$ *party-state* $\Rightarrow$
int64 $\Rightarrow$ *exchange-rounding* $\Rightarrow$ *exchange-options* $\Rightarrow$ *bool*

where

maker-non-interference-at price-n price-d amount maker taker
taker-amount rounding options $\longleftrightarrow$
(case (cross-offer-v10 price-n price-d amount maker taker taker-amount
rounding options,
exchange-unconstrained-maker price-n price-d amount taker
taker-amount rounding options) of
(Cxx-Ok crossed, Cxx-Ok unconstrained) $\Rightarrow$
cross-wheat-received crossed = num-wheat-received unconstrained $\wedge$
cross-sheep-send crossed = num-sheep-send unconstrained $\wedge$
cross-wheat-stays crossed = result-wheat-stays unconstrained
| - $\Rightarrow$ False)

definition *posted-offers-trade-as-written* :: *exchange-options* $\Rightarrow$ *bool*

where

posted-offers-trade-as-written options $\longleftrightarrow$
($\forall$ price-n price-d amount maker-at-post posted maker-after
maker-at-cross taker taker-amount.
party-state-wf maker-at-post $\wedge$
party-state-wf maker-at-cross $\wedge$
party-state-wf taker $\wedge$
post-offer price-n price-d amount maker-at-post options =
Cxx-Ok (Post-Created posted maker-after) $\wedge$
maker-covers-offer-liabilities price-n price-d posted
maker-at-cross $\wedge$
0 < sint (can-buy-at-most taker) $\wedge$
0 < sint (signed-min64 taker-amount (can-sell-at-most taker)) $\longrightarrow$
maker-non-interference-at price-n price-d posted maker-at-cross
taker taker-amount Exchange-Normal options)

maker-non-interference-at compares a real crossing with that unconstrained-maker benchmark: both calls must succeed and agree on the amount the maker sells, the amount the taker sends, and which side remains. *posted-offers-trade-as-written* lifts the comparison to every successful post, every well-formed crossing state that still covers the offer's reservation, and every admissible normal-mode taker.

These predicates capture a useful, strong comparison, but choosing the unconstrained-maker fill as normative would additionally require maker-side headroom and the resulting *wheatStays* decision to be observationally irrelevant. The counterexamples below record that protocol 28 and protocol 29

do not satisfy this comparison; they do not by themselves establish that the implementation should be changed to satisfy it.

6.13.1 The shipped protocol differs from the benchmark

The grieving replay refutes *posted-offers-trade-as-written* ($\langle \text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False} \rangle$), the benchmark comparison for the shipped protocol. Instantiate it with the walkthrough numbers: Alice posts one TOKEN at 101/100, her state at the crossing step is the one Carol's payment produces, and Bob is the taker. Every premise holds by evaluation, while the crossing fills nothing and the unconstrained-maker exchange fills one TOKEN against one USD. Proof sketch: evaluate the premises and the pointwise benchmark comparison at that instance, then specialize the universal comparison.

lemma *posted-offers-do-not-trade-as-written*:

$\neg \text{posted-offers-trade-as-written } (\langle \text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False} \rangle)$

6.13.2 The exact receive cap repairs the walkthrough, not the benchmark comparison

At protocol 29, the walkthrough grieving disappears: the preventative adjustment values Alice's one-USD headroom as enough for her last unit of wheat, so the same crossing that filled nothing under the plain cap now fills her offer completely. The lemma repeats *carol-grieves-alice* at *repaired-exchange-options* and is proved by evaluating both sides.

lemma *exact-receive-cap-repairs-the-walkthrough*:

post-then-cross 101 100 1
 $\langle \text{sell-balance} = 1, \text{sell-liabilities} = 0,$
 $\text{buy-limit} = 2, \text{buy-balance} = 0, \text{buy-liabilities} = 0 \rangle$
 $\langle \text{sell-balance} = 1, \text{sell-liabilities} = 1,$
 $\text{buy-limit} = 2, \text{buy-balance} = 1, \text{buy-liabilities} = 1 \rangle$
 $\langle \text{sell-balance} = 2, \text{sell-liabilities} = 0,$
 $\text{buy-limit} = 1000, \text{buy-balance} = 0, \text{buy-liabilities} = 0 \rangle$
 $2 \text{ Exchange-Normal repaired-exchange-options} =$
Cxx-Ok
(Post-Created 1
 $\langle \text{sell-balance} = 1, \text{sell-liabilities} = 1,$
 $\text{buy-limit} = 2, \text{buy-balance} = 0, \text{buy-liabilities} = 1 \rangle,$
Some
 $\langle \text{cross-wheat-received} = 1,$
 $\text{cross-sheep-send} = 1,$
 $\text{cross-wheat-stays} = \text{False},$
 $\text{cross-offer-amount} = 0,$
 $\text{cross-maker} =$

$$\begin{aligned}
& (\text{sell-balance} = 0, \text{sell-liabilities} = 0, \\
& \quad \text{buy-limit} = 2, \text{buy-balance} = 2, \text{buy-liabilities} = 0), \\
& \text{cross-taker} = \\
& (\text{sell-balance} = 1, \text{sell-liabilities} = 0, \\
& \quad \text{buy-limit} = 1000, \text{buy-balance} = 1, \text{buy-liabilities} = 0)
\end{aligned}$$

The benchmark comparison nevertheless stays false, because protocol 29 deliberately leaves the *wheatStays* comparison in *exchangeV10WithoutPriceErrorThresholds* on the plain truncating values. A maker whose sheep headroom equals its booked buying liability presents a wheat value up to $\text{mint price} - d - 1$ below the one an unconstrained maker would present, and a taker whose sheep value lands in that gap flips the comparison: against the constrained maker the resting offer is the smaller side and is taken completely, while against the unconstrained benchmark it is the larger side and the fill is zero. The witness below reuses the walkthrough state after Carol's payment; the taker is Bob willing to sell only one USD, whose sheep value 100 lands in the gap between Alice's constrained wheat value 100 and her unconstrained wheat value 101. The crossing then fills one TOKEN against one USD while the benchmark fills nothing, so the maker is not grieved—the offer fills more, not less—but the fill still depends on maker state that the ledger invariants admit. Both exchange outcomes were confirmed against the stellar-core C++. Proof sketch: as in *posted-offers-do-not-trade-as-written*, every premise holds by evaluation. Evaluate the real crossing and the unconstrained benchmark separately: their wheat amounts are one and zero, their sheep amounts are one and zero, and their stay flags are false and true. The proof then exposes these three unequal selector values before unfolding the conjunction required by *maker-non-interference-at*.

lemma *exact-receive-cap-does-not-restore-trade-as-written*:
 $\neg \text{posted-offers-trade-as-written repaired-exchange-options}$

6.14 Proofs and counterexamples: fully taken posted offers

This subsection proves the exact-amount property stated with the intended lifecycle predicates above. It is placed after the benchmark material so the proof can reuse all established posting, coverage, adjustment, and exchange contracts. The key arithmetic observation holds uniformly for every option record: once the resting wheat offer is known not to stay, two calls under the same options no longer differ because of the taker's caps or the rounding mode. The exact receive cap is needed one layer higher, to make liability coverage preserve the posted offer's positive adjustment after the maker state changes. The symmetric receive cap is irrelevant to both adjustments and to the non-staying branch, so the strongest theorem quantifies its value and a final corollary supplies the requested both-enabled record.

lemma *check-exchange-v10-result-success-record:*

assumes *result:*

check-exchange-v10-result max-wheat-send max-wheat-receive

max-sheep-send max-sheep-receive wheat-stays amounts = Cxx-Ok outcome

shows *outcome = make-exchange-result (fst amounts) (snd amounts) wheat-stays*

Proof sketch: unfold the final bounds checker. Every successful branch returns exactly the record made from the checked pair and the supplied stay flag.

lemma *exchange-v10-amounts-nonstaying-independent:*

exchange-v10-amounts price-n price-d wheat-value sheep-value1

max-wheat-send max-wheat-receive1 max-sheep-send1 max-sheep-receive

False rounding1 options =

exchange-v10-amounts price-n price-d wheat-value sheep-value2

max-wheat-send max-wheat-receive2 max-sheep-send2 max-sheep-receive

False rounding2 options

Proof sketch: unfold the amount calculation at a false wheat-stays flag. Both remaining price branches use only the wheat value and maker-side caps; the sheep value, taker-side caps, rounding mode, and symmetric option are unreachable or unused.

lemma *exchange-without-nonstaying-amounts-independent:*

assumes *first:*

exchange-v10-without-price-error-thresholds-with-options price-n price-d

max-wheat-send max-wheat-receive1 max-sheep-send1 max-sheep-receive

rounding1 options = Cxx-Ok result1

and *second:*

exchange-v10-without-price-error-thresholds-with-options price-n price-d

max-wheat-send max-wheat-receive2 max-sheep-send2 max-sheep-receive

rounding2 options = Cxx-Ok result2

and *first-nonstaying: $\neg$ result-wheat-stays result1*

and *second-nonstaying: $\neg$ result-wheat-stays result2*

shows *num-wheat-received result1 = num-wheat-received result2 $\wedge$*

num-sheep-send result1 = num-sheep-send result2

Proof sketch: invert the two successful pre-threshold exchanges through their common wheat-value calculation and final checked constructors. Their false stay flags put both calls in the independent amount branch above, so the amount pairs and hence both result fields agree.

lemma *exchange-v10-nonstaying-matches-positive-adjustment:*

assumes *pn: $0 < \text{sint price-n}$*

and *pd: $0 < \text{sint price-d}$*

```

and wheat-positive:  $0 < \text{sint max-wheat-send}$ 
and sheep-nonnegative:  $0 \leq \text{sint max-sheep-receive}$ 
and adjustment:
  adjust-offer-with-options price-n price-d max-wheat-send max-sheep-receive
options =
  Cxx-Ok max-wheat-send
and exchange:
  exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive
  max-sheep-send max-sheep-receive rounding options = Cxx-Ok result
and nonstaying:  $\neg \text{result-wheat-stays result}$ 
shows num-wheat-received result = max-wheat-send

```

Proof sketch: *adjust-offer-with-options* is the same exchange against an unlimited counterparty in normal mode. A positive fixed adjustment therefore supplies a successful non-staying reference exchange that returns the full wheat cap. Any other successful non-staying exchange with the same maker caps has the same pre-threshold pair. Normal mode makes the threshold calls identical; either path mode must preserve that positive pair on success. Thus arbitrary taker caps and arbitrary rounding return the reference wheat amount.

```

lemma post-offer-symmetric-irrelevant-if-created:
assumes maker-wf: party-state-wf maker
and post:
  post-offer price-n price-d amount maker
  ( $\text{exact-receive-cap} = \text{cap}$ ,  $\text{symmetric-exact-receive-cap} = \text{sym-cap}$ ) =
  Cxx-Ok (Post-Created posted maker-after)
shows
  post-offer price-n price-d amount maker
  ( $\text{exact-receive-cap} = \text{cap}$ ,  $\text{symmetric-exact-receive-cap} = \text{False}$ ) =
  Cxx-Ok (Post-Created posted maker-after)

```

Proof sketch: invert the created result to expose the ready pre-flight caps, the positive adjustment, and liability acquisition. Well-formedness makes both adjustment caps non-negative, so *adjust-offer-symmetric-irrelevant* replaces the symmetric option by false without changing the adjusted amount or the acquired maker state.

```

lemma cover-implies-adjust-stable-exact-any-symmetric:
fixes sym-cap :: bool
shows cover-implies-adjust-stable
  ( $\text{exact-receive-cap} = \text{True}$ ,  $\text{symmetric-exact-receive-cap} = \text{sym-cap}$ )

```

Proof sketch: normalize a successful post to a false symmetric option. The normalized post can be replayed as an unlimited adjustment returning the

posted amount. Liability coverage and release expose enough selling capacity for that amount and enough buying capacity for its booked rounded payment. The exact receive cap turns those two capacity bounds back into the same adjustment result, so at a false symmetric option the crossing-time preventative adjustment is the identity. Covered liability release yields a well-formed released state and non-negative adjustment caps; symmetric irrelevance then transports the stable result back to the original arbitrary symmetric option.

theorem *cover-implies-adjust-stable-repaired:*

cover-implies-adjust-stable

$(\text{exact-receive-cap} = \text{True}, \text{symmetric-exact-receive-cap} = \text{True})$

Proof sketch: instantiate the symmetric-option-independent exact-cap result at true, the record requested for both repairs enabled.

lemma *successful-nonstaying-cross-fields:*

assumes *cross:*

cross-offer-v10 price-n price-d offer-amount maker taker taker-amount
rounding options = Cxx-Ok crossed

and *nonstaying:* $\neg \text{cross-wheat-stays crossed}$

obtains *released adjusted exchanged where*

release-offer-liabilities price-n price-d offer-amount maker =
Cxx-Ok released

adjust-offer-with-options price-n price-d
(signed-min64 offer-amount (can-sell-at-most released))
(can-buy-at-most released) options = Cxx-Ok adjusted

exchange-v10-with-options price-n price-d
(signed-min64 adjusted (can-sell-at-most released))
(can-buy-at-most taker)

(signed-min64 taker-amount (can-sell-at-most taker))
(can-buy-at-most released) rounding options = Cxx-Ok exchanged

cross-wheat-received crossed = num-wheat-received exchanged

$\neg \text{result-wheat-stays exchanged}$

cross-offer-amount crossed = 0

Proof sketch: invert the successful crossing through liability release, preventative adjustment, exchange, and the four balance moves. A true inner stay flag always constructs a result whose exposed crossing flag is true, including the dust-erasure branch, contradicting the premise. The remaining branch copies the inner wheat amount, preserves the false flag, and writes a zero remaining offer amount.

theorem *fully-taken-posted-offer-exchanges-posted-amount-exact-any-symmetric:*

fixes *sym-cap* :: *bool*
shows *fully-taken-posted-offer-exchanges-posted-amount*
 $\langle \text{exact-receive-cap} = \text{True}, \text{symmetric-exact-receive-cap} = \text{sym-cap} \rangle$

Proof sketch: invert the successful non-staying cross, which also exposes that its branch sets the remaining offer amount to zero. The both-state lifecycle premises and the exact cap make the preventative adjustment return the positive posted amount. Positive adjustment idempotence proves that the maker's send cap also equals that amount. The flag-independent exchange lemma then shows that the arbitrary taker's successful non-staying exchange returns the whole posted amount for every rounding mode. The symmetric flag remains universally quantified because neither adjustment nor this exchange branch consults it.

theorem *fully-taken-posted-offer-exchanges-posted-amount-repaired:*
fully-taken-posted-offer-exchanges-posted-amount
 $\langle \text{exact-receive-cap} = \text{True}, \text{symmetric-exact-receive-cap} = \text{True} \rangle$

Proof sketch: instantiate the exact-cap theorem at a true symmetric cap, retaining the user-requested both-enabled packaging.

theorem *fully-taken-posted-offer-exchanges-posted-amount-legacy-false:*
 $\neg$ *fully-taken-posted-offer-exchanges-posted-amount*
 $\langle \text{exact-receive-cap} = \text{False}, \text{symmetric-exact-receive-cap} = \text{False} \rangle$

Proof sketch: instantiate the existing Alice-and-Carol reservation anomaly. Alice's well-formed post creates one unit, Carol's intervening payment leaves a well-formed maker state that still covers both liabilities, and the later successful cross reports a false stay flag and erases the offer while exchanging zero wheat. This contradicts the claimed equality with the posted one unit.

lemma *fully-taken-incoming-offer-positive-partial-witness:*
party-state-wf
 $\langle \text{sell-balance} = 4, \text{sell-liabilities} = 0, \\ \text{buy-limit} = 10, \text{buy-balance} = 0, \text{buy-liabilities} = 0 \rangle \wedge$
party-state-wf
 $\langle \text{sell-balance} = 4, \text{sell-liabilities} = 4, \\ \text{buy-limit} = 10, \text{buy-balance} = 0, \text{buy-liabilities} = 6 \rangle \wedge$
post-offer 3 2 4
 $\langle \text{sell-balance} = 4, \text{sell-liabilities} = 0, \\ \text{buy-limit} = 10, \text{buy-balance} = 0, \text{buy-liabilities} = 0 \rangle$
 $\langle \text{exact-receive-cap} = \text{True}, \\ \text{symmetric-exact-receive-cap} = \text{True} \rangle =$
Cxx-Ok

(Post-Created 4
 (|sell-balance = 4, sell-liabilities = 4,
 buy-limit = 10, buy-balance = 0, buy-liabilities = 6|) ∧
 maker-covers-offer-liabilities 3 2 4
 (|sell-balance = 4, sell-liabilities = 4,
 buy-limit = 10, buy-balance = 0, buy-liabilities = 6|) ∧
 party-state-wf
 (|sell-balance = 4, sell-liabilities = 0,
 buy-limit = 10, buy-balance = 0, buy-liabilities = 0|) ∧
 preflight-offer 2 3 4
 (|sell-balance = 4, sell-liabilities = 0,
 buy-limit = 10, buy-balance = 0, buy-liabilities = 0|) =
 Cxx-Ok (Preflight-Ready 4 10) ∧
 (case cross-offer-v10 3 2 4
 (|sell-balance = 4, sell-liabilities = 4,
 buy-limit = 10, buy-balance = 0, buy-liabilities = 6|)
 (|sell-balance = 4, sell-liabilities = 0,
 buy-limit = 10, buy-balance = 0, buy-liabilities = 0|)
 4 Exchange-Normal
 (|exact-receive-cap = True,
 symmetric-exact-receive-cap = True|) of
 Cxx-Ok crossed ⇒
 cross-wheat-received crossed = 2 ∧
 cross-sheep-send crossed = 3 ∧
 cross-offer-amount crossed = 2 ∧
 cross-wheat-stays crossed
 | Cxx-Err - ⇒ False)

Proof sketch: executable evaluation confirms the full resting lifecycle: the initial and later maker states are well formed, posting creates the actual four-wheat offer and acquires its four selling and six buying liabilities, and the later state covers them. Reciprocal *ManageSellOffer* pre-flight accepts the raw four-sheep request without clipping and with ten units of wheat receive headroom, so the request is send-amount limited rather than receive-headroom limited. At price three halves, three sheep buy two whole wheat at the exact price while the fourth sheep cannot buy another whole wheat. Normal price-error filtering keeps this positive result and its true wheat-stays flag. Two wheat remain on the resting offer, while the accepted transfer is two wheat for only three of the four incoming sheep.

theorem *fully-taken-incoming-offer-exchanges-incoming-amount-repaired-false:*
 ¬ *fully-taken-incoming-offer-exchanges-incoming-amount*
 (|exact-receive-cap = True, symmetric-exact-receive-cap = True|)

Proof sketch: specialize the universal property to the executable witness above. Its well-formed maker posts four wheat, the well-formed crossing state still covers that actual offer's liabilities, its well-formed taker's positive

request passes reciprocal pre-flight with the request itself as the send cap, and successful crossing reports that wheat stays. The property would therefore equate the observed three-sheep transfer with the incoming amount four, a contradiction. Hence enabling both exact receive caps does not make the wheat-stays flag an exact full-send certificate after integer exchange rounding, even when the accepted exchange is positive and exactly priced.

lemma *exchange-v10-bounds-any-cap:*

assumes *exchange:*

*exchange-v10-with-options price-n price-d max-wheat-send max-wheat-receive
max-sheep-send max-sheep-receive rounding options = Cxx-Ok result*

shows

$0 \leq \text{sint } (\text{num-wheat-received result})$
 $\text{sint } (\text{num-wheat-received result}) \leq$
 $\min (\text{sint max-wheat-receive}) (\text{sint max-wheat-send})$
 $0 \leq \text{sint } (\text{num-sheep-send result})$
 $\text{sint } (\text{num-sheep-send result}) \leq$
 $\min (\text{sint max-sheep-receive}) (\text{sint max-sheep-send})$

Proof sketch: decompose the full exchange into its checked pre-threshold result and the common threshold call. The checked result satisfies all four cap bounds. Every successful threshold branch returns either that original amount pair or the explicit zero pair, in all three rounding modes, so the same inequalities hold without assuming a positive normal result.

7 Strict-mode contracts across the exact-cap boundary

Every strict-mode result of the arithmetic theory is pinned to *legacy-exchange-options*, because it is derived from *exchange-v10-amounts-int*, the exact-integer abstraction that hard-codes the legacy branch formulas. Both strict modes can nevertheless reach a repaired branch: strict send and strict receive both bypass the symmetric wheat-stays branch, but neither bypasses the *exact-receive-cap* branch taken when the wheat offer does not stay and wheat is the more valuable asset. So nothing about path payments was proved on the protocol-29 side of the boundary.

This section closes that gap without an options-parametric integer abstraction, by observing where the strict-mode guarantees actually come from. *apply-price-error-thresholds* does not inspect the options, and *exchange-without-thresholds-bounds-any-cap* already proves the four cap bounds for every options record, because the pre-threshold exchange passes its amount pair through one common checked constructor whatever the branch com-

puted. The strict-mode contracts are consequences of exactly those two facts, so they hold at every options record — the two real protocols and the two diagnostic mutants alike — and need neither *exchange-v10-pre* nor any branch formula.

lemma *exchange-v10-decompose-any-options:*

assumes *result:*

exchange-v10-with-options price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding options =
Cxx-Ok exchange-result

obtains *before-thresholds where*

exchange-v10-without-price-error-thresholds-with-options price-n price-d
max-wheat-send max-wheat-receive max-sheep-send max-sheep-receive
rounding options = Cxx-Ok before-thresholds

and *apply-price-error-thresholds price-n price-d*

(num-wheat-received before-thresholds)
(num-sheep-send before-thresholds)
(result-wheat-stays before-thresholds) rounding =
Cxx-Ok exchange-result

Proof sketch: the full exchange is one monadic bind of the pre-threshold calculation into the threshold pass, so a successful outcome forces both calls to have succeeded, with the second applied to the fields of the first.

7.1 Strict send

theorem *exchange-v10-strict-send-sheep-positive-any-options:*

assumes *result: exchange-v10-with-options price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
Exchange-Strict-Send options = Cxx-Ok exchange-result

shows $0 < \text{sint } (\text{num-sheep-send } \text{exchange-result})$

— A successful strict-send exchange always sends a strictly positive amount of sheep, at every exact-cap configuration.

Proof sketch: decompose the exchange into its pre-threshold result and the threshold pass. Strict-send success returns the pre-threshold record unchanged and proves its sheep word nonzero, and the common checked constructor already made that word a non-negative signed amount, so nonzero strengthens to strictly positive. Neither step looks at the options.

corollary *exchange-v10-strict-send-sheep-positive-repaired:*

assumes *result: exchange-v10-with-options price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
Exchange-Strict-Send repaired-exchange-options = Cxx-Ok exchange-result

shows $0 < \text{sint } (\text{num-sheep-send } \text{exchange-result})$

— The protocol-29 instance of strict-send positivity.

exchange-v10-strict-send-sheep-positive-any-options is a strict generalization of *exchange-v10-strict-send-sheep-positive*: it drops both the options record and the *exchange-v10-pre* hypothesis. The legacy theorem needed the precondition only to route the argument through the exact-integer abstraction, which is what tied it to one configuration; the shorter route above needs neither.

The corresponding *iff* results, *exchange-v10-strict-send-sheep-positive-iff* and *exchange-v10-without-strict-send-sheep-positive-iff*, do not generalize this way. They compute the exact positivity threshold from the branch formulas, and the repaired non-staying branch replaces the plain wheat value by the relaxed exact-cap value, so the right-hand side changes. Deciding the repaired threshold requires the options-parametric integer abstraction and is not attempted here; what is proved instead is the positivity direction, which is the direction the lifecycle results consume.

7.2 Strict receive, and simultaneous zero in general

theorem *apply-price-error-thresholds-zero-iff*:

assumes *not-strict-send*: *rounding* $\neq$ *Exchange-Strict-Send*

and result:

apply-price-error-thresholds price-n price-d wheat-receive sheep-send

wheat-stays rounding = Cxx-Ok exchange-result

shows (*num-wheat-received exchange-result = 0*) =

(*num-sheep-send exchange-result = 0*)

— Outside strict-send mode the threshold pass never returns a half-zero trade, whatever amount pair it was given.

Proof sketch: read the total decision tree. The unchanged record is returned only under the guard that both incoming amounts are strictly positive, so both its words are nonzero; every other successful branch outside strict-send mode is the explicit zero record. Strict send is excluded because it is the one branch that returns the unchanged record without that guard.

theorem *exchange-v10-zero-iff-any-options*:

assumes *not-strict-send*: *rounding* $\neq$ *Exchange-Strict-Send*

and result: *exchange-v10-with-options price-n price-d max-wheat-send*

max-wheat-receive max-sheep-send max-sheep-receive rounding options =

Cxx-Ok exchange-result

shows (*num-wheat-received exchange-result = 0*) =

(*num-sheep-send exchange-result = 0*)

— Outside strict-send mode a successful exchange transfers either both assets or neither, at every exact-cap configuration.

Proof sketch: decompose the exchange and apply the threshold pass's own simultaneous-zero property to whatever amount pair the branch block produced.

exchange-v10-zero-iff-any-options subsumes *exchange-v10-zero-iff*, dropping both the options record and *exchange-v10-pre*. It is the load-bearing half of the strict-receive contract: the amount-block branch formulas do not enter, so the repaired branches are covered for free.

theorem *exchange-v10-positive-trade-contract-any-options*:

assumes *result: exchange-v10-with-options price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive rounding options =
Cxx-Ok exchange-result

and *positive*:

$0 < \text{sint } (\text{num-wheat-received } \text{exchange-result}) \wedge$
 $0 < \text{sint } (\text{num-sheep-send } \text{exchange-result})$

shows *favoured-seller-ok price-n price-d*
(num-wheat-received exchange-result)
(num-sheep-send exchange-result)
(result-wheat-stays exchange-result)

and *if rounding = Exchange-Normal*
then price-error-bound-spec price-n price-d
(num-wheat-received exchange-result)
(num-sheep-send exchange-result) False
else price-error-bound-spec price-n price-d
(num-wheat-received exchange-result)
(num-sheep-send exchange-result) True

— A positive trade favours whichever offer stays and meets its rounding mode's price-error bound, at every exact-cap configuration.

Proof sketch: decompose the exchange. A positive returned trade cannot be the explicit zero record, so the threshold choice theorem identifies it with the pre-threshold record, whose amounts are therefore positive too. The threshold caller's favoured-seller and price-bound contracts then apply and their fields are the returned ones.

theorem *exchange-v10-strict-receive-contract-any-options*:

assumes *result: exchange-v10-with-options price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
Exchange-Strict-Receive options = Cxx-Ok exchange-result

shows $(\text{num-wheat-received } \text{exchange-result} = 0) =$
 $(\text{num-sheep-send } \text{exchange-result} = 0)$

and $0 \leq \text{sint } (\text{num-wheat-received } \text{exchange-result})$

and $\text{sint } (\text{num-wheat-received } \text{exchange-result}) \leq$
 $\min (\text{sint } \text{max-wheat-receive}) (\text{sint } \text{max-wheat-send})$

and $0 \leq \text{sint } (\text{num-sheep-send } \text{exchange-result})$

and $\text{sint } (\text{num-sheep-send } \text{exchange-result}) \leq$
 $\min (\text{sint } \text{max-sheep-receive}) (\text{sint } \text{max-sheep-send})$

and $0 < \text{sint } (\text{num-wheat-received } \text{exchange-result}) \implies$
favoured-seller-ok price-n price-d
(num-wheat-received exchange-result)
(num-sheep-send exchange-result)

$(\text{result-wheat-stays } \text{exchange-result}) \wedge$
 $\text{price-error-bound-spec } \text{price-n } \text{price-d}$
 $(\text{num-wheat-received } \text{exchange-result})$
 $(\text{num-sheep-send } \text{exchange-result}) \text{ True}$

— The strict-receive contract: a successful strict-receive exchange transfers both assets or neither, respects all four caps, and, when it transfers anything, favours whichever offer stays and meets the tight one-sided price-error bound.

Proof sketch: the first claim is the general simultaneous-zero theorem, the four cap claims are the option-free bounds of a successful exchange, and the last is the positive-trade contract, whose non-normal branch is the tight bound. A positive wheat amount forces a positive sheep amount through the first claim and the non-negativity of the sheep amount.

corollary *exchange-v10-strict-receive-contract-repaired:*

assumes *result: exchange-v10-with-options price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
Exchange-Strict-Receive repaired-exchange-options =
Cxx-Ok exchange-result

shows $(\text{num-wheat-received } \text{exchange-result} = 0) =$

$(\text{num-sheep-send } \text{exchange-result} = 0)$

and $0 < \text{sint } (\text{num-wheat-received } \text{exchange-result}) \implies$

$\text{price-error-bound-spec } \text{price-n } \text{price-d}$
 $(\text{num-wheat-received } \text{exchange-result})$
 $(\text{num-sheep-send } \text{exchange-result}) \text{ True}$

— The protocol-29 instance of the strict-receive contract.

8 Protocol-29 property catalogue

The results above are stated for an *exchange-options* record, so each covers both real protocols and the two diagnostic mutants at once. This section packages the ones that matter at the protocol boundary itself, phrased through *exchange-options-at-version* so that the hypothesis is a ledger version rather than a choice of flags. Each is a short specialization: no property is re-proved here.

Takeability after maker-state changes and limit-adjustment stability need the stored-offer invariant, so their protocol-29 forms are proved in the stored-offer theory: *posted-offers-remain-takeable-p29*, *limit-adjustment-stable-p29* and *limit-adjustment-stable-p28-false* for the ManageSell route, *posted-buy-offers-remain-takeable-p29* and *buy-limit-adjustment-stable-p29* for the ManageBuy route, and *posted-request-offers-remain-takeable-p29* for both.

corollary *positive-normal-crosses-are-maximal-p29:*

protocol-version-starts-from ledger-version protocol-version-v29 $\implies$

positive-normal-crosses-are-maximal
(exchange-options-at-version ledger-version)

— From protocol 29 on, a positive normal cross transfers as much wheat as any admissible trade could.

corollary *cover-implies-adjust-stable-p29:*

protocol-version-starts-from ledger-version protocol-version-v29 $\implies$
cover-implies-adjust-stable (exchange-options-at-version ledger-version)

— From protocol 29 on, an offer whose liabilities its maker still covers is not reduced by adjustment.

corollary *fully-taken-posted-offer-exchanges-posted-amount-p29:*

protocol-version-starts-from ledger-version protocol-version-v29 $\implies$
fully-taken-posted-offer-exchanges-posted-amount
(exchange-options-at-version ledger-version)

— From protocol 29 on, a resting offer that does not stay transfers its actual stored amount.

corollary *fully-taken-posted-offer-exchanges-posted-amount-p28-false:*

$\neg$ *protocol-version-starts-from ledger-version protocol-version-v29* $\implies$
 $\neg$ *fully-taken-posted-offer-exchanges-posted-amount*
(exchange-options-at-version ledger-version)

— The same property is false at protocol 28, so this is a behavioral repair and not merely a restatement.

The pair above is the sharpest statement of what protocol 29 changes in the lifecycle: at protocol 28 a fully taken resting offer need not transfer its stored amount, and from protocol 29 it always does.

corollary *exchange-v10-p29-positive-normal-refines-ideal:*

assumes *version:*

protocol-version-starts-from ledger-version protocol-version-v29

and *pre: exchange-v10-pre price-n price-d max-wheat-send*

max-wheat-receive max-sheep-send max-sheep-receive

and *result: exchange-v10 ledger-version price-n price-d max-wheat-send*

max-wheat-receive max-sheep-send max-sheep-receive Exchange-Normal =

Cxx-Ok exchange-result

and *wheat-positive: 0 < sint (num-wheat-received exchange-result)*

and *sheep-positive: 0 < sint (num-sheep-send exchange-result)*

shows *normal-result-refines-ideal price-n price-d max-wheat-send*

max-wheat-receive max-sheep-send max-sheep-receive exchange-result

— A positive normal protocol-29 exchange refines the ideal specification.

Proof sketch: the protocol mapping turns the public call at a version from 29 on into the repaired kernel, and the existing repaired refinement theorem applies to that call unchanged.

corollary *rounding-favors-offer-that-stays-p29:*

rounding-favors-offer-that-stays (exchange-options-at-version ledger-version)

— Rounding favours whichever offer stays, at every ledger version. The protocol boundary does not enter: the guarantee comes from the threshold pass, which never inspects the exact-cap options.

corollary *adjust-offer-positive-idempotent-p29:*

assumes *pn*: $0 < \text{sint } \text{price-n}$

and *pd*: $0 < \text{sint } \text{price-d}$

and *wheat-nonnegative*: $0 \leq \text{sint } \text{max-wheat}$

and *sheep-nonnegative*: $0 \leq \text{sint } \text{max-sheep}$

and *adjusted*:

adjust-offer ledger-version price-n price-d max-wheat max-sheep =
Cxx-Ok result

and *result-positive*: $0 < \text{sint } \text{result}$

shows

adjust-offer ledger-version price-n price-d result max-sheep =
Cxx-Ok result $\wedge$

sint result $\leq \text{sint } \text{max-wheat}$

— Re-adjusting a positive adjustment result changes nothing, at every ledger version.

Proof sketch: the underlying idempotence result already quantifies over the options record, so the protocol-facing call is an instance of it at the record the version selects.

corollary *exchange-v10-p29-integer-characterization:*

fixes *amounts* :: $\text{int} \times \text{int}$

assumes *version*:

protocol-version-starts-from ledger-version protocol-version-v29

and *pre*: *exchange-v10-pre price-n price-d max-wheat-send*

max-wheat-receive max-sheep-send max-sheep-receive

defines *amounts* $\equiv$

exchange-v10-amounts-int-repaired price-n price-d max-wheat-send

max-wheat-receive max-sheep-send max-sheep-receive

shows *exchange-v10 ledger-version price-n price-d max-wheat-send*

max-wheat-receive max-sheep-send max-sheep-receive Exchange-Normal =

apply-price-error-thresholds price-n price-d

(word-of-int (fst amounts)) (word-of-int (snd amounts))

(exchange-wheat-value-int price-n price-d max-wheat-send

max-sheep-receive >

exchange-sheep-value-int price-n price-d max-sheep-send

max-wheat-receive)

Exchange-Normal

— The protocol-29 public call is exactly the repaired exact-integer calculation followed by the threshold pass.

Proof sketch: the version hypothesis reduces the public call to the repaired kernel, where the established integer characterization applies unchanged.

exchange-v10-p29-integer-characterization carries three of the protocol-29 obligations at once. It is the totality and determinism statement for the protocol-29 public function, since the right-hand side is a branch-free calculation over exact integers that is defined for every input satisfying *exchange-v10-pre*. It is the C++ error characterization, because *apply-price-error-thresholds* is where the modeled failures are produced. And the retained-offer flag it passes is the comparison of the two *plain* offer values, so the *wheatStays* decision is visibly unchanged by the repair.

8.1 The exact-integer image at the protocol interface

corollary *exchange-v10-p29-options-integer-characterization*:

fixes *amounts* :: *int* × *int*
and *rounding* :: *exchange-rounding* **and** *ledger-version* :: *uint32*
assumes *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
defines *amounts* ≡
exchange-v10-amounts-int-with-options price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive rounding
(exchange-options-at-version ledger-version)
shows *exchange-v10 ledger-version price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive rounding =
apply-price-error-thresholds price-n price-d
(word-of-int (fst amounts)) (word-of-int (snd amounts))
(exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive >
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive)
rounding

— The protocol-facing public call, at every ledger version and in all three rounding modes, is the exact-integer calculation for that version followed by the threshold pass.

Proof sketch: the public call is the kernel at the options record the version selects, where the options-parametric integer characterization applies unchanged.

exchange-v10-p29-options-integer-characterization is the version of *exchange-v10-p29-integer-characterization* that drops both restrictions of the latter: it holds in the two strict rounding modes as well as normal mode, and before the boundary as well as from protocol 29 on. The normal-mode protocol-29 instance is the earlier theorem, because the collapse equation *exchange-v10-amounts-int-with-options-repaired-normal* identifies the two abstractions there.

corollary *exchange-v10-p29-strict-send-sheep-positive-iff*:

assumes *version*:
protocol-version-starts-from ledger-version protocol-version-v29

and *pre*: *exchange-v10-pre price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
shows ($0 < \text{snd}$
(exchange-v10-amounts-int-with-options price-n price-d max-wheat-send
max-wheat-receive max-sheep-send max-sheep-receive
Exchange-Strict-Send
(exchange-options-at-version ledger-version))) =
(let wheat-value =
exchange-wheat-value-int price-n price-d max-wheat-send
max-sheep-receive;
sheep-value =
exchange-sheep-value-int price-n price-d max-sheep-send
max-wheat-receive
in if wheat-value > sheep-value
then $0 < \text{sint max-sheep-send} \wedge 0 < \text{sint max-sheep-receive}$
else if $\text{sint price-n} > \text{sint price-d}$
then $\text{sint price-n} \leq$
exact-trade-value-int price-n price-d max-wheat-send
max-sheep-receive
else $\text{sint price-d} \leq \text{wheat-value}$)
 — The protocol-29 strict-send positivity threshold, stated through the protocol mapping.

Proof sketch: the version hypothesis reduces the mapped options to the repaired record, where the established threshold applies.

8.2 Strict-mode contracts through the protocol mapping

The strict-mode results of the preceding section hold at every options record, so their protocol-facing forms need no version hypothesis at all. That is the point worth recording: path payments were the part of the interface with no protocol-29 statement, and what the boundary changes for them is nothing.

corollary *exchange-v10-strict-send-sheep-positive-p29*:

assumes *result*: *exchange-v10 ledger-version price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
Exchange-Strict-Send = Cxx-Ok exchange-result

shows $0 < \text{sint} (\text{num-sheep-send exchange-result})$

— A successful strict-send exchange sends a positive amount of sheep, at every ledger version and in particular from protocol 29 on.

corollary *exchange-v10-zero-iff-p29*:

assumes *not-strict-send*: *rounding $\neq$ Exchange-Strict-Send*

and *result*: *exchange-v10 ledger-version price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive rounding =
Cxx-Ok exchange-result

shows *(num-wheat-received exchange-result = 0) =*
(num-sheep-send exchange-result = 0)

— Outside strict-send mode a successful exchange transfers both assets or neither, at every ledger version.

corollary *exchange-v10-strict-receive-contract-p29*:

assumes *result: exchange-v10 ledger-version price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive
Exchange-Strict-Receive = Cxx-Ok exchange-result

shows $(\text{num-wheat-received } \text{exchange-result} = 0) =$
 $(\text{num-sheep-send } \text{exchange-result} = 0)$
and $0 \leq \text{sint } (\text{num-wheat-received } \text{exchange-result})$
and $\text{sint } (\text{num-wheat-received } \text{exchange-result}) \leq$
 $\min (\text{sint } \text{max-wheat-receive}) (\text{sint } \text{max-wheat-send})$
and $0 \leq \text{sint } (\text{num-sheep-send } \text{exchange-result})$
and $\text{sint } (\text{num-sheep-send } \text{exchange-result}) \leq$
 $\min (\text{sint } \text{max-sheep-receive}) (\text{sint } \text{max-sheep-send})$
and $0 < \text{sint } (\text{num-wheat-received } \text{exchange-result}) \implies$
avored-seller-ok price-n price-d
 $(\text{num-wheat-received } \text{exchange-result})$
 $(\text{num-sheep-send } \text{exchange-result})$
 $(\text{result-wheat-stays } \text{exchange-result}) \wedge$
price-error-bound-spec price-n price-d
 $(\text{num-wheat-received } \text{exchange-result})$
 $(\text{num-sheep-send } \text{exchange-result}) \text{ True}$

— The strict-receive contract at the protocol-facing interface, at every ledger version.

corollary *exchange-v10-positive-trade-contract-p29*:

assumes *result: exchange-v10 ledger-version price-n price-d max-wheat-send*
max-wheat-receive max-sheep-send max-sheep-receive rounding =
Cxx-Ok exchange-result

and *positive*:
 $0 < \text{sint } (\text{num-wheat-received } \text{exchange-result}) \wedge$
 $0 < \text{sint } (\text{num-sheep-send } \text{exchange-result})$

shows *avored-seller-ok price-n price-d*
 $(\text{num-wheat-received } \text{exchange-result})$
 $(\text{num-sheep-send } \text{exchange-result})$
 $(\text{result-wheat-stays } \text{exchange-result})$
and *if rounding = Exchange-Normal*
then price-error-bound-spec price-n price-d
 $(\text{num-wheat-received } \text{exchange-result})$
 $(\text{num-sheep-send } \text{exchange-result}) \text{ False}$
else price-error-bound-spec price-n price-d
 $(\text{num-wheat-received } \text{exchange-result})$
 $(\text{num-sheep-send } \text{exchange-result}) \text{ True}$

— The positive-trade contract at the protocol-facing interface, in all three rounding modes and at every ledger version.

8.3 Properties protocol 29 does not promise

corollary *fully-taken-incoming-offer-exchanges-incoming-amount-p29-false:*

protocol-version-starts-from ledger-version protocol-version-v29 $\implies$

*$\neg$ fully-taken-incoming-offer-exchanges-incoming-amount
(exchange-options-at-version ledger-version)*

— Consuming the incoming offer still does not certify that its entire raw requested amount was exchanged.

This negative result is retained deliberately. The repair is about the resting offer’s stored amount, not about the incoming request: a positive cross that leaves the resting offer in the book can still transfer less wheat than the taker asked for, and no definition should be weakened to make the incoming-side claim true. Stating it against *exchange-options-at-version* keeps the distinction visible to reviewers between what protocol 29 repairs and what it does not.

end

theory *Offer-Exchange-Stored-Offers*

imports *Offer-Exchange-Lifecycle*

begin

9 Stored offers

This theory studies a single resting offer through the stored-offer invariant *stored-offer*: positive price components, a positive stored amount, and an unlimited adjustment that leaves the amount unchanged. The invariant mentions no party state and does not depend on the protocol version, because both option records that the protocol mapping selects adjust an offer identically against an unlimited cap (*stored-offer-iff-repaired* below).

The results below show that a covered offer satisfying the invariant is safe to cross at protocol 29 and has a maximum-capacity full-take witness, and that any positive remainder left by a successful crossing satisfies the invariant again. The last two sections derive the protocol-29 takeability and limit-adjustment stability properties for the ManageSell and ManageBuy routes. The migration theory applies the same results to offers posted at protocol 28 and crossed at protocol 29.

In real ledger state, the local assumption *maker-covers-offer-liabilities* is supplied by the global invariant relating aggregate liabilities to all open offers.

definition *stored-offer* ::

int32 $\Rightarrow$ int32 $\Rightarrow$ int64 $\Rightarrow$ bool **where**

stored-offer price-n price-d posted $\longleftrightarrow$

0 < sint price-n $\wedge$

0 < sint price-d $\wedge$

$0 < \text{ sint posted} \wedge$
 $\text{adjust-offer-with-options price-n price-d posted int64-max}$
 $\text{legacy-exchange-options} = \text{Cxx-Ok posted}$

stored-offer is deliberately offer-local: it records positive price components, a positive stored amount, and the unlimited legacy adjustment fixed point that crossing needs. It does not duplicate *adjust-stable*, which also depends on a particular party state and its current capacities. The same unlimited fixed-point equation already occurs as premises or conclusions of lifecycle lemmas, but the imported theories do not give that conjunction a name.

The following evaluations cover the legacy Alice offer from *alice-posts-her-offer* and the non-unit three-halves resting offer used by *fully-taken-incoming-offer-positive-partial-witness*. They confirm that the intrinsic predicate accepts representative existing witnesses on both the near-unit and larger-price cases.

lemma *stored-offer-representative-examples*:

$\text{stored-offer } 101 \ 100 \ 1$
 $\text{stored-offer } 3 \ 2 \ 4$

Proof sketch: unfold the executable predicate and evaluate the two positive unlimited legacy adjustments.

9.1 Lifecycle results used here

The proofs below reuse the following lifecycle results rather than unfold the bit-precise arithmetic again.

- Protocol boundary and provenance: *legacy-exchange-options*, *repaired-exchange-options*, *post-offer*, *post-buy-offer*, *cross-offer-v10*, and *post-created-positive-facts*. Legacy request routes can additionally use *post-offer-request-created-facts*. The unlimited fixed point of a successful legacy posting comes from the legacy replay lemma proved below, with no admission filter involved.
- Persistent liabilities and preventative adjustment: *offer-liabilities-result*, *covered-offer-release*, *unlimited-positive-adjustment-identifies-liabilities*, *exact-cap-replays-unlimited-positive-adjustment*, and *adjust-offer-symmetric-irrelevant*. The same-version packaging in *cover-implies-adjust-stable-exact-any-symmetric* is a useful proof template, but its posting premise must not be reused as if it crossed a protocol boundary.
- Option compatibility at unlimited caps: *exchange-v10-amounts-exact-irrelevant-if-not-wheat-more*, *exchange-v10-exact-irrelevant-if-not-wheat-more*, *exchange-v10-amounts-symmetric-irrelevant*

if-not-wheat-stays, and *exact-receive-cap-value-int*. The repaired integer characterizations *exchange-v10-without-repaired-integer-characterization* and *exchange-v10-repaired-integer-characterization* expose the retained saturation clamp when a direct compatibility proof needs it.

- Successful-cross safety: *exchange-without-thresholds-bounds-any-cap*, *exchange-normal-positive-result-bounds-any-cap*, *party-receive-within-capacity*, *party-spend-within-capacity*, and *successful-nonstaying-cross-fields*. These separate cap safety and checked party updates from the repaired branch arithmetic.
- Full-take and existential witness: *exchange-v10-nonstaying-matches-positive-adjustment*, *exchange-against-unlimited-counterparty-does-not-leave-wheat*, *maximum-capacity-taker-equations*, and the proof structure of *fully-taken-posted-offer-exchanges-posted-amount-exact-any-symmetric*.

9.2 Persistent liability compatibility

lemma *unrestricted-repaired-amounts-match-legacy*:

assumes *wheat-value*:

calculate-offer-value price-n price-d posted int64-max =
Cxx-Ok wheat-value

and *sheep-value*:

calculate-offer-value price-d price-n int64-max int64-max =
Cxx-Ok sheep-value

shows

exchange-v10-amounts price-n price-d wheat-value sheep-value posted
int64-max int64-max int64-max wheat-stays Exchange-Normal
repaired-exchange-options =
exchange-v10-amounts price-n price-d wheat-value sheep-value posted
int64-max int64-max int64-max wheat-stays Exchange-Normal
legacy-exchange-options

Proof sketch: split on the stored-offer side and relative price order. The two option-sensitive branches then use an unrestricted receive cap, where the retained clamp makes the exact value equal the plain value.

lemma *unrestricted-repaired-exchange-matches-legacy*:

exchange-v10-without-price-error-thresholds-with-options
price-n price-d posted int64-max int64-max int64-max
Exchange-Normal repaired-exchange-options =
exchange-v10-without-price-error-thresholds-with-options
price-n price-d posted int64-max int64-max int64-max

Exchange-Normal legacy-exchange-options

Proof sketch: both option-sensitive receive caps are the signed maximum. The retained global clamp therefore collapses each relaxed offer-value call to its plain counterpart, after which both branch calculations and final bounds checks are syntactically identical.

corollary *unrestricted-repaired-adjustment-matches-legacy:*

$$\begin{aligned} & \text{adjust-offer-with-options } \text{price-n } \text{price-d } \text{posted int64-max} \\ & \text{repaired-exchange-options} = \\ & \text{adjust-offer-with-options } \text{price-n } \text{price-d } \text{posted int64-max} \\ & \text{legacy-exchange-options} \end{aligned}$$

Proof sketch: adjustment projects the wheat field of the unrestricted normal exchange, so the preceding equality applies directly.

Stored liabilities themselves remain option-free. The preceding results additionally show that explicitly threading both repaired flags through an unrestricted projection would compute the same exchange record as the legacy projection; this is the clamp-dependent protocol-boundary fact.

lemma *stored-offer-iff-repaired:*

$$\begin{aligned} & \text{stored-offer } \text{price-n } \text{price-d } \text{posted} \longleftrightarrow \\ & 0 < \text{sint } \text{price-n} \wedge 0 < \text{sint } \text{price-d} \wedge 0 < \text{sint } \text{posted} \wedge \\ & \text{adjust-offer-with-options } \text{price-n } \text{price-d } \text{posted int64-max} \\ & \text{repaired-exchange-options} = \text{Cxx-Ok } \text{posted} \end{aligned}$$

stored-offer is stated with *legacy-exchange-options* because the proofs below consume it in that form. The invariant is the same at both protocols: an unlimited adjustment cannot tell the two option records apart. Proof sketch: unfold the invariant and rewrite with the preceding corollary.

9.3 Unlimited replay of a protocol-28 adjustment

lemma *legacy-positive-adjustment-replays-unlimited:*

$$\begin{aligned} & \text{assumes } \text{pn: } 0 < \text{sint } \text{price-n} \\ & \text{and } \text{pd: } 0 < \text{sint } \text{price-d} \\ & \text{and } \text{wheat-nonnegative: } 0 \leq \text{sint } \text{max-wheat} \\ & \text{and } \text{sheep-nonnegative: } 0 \leq \text{sint } \text{max-sheep} \\ & \text{and } \text{adjusted:} \\ & \quad \text{adjust-offer-with-options } \text{price-n } \text{price-d } \text{max-wheat } \text{max-sheep} \\ & \quad \text{legacy-exchange-options} = \text{Cxx-Ok } \text{result} \\ & \text{and } \text{result-positive: } 0 < \text{sint } \text{result} \\ & \text{shows } \text{adjust-offer-with-options } \text{price-n } \text{price-d } \text{result int64-max} \\ & \quad \text{legacy-exchange-options} = \text{Cxx-Ok } \text{result} \end{aligned}$$

Proof sketch: when wheat is not more valuable, the exact option is inert, so the established positive exact-adjustment replay theorem applies. When

wheat is more valuable, characterize the successful legacy result in integers. Its wheat quotient proves that the returned amount-price product fits under the original receive cap and hence under the retained global clamp. Replacing both caps by the signed maximum therefore reproduces the same amount pair and the same successful threshold decision.

9.4 Preventative adjustment at protocol 29

lemma *stored-offer-repaired-adjust-stable:*

assumes *stored: stored-offer price-n price-d posted*

and *maker-wf: party-state-wf maker-at-cross*

and *cover:*

maker-covers-offer-liabilities price-n price-d posted maker-at-cross

and *release:*

*release-offer-liabilities price-n price-d posted maker-at-cross =
Cxx-Ok released*

shows

*adjust-stable price-n price-d posted released
repaired-exchange-options*

Proof sketch: the intrinsic invariant supplies the positive unlimited legacy fixed point. Covered release identifies its selling liability with the stored amount and exposes at least that selling capacity plus the booked buying capacity. The exact-cap replay lemma therefore preserves the full amount at the released receive cap. Adjustment never takes the symmetric branch against its unlimited counterparty, so enabling the second repaired flag preserves the result as well.

9.5 Successful-cross safety infrastructure

lemma *party-receive-within-capacity-nonnegative:*

assumes *wf: party-state-wf party*

and *delta-nonnegative: $0 \leq \text{sint } \delta$*

and *within: $\text{sint } \delta \leq \text{sint } (\text{can-buy-at-most } \text{party})$*

and *receive: party-receive-buy-asset party $\delta = \text{Cxx-Ok}$ after*

shows

*party-state-wf after $\wedge$
can-sell-at-most after = can-sell-at-most party*

Proof sketch: a zero delta is the identity. A nonzero non-negative signed word is positive, so the established checked-receive lemma supplies the unique successful state and its unchanged selling capacity.

lemma *party-spend-within-capacity-nonnegative:*

assumes *wf: party-state-wf party*

and *delta-nonnegative: $0 \leq \text{sint } \delta$*

and *within*: $\text{ sint } \delta \leq \text{ sint } (\text{ can-sell-at-most party })$
and *spend*: $\text{ party-spend-sell-asset party } \delta = \text{ Cxx-Ok after }$
shows *party-state-wf after*

Proof sketch: spending zero is the identity. Otherwise non-negativity makes the delta positive, and the established checked-spend lemma supplies the unique well-formed result.

lemma *add-liability-checked-success-bounds*:
assumes *result*:
 add-liability-checked cap current delta = Cxx-Ok updated
and *cap-max*: $\text{ cap } \leq \text{ int64-max-int }$
shows $0 \leq \text{ sint updated sint updated } \leq \text{ cap }$

Proof sketch: a successful checked addition has an exact integer total between zero and its supplied cap. Since that cap is within the positive signed range, conversion back to the signed word preserves the total.

lemma *acquire-offer-liabilities-preserves-wf*:
assumes *wf*: *party-state-wf maker*
and *acquire*:
 acquire-offer-liabilities price-n price-d amount maker =
 Cxx-Ok maker-after
shows *party-state-wf maker-after*

Proof sketch: invert the two liability projections and checked additions. The selling cap is the maker's balance and the buying cap is its limit minus balance; well-formedness places both inside the signed maximum. Successful checked additions leave the new totals between zero and those caps, which is precisely the party-state invariant after updating the two liability fields.

theorem *stored-offer-repaired-cross-preserves-invariants*:
assumes *stored*: *stored-offer price-n price-d posted*
and *maker-wf*: *party-state-wf maker-at-cross*
and *cover*:
 maker-covers-offer-liabilities price-n price-d posted maker-at-cross
and *taker-wf*: *party-state-wf taker*
and *cross*:
 cross-offer-v10 price-n price-d posted maker-at-cross taker
 taker-amount rounding repaired-exchange-options = Cxx-Ok crossed
shows
 party-state-wf (cross-maker crossed) ∧
 party-state-wf (cross-taker crossed) ∧
 (cross-offer-amount crossed = 0 ∨
 maker-covers-offer-liabilities price-n price-d

(cross-offer-amount crossed) (cross-maker crossed))

Proof sketch: covered release yields a well-formed maker with the stored liabilities restored to live capacity. The common exchange bounds apply to every successful repaired result and every rounding mode, including zero results. They justify each checked maker and taker balance move, preserving both party invariants. A non-staying or dust-erasure branch has zero remainder. Otherwise the positive branch succeeds only after reacquiring the adjusted remainder's liabilities; checked acquisition preserves maker well-formedness and makes that exact remainder covered.

9.6 No phantom full take

lemma *stored-offer-nonstaying-repaired-cross-is-full-take:*

assumes *stored: stored-offer price-n price-d posted*

and *maker-wf: party-state-wf maker-at-cross*

and *cover:*

maker-covers-offer-liabilities price-n price-d posted maker-at-cross

and *cross:*

cross-offer-v10 price-n price-d posted maker-at-cross taker

taker-amount rounding repaired-exchange-options = Cxx-Ok crossed

and *nonstaying: $\neg$ cross-wheat-stays crossed*

shows

cross-wheat-received crossed = posted $\wedge$

cross-offer-amount crossed = 0

Proof sketch: invert the non-staying crossing to expose liability release, preventative adjustment, exchange, and the branch's zero remainder. The repaired stability lemma makes the preventative adjustment return the positive stored amount. Positive idempotence identifies its send cap with that amount, and the flag-independent non-staying exchange theorem then forces every successful rounding mode to transfer the actual stored amount.

9.7 Existential safe full take

theorem *stored-offer-has-safe-full-cross-repaired:*

assumes *stored: stored-offer price-n price-d posted*

and *maker-wf: party-state-wf maker-at-cross*

and *cover:*

maker-covers-offer-liabilities price-n price-d posted maker-at-cross

shows

$\exists$ *crossed.*

cross-offer-v10 price-n price-d posted maker-at-cross

maximum-capacity-taker int64-max Exchange-Normal

repaired-exchange-options = Cxx-Ok crossed $\wedge$

cross-wheat-received crossed = posted $\wedge$

0 < sint (cross-sheep-send crossed) $\wedge$

$\neg$ cross-wheat-stays crossed $\wedge$

cross-offer-amount crossed = 0 $\wedge$
party-state-wf (cross-maker crossed) $\wedge$
party-state-wf (cross-taker crossed)

Proof sketch: covered release restores at least the stored offer's selling and buying liabilities as live capacities. Repaired adjustment stability and positive idempotence make the preventative send cap exactly the stored amount. The maximum-capacity taker realizes the same repaired exchange as adjustment, hence transfers all stored wheat and a positive sheep amount. Generic cap bounds justify all four checked balance moves, and an unlimited counterparty cannot leave wheat resting, so the constructed successful result has no remainder and both parties remain well formed.

9.8 Positive remainder provenance and recursive takeability

lemma *positive-repaired-adjustment-is-stored-offer:*

assumes *pn: 0 < sint price-n*
and *pd: 0 < sint price-d*
and *adjustment:*
 adjust-offer-with-options price-n price-d max-wheat max-sheep
 repaired-exchange-options = Cxx-Ok result
and *result-positive: 0 < sint result*
shows *stored-offer price-n price-d result*

Proof sketch: expose the full exchange behind the positive adjustment. Its common cap bounds make both adjustment caps non-negative, and its positive wheat result implies a positive sheep result. Symmetric irrelevance reduces the repaired adjustment to the exact-only form; the established positive exact replay then supplies the unlimited legacy fixed point required by the intrinsic stored-offer invariant.

lemma *positive-legacy-adjustment-is-stored-offer:*

assumes *pn: 0 < sint price-n*
and *pd: 0 < sint price-d*
and *adjustment:*
 adjust-offer-with-options price-n price-d max-wheat max-sheep
 legacy-exchange-options = Cxx-Ok result
and *result-positive: 0 < sint result*
shows *stored-offer price-n price-d result*

Proof sketch: the successful positive adjustment's exchange bounds imply non-negative send and receive caps. The legacy replay theorem then lifts the returned amount directly to its unlimited fixed point.

lemma *successful-positive-cross-remainder-has-adjustment:*

assumes *cross*:

cross-offer-v10 price-n price-d offer-amount maker taker taker-amount

rounding options = Cxx-Ok crossed

and *remainder-positive: 0 < sint (cross-offer-amount crossed)*

obtains *max-wheat max-sheep where*

adjust-offer-with-options price-n price-d max-wheat max-sheep options =
Cxx-Ok (cross-offer-amount crossed)

Proof sketch: invert a successful crossing through release, preventative adjustment, exchange, and all four balance moves. A non-staying branch and the dust-erasure sub-branch both expose a zero remaining amount, contradicting positivity. The only remaining branch is the successful post-trade adjustment followed by liability acquisition, and its adjusted amount is exactly the positive amount stored in the crossing result.

lemma *repaired-positive-cross-remainder-is-stored-offer*:

assumes *pn: 0 < sint price-n*

and *pd: 0 < sint price-d*

and *cross*:

cross-offer-v10 price-n price-d offer-amount maker taker taker-amount

rounding repaired-exchange-options = Cxx-Ok crossed

and *remainder-positive: 0 < sint (cross-offer-amount crossed)*

shows

stored-offer price-n price-d (cross-offer-amount crossed)

Proof sketch: a positive remainder exposes the repaired post-trade adjustment that wrote it; every positive repaired adjustment result is an unlimited legacy fixed point because the clamp makes unrestricted repaired and legacy projections coincide.

lemma *legacy-positive-cross-remainder-is-stored-offer*:

assumes *pn: 0 < sint price-n*

and *pd: 0 < sint price-d*

and *cross*:

cross-offer-v10 price-n price-d offer-amount maker taker taker-amount

rounding legacy-exchange-options = Cxx-Ok crossed

and *remainder-positive: 0 < sint (cross-offer-amount crossed)*

shows

stored-offer price-n price-d (cross-offer-amount crossed)

Proof sketch: the same successful-remainder inversion exposes a positive legacy adjustment, whose result is an unlimited legacy fixed point by the legacy replay theorem.

theorem *repaired-cross-remainder-remains-safely-takeable*:
assumes *stored*: *stored-offer price-n price-d posted*
and *maker-wf*: *party-state-wf maker-at-cross*
and *cover*:
 maker-covers-offer-liabilities price-n price-d posted maker-at-cross
and *taker-wf*: *party-state-wf taker*
and *cross*:
 cross-offer-v10 price-n price-d posted maker-at-cross taker
 taker-amount rounding repaired-exchange-options = Cxx-Ok crossed
and *remainder-positive*: $0 < \text{sint } (\text{cross-offer-amount crossed})$
shows
 maker-covers-offer-liabilities price-n price-d
 $(\text{cross-offer-amount crossed}) (\text{cross-maker crossed}) \wedge$
 $(\exists \text{next.}$
 cross-offer-v10 price-n price-d (cross-offer-amount crossed)
 $(\text{cross-maker crossed}) \text{maximum-capacity-taker int64-max}$
 Exchange-Normal repaired-exchange-options = Cxx-Ok next $\wedge$
 cross-wheat-received next = cross-offer-amount crossed $\wedge$
 $0 < \text{sint } (\text{cross-sheep-send next}) \wedge$
 $\neg \text{cross-wheat-stays next} \wedge$
 cross-offer-amount next = 0 $\wedge$
 party-state-wf (cross-maker next) $\wedge$
 party-state-wf (cross-taker next))

Proof sketch: universal successful-cross safety makes the final maker well formed and covers every nonzero reacquired remainder. Successful positive remainder provenance shows that the repaired post-trade adjustment wrote an intrinsic legacy fixed point. Applying the existential theorem to that final amount and maker state constructs a second safe repaired cross that consumes the entire remainder with the maximum-capacity taker.

10 Protocol-29 takeability and limit-adjustment stability

Two properties of the predicate catalogue follow from the existential full-take theorem above, because it already produces a protocol-29 cross by a maximum-capacity counterparty. They are stated here rather than in the lifecycle theory because that theorem lives here.

lemma *maker-covers-offer-liabilities-buy-limit-update [simp]*:
 maker-covers-offer-liabilities price-n price-d amount
 $(\text{maker}(\text{buy-limit} := \text{new-limit})) =$
 maker-covers-offer-liabilities price-n price-d amount maker
 — Coverage reads only the maker’s booked liabilities, so changing the buying limit cannot disturb it.

theorem *posted-offers-remain-takeable-repaired*:

posted-offers-remain-takeable repaired-exchange-options

— From protocol 29 on, an offer created by a successful posting can still be crossed for positive amounts by some well-formed taker, whenever the maker covers its liabilities.

Proof sketch: the symmetric option is inert for a successful posting, so normalize the repaired post to a false symmetric option and replay it to the unlimited legacy fixed point. With positive price and amount that is exactly the intrinsic stored-offer invariant, so the existential full-take theorem applies to the covered offer and its maximum-capacity counterparty is the required witness.

theorem *limit-adjustment-stable-repaired:*

limit-adjustment-stable repaired-exchange-options

— From protocol 29 on, changing a maker’s buying limit to any value that leaves it well formed cannot make its posted offer untakeable.

Proof sketch: a successful posting acquires the offer’s liabilities, so the resulting maker covers them. Changing only the buying limit leaves those booked liabilities untouched, so coverage survives the change, and protocol-29 takeability then supplies the taker.

corollary *posted-offers-remain-takeable-p29:*

protocol-version-starts-from ledger-version protocol-version-v29 $\implies$

posted-offers-remain-takeable (exchange-options-at-version ledger-version)

— The takeability property, stated through the protocol mapping.

corollary *limit-adjustment-stable-p29:*

protocol-version-starts-from ledger-version protocol-version-v29 $\implies$

limit-adjustment-stable (exchange-options-at-version ledger-version)

— Limit-adjustment stability, stated through the protocol mapping.

corollary *limit-adjustment-stable-p28-false:*

$\neg$ *protocol-version-starts-from ledger-version protocol-version-v29 $\implies$*

$\neg$ *limit-adjustment-stable (exchange-options-at-version ledger-version)*

— The same property is false before protocol 29, so this is a behavioral repair rather than a restatement.

limit-adjustment-stable-p29 together with *limit-adjustment-stable-p28-false* explains a design choice in the protocol-29 change that is otherwise unexplained by this model. Before protocol 29, reducing a maker’s buying limit to exactly its booked liability could make a posted offer untakeable, which is why stellar-core refuses such offers at overlay admission. From protocol 29 the property holds outright, which is why *doCheckValidForOverlay* returns immediately at that version: the filter has nothing left to prevent.

The overlay filter itself is out of scope here, and this pair is not a proof about it. It is the statement of the ledger-level property the filter existed to protect, before and after the boundary.

11 Protocol-29 takeability and limit stability for ManageBuy

The provenance and catalogue results above enter through *post-offer*, the ManageSell route. A *ManageBuyOffer* reaches the book through *post-buy-offer*, which differs in three ways that matter here: the resting offer sits at the inverted canonical price (*price-d*, *price-n*), the operation caps are an unlimited send cap and a finite receive cap of *buy-amount*, and the request-time liabilities are versioned. The results about a covered offer satisfying *stored-offer* apply to such an offer once it is known to satisfy the invariant; the ManageSell provenance and catalogue results do not apply to it.

This section supplies the missing provenance. Its pivot is that neither *positive-legacy-adjustment-is-stored-offer* nor *positive-repaired-adjustment-is-stored-offer* mentions a party state or a cap shape: both recover cap non-negativity from the exchange bounds of the successful adjustment itself. Composing either with the core created-branch inversion therefore gives stored-offer provenance for *any* posting through *post-offer-core*, at either side of the protocol boundary, with no well-formedness hypothesis at all.

11.1 Stored-offer provenance for the common posting core

lemma *exchange-options-at-version-cases*:

exchange-options-at-version ledger-version = *legacy-exchange-options* $\vee$
exchange-options-at-version ledger-version = *repaired-exchange-options*
 — The protocol mapping has exactly two configurations.

lemma *post-offer-core-created-is-stored-offer*:

assumes *post*:

post-offer-core price-n price-d request-valid buying-liability
selling-liability max-send-cap max-receive-cap maker options =
Cxx-Ok (Post-Created posted maker-after)

and *options*:

options = *legacy-exchange-options* $\vee$
options = *repaired-exchange-options*

shows *stored-offer price-n price-d posted*

Proof sketch: the created-branch inversion supplies positive price components, a positive posted amount, and the adjustment call that returned it. Whichever of the two configurations the post used, the matching positive-adjustment provenance lemma turns that successful adjustment into the

unlimited legacy fixed point, recovering both non-negative caps from the adjustment's own exchange bounds. No party-state well-formedness and no assumption about the cap shape is needed.

theorem *buy-post-created-is-stored-offer:*

assumes *post:*

*post-buy-offer ledger-version price-n price-d buy-amount maker =
Cxx-Ok (Post-Created posted maker-after)*

shows *stored-offer price-d price-n posted*

— An offer created by a *ManageBuyOffer* satisfies the intrinsic stored-offer invariant at its inverted canonical price.

Proof sketch: unfold the buy wrapper into the common posting core and apply the core provenance lemma, whose two admissible configurations are exactly the range of the protocol mapping. The core's price arguments are the submitted price components in the opposite order, which is where the inverted conclusion comes from.

11.2 Takeability and limit stability

theorem *posted-buy-offers-remain-takeable-p29:*

assumes *activation:*

protocol-version-starts-from ledger-version protocol-version-v29

shows *posted-buy-offers-remain-takeable ledger-version*

— From protocol 29 on, an offer created by a successful *ManageBuyOffer* can still be crossed for positive amounts by some well-formed taker, whenever the maker covers its liabilities.

Proof sketch: buy provenance turns the created post directly into the intrinsic stored-offer invariant at the inverted price — no symmetric normalization and no replay step is needed here, because the provenance lemma already absorbed both. The existential full-take theorem then applies to the covered offer, and its maximum-capacity counterparty is the required witness.

theorem *buy-limit-adjustment-stable-p29:*

assumes *activation:*

protocol-version-starts-from ledger-version protocol-version-v29

shows *buy-limit-adjustment-stable ledger-version*

— From protocol 29 on, changing a maker's buying limit to any value that leaves it well formed cannot make its *ManageBuyOffer* untakeable.

Proof sketch: a successful buy posting acquires the created offer's liabilities at the inverted canonical price, so the resulting maker covers them. Changing only the buying limit leaves those booked liabilities untouched, so

coverage survives the change, and protocol-29 ManageBuy takeability then supplies the taker.

Both routes are now covered at protocol 29. The request-level corollaries below say so in one statement, over the *offer-request* constructor that *post-offer-request* dispatches on, without introducing a new predicate: the per-route theorems are the two cases.

corollary *posted-request-offers-remain-takeable-p29*:

assumes *activation*:

protocol-version-starts-from ledger-version protocol-version-v29

and *post-wf*: *party-state-wf maker-at-post*

and *cross-wf*: *party-state-wf maker-at-cross*

and *post*:

post-offer-request ledger-version request maker-at-post =

Cxx-Ok (Post-Created posted maker-after)

and *cover*:

maker-covers-offer-liabilities (request-canonical-price-n request)

(request-canonical-price-d request) posted maker-at-cross

shows

$\exists$ *taker taker-amount crossed*.

party-state-wf taker $\wedge$

$0 < \text{sint } (\text{can-buy-at-most taker}) \wedge$

$0 < \text{sint } (\text{signed-min64 taker-amount } (\text{can-sell-at-most taker})) \wedge$

cross-offer-v10 (request-canonical-price-n request)

(request-canonical-price-d request) posted maker-at-cross taker

taker-amount Exchange-Normal

(exchange-options-at-version ledger-version) = Cxx-Ok crossed $\wedge$

$0 < \text{sint } (\text{cross-wheat-received crossed}) \wedge$

$0 < \text{sint } (\text{cross-sheep-send crossed})$

— From protocol 29 on, every offer created by either posting operation remains takeable while its maker covers it.

Proof sketch: split on the request constructor. The sell case is the established repaired takeability theorem applied through the sell-wrapper refinement, the buy case is the ManageBuy theorem above, and the canonical price projections are the arguments each route already uses.

end

theory *Offer-Exchange-Migration*

imports *Offer-Exchange-Stored-Offers*

begin

12 Exact-cap protocol migration

This theory studies a protocol boundary rather than a same-version lifecycle. A resting offer may have been admitted and had its liabilities acquired using

legacy-exchange-options; after activation, liability release is still flag-free, while preventative adjustment and *cross-offer-v10* use *repaired-exchange-options*. The stored-offer theory shows that every covered offer satisfying the invariant *stored-offer* remains safe to cross and has a maximum-capacity full-take witness at protocol 29. This theory applies those results to offers posted at protocol 28: it establishes the invariant for them and restates the results through the protocol mapping.

The invariant is proved for the modeled creation routes: a successful protocol-28 *post-offer*, a successful *post-buy-offer* at any ledger version, and any positive remainder left by a successful crossing under either option record. The local model does not prove that every historical ledger entry has such provenance. Applying an intrinsic result to passive offers, offer updates, or offers that survived earlier protocol migrations or an administrative upgrade pass additionally requires a reachability argument that those live entries satisfy the intrinsic invariant. In real ledger state, the local assumption *maker-covers-offer-liabilities* is supplied by the global invariant relating aggregate liabilities to all open offers.

12.1 Counterexample-first executable probes

definition *migration-probe-maker* :: *party-state* **where**

migration-probe-maker =
 $\langle \langle \text{sell-balance} = 8, \text{sell-liabilities} = 0, \\ \text{buy-limit} = 32, \text{buy-balance} = 0, \text{buy-liabilities} = 0 \rangle \rangle$

definition *small-upgrade-case* ::

int32 $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *bool*

where

small-upgrade-case price-n price-d amount $\longleftrightarrow$
 (case *post-offer price-n price-d amount migration-probe-maker*
legacy-exchange-options of
Cxx-Ok (Post-Created posted maker-after) $\Rightarrow$
stored-offer price-n price-d posted $\wedge$
party-state-wf maker-after $\wedge$
maker-covers-offer-liabilities price-n price-d posted
maker-after $\wedge$
 (case *cross-offer-v10 price-n price-d posted maker-after*
maximum-capacity-taker int64-max Exchange-Normal
repaired-exchange-options of
Cxx-Ok crossed $\Rightarrow$
cross-wheat-received crossed = *posted* $\wedge$
 $0 < \text{sint } (\text{cross-sheep-send crossed}) \wedge$
 $\neg \text{cross-wheat-stays crossed} \wedge$
cross-offer-amount crossed = 0 $\wedge$
party-state-wf (cross-maker crossed) $\wedge$
party-state-wf (cross-taker crossed)
 | *Cxx-Err* - $\Rightarrow$ *False*)

| - $\Rightarrow$ True)

This bounded search covers all positive price numerators, denominators, and amounts from one through four. Every created legacy offer in that grid is an unlimited legacy fixed point, its acquired maker state is well formed and covers the stored liabilities, and a repaired maximum-capacity cross is a safe positive full take. Thus the small search found no instance of the liability, preventative-adjustment, maximum-taker, invariant, or phantom-erasure failure classes. Non-created requests are outside the implication, exactly as in the later provenance results.

lemma *bounded-small-upgrade-search:*

list-all (λ price-n.
list-all (λ price-d.
list-all (λ amount.
small-upgrade-case price-n price-d amount)
([1, 2, 3, 4] :: int64 list))
([1, 2, 3, 4] :: int32 list))
([1, 2, 3, 4] :: int32 list)

Proof sketch: execute the finite sixty-four-case grid against the bit-precise posting, liability, adjustment, crossing, and party-state functions.

The first concrete boundary witness is the historical one-unit offer at price *real-of-int 101 / 100*. Its maker is exactly at both booked liabilities after an intervening balance increase. The legacy crossing erased it at zero, whereas the repaired crossing transfers the stored unit and leaves both parties well formed.

lemma *legacy-alice-offer-crosses-safely-after-upgrade:*

party-state-wf
($\text{sell-balance} = 1, \text{sell-liabilities} = 1,$
 $\text{buy-limit} = 2, \text{buy-balance} = 1, \text{buy-liabilities} = 1$) $\wedge$
maker-covers-offer-liabilities 101 100 1
($\text{sell-balance} = 1, \text{sell-liabilities} = 1,$
 $\text{buy-limit} = 2, \text{buy-balance} = 1, \text{buy-liabilities} = 1$) $\wedge$
(*case cross-offer-v10 101 100 1*
($\text{sell-balance} = 1, \text{sell-liabilities} = 1,$
 $\text{buy-limit} = 2, \text{buy-balance} = 1, \text{buy-liabilities} = 1$)
maximum-capacity-taker int64-max Exchange-Normal
repaired-exchange-options of
Cxx-Ok crossed $\Rightarrow$
cross-wheat-received crossed = 1 $\wedge$
cross-sheep-send crossed = 1 $\wedge$
 $\neg$ *cross-wheat-stays crossed* $\wedge$
cross-offer-amount crossed = 0 $\wedge$
party-state-wf (*cross-maker crossed*) $\wedge$
party-state-wf (*cross-taker crossed*)
| *Cxx-Err* - $\Rightarrow$ False)

Proof sketch: evaluate liability coverage, repaired exact-cap arithmetic, the four checked balance moves, and the final no-remainder record.

A reciprocal-side witness exercises the symmetric repaired branch. The resting offer is priced at one hundred sheep per one hundred one wheat. A taker capped at ninety-nine units on both sides produces a positive ninety-nine-for-ninety-nine partial fill; the maker remains well formed and covers the positive remainder.

lemma *symmetric-repaired-branch-partial-remainder-probe:*

```
(case post-offer 100 101 200
  (sell-balance = 200, sell-liabilities = 0,
   buy-limit = 199, buy-balance = 0, buy-liabilities = 0)
  legacy-exchange-options of
  Cxx-Ok (Post-Created posted maker-after) ⇒
  (case cross-offer-v10 100 101 posted maker-after
    (sell-balance = 99, sell-liabilities = 0,
     buy-limit = 99, buy-balance = 0, buy-liabilities = 0)
    99 Exchange-Normal repaired-exchange-options of
    Cxx-Ok crossed ⇒
    cross-wheat-received crossed = 99 ∧
    cross-sheep-send crossed = 99 ∧
    cross-wheat-stays crossed ∧
    0 < sint (cross-offer-amount crossed) ∧
    party-state-wf (cross-maker crossed) ∧
    party-state-wf (cross-taker crossed) ∧
    maker-covers-offer-liabilities 100 101
    (cross-offer-amount crossed) (cross-maker crossed)
  | Cxx-Err - ⇒ False)
| - ⇒ False)
```

Proof sketch: execute legacy posting followed by a repaired constrained cross, including repaired post-trade adjustment and liability reacquisition.

A particular incoming offer may still exchange nothing. Here the maker has unrelated extra buying headroom, while a one-sheep incoming cap is too small for an integer unit at price *real-of-int 101 / 100*. The safe outcome keeps the covered resting unit; this is a particular-taker liveness limit, not a migration safety failure or an intrinsically frozen offer.

lemma *repaired-particular-taker-zero-fill-is-safe:*

```
(case cross-offer-v10 101 100 1
  (sell-balance = 1, sell-liabilities = 1,
   buy-limit = 3, buy-balance = 0, buy-liabilities = 1)
  (sell-balance = 1, sell-liabilities = 0,
   buy-limit = 10, buy-balance = 0, buy-liabilities = 0)
  1 Exchange-Normal repaired-exchange-options of
  Cxx-Ok crossed ⇒
  cross-wheat-received crossed = 0 ∧
  cross-sheep-send crossed = 0 ∧
```

$$\begin{aligned}
& \text{cross-wheat-stays crossed} \wedge \\
& \text{cross-offer-amount crossed} = 1 \wedge \\
& \text{party-state-wf} (\text{cross-maker crossed}) \wedge \\
& \text{party-state-wf} (\text{cross-taker crossed}) \wedge \\
& \text{maker-covers-offer-liabilities } 101 \ 100 \ 1 (\text{cross-maker crossed}) \\
& | \text{Cxx-Err} - \Rightarrow \text{False}
\end{aligned}$$

Proof sketch: evaluate the integer-rounding zero trade, the unchanged balance moves, repaired remainder adjustment, and liability reacquisition.

The two path-payment modes are exercised separately from normal mode. With a maximum-capacity taker they both fully consume a representative legacy fixed point and cannot bypass the final cap checks.

lemma *repaired-strict-mode-full-take-probes:*

$$\begin{aligned}
& (\forall \text{rounding} \in \\
& \quad \{ \text{Exchange-Strict-Send}, \text{Exchange-Strict-Receive} \}. \\
& \text{case cross-offer-v10 } 3 \ 2 \ 4 \\
& \quad (\text{sell-balance} = 4, \text{sell-liabilities} = 4, \\
& \quad \text{buy-limit} = 6, \text{buy-balance} = 0, \text{buy-liabilities} = 6) \\
& \quad \text{maximum-capacity-taker int64-max rounding} \\
& \quad \text{repaired-exchange-options of} \\
& \text{Cxx-Ok crossed} \Rightarrow \\
& \quad \text{cross-wheat-received crossed} = 4 \wedge \\
& \quad 0 < \text{sint} (\text{cross-sheep-send crossed}) \wedge \\
& \quad \neg \text{cross-wheat-stays crossed} \wedge \\
& \quad \text{cross-offer-amount crossed} = 0 \wedge \\
& \quad \text{party-state-wf} (\text{cross-maker crossed}) \wedge \\
& \quad \text{party-state-wf} (\text{cross-taker crossed}) \\
& | \text{Cxx-Err} - \Rightarrow \text{False}
\end{aligned}$$

Proof sketch: enumerate the two strict rounding constructors and execute both complete repaired crossings.

Finally, equality and fixed-point evaluation at the largest signed words exercises the retained saturation clamp rather than only small arithmetic.

lemma *signed-saturation-upgrade-probe:*

$$\begin{aligned}
& \text{stored-offer } 2147483647 \ 2147483647 \text{ int64-max} \wedge \\
& \text{exchange-v10-without-price-error-thresholds-with-options} \\
& \quad 2147483647 \ 2147483647 \text{ int64-max int64-max int64-max int64-max} \\
& \quad \text{Exchange-Normal repaired-exchange-options} = \\
& \text{exchange-v10-without-price-error-thresholds-with-options} \\
& \quad 2147483647 \ 2147483647 \text{ int64-max int64-max int64-max int64-max} \\
& \quad \text{Exchange-Normal legacy-exchange-options}
\end{aligned}$$

Proof sketch: execute both unlimited calculations at maximal signed price and amount, where the products require the modeled unsigned wide arithmetic.

12.2 Dependency map for the migration proofs

The results below apply the stored-offer theory to offers posted at protocol 28. They reuse *stored-offer*, *legacy-positive-adjustment-replays-unlimited*, *stored-offer-repaired-adjust-stable*, *stored-offer-nonstaying-repaired-cross-is-full-take*, *stored-offer-has-safe-full-cross-repaired*, *stored-offer-repaired-cross-preserves-invariants*, *repaired-cross-remainder-remains-safely-takeable*, and *buy-post-created-is-stored-offer*. The lifecycle results those proofs rely on are listed at the start of the stored-offer theory.

12.3 Offers posted at protocol 28

A successful protocol-28 *post-offer* establishes the stored-offer invariant. Composed with the stored-offer theory, this gives preventative adjustment stability, the absence of a phantom full take, and an existential safe full take for the posted offer at protocol 29.

lemma *legacy-post-created-is-stored-offer*:

assumes *maker-wf*: *party-state-wf maker-at-post*

and *post*:

post-offer price-n price-d amount maker-at-post

legacy-exchange-options =

Cxx-Ok (Post-Created posted maker-after)

shows *stored-offer price-n price-d posted*

Proof sketch: invert the created posting branch to recover its positive legacy adjustment and the actual posted amount. Well-formedness makes both posting capacities non-negative. *legacy-positive-adjustment-replays-unlimited* lifts that positive result to the unlimited legacy fixed point, while the created- outcome facts supply positivity of the price and stored amount.

theorem *legacy-post-cover-implies-repaired-adjust-stable*:

assumes *post-wf*: *party-state-wf maker-at-post*

and *cross-wf*: *party-state-wf maker-at-cross*

and *post*:

post-offer price-n price-d amount maker-at-post

legacy-exchange-options =

Cxx-Ok (Post-Created posted maker-after)

and *cover*:

maker-covers-offer-liabilities price-n price-d posted maker-at-cross

and *release*:

release-offer-liabilities price-n price-d posted maker-at-cross =

Cxx-Ok released

shows

adjust-stable price-n price-d posted released

repaired-exchange-options

Proof sketch: legacy posting establishes the intrinsic stored-offer invariant, after which the cross-version stability theorem uses only the crossing-time well-formedness, coverage, and actual release result.

theorem *legacy-fully-taken-posted-offer-exchanges-posted-amount-repaired:*

assumes *post-wf: party-state-wf maker-at-post*

and *cross-wf: party-state-wf maker-at-cross*

and *post:*

post-offer price-n price-d amount maker-at-post

legacy-exchange-options =

Cxx-Ok (Post-Created posted maker-after)

and *cover:*

maker-covers-offer-liabilities price-n price-d posted maker-at-cross

and *cross:*

cross-offer-v10 price-n price-d posted maker-at-cross taker

taker-amount rounding repaired-exchange-options = Cxx-Ok crossed

and *nonstaying: $\neg$ cross-wheat-stays crossed*

shows

cross-wheat-received crossed = posted $\wedge$

cross-offer-amount crossed = 0

Proof sketch: legacy posting establishes the intrinsic fixed point; *stored-offer-nonstaying-repaired-cross-is-full-take* then applies to the independently supplied covered crossing state, arbitrary taker, and arbitrary rounding mode.

corollary *legacy-posted-offer-has-safe-full-cross-repaired:*

assumes *post-wf: party-state-wf maker-at-post*

and *cross-wf: party-state-wf maker-at-cross*

and *post:*

post-offer price-n price-d amount maker-at-post

legacy-exchange-options =

Cxx-Ok (Post-Created posted maker-after)

and *cover:*

maker-covers-offer-liabilities price-n price-d posted maker-at-cross

shows

$\exists$ *crossed.*

cross-offer-v10 price-n price-d posted maker-at-cross

maximum-capacity-taker int64-max Exchange-Normal

repaired-exchange-options = Cxx-Ok crossed $\wedge$

cross-wheat-received crossed = posted $\wedge$

0 < sint (cross-sheep-send crossed) $\wedge$

$\neg$ cross-wheat-stays crossed $\wedge$

cross-offer-amount crossed = 0 $\wedge$

party-state-wf (cross-maker crossed) $\wedge$

party-state-wf (cross-taker crossed)

Proof sketch: successful legacy ManageSell posting establishes the intrinsic

stored-offer premise, and the existential theorem uses only the later well-formed covered maker state.

12.4 Reachability boundary

The intrinsic theorems of the stored-offer theory apply to an offer from any origin once *stored-offer* is established. This theory proves that provenance for every successful modeled legacy *post-offer*, which is the ManageSell creation route, and the stored-offer theory proves it for every positive legacy or repaired crossing remainder. Neither derivation uses any overlay-admission filter: a successful posting already yields the unlimited legacy fixed point.

ManageBuy creation is a separate route, because *post-buy-offer* inverts the submitted price and computes its request-time liabilities from the finite buy amount rather than from an unlimited receive cap. The stored-offer theory establishes its provenance separately, as *buy-post-created-is-stored-offer*.

Applying the intrinsic results to arbitrary historical ledger entries still assumes the real ledger's global liabilities-match-offers invariant, which supplies local *maker-covers-offer-liabilities*, and a protocol-upgrade reachability argument that every live entry came from a covered modeled origin or otherwise satisfies *stored-offer*. Native reserve, authorization, sponsorship, order-book selection, and aggregation across multiple offers remain outside this single-offer lifecycle model. Thus the results establish the arithmetic and local-ledger migration obligation, not by themselves the complete network-state reachability argument.

13 The migration ladder at the real protocol boundary

The ladder above and the stored-offer results are phrased with *legacy-exchange-options* for creation and provenance and *repaired-exchange-options* for adjustment and crossing. Those are exactly the two configurations *exchange-options-at-version* selects, and *exchange-options-at-version-legacy* and *exchange-options-at-version-repaired* are simplification rules, so each of those theorems applies verbatim once a ledger version and its side of the boundary are known.

The definitions below name the two sides, and the corollaries restate the two load-bearing results in those terms: the universal safety of a protocol-29 cross of a protocol-28 offer, and the existence of a counterparty that takes such an offer completely.

definition *ledger-version-before-v29* :: *uint32* $\Rightarrow$ *bool*
where
ledger-version-before-v29 ledger-version $\longleftrightarrow$

$\neg$ *protocol-version-starts-from ledger-version protocol-version-v29*

definition *ledger-version-from-v29* :: *uint32* $\Rightarrow$ *bool*

where

ledger-version-from-v29 ledger-version $\longleftrightarrow$

protocol-version-starts-from ledger-version protocol-version-v29

lemma *options-before-v29* [simp]:

ledger-version-before-v29 ledger-version $\implies$

exchange-options-at-version ledger-version = *legacy-exchange-options*

lemma *options-from-v29* [simp]:

ledger-version-from-v29 ledger-version $\implies$

exchange-options-at-version ledger-version = *repaired-exchange-options*

definition *stored-offer-created-before-v29* ::

uint32 $\Rightarrow$ *int32* $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *bool*

where

stored-offer-created-before-v29 creation-version price-n price-d

posted $\longleftrightarrow$

$0 < \text{sint } \text{price-n} \wedge$

$0 < \text{sint } \text{price-d} \wedge$

$0 < \text{sint } \text{posted} \wedge$

adjust-offer creation-version price-n price-d posted int64-max =

Cxx-Ok posted

stored-offer-created-before-v29 is the intrinsic stored-offer invariant expressed through the protocol-facing *adjust-offer*: a positive stored amount that a pre-29 unlimited adjustment leaves unchanged. It is *stored-offer* with the option record replaced by the ledger version that selects it.

lemma *stored-offer-created-before-v29-eq* [simp]:

ledger-version-before-v29 creation-version $\implies$

stored-offer-created-before-v29 creation-version price-n price-d posted =

stored-offer price-n price-d posted

corollary *p28-offer-crossed-at-p29-preserves-invariants*:

assumes *creation*: *ledger-version-before-v29 creation-version*

and *activation*: *ledger-version-from-v29 crossing-version*

and *stored*:

stored-offer-created-before-v29 creation-version price-n price-d posted

and *maker-wf*: *party-state-wf maker-at-cross*

and *cover*:

maker-covers-offer-liabilities price-n price-d posted maker-at-cross

and *taker-wf*: *party-state-wf taker*

and *cross*:

cross-offer-v10 price-n price-d posted maker-at-cross taker

taker-amount rounding

$(\text{exchange-options-at-version crossing-version}) = \text{Cxx-Ok crossed}$

shows *party-state-wf (cross-maker crossed) $\wedge$*

party-state-wf (cross-taker crossed)
 — Every successful protocol-29 cross of a covered protocol-28 offer leaves both parties well formed.

Proof sketch: the activation hypothesis turns the versioned options into the repaired record, and the ladder's successful-cross safety theorem applies to the resulting call unchanged.

corollary *p28-stored-offer-has-safe-full-cross-at-p29:*

assumes *creation: ledger-version-before-v29 creation-version*

and *activation: ledger-version-from-v29 crossing-version*

and *stored:*

stored-offer-created-before-v29 creation-version price-n price-d posted

and *maker-wf: party-state-wf maker-at-cross*

and *cover:*

maker-covers-offer-liabilities price-n price-d posted maker-at-cross

shows

$\exists$ *crossed.*

cross-offer-v10 price-n price-d posted maker-at-cross

maximum-capacity-taker int64-max Exchange-Normal

(exchange-options-at-version crossing-version) = Cxx-Ok crossed $\wedge$

cross-wheat-received crossed = posted $\wedge$

0 < sint (cross-sheep-send crossed) $\wedge$

$\neg$ cross-wheat-stays crossed $\wedge$

cross-offer-amount crossed = 0 $\wedge$

party-state-wf (cross-maker crossed) $\wedge$

party-state-wf (cross-taker crossed)

— Every covered protocol-28 stored offer still has a maximum-capacity protocol-29 counterparty that takes it completely.

Proof sketch: as above, the activation hypothesis reduces the versioned options to the repaired record and the ladder's existential full-take theorem supplies the witness.

corollary *p29-cross-remainder-remains-safely-takeable:*

assumes *creation: ledger-version-before-v29 creation-version*

and *activation: ledger-version-from-v29 crossing-version*

and *stored:*

stored-offer-created-before-v29 creation-version price-n price-d posted

and *maker-wf: party-state-wf maker-at-cross*

and *cover:*

maker-covers-offer-liabilities price-n price-d posted maker-at-cross

and *taker-wf: party-state-wf taker*

and *cross:*

cross-offer-v10 price-n price-d posted maker-at-cross taker

taker-amount rounding

(exchange-options-at-version crossing-version) = Cxx-Ok crossed

and *remainder-positive*: $0 < \text{sint } (\text{cross-offer-amount crossed})$
shows

maker-covers-offer-liabilities price-n price-d
 $(\text{cross-offer-amount crossed}) (\text{cross-maker crossed}) \wedge$
 $(\exists \text{next}.$
 $\text{cross-offer-v10 price-n price-d } (\text{cross-offer-amount crossed})$
 $(\text{cross-maker crossed}) \text{ maximum-capacity-taker int64-max}$
 $\text{Exchange-Normal } (\text{exchange-options-at-version crossing-version}) =$
 $\text{Cxx-Ok next} \wedge$
 $\text{cross-wheat-received next} = \text{cross-offer-amount crossed} \wedge$
 $0 < \text{sint } (\text{cross-sheep-send next}) \wedge$
 $\neg \text{cross-wheat-stays next} \wedge$
 $\text{cross-offer-amount next} = 0 \wedge$
 $\text{party-state-wf } (\text{cross-maker next}) \wedge$
 $\text{party-state-wf } (\text{cross-taker next}))$

— A positive remainder left by a protocol-29 cross of a protocol-28 offer is itself covered, and is taken completely by a maximum-capacity counterparty.

Proof sketch: the activation hypothesis reduces the versioned options to the repaired record, and the ladder’s remainder-closure theorem then applies to the resulting call unchanged.

Reachability. The corollaries above assume *stored-offer*, the intrinsic stored-offer invariant, and *maker-covers-offer-liabilities*. The first is established for the modeled fresh manage-sell route by *legacy-post-created-is-stored-offer* from a successful protocol-28 *post-offer*, for manage-buy creation by *buy-post-created-is-stored-offer*, and for any positive remainder of a successful crossing by *legacy-positive-cross-remainder-is-stored-offer* and *repaired-positive-cross-remainder-is-stored-offer*; the last three are in the stored-offer theory. It is *not* established for passive sell offers, offer updates, or offers that survived earlier protocol migrations or an administrative upgrade pass. Applying these results to such an entry requires a separate argument that it satisfies the intrinsic invariant.

The second assumption is supplied in real execution by the global liabilities-match-offers invariant. The local model does not prove that global invariant, so these are local safety results about a covered offer, not a proof of complete network migration safety. The missing induction over historical ledger state is recorded as follow-up work.

14 The ManageBuy route at protocol 29

The ladder assumes only *stored-offer*, whatever route created the offer, but the protocol-28 corollaries above discharge that premise through *post-offer*, the ManageSell route. The stored-offer theory establishes the invariant for offers created by *post-buy-offer* at any ledger version, as *buy-post-created-is-*

stored-offer. The probes below test ManageBuy-created offers on both sides of the boundary, and the corollaries apply the ladder to them.

14.1 Counterexample-first executable probes

definition *small-buy-upgrade-case* ::

uint32 $\Rightarrow$ *int32* $\Rightarrow$ *int32* $\Rightarrow$ *int64* $\Rightarrow$ *bool*

where

small-buy-upgrade-case *ledger-version* *price-n* *price-d* *buy-amount* $\longleftrightarrow$
 (case *post-buy-offer* *ledger-version* *price-n* *price-d* *buy-amount*
migration-probe-maker of
Cxx-Ok (*Post-Created* *posted* *maker-after*) $\Rightarrow$
stored-offer *price-d* *price-n* *posted* $\wedge$
party-state-wf *maker-after* $\wedge$
maker-covers-offer-liabilities *price-d* *price-n* *posted*
maker-after $\wedge$
 (case *cross-offer-v10* *price-d* *price-n* *posted* *maker-after*
maximum-capacity-taker *int64-max* *Exchange-Normal*
(exchange-options-at-version *ledger-version)* of
Cxx-Ok *crossed* $\Rightarrow$
cross-wheat-received *crossed* = *posted* $\wedge$
0 < *sint* (*cross-sheep-send* *crossed*) $\wedge$
 $\neg$ *cross-wheat-stays* *crossed* $\wedge$
cross-offer-amount *crossed* = *0* $\wedge$
party-state-wf (*cross-maker* *crossed*) $\wedge$
party-state-wf (*cross-taker* *crossed*)
 | *Cxx-Err* - $\Rightarrow$ *False*)
 | - $\Rightarrow$ *True*)

small-buy-upgrade-case is the ManageBuy counterpart of *small-upgrade-case*. It differs in the three ways the buy route differs: the request goes through *post-buy-offer* so its liabilities are the versioned ManageBuy ones and its receive cap is the requested buy amount, every stored-offer claim is made at the inverted price (*price-d*, *price-n*), and the crossing options come from the same ledger version rather than being fixed. The search below therefore probes the created offer for the same five failure classes on both sides of the boundary at once.

lemma *bounded-small-buy-upgrade-search*:

list-all (λ *ledger-version*.

list-all (λ *price-n*.

list-all (λ *price-d*.

list-all (λ *buy-amount*.

small-buy-upgrade-case *ledger-version* *price-n* *price-d* *buy-amount*)

(*[1, 2, 3, 4]* :: *int64* *list*))

(*[1, 2, 3, 4]* :: *int32* *list*))

(*[1, 2, 3, 4]* :: *int32* *list*))

(*[28, 29]* :: *uint32* *list*)

Proof sketch: execute the finite one-hundred-and-twenty-eight-case grid against the bit-precise ManageBuy posting, liability, adjustment, crossing, and party-state functions.

The grid is not vacuous: eighty-five of its one hundred and twenty-eight requests create an offer, so the conjunction above is actually tested. The two witnesses below pin that down for a single case on the protocol-29 side, where the created offer rests at the inverted price 2 over 3 and the maximum-capacity counterparty takes it whole.

lemma *post-buy-offer-created-at-v29-witness:*

post-buy-offer 29 3 2 4 migration-probe-maker =
Cxx-Ok (Post-Created 6
(sell-balance = 8, sell-liabilities = 6,
buy-limit = 32, buy-balance = 0, buy-liabilities = 4))

Proof sketch: evaluate the executable ManageBuy posting path at protocol 29.

lemma *small-buy-upgrade-case-v29-witness:*

small-buy-upgrade-case 29 3 2 4

Proof sketch: evaluate the probe on the created case above.

14.2 ManageBuy-created offers crossed at protocol 29

buy-post-created-is-stored-offer holds at every ledger version, not just before protocol 29, because the intrinsic invariant is about the *stored* projection and both configurations of the adjustment produce a legacy unlimited fixed point when they succeed positively. Unlike *legacy-post-created-is-stored-offer*, which is stated for ManageSell posting in the legacy configuration, it covers every creation version and needs no well-formedness assumption on the posting state. The request-time liabilities remain genuinely version-dependent, as *manage-buy-request-liabilities-change-at-activation* records; they influence *whether* a buy request is admitted, not what the created offer looks like once it is.

With provenance in hand, every ladder theorem applies to a ManageBuy-created offer verbatim. The two load-bearing ones are restated below.

corollary *buy-post-created-offer-has-safe-full-cross-at-p29:*

assumes *post:*

post-buy-offer creation-version price-n price-d buy-amount
maker-at-post = Cxx-Ok (Post-Created posted maker-after)

and *activation: ledger-version-from-v29 crossing-version*

and *maker-wf: party-state-wf maker-at-cross*

and *cover:*

maker-covers-offer-liabilities price-d price-n posted maker-at-cross
shows
 $\exists$ *crossed*.
cross-offer-v10 price-d price-n posted maker-at-cross
maximum-capacity-taker int64-max Exchange-Normal
(exchange-options-at-version crossing-version) = Cxx-Ok crossed $\wedge$
cross-wheat-received crossed = posted $\wedge$
 $0 < \text{sint } (\text{cross-sheep-send } \text{crossed})$ $\wedge$
 $\neg \text{cross-wheat-stays } \text{crossed}$ $\wedge$
cross-offer-amount crossed = 0 $\wedge$
party-state-wf (cross-maker crossed) $\wedge$
party-state-wf (cross-taker crossed)
 — A covered ManageBuy-created offer still has a maximum-capacity protocol-29
 counterparty that takes it completely.

Proof sketch: buy provenance gives the intrinsic invariant at the inverted price, the activation hypothesis reduces the versioned options to the repaired record, and the ladder's existential full-take theorem supplies the witness.

corollary *buy-post-created-offer-crossed-at-p29-preserves-invariants:*
assumes *post:*
post-buy-offer creation-version price-n price-d buy-amount
maker-at-post = Cxx-Ok (Post-Created posted maker-after)
and *activation: ledger-version-from-v29 crossing-version*
and *maker-wf: party-state-wf maker-at-cross*
and *cover:*
maker-covers-offer-liabilities price-d price-n posted maker-at-cross
and *taker-wf: party-state-wf taker*
and *cross:*
cross-offer-v10 price-d price-n posted maker-at-cross taker
taker-amount rounding
(exchange-options-at-version crossing-version) = Cxx-Ok crossed
shows *party-state-wf (cross-maker crossed)* $\wedge$
party-state-wf (cross-taker crossed)
 — Every successful protocol-29 cross of a covered ManageBuy-created offer leaves
 both parties well formed.

Proof sketch: as above, with the ladder's successful-cross safety theorem in place of the existential full take.

end